Principia Softwarica: The Plan 9 Shell `rc`

version 0.1

Yoann Padioleau
yoann.padioleau@gmail.com

with code from
Tom Duff

March 13, 2026

The text and figures are Copyright © 2014–2026 Yoann Padioleau.
All rights reserved.

The source code is Copyright © 2021 Plan 9 Foundation.
MIT license.

Contents

1 Introduction **8**
- 1.1 Motivations . . . 8
- 1.2 The Plan 9 shell: `rc` . . . 9
- 1.3 Other shells . . . 9
- 1.4 Getting started . . . 11
- 1.5 Requirements . . . 11
- 1.6 About this document . . . 12
- 1.7 Copyright . . . 12
- 1.8 Acknowledgments . . . 12

2 Overview **13**
- 2.1 Essential shell features . . . 13
  - 2.1.1 Running commands . . . 13
  - 2.1.2 Redirections . . . 14
  - 2.1.3 Pipes . . . 14
  - 2.1.4 Storing commands in a script . . . 14
  - 2.1.5 A shell as a scripting language . . . 15
- 2.2 `rc` command-line interface . . . 15
- 2.3 `boot.rc` . . . 16
  - 2.3.1 `#!` . . . 16
  - 2.3.2 Initialization script . . . 16
  - 2.3.3 Variables . . . 17
  - 2.3.4 `$path` . . . 17
  - 2.3.5 Comments . . . 17
  - 2.3.6 Quoting and escaping . . . 17
  - 2.3.7 The environment . . . 17
  - 2.3.8 Builtins . . . 18
  - 2.3.9 Other features . . . 18
- 2.4 Code organization . . . 18
- 2.5 Software architecture . . . 18
  - 2.5.1 The command-line user interface . . . 18
  - 2.5.2 `rc`'s components . . . 21
  - 2.5.3 Trace of a simple command: `ls /` . . . 23
- 2.6 Book structure . . . 23

3 Core Data Structures **24**
- 3.1 `Token` . . . 24
- 3.2 Abstract syntax `Tree` . . . 24
- 3.3 Byte`Code` and `codebuf` . . . 27
- 3.4 `Thread` and `runq`eue . . . 28

3.5 Words and lists of words . . . 29
3.5.1 Words . . . 29
3.5.2 List of sequence of words . . . 31
3.6 Variables . . . 31

4 main() . . . 34
4.1 Overview . . . 34
4.2 Command-line arguments processing . . . 34
4.2.1 Login mode: rc -l . . . 35
4.2.2 Interactive mode: rc -i . . . 35
4.3 Initialization . . . 36
4.4 Bootstrapping bytecodes (simplified) . . . 36
4.5 Setting runq . . . 36
4.6 Setting runq->argv . . . 37
4.7 Bytecode interpreter loop . . . 38
4.8 Reading commands: Xrdcmds() . . . 38

5 Input . . . 41
5.1 Overview . . . 41
5.2 Reading a character: getnext() . . . 41
5.2.1 End-of-file management . . . 42
5.2.2 Multiple lines commands and escaped newlines . . . 42
5.2.3 Displaying the prompt . . . 43
5.3 Looking ahead: nextc() and advance() . . . 44

6 Lexing . . . 46
6.1 yylex() . . . 46
6.2 Spaces and comments ('#') . . . 47
6.3 Newlines . . . 48
6.4 Operators ('&', '&&', '|', '||', '<', '>', '$', ...) . . . 48
6.5 Quoted strings ('...') . . . 51
6.6 Keywords and identifiers (if, for, while, switch, fn, ...) . . . 52
6.7 Array subscript (<arr>(<n>)) . . . 54

7 Parsing . . . 55
7.1 Overview . . . 55
7.2 Simple commands (<cmd> <arg1>...<argn>) . . . 56
7.3 Operators . . . 58
7.3.1 Sequences (';', '&') . . . 58
7.3.2 Logical operators ('&&', '||', '!') . . . 58
7.3.3 String matching ('~') . . . 59
7.3.4 Pipe ('|') . . . 59
7.3.5 Redirections ('>', '<') . . . 59
7.4 Control flow statements (if, if not, while, switch, for) . . . 60
7.5 Functions (fn) . . . 61
7.6 Variables (<x> = ...) . . . 61
7.7 Lists ((...)) . . . 61

8 Bytecode Generation and Interpretation **62**
8.1 Overview . . . 62
8.1.1 `emitxxx()` . . . 62
8.1.2 `compile()` . . . 63
8.1.3 `outcode()` . . . 63
8.1.4 `argv` management . . . 64
8.1.5 Process status management . . . 65
8.1.6 Subprocesses management . . . 66
8.2 Simple commands . . . 67
8.2.1 Bytecode generation . . . 67
8.2.2 `Xsimple()` . . . 68
8.2.3 Fork . . . 70
8.2.4 Exec . . . 70
8.2.5 `$path` management . . . 71
8.2.6 Wait . . . 72
8.2.7 Fork optimization . . . 73
8.3 Operators . . . 73
8.3.1 Basic sequence . . . 73
8.3.2 Logical operators . . . 74
8.3.3 String matching . . . 75
8.3.4 Redirection . . . 76
8.3.5 Pipe . . . 81
8.3.6 Asynchronous execution . . . 85
8.4 Control flow statements . . . 86
8.4.1 `if` . . . 86
8.4.2 `while` . . . 87
8.4.3 `for` . . . 88
8.4.4 `switch` . . . 89
8.4.5 Blocks: `'{...}'` . . . 91
8.5 Functions . . . 92
8.5.1 Function definitions (`fn <foo> ... )` . . . 92
8.5.2 Function uses (`<foo>(...)`) . . . 93
8.6 Variables . . . 94
8.6.1 Variable definitions (`<x>=...`) . . . 94
8.6.2 Variable uses (`$<x>`) . . . 96
8.6.3 Special variables . . . 99

9 Builtins **100**
9.1 Overview . . . 100
9.2 `$ cd` . . . 101
9.3 `$ exit` . . . 103
9.4 `$ .` . . . 103
9.5 `$ eval` . . . 105
9.6 `$ wait` . . . 106

10 Environment **108**
10.1 `Updenv()` . . . 108
10.2 `Vinit()` . . . 109

11 Signals 111
11.1 Overview . . . 111
11.2 Registering handlers: `Trapinit()` . . . 112
11.3 Trap dispatch: `dotrap()` . . . 113

12 Initialization 115
12.1 Actual bootstrapping code . . . 115
12.2 Intitialization script and `rc -m /path/to/rcmain` . . . 116
12.3 Actual environment . . . 116

13 Globbing 117
13.1 Lexing globbing characters . . . 117
13.2 Expanding globbing characters . . . 118
13.3 `glob()` . . . 119
13.4 `match()` . . . 122

14 Advanced Topics 126
14.1 Reading commands from a string: `rc -c` . . . 126
14.2 Failing fast: `rc -e` . . . 126
14.3 Unicode . . . 127
14.4 Advanced constructs . . . 127
14.4.1 Subshell: `@ <cmd>` . . . 127
14.4.2 Here documents: `<< <HERE>` . . . 128
14.4.3 Read-write redirections: `<> <file>` . . . 132
14.4.4 General redirections: `>[2] <file>` . . . 133
14.4.5 Advanced dup: `>[<fd0>=<fd1>]`, `<>[<fd0>=<fd1>]`, `<[<fd0>=<fd1>]` . . . 134
14.4.6 Advanced pipes: `|[<fd>]`, `|[<fd0>=<fd1>]` . . . 136
14.4.7 Command output as a file: `'<{<cmd>}'` . . . 136
14.4.8 Command substitution: `` `{<cmd>} `` . . . 137
14.4.9 Stringification of variables: `$"<foo>` . . . 139
14.5 Advanced builtins . . . 140
14.5.1 `$ exec` . . . 140
14.5.2 `$ whatis` . . . 140
14.5.3 `$ rfork` . . . 142
14.5.4 `$ flag` . . . 143
14.5.5 `$ shift` . . . 144
14.5.6 `$ finit` . . . 144

15 Conclusion 146
15.1 Connections to other books . . . 146
15.2 Missing features . . . 146

A Debugging 148
A.1 AST dumper . . . 148
A.2 Bytecode generator trace: `rc -r` . . . 150
A.3 Printing subprocesses status: `rc -s` . . . 152
A.4 Printing commands: `rc -x` . . . 152
A.5 Printing characters: `rc -v` . . . 152
A.6 Printing all characters: `rc -V` . . . 152

B Error Management 153

C Utilities **155**
C.1 Memory management . . . 155
C.2 Command-line arguments . . . 155
C.3 Buffered IO . . . 160
C.4 Format . . . 163
C.5 String conversions . . . 166
C.6 Misc . . . 166

D Examples of `rc` scripts **168**
D.1 `/rc/lib/rcmain` . . . 168
D.2 `/usr/glenda/lib/profile` . . . 169

E Shell Companion Utilities **171**
E.1 `echo` . . . 171
E.2 `pwd` . . . 172
E.3 `test` . . . 172
E.4 `sh`, the simple shell . . . 179

F Extra Code **189**
F.1 `misc/` . . . 189
F.1.1 `misc/echo.c` . . . 189
F.1.2 `misc/pwd.c` . . . 189
F.1.3 `misc/test.c` . . . 189
F.2 `sh/` . . . 190
F.2.1 `sh/sh.c` . . . 190
F.3 `rc/` . . . 192
F.3.1 `rc/fns.h` . . . 192
F.3.2 `rc/getflags.h` . . . 194
F.3.3 `rc/io.h` . . . 194
F.3.4 `rc/rc.h` . . . 194
F.3.5 `rc/exec.h` . . . 197
F.3.6 `rc/globals.c` . . . 197
F.3.7 `rc/getflags.c` . . . 198
F.3.8 `rc/io.c` . . . 198
F.3.9 `rc/input.c` . . . 199
F.3.10 `rc/var.c` . . . 200
F.3.11 `rc/env.c` . . . 200
F.3.12 `rc/glob.c` . . . 200
F.3.13 `rc/executils.c` . . . 201
F.3.14 `rc/exec.c` . . . 201
F.3.15 `rc/processes.c` . . . 204
F.3.16 `rc/tree.c` . . . 204
F.3.17 `rc/lex.c` . . . 205
F.3.18 `rc/trap.c` . . . 205
F.3.19 `rc/simple.c` . . . 206
F.3.20 `rc/pcmd.c` . . . 206
F.3.21 `rc/here.c` . . . 206
F.3.22 `rc/code.c` . . . 207
F.3.23 `rc/pfnc.c` . . . 207
F.3.24 `rc/utils.c` . . . 208

F.3.25 rc/status.c 208
F.3.26 rc/builtins.c 208
F.3.27 rc/path.c 209
F.3.28 rc/fmt.c 209
F.3.29 rc/words.c 210
F.3.30 rc/error.c 210
F.3.31 rc/main.c 211
F.3.32 rc/plan9.c 211
F.3.33 rc/unix.c 212

Glossary **222**

Index **223**

References **231**

Chapter 1

Introduction

The goal of this book is to explain with full details the source code of a shell.

1.1 Motivations

Why a shell? Because I think you are a better programmer if you fully understand how things work under the hood, and the shell is the central piece of the *command-line user interface* (CLI). The shell is a thin layer around the kernel (hence its name) allowing you to run commands in a terminal.

Most users now use a *graphical user interface* (GUI) to execute programs (e.g., macOS, Microsoft Windows, X Window), but most programmers still spend a significant portion of their time in a shell to run compilation commands, editors, debuggers, or *scripts* to automate repetitive tasks. Integrated Development Environments (IDEs) can handle some of those use cases, but the shell still reigns when a programmer needs more flexibility. The power of pipes, redirections, variables, and basic control flow constructs allows sometimes in one command-line to perform tasks that would require hundreds of lines in a regular programming language[1].

In fact, the rise of AI coding assistants like Claude Code has given the command-line a second wind. These tools operate as shell programs, composing existing commands through pipes and scripts rather than through graphical menus. The shell's text-based, composable nature turns out to be a perfect interface for AI agents that can read, write, and chain commands together—something much harder to automate inside a GUI-based IDE.

There are very few books explaining how a shell works. I can cite *Advanced UNIX Programming* [Roc04], but it explains only the code of a mini-shell. This is a pity because the implementation of a real shell covers many interesting topics (e.g., programming language design, compilation, interpretation, system programming, etc.) as you will see soon in this book.

Here are a few questions I hope this book will answer:

- What is the difference between a shell and a terminal? What is the difference between a shell and a login program?
- What happens when the user types `ls` in a terminal? What is the trace of such a command through the different layers of the software stack?
- What are the main features of a shell?
- How are redirections and pipes implemented? What are the system calls involved?
- Why `ls` and `rm` are regular programs but not `cd`? Why `cd` has to be a shell builtin? What is a shell builtin?

[1] For example, [BKM86] contrasts writing a program to count words in Pascal and in a shell.

- How does `C-c`, which interrupts a process, work? Which process receives the signal after an interruption? The shell or the command run from the shell?

1.2 The Plan 9 shell: `rc`

I will explain in this book the code of the Plan 9 shell `rc` [Duf90][2] (for `R`un `C`ommands), which contains about 6700 lines of code (LOC). `rc` is written mostly in C, with its parser using also Yacc [Joh79].

Like for most books in Principia Softwarica, I chose a Plan 9 program because those programs are simple, small, elegant, open source, and they form together a coherent set. Moreover, `rc` is arguably simpler to use and to understand than `sh` [Bou79], the UNIX shell, or any of its derivatives (e.g., `bash` [Ram94], the most popular shell under Linux). For example, by treating the content of any variables uniformly as a list of strings, `rc` does not need the extra operator `$@` used in `sh` (`$*` is enough). Moreover, the syntax of `rc`, specified formally and succintly by a small grammar, is also easier to learn than the (unspecified) syntax of `sh`, partly because `rc` is inspired by the syntax of C with its curly braces (`sh`, instead, is using multiple keywords inspired by Algol).

`rc` itself lacks many of the interactive features found in other shells (e.g., `bash`) such as filename completion, command-line editing, job control, etc. This is partly because under Plan 9, the terminal of the windowing system `rio` is providing instead those features (see the WINDOWS book [Pad16c]).

1.3 Other shells

Here are a few shells that I considered for this book, but which I ultimately discarded:

- The first UNIX shell, originally called `sh`, was written by Ken Thompson, the original author of the UNIX kernel. `sh` started as an assembly program in UNIX V1[3] and finished as a C program in UNIX V6[4]. Ken Thompson's shell introduced the pipe (`'|'`), redirections (`'>'` and `'<'`), as well as wildcard matching[5] (`'*'`). The syntax of those features remained the same in all subsequent shells. Ken Thompson's shell contains only 899 LOC, but it is just a basic command interpreter, not a full scripting language like `rc`.

- The Bourne shell [Bou79], also called `sh`, superseded Ken Thompson's shell (which was renamed `osh`) in UNIX V7[6]. It was written by Stephen Bourne and contains 4145 LOC of C. It is arguably the most famous shell because it defined first the main features that we expect now from a shell: the ability to run commands with pipes and redirections, but also the use of variables and control flow constructs to write scripts. The Bourne shell is both an interactive command interpreter and a full programming language. Most subsequent shells tried to remain backward compatible with the Bourne shell.

  The syntax of the Bourne shell was inspired by Algol with pair of keywords (e.g., `begin/end`, `do/done`, `case/esac`) instead of the curly braces of C[7]. The code of `sh` is smaller than the code of `rc`, but it is also harder to understand. as in `rc`, but instead a complex recursive descent parser.

- Bash [Ram94, New95][8] (for Bourne-Again Shell) is an open-source shell similar to the Bourne shell (hence its name) from the GNU project[9]. It is the default shell in most Linux distributions. It provides many interactive features not found in the Bourne shell such as filename completion, command-line editing,

[2] See http://plan9.bell-labs.com/magic/man2html/1/rc for its manual page.
[3] http://minnie.tuhs.org/cgi-bin/utree.pl?file=V1/sh.s
[4] http://minnie.tuhs.org/cgi-bin/utree.pl?file=V6/usr/source/s2/sh.c
[5] Also known as globbing.
[6] http://minnie.tuhs.org/cgi-bin/utree.pl?file=V7/usr/src/cmd/sh/
[7] Stephen Bourne was such as fan of Algol that the C source code of `sh` itself looks like Algol, thanks to macros such as `THEN`, `BEGIN`, `END`, etc.
[8] https://www.gnu.org/software/bash/
[9] http://www.gnu.org

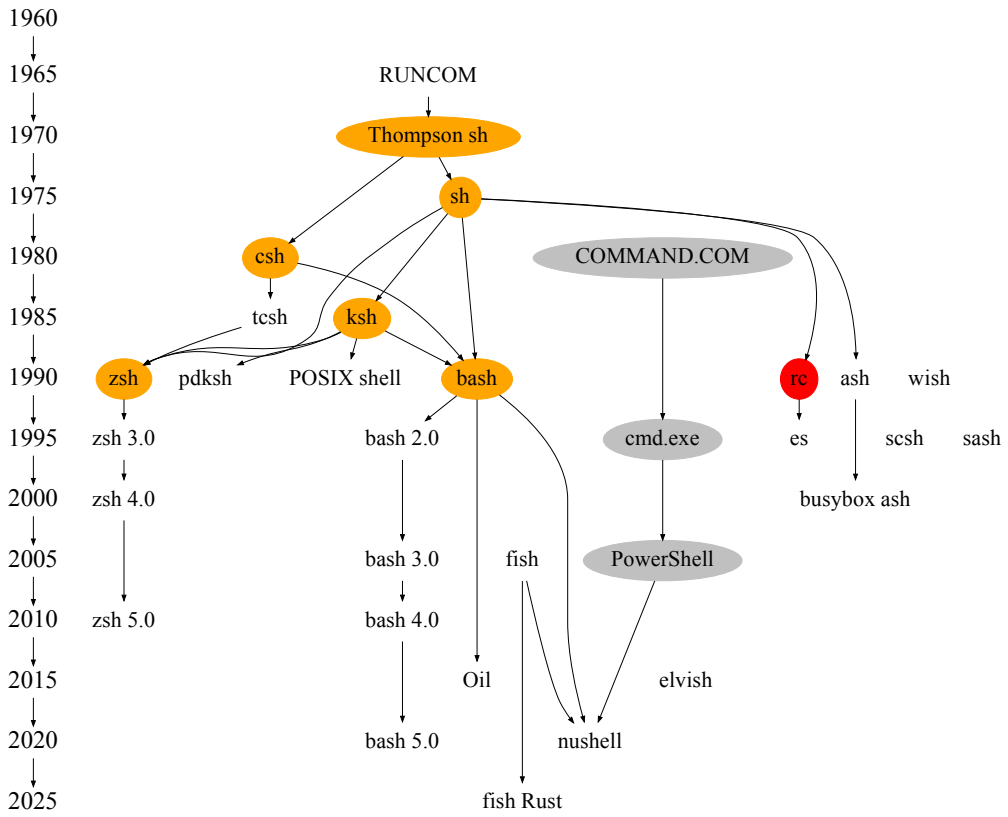

Figure 1.1: Shells timeline

interactive history, job control, etc. Many of those features were partly inspired by the C shell [Joy86], an older shell originating in BSD UNIX using a syntax more similar to C.

Bash relies on the GNU Readline library[10] for many of those interactive features (`rc` relies instead on `rio`'s terminal to provide similar features). However, the codebase of Bash is very large with more than 100 000 LOC (not including the code of the Readline library, which would add another 33 000 LOC), which is more than an order of magnitude more code than in `rc`. Bash's grammar file `parse.y` contains alone 6268 LOC.

The 15× size difference between Bash and `rc` is not just about interactive features. Much of the complexity comes from backward compatibility with the Bourne shell (and POSIX), which forces Bash to support multiple quoting styles, here-documents, arithmetic expansion, brace expansion, parameter expansion with numerous operators (`$x:-default`, `$x##pattern`, etc.), and the context-sensitive grammar inherited from `sh`.

- The Z shell [Kid05][11] is another open-source shell popular among advanced Linux users. It is extensible through plugins and themes[12]. It contains most of the features found in other shells (e.g., Bash, the C Shell, the Korn Shell [BK89, Kor94]) and introduced many new interactive features such as programmable completion, recursive wildcarding with `**/*` (eliminating the need for the program `find`), and more. However, all those features come at a price: the code of the Z shell contains more than 145 000 LOC (not including the tests).

[10] https://tiswww.case.edu/php/chet/readline/rltop.html
[11] http://www.zsh.org/
[12] See https://github.com/robbyrussell/oh-my-zsh for a large repository of such contributions.

Figure 1.1 presents a timeline of major UNIX shells (and a few non-UNIX shells). I think `rc` represents the best compromise for this book: it implements the essential features of a shell while still having a small and understandable codebase (6700 LOC).

1.4 Getting started

To play with `rc`, you will first need to install the Plan 9 fork used in Principia Softwarica (see https://www.principia-softwarica.org/getting-started.html). Once installed, you do not need to do anything to run `rc` because it is the program started by default by the kernel (see the KERNEL book [Pad14]). Once the kernel finished to boot, you should see a percent sign, called the *prompt*, after which you can type any commands as in the following:

```
1   % ls /
2   bin
3   boot
4   ...
5   srv
6   % rc -help
7   Usage: rc [-SsrdiIlxepvV] [-c arg] [-m command] [file [arg...]]
8   % rc
9   % prompt='$ '
10  $ ls /
11  bin
12  boot
13  ...
14  srv
15  $ exit
16  %
```

Line 1 runs the program `ls` to list the content of the root directory. Line 6 and Line 8 show that `rc` is a regular program (just like `ls` at Line 1): you can run a shell program under a shell. Line 8 and Line 9 seem to indicate that typing `rc` has no effect, but the percent sign shown at the beginning of Line 9 is in fact displayed by the `rc` process started by Line 8, not the original `rc` process started by the kernel. Line 9 modifies the *special variable* `prompt` (see Section 8.6.3 for a list of those special variables) to better differentiate the two shell processes. Indeed, Line 10 shows that `'$ '` is the new prompt replacing the percent sign. Finally, Line 15 shows the use of the *builtin* `exit` (see Chapter 9 for more information on builtins) to exit from the shell. Doing so goes back to the preceding shell process (the one launched by the kernel) with the original percent prompt at Line 16.

1.5 Requirements

Because most of this book is made of C source code, you will need a good knowledge of the C programming language [KR88] to understand it. To understand Chapter 7, you will also need to know Yacc [BLM92].

Note that this book is not an introduction to the shell or to shell scripting. I assume you are already familiar with at least one shell, for instance a derivative of `sh` such as `bash`, and so are familiar with concepts such as a pipe, a redirection, what `#!` means, or what a script is. If not, I suggest you to read either the book introducing the UNIX programming environment [KP84], or the original `sh` tutorial [Bou79], or any more recent books on shell scripting [Mic08, New95, Kid05].

A shell is a thin layer on top of a kernel, and so a shell relies heavily on the services offered by the kernel: system calls (e.g., `rfork()` `exec()`, `wait()`, `chdir()`), but also device files (e.g., `/dev/cons` to read and write characters on the terminal). Thus, it can be useful to know how the Plan 9 kernel works (see the KERNEL book [Pad14]), or at least be familiar with sytem programming under UNIX [Roc04, Ste94], to fully understand some of the code in this book.

A shell is also a full programming language, and as you will see soon, `rc` is internally both a compiler and a bytecode interpreter. I assume you also have a basic understanding of how a compiler works, and for example that you know what a lexer or parser is (see the COMPILER book [Pad16b]).

If, while reading this book, you have specific questions on the syntax and interface of `rc`, I suggest you to consult the man page of `rc` at `docs/man/1/rc` in my Plan 9 repository.

Note that the `shells/docs/` directory in my Plan 9 repository contains documents describing either `rc` [Duf00] or `sh` [Bou15]. Those documents are useful to understand some of the design decisions presented in this book, especially how and why `rc` differs from `sh`.

1.6 About this document

This document is a *literate program* [Knu92]. It derives from a set of files processed by a tool, `syncweb` [Pad09], generating either this book or the actual source code of the program. The code and its documentation are thus strongly connected.

1.7 Copyright

Most of this document is made of source code from Plan 9, so those parts are copyright by Plan 9 Foundation. The prose is mine and is licensed under the All Rights Reserved License.

1.8 Acknowledgments

I would like to acknowledge of course the author of `rc`, Tom Duff, who wrote in some sense most of this book.

Chapter 2

Overview

Before showing the source code of `rc` in the following chapters, I first give an overview in this chapter of the general features of a shell. I also quickly describe the command-line interface of `rc` and the specific language supported by `rc`. I also define terms, explain how the code is organized, and more generally give the background necessary to understand the code I will show later.

2.1 Essential shell features

The following sections will explain the essential features of a shell. A shell is a strange beast: it must be both an interactive command interpreter, allowing to run commands easily on one line in a terminal, and a proper programming language, allowing to write complex scripts in a file.

2.1.1 Running commands

The first job of a shell is to allow the user to run commands[1]. Most shells offer a minimalist syntax for running commands, so we can type those commands quickly. Indeed, contrast the shell command `ls /` with the equivalent C program:

⟨`lsroot.c` 13⟩≡

```
#include <u.h>
#include <libc.h>

void main(){
  char* args[] = {"/", nil};
  exec("/bin/ls", args);
}
```

Here are a few notes on the minimalist syntax used by shells:

- *No Parenthesis:* to call a program, you do not need any parenthesis; just type the program name next to its arguments.
- *No quotes:* to pass arguments to the program, you usually do not need any quotes or commas; just type the arguments separated by space. For example, `foo`, `--help`, `42`, `/a/b/c` are all valid arguments[2]. The only time you need to enclose an argument in a quote is when you want to use one of the special character used by the shell (e.g., `'#'`, `'$'`, `'&'`, etc. with `rc`), also known as *meta-characters*.

[1] The very first shell, which was written for the CTSS operating system in 1964, was called `RUNCOM`[Pou00].

[2] You can even sometimes avoid to specify fully all the arguments by using shortcuts, for example, wildcards as explained in Chapter 13.

- *No full path:* to call a program, you do not need to specify its path in the filesystem; just type the name of the program and the shell will automatically find its location. Under UNIX, the `PATH` environment variable stores the candidate locations to find programs.

- *No special ending character:* to finish your command, you do not need any special character like a semicolon; just type a newline and the shell will start interpreting your command[3].

- *No types:* to enter a command, you do not need to specify any types such as `char*` as in C. The arguments are always strings.

Once the command you ran finished, the shell gets back in control and displays another *prompt* to indicate that you can type another command.

2.1.2 Redirections

The second important feature of a shell is to allow to *redirect* the output and input of a program, as in `ls / > listing.txt`.

Many programs (e.g., `ls`, `find`, `rm`, `grep`) live in the command-line world and simply use text for their input and output. By using redirections, you can easily save the output of those programs in a file, or use a previously saved file as input for those programs[4], without even changing the code of those programs. In fact, because under UNIX and Plan 9 "everything is a file", including devices, you can use the same program in many different ways. For example, you can print the listing of the root directory by simply typing `ls / > /dev/printer`.

2.1.3 Pipes

Because of the universality of plain text, you can easily combine many command-line programs. For example, you can list all the C files in your home directory by combining `find` and `grep` with the two commands `find /home/pad/ > /tmp/list.txt` and `grep '\.c$' < /tmp/list.txt`. In fact, thanks to another great feature of the shell, the *pipe*, you can just use one command: `find /home/pad | grep '\.c$'`. This is not only shorter, but also more efficient, and it gives results more quickly. You can even use multiple pipes in the same command as in `find | grep '\.c$' | xargs cat | grep foo`.

Pipes allow to combine easily full programs, just like you can combine functions in a functional language. Pipes are one of the greatest innovations introduced by UNIX, and one of the first programming construct allowing a form of *component-oriented programming*. Each program can be treated as a component, which can be combined with other components. You do not program at the granularity of functions but at the higher granularity of programs.

2.1.4 Storing commands in a script

Just like you can type a series of commands separated by newlines in a terminal, you can save those same commands in a file, a *script*, and execute this script with the shell. As the author of the first shell said [Pou00]: "commands should be usable as building blocks for writing more commands, just like subroutine libraries".

Scripts allow to automate easily repetitive tasks. Moreover, because scripts are commands themselves, they can also be combined, leading to more forms of components-oriented programming.

[3] You can also enter commands on multiple lines, which requires to *escape* the newline as explained in Section 5.2.2.

[4] `rc` allows to redirect not just the standard input and output, as explained in Section 14.4.4.

2.1.5 A shell as a scripting language

Once you can write a sequence of commands in a script, you quickly want more, such as the ability to use conditionals or loops. Thus, most shells are also full programming languages, often referred to as *scripting languages*, with *variables* and many *control flow* constructs, and even function definitions. A scripting language acts like a glue, allowing to combine many programs together.

Because a scripting language must also support the basic commands you type on the command-line, it shares many characteristics with those basic commands: a scripting language does not use types and is interpreted.

A scripting language operates mainly on strings. Because the string arguments you type on the command-line do not usually require any quotes, the use of variables requires then a special operator. Most shells use `'$'` as a prefix to differentiate variables from regular string arguments, for example, `find $home`.

The tension between being an interactive command interpreter and a full programming language has shaped many design choices in shells. Ken Thompson's original V6 shell was purely interactive—it could not be scripted. The Bourne shell added scripting, but the need to keep the minimalist command-line syntax (no quotes, no types, newlines as terminators) forced compromises: variables need the `$` prefix to distinguish them from bare-word arguments, newlines must sometimes be "skipped" after binary operators so that multi-line statements work, and values are always strings.

In `rc`, the basis is even cleaner: all values are lists of strings. This eliminates the `sh` distinction between `$*` (all arguments as one string) and `$@` (all arguments as separate words), and means that `$path` does not need the colon-separated encoding that `$PATH` uses in `sh`.

There are no booleans: control flow is based on the exit status of the command you run. The only comparison operator is `~` (pattern matching on strings), and for numerical tests, the external program `test` is used.

A shell is also a kind of universal read-eval-print loop (REPL): you call programs (like functions), pass arguments (like parameters), use environment variables (like globals), and get results back (via exit status and stdout). The core loop is simply: read a line, parse it, fork, exec, wait, repeat.

2.2 `rc` command-line interface

I just described the main features of a shell. I will now focus exclusively on `rc` and give more details about its command-line interface.

It is rare to run `rc` itself on the command-line, like you run `ls`, `cp`, `grep`, etc. After all, if you have a command-line, you already are in a shell. However, as I said in Section 1.4, the shell under UNIX (and Plan 9) is a regular program. You can run a shell under a shell, which can be useful for example if you do not like the default shell when you log-in[5]. To run `rc`, simply type `rc` on the command-line without any arguments, as I did in Section 1.4 Line 8.

You can also pass to `rc` the name of a script as an argument, as well as the arguments of this script, for example, `rc foo.rc arg1 arg2`. However, this is usually not necessary because most scripts use `#!/bin/rc` at the beginning, which is recognized by the kernel (see the KERNEL book [Pad14]).

Here is the full command-line interface of `rc`:

```
% rc -help
Usage: rc [-SsrdiIlxepvV] [-c arg] [-m command] [file [arg...]]
```

The `-c` flag allows to execute commands from a string passed as an argument instead of from the content of a script (see Section 14.1 for the code handling `rc -c`).

The `-m` flag allows to specify the initialization script of `rc`. I will explain fully the complex initialization process of `rc` and the code of `rc -m` in Chapter 12.

Finally, `rc` supports a few options to provide advanced features or to help debug `rc` itself. I will present gradually those options in this book.

[5] Under UNIX, you can also use the special program `chsh` to change your login shell.

2.3 boot.rc

In this section, I will present an example of an rc script showing a few features of rc's scripting language. The goal here is to illustrate the general features of a shell you have seen in Section 2.1 with the concrete syntax of rc's scripting language. This will also help you understand the grammar of rc I will present in Chapter 7. Finally, this example will introduce a few concepts specifics to rc that are useful to have in mind while I will explain the code of rc in the rest of the document.

boot.rc, below, is the first program executed by the kernel when running on an ARM machine (see the KERNEL book [Pad14]). The following sections will explain the main features of rc used in this script.

⟨*kernel/init/user/boot/arm/boot.rc* 16⟩≡

```
#!/boot/rc -m /boot/rcmain

/boot/echo booooooooting...

path=(/bin /boot)

# basic devices
bind -c '#e' /env
...

# storage
bind -a '#S' /dev
fdisk -p /dev/sdM0/data >/dev/sdM0/ctl
dossrv
mount -c /srv/dos /root /dev/sdM0/dos
bind -a -c /root /

bind -a /arm/bin /bin
bind -a /rc/bin /bin
...

# to use 5c, 5a, 5l by default in mk
objtype=arm
...

exec /boot/rc -m /boot/rcmain -i
```

2.3.1 #!

The first line of boot.rc is #!/boot/rc -m /boot/rcmain. This is a *shebang* line: when the kernel encounters the #! marker at the start of a file being exec()-ed, it runs the specified interpreter with the file as argument instead of treating the file as a binary.

This mechanism, introduced in UNIX V8, is what makes scripts indistinguishable from compiled programs: they can be used as filters in pipes, as components in other scripts, or as standalone commands. Before the kernel handled shebangs, only the shell could run scripts—you had to type rc foo.rc explicitly, and a C program could not exec() a script.

Note that #! also works as a comment, since # starts a comment in rc, so the shebang line is simply ignored when the file is read as a script.

2.3.2 Initialization script

The -m /boot/rcmain flag specifies the initialization script. By default rc sources /rc/lib/rcmain at startup, but during boot the root filesystem is not yet mounted, so the boot script must use -m to point to a copy stored in the boot partition at /boot/rcmain.

Under UNIX, the equivalent mechanism is the cascade of initialization files: `.profile` for the Bourne shell, `.login` for the C shell, `.bashrc` for Bash, etc. In Plan 9, there is a single initialization script `rcmain` shared by all users (see Section D.1 and Chapter 12 for details).

2.3.3 Variables

The `boot.rc` script above shows a variable assignment: `path=(/bin /boot)`. In `rc`, variables hold *lists* of strings, and parentheses are used to create lists: `(a b c)` is a three-element list. This is a key design difference from `sh`, where variables hold a single string and lists are simulated by splitting on `IFS` characters. By treating all values as lists, `rc` avoids many of the quoting pitfalls that plague `sh` scripts.

2.3.4 `$path`

The variable `$path` is a special variable: it tells `rc` where to look for programs. When you type `ls`, the shell searches each directory in `$path` until it finds an executable named `ls`.

In `sh` and `bash`, the equivalent variable is `$PATH`, which encodes the list as a colon-separated string (e.g., `/bin:/usr/bin`). In `rc`, because variables are already lists, `$path` is simply `(/ /bin)`—no special separator needed.

Under Plan 9, the path list is typically short because the system uses *union directories*: multiple directories are bound together into a single mount point like `/bin`, so there is no need for a long search path. This also means you never need the `/usr/bin/env` trick commonly used in UNIX shebang lines. Under UNIX, different systems install programs in different locations (e.g., `/usr/bin/python` vs. `/usr/local/bin/python`), so script authors write `#!/usr/bin/env python` instead of hardcoding the path: `env` searches `$PATH` and runs the first match. Under Plan 9, with union directories, `/bin/rc` is always the right path regardless of the architecture.

See Section 8.2.5 for the implementation.

2.3.5 Comments

Comments in `rc` start with `#` and extend to the end of the line. This is why the `#!` shebang works as described above: the line is simply a comment from `rc`'s perspective.

2.3.6 Quoting and escaping

The `boot.rc` script uses single quotes in `bind -c '#e' /env`: the `#` must be quoted, otherwise the shell would treat it as a comment.

In `rc`, single quotes are the only quoting mechanism. To include a literal single quote inside a quoted string, you double it: `'it''s'`. There are no backslash escapes and no double quotes (the `"|` character has a different meaning in `rc`: variable stringification, see Section 6.5).

2.3.7 The environment

The `boot.rc` script also assigns `objtype=arm`. This is not just a local shell variable: in Plan 9, all shell variables are automatically exported to the environment—there is no need for an `export` command like in `bash`[6]. When `mk` (the build tool) is later run from this shell, it inherits `$objtype` and uses it to select the correct compiler and assembler (e.g., `5c`, `5a`, `5l` for ARM).

Under Plan 9, the environment is stored as files in the `/env` filesystem: each variable becomes a file `/env/varname` whose content is the variable's value. See Chapter 10 for the implementation.

[6] If you need environment isolation, `rfork e` creates a private copy of `/env/`; see Section 14.5.3.

2.3.8 Builtins

The last line of `boot.rc` is `exec /boot/rc -m /boot/rcmain -i`. `exec` is a shell *builtin*: a command that is executed inside the shell process rather than in a child. Builtins exist because some operations must modify the shell's own state. For example, `cd` changes the shell's current directory—if `cd` were a regular program, it would run in a forked child, the child would change *its* directory and exit, and the parent shell would be unaffected.

The `exec` builtin replaces the current shell process with the specified command. Here it replaces the boot shell with an interactive instance of `rc` (the `-i` flag). Without `exec`, the boot shell would fork a child `rc`, wait for it to finish, and then continue—but there is nothing left to do, so the extra process would just waste a process slot until the interactive shell exits. See Chapter 9 for the full list of builtins.

2.3.9 Other features

The `boot.rc` script is simple enough that it does not use many of `rc`'s advanced features: no pipes, no control flow, no functions. Those features—pipes, redirections, conditionals, loops, pattern matching, functions, command substitution, and glob patterns—will be presented gradually in the following chapters.

2.4 Code organization

Table 2.1 presents short descriptions of the source files of `rc`, together with the main entities (e.g., structures, functions, globals) the file defines, and the corresponding chapter in this document in which the code contained in the file is primarily discussed.

Every source file in `rc` includes the same set of headers. `rc.h` defines the main data structures (`Tree`, `Code`, `Word`, `Var`), `exec.h` defines the execution data structures (`Thread`, `List`, `Redir`[77a]), `fns.h` contains all function prototypes, `getflags.h` provides the command-line flag parsing API, and `io.h` defines the buffered IO layer.

⟨*includes* 18⟩≡ (211 210 209 208 207 206 205 204 201 200 199 198b 197b)

```
#include "rc.h"
#include "exec.h"
#include "fns.h"
#include "getflags.h"
#include "io.h"
```

2.5 Software architecture

Before describing the internal architecture of `rc` itself, I will first present the broader context in which `rc` operates: the command-line user interface and the other system components (kernel, device drivers, terminal) that `rc` interacts with. Then I will describe `rc`'s own components and trace the execution of a simple command through them.

2.5.1 The command-line user interface

The shell is only one component of the command-line user interface (CLI). Figure 2.1 shows the components supporting the CLI under Plan 9 (the components for the CLI under UNIX are very similar).

To run commands, the shell needs first a kernel to create processes. The kernel provides services to applications (including the shell) through its *system call interface* (also known as *syscalls*). For example, under Plan 9, `rfork()` creates a new process in which the shell can then `exec()` a program (see Kernel book [Pad14]).

A shell needs also device drivers in the kernel handling the terminal with its keyboard and monitor. To access those devices, the kernel provides a *filesystem interface* (called a *namespace* under Plan 9). An application can `open()`, `read()`, `write()`, or `close()` files in this filesystem. Under UNIX and Plan 9, devices are represented

```
filesystem           +---------------+ +-------------------+
 interface           |               | |                   |  A
|-- bin              |     Shell     | |command (e.g., ls) |  P
|   |-- cp  +--------+-> stdin       | |      stdin        |  P
|   |-- ls  |      +-+---stdout      | |      stdout       |  S
|   |-- rc  |      | +---------------+ +-------------------+
|   `-- ... |      |------------syscall interface------------
|-- boot    |      | +-------------------------------------+
|-- dev     |      | |               Kernel                |
|   |--  cons <----+ |                                     |  K
|   |         < - - +|  device drivers-+                   |  E
|   |--  draw        |  |              |                   |  R
|   |--  mouse      +|- + >console     |                   |  N
|   `--  ...         |  |              |                   |  E
|-- env           + -|- + -display     |                   |  L
|-- sys              |  |  keyboard- - |                   |
`-- ...           |  |  |  mouse - - -|+ - - - - - - +     |
                   |  |              |                   |
                  |  |  +-------------++            |     |
                   +-------------------------------------+
                  v                   |              |        D
            + - - - - - - -                                   E
                            |         v              v        V
            |   Monitor      + - - - - -      + - - - - -    I
                            |   keyboard |        mouse   |   C
            + - - - - - - -  + - - - - -      + - - - - -    E
                        Terminal                              S
                       (console)
```

Figure 2.1: Components of the command-line user interface under Plan 9.

Function	Ch.	File	Entities	LOC
main data structures	3	`rc.h`	`Tree`[194c] `Code`[25a] `Word`[194c] `Var`[50d]	
execution data structures	3	`exec.h`	`Thread`[28b] `List`[31b]	
AST helpers	3	`tree.c`	`newtree()`[25e] `tree1()`[26b] `tree2()`[26c]	
globals	3	`globals.c`	`codebuf`[27b] `runq`[28c]	
list of strings (words)	3	`words.c`	`newword()`[29d] `count()`[30a] `freewords()`[30d]	
shell variables	3	`var.c`	`setvar()`[32a] `vlook()`[32b] `gvar`[32d] `gvlook()`[32e]	
function prototypes	3	`fns.h`		
entry point	4	`main.c`	`main()`[35e]	
execution utilities	4	`executils.c`	`start()`[36e] `pushlist()`[38a] `pushword()`[38b]	
character input and prompt	5	`input.c`	`getnext()`[41] `pprompt()`[43g] `nextc()`[44e]	
lexer	6	`lex.c`	`yylex()`[46a]	
parser	7	`syn.y`	`yyparse()`[55b]	
bytecode generation	8	`code.c`	`emitf()`[62d] `compile()`[63a] `outcode()`[64a]	
process status	8	`status.c`	`setstatus()`[65d] `getstatus()`[65e]	
simple command bytecode	8	`simple.c`	`Xsimple()`[68c] `execexec()`[70b] `doredir()`[77e]	
`$path` management	8	`path.c`	`searchpath()`[71b]	
process-related bytecodes	8	`processes.c`	`execforkexec()`[70a] `Xpipe()`[137c] `Xasync()`[85b]	
other bytecodes	8	`exec.c`	`Xtrue()`[74e] `Xjump()`[88c] `Xfn()`[93a]	
shell builtins	9	`builtins.c`	`execcd()`[101c] `execexit()`[103c] `execdot()`[103d]	
shell environment	10	`env.c`	`addenv()`[109a] `Vinit()`[109b]	
signal management	11	`trap.c`	`dotrap()`[113b] `Trapinit()`[112d]	
initialization (bootstrap)	12	`main.c`	`Rcmain`[211a] `dotcmds`[105b]	
wildcard matching	13	`glob.c`	`glob()`[119c] `match()`[122b]	
here documents	14	`here.c`	`readhere()`[129h] `heredoc()`[131e]	
AST dumper	A	`pcmd.c`	`pcmd()`[148e]	
bytecode dumper	A	`pfnc.c`	`pfnc()`[151]	
error management	B	`error.c`	`panic()`[154b]	
memory management	C	`utils.c`	`emalloc()`[155b] `efree()`[155d]	
command-line flags	C	`getflags.c`	`getflags()`[156h] `usage()`[158]	
buffered IO	C	`io.c`	`openfd()`[160f] `rchr()`[161c]	
pretty printer	C	`fmt.c`	`pfmt()`[163b]	
Total				6800

Table 2.1: Chapters and associated `rc` source files.

as files. For example, `/dev/cons` (for console) is a file representing the terminal. To read characters from the keyboard, an application can simply read characters from `/dev/cons`. To write characters on the monitor, an application can simply write characters in `/dev/cons`.

A key UNIX innovation is that the shell is a regular user program, not part of the kernel. When the kernel boots, it starts an initial process whose stdin and stdout are connected to `/dev/cons`. Every process forked from it inherits this connection, including `rc`. Because the shell is just a program, you can replace it—or run a different shell inside the current one—without changing the kernel.

```
Input characters --> tokens --> AST --> bytecodes --> threads --> processes
```

Figure 2.2: Data flow diagram of `rc`.

2.5.2 `rc`'s components

Figure 2.2 describes the main data flow of `rc`, whereas Figure 2.3 describes the main control flow of `rc`. At its core, `rc` is both a compiler and a bytecode interpreter. As shown by Figure 2.2, given a series of characters (coming either from what you typed in the terminal or from the content of a script), `rc` first groups those characters in tokens (with `yylex()`[46a]). Then, `rc` parses those tokens (with `yyparse()`[55b]) and builds an abstract syntax tree (AST) of the program (see `Tree`[194c]). Finally, `rc` transforms this tree in a series of bytecodes (see `Code`[25a]). This is similar to what a compiler such as `5c` does (see COMPILER book [Pad16b]), except a bytecode here is not an instruction from a concrete machine but from a *virtual machine*.

After this compilation, `rc` goes through the series of bytecodes and interprets them. `rc` keeps track of what is currently executing in a `Thread`[28b] data structure and in a queue `runq`[28c]. This data structure is called a "thread" because `rc` must sometimes manage multiple threads of execution. Indeed, some of the bytecodes can create new processes running concurrently (e.g., `Xpipe()`[137c], the bytecode handling pipes).

In fact, `rc` does not start by compiling, but by interpreting. Indeed, `rc` starts first by interpreting some special bytecodes, called the *bootstrap*, that contains a bytecode (`Xrdcmds()`[39a]) that then triggers the compiler. I will now explain briefly the control flow of `rc`, starting from the top of Figure 2.3.

After some basic initializations, `main()`[35e] sets its local `bootstrap`[34a] to contain the initial set of bytecodes to execute. It then calls `start()`[36e] with `bootstrap` as an argument, to start interpreting this series of bytecodes. Internally, `start()` just modifies the global `runq` to point to a newly created `Thread`, and sets the field `Thread.code`[28b] to the content of `bootstrap`. After `start()` returns, `main()` goes in a loop that interprets the bytecodes in `runq->code`.

The most important bytecode in `bootstrap` is `Xrdcmds()`, which reads a command (hence its name) and starts a new `Thread`. `Xrdcmds()` first calls the parser `yyparse()`, which calls the lexer `yylex()`, which calls the input routine `nextc()`[44e] to get the next character. By default, `nextc()` reads characters from the standard input, and so it waits for the characters you type in a terminal. Once you finish entering a command with a newline, `yyparse()` will call `compile()`[63a] with the tree it built during parsing as an argument. Internally, `compile()` modifies the global `codebuf`[27b] to store the bytecodes deriving from the tree. Then, after `yyparse()` returned, `Xrdcmds()` calls `start()` to start a new thread with `codebuf` as a parameter. `start()` then modifies again the global `runq`, and when `Xrdcmds()` returns, the main bytecode interpreter loop will process a new series of bytecodes stored in `runq.code`.

Note that before modifying `runq`, `start()` first links the newly created thread with the old thread (through the `Thread.ret`[28d] field). Moreover, `compile()` adds the bytecode `Xreturn()`[82d] at the end of the series of bytecodes deriving from your command. Thus, after `main()` finished interpreting the bytecodes of your command, it will process `Xreturn()`, which will modify `runq` to point to the old thread, the one containing the bootstrap bytecodes. Then, after `Xreturn()` returns, the main bytecode interpreter loop will process again the bytecodes from the bootstrap, which will read another command through `Xrdcmds()`.

In `rc`, the bytecodes are represented by regular C functions starting by convention with an `X` (e.g., `Xrdcmds()`, `Xpipe()`, `Xif()`[86b]). Thus, the bytecode interpreter is mainly a function dispatcher. Internally those functions perform system calls to the kernel to create a pipe (`pipe()`), to fork a new process (`fork()`), to wait for a child process (`wait()`), or to change directory (`chdir()`). An important bytecode is `Xsimple()`[68c], which `rc` uses to run a "simple" command. `Xsimple()` represents the essence of a shell: with the series of system calls `fork()`, `exec()`, and `wait()`, `rc` can run a command in a new process and wait for its termination (or interruption). `Xsimple()` is also responsible for managing the multiple shell builtins. Indeed, if the name of the "simple" command is a builtin (e.g., `cd`), then `Xsimple()` dispatches the appropriate function (e.g., `execcd`) instead of forking a new process.

```
                          +------------+
                          |    main    |
                          +-+----+-----+
                            |    |
                 .------------+----v----------------.
                (      bytecod| interpreter loop      ) (interpreting)
                 `------------+----+----------------'
                            |    |
            +------------------++---+-------------+--------------------+
            |        +---------+|                 |                    |
     +------v-----+  |   +------v-----+   +-------v----+        +------v-----+
     |  Xrdcmds   |  |   |   Xpipe    |   |  Xsimple   |   ...  |    Xif     | bytecodes
     +-----+------+  |   +--+---------+   +--+-----+---+|       +------------+
           ||        |      |   |            |     |    |
           |+--------+---+  |   | +----------v-+   |   +v-----------+
     +-----+         |   |  |   | |execforkexec|  .|.  |   execcd   | shell builtins
     |            +--v---v--v--+| +------------+   |   +----------+-+
     |            |   start    ||        |         |              |
     v            +------------+|        |         |              |
+---------+                     |        |         |              |
| yyparse | (parsing)   +-------+-+      |         +----+         |
+----+-+--+             |         |      |              |         |
     | +-------+        |         |  +---+----+         |         |
+----v--+  +---v-----+  |         |  |        |         |         |
| yylex |  | compile |  |         |  |        |         |         |
+----+--+  +---------+  |         |  |        |         |         |
     |     (compiling)  |         |  |        |         |         |
+----v--+               |         |  |        |         |         |
| nextc |               v         v  v        v         v         v
+-------+        +--------+  +--------+  +--------+  +--------+  +--------+
                 |  pipe  |  |  fork  |  |  exec  |  |  wait  |  | chdir  |  syscalls
                 +--------+  +--------+  +--------+  +--------+  +--------+
```

Figure 2.3: Control flow diagram of `rc`.

2.5.3 Trace of a simple command: `ls /`

You can see the bytecodes and the threads created internally by `rc` by running `rc` with the `-r` flag. Here is an example of a trace of the simple command `ls /`:

```
% rc -r
pid 39 cycle 0002D930 1 Xmark ()
...
pid 38 cycle 0001984C 9 Xrdcmds
% ls /
pid 38 cycle 0002CBD0 1 Xmark
pid 38 cycle 0002CBD0 2 Xword ()
pid 38 cycle 0002CBD0 4 Xword (/)
pid 38 cycle 0002CBD0 6 Xsimple (ls /)
bin
boot
...
srv
pid 38 cycle 0002CBD0 7 Xreturn
pid 38 cycle 0001984C 9 Xrdcmds
```

It is not important to fully understand the format of this trace (see Section A.2 for the full explanation and for the code handling `rc -r`), but you can recognize a few of the bytecodes I mentioned before: `Xrdcmds()`[39a], `Xsimple()`[68c], and `Xreturn()`[82d]. I will explain `Xmark()`[64c] and `Xword()`[64b] later in this document.

2.6 Book structure

You now have enough background to understand the source code of `rc`. The rest of the book is organized as follows. I will start by describing the core data structures of `rc` in Chapter 3. Then, I will use a top-down approach, starting with Chapter 4 with the description of `main()`[35e], the initialization of `rc`, the bytecode interpreter loop, and the function `Xrdcmds()`[39a]. The following chapters will describe the main components of the compiler pipeline: Chapter 5 will present the input routines, Chapter 6 the lexer, Chapter 7 the parser, and finally Chapter 8 the bytecode generator and bytecode interpreter for the main features of `rc`. In Chapter 9, I will present the code of the different shell builtins (e.g., `execcd()`[101c] for `cd`). Chapter 10 contains the code to manage the environment of the processes launched from the shell, and Chapter 11 the code to manage the signals sent to the processes (also known as *notes* under Plan 9). Chapter 12 presents the actual code initializing `rc`. Indeed, Chapter 4 presents only a simplified initialization to not introduce too much complexity early-on. Then, I will present advanced features of `rc` that I did not present before to simplify the explanations, for instance, wildcard matching (e.g., `ls *.c`) in Chapter 13, or command substitutions (e.g., `` `{ls} ``) in Chapter 14. Those advanced features tend to crosscut many components of `rc` with extensions to the lexer, the parser, the bytecode generator, and the bytecode interpreter. Finally, Chapter 15 concludes and gives pointers to other books in the Principia Softwarica series.

Some appendices present the code of non-functional properties: code to help debug `rc` itself in Appendix A and code to manage errors in Appendix B. Appendix C contains the code of utility functions used by `rc` but that are not specific to `rc` (e.g., a library to manage string buffers). Finally, Appendix D presents examples of `rc` scripts.

Chapter 3

Core Data Structures

Show me your code and conceal your data structures, and I shall continue to be mystified. Show me your data structures, and I won't usually need your code; it'll be obvious.

Fred Brooks

In this chapter, I will present the core data structures of `rc`. Most of those data structures are listed in Figure 2.2, which presented the data flow of `rc`. The first three sections will present the data structures of the compiler part of `rc`: the token, the abstract syntax tree, and the bytecode. The bytecode is a data structure used both by the compiler and interpreter. The following sections will present the remaining data structures of the interpreter part of `rc`: the thread, and the variable, which is itself defined by a list of words.

3.1 `Token`

As mentioned in the previous chapter, the first step of the compilation pipeline is to transform a stream of characters into a stream of tokens (also called lexemes). A token groups one or more characters that logically belong together (e.g., `while` is a single keyword token, not five character tokens). The lexer `yylex()`[46a] (described in Chapter 6) is responsible for this transformation.

The token kinds in `rc` are declared below using the `yacc` syntax `%token` (see COMPILER book [Pad16b]). I have grouped them in categories: the keyword tokens (`FOR`, `WHILE`, `IF`, etc.), the operator tokens (`REDIR`, `PIPE`, `ANDAND`, etc.), and the catch-all `WORD` token for anything else (e.g., command names, arguments, numbers, or file paths). Note that some single-character operators like `'$'`, `';'`, or `'='` do not have a dedicated token kind; the lexer simply returns their ASCII value directly (see `yylex()` and `pcmd()`[148e]).

⟨*token declarations* 24⟩≡ (55a) 25b ▷

```
%token FOR IN WHILE IF NOT SWITCH FN
%token TWIDDLE BANG  /** ~ ! */
%token REDIR PIPE /** {>, <, <<, >>} | */
%token ANDAND OROR /** && || */
%token COUNT SUB /** $# ( */
%token WORD /** anything else (e.g. foo, --help, 42, /a/b/c, etc) */
```

3.2 Abstract syntax `Tree`

The second step of the compilation pipeline transforms the stream of tokens into an abstract syntax tree (AST). The parser `yyparse()`[55b] (described in Chapter 7) is responsible for this transformation. `rc` has a *command language*: it has variables, control flow statements (`if`, `while`, `for`, `switch`), functions (`fn`), and operators (pipes, redirections, etc.). All those constructs are represented in the AST using the `Tree` structure below.

The `Tree` structure is a generic tree node with a `type` field indicating what kind of node it is (e.g., a `PIPE`, an `IF`, a `WORD`, or even a single character like `';'` for sequencing), a `str` field to store the string content of the token (e.g., the command name or the variable name), and up to three children. Three children are enough to represent all the constructs of `rc`: for example, an `if` uses `child[0]` for the condition, `child[1]` for the then-body, and leaves `child[2]` unused. A `for` uses all three: `child[0]` for the variable, `child[1]` for the list, and `child[2]` for the body.

⟨*struct* `Tree` 25a⟩≡ (194c)

```
struct Tree {

    // either<enum<Token_kind>, char>
    int type;
    // string of the token or AST dump of the whole subtree for certain nodes
    char *str;

    // array<option<ref_own<Tree>>
    tree *child[3];

    ⟨Tree redirection and pipe specific fields 50a⟩
    ⟨Tree word specific fields 52a⟩

    // Extra
    ⟨Tree extra fields 25d⟩
};
```

The parser also introduces a few extra node types that do not correspond to tokens produced by the lexer, but that are useful to structure the AST. For example, `SIMPLE` represents a simple command with its arguments, and `ARGLIST` groups those arguments into a list.

⟨*token declarations* 25b⟩+≡ (55a) ◁24 61a▷

```
/* not used in syntax */
%token SIMPLE
%token ARGLIST WORDS
%token BRACE PAREN
```

All tree nodes allocated during parsing are linked together in a global list `treenodes` through the `Tree.next` field below. This makes it easy to free all nodes at once (via `freenodes()`[26a]) after the tree has been compiled to bytecodes in `Xrdcmds()`[39a], instead of having to walk the tree recursively.

⟨*global* `treenodes` 25c⟩≡ (204b)

```
// list<ref_own<Tree>> (next = Tree.next)
tree *treenodes;
```

⟨`Tree` *extra fields* 25d⟩≡ (25a)

```
tree *next;
```

The functions below are used by the parser to build AST nodes. `newtree()`[25e] allocates a fresh node and links it into `treenodes`. The convenience wrappers `tree1()`[26b], `tree2()`[26c], and `tree3()`[26d] create a node with a given type and one, two, or three children respectively.

⟨*function* `newtree` 25e⟩≡ (204b)

```
/*
 * create and clear a new tree node, and add it
 * to the node list.
 */
tree*
newtree(void)
{
    tree *t = new(tree);
    t->str = nil;
    t->child[0] = t->child[1] = t->child[2] = nil;
```

```
    // add_list(t, treenodes)
    t->next = treenodes;
    treenodes = t;

    return t;
}
```

Uses `nil` and `treenodes` 25c.

⟨*function* `freenodes` 26a⟩≡ (204b)

```
void
freenodes(void)
{
    tree *t, *u;
    for(t = treenodes;t;t = u){
        u = t->next;
        if(t->str)
            efree(t->str);
        efree((char *)t);
    }
    treenodes = nil;
}
```

Uses `efree()`, `nil`, and `treenodes` 25c.

⟨*function* `tree1` 26b⟩≡ (204b)

```
tree*
tree1(int type, tree *c0)
{
    return tree3(type, c0, (tree *)nil, (tree *)nil);
}
```

Uses `nil` and `tree3()` 26d.

⟨*function* `tree2` 26c⟩≡ (204b)

```
//@Scheck: used by syn.y
tree* tree2(int type, tree *c0, tree *c1)
{
    return tree3(type, c0, c1, (tree *)nil);
}
```

Uses `nil` and `tree3()` 26d.

⟨*function* `tree3` 26d⟩≡ (204b)

```
tree*
tree3(int type, tree *c0, tree *c1, tree *c2)
{
    tree *t;

    ⟨tree3 if some empty sequence 27a⟩
    // else
    t = newtree();
    t->type = type;
    t->child[0] = c0;
    t->child[1] = c1;
    t->child[2] = c2;
    return t;
}
```

Uses `newtree()` 25e.

The special case below is an optimization for the sequencing operator `';'`: if one side of a sequence is empty (which happens frequently since each newline produces an empty command), `tree3()` simply returns the other side instead of creating a useless node.

⟨`tree3` *if some empty sequence* 27a⟩≡ (26d)

```
if(type==';'){
    if(c0==nil)
        return c1;
    if(c1==nil)
        return c0;
}
```

Uses `nil`.

3.3 ByteCode and codebuf

The third step of the compilation pipeline transforms the AST into a series of bytecodes. The function `compile()`[63a] (described in Chapter 8) is responsible for this transformation, and stores the resulting bytecodes in the global `codebuf`[27b] below. Like Java or Python, `rc` does not interpret the AST directly but compiles it first into a lower-level representation that is then interpreted by a small stack-based virtual machine (described in Chapter 4).

⟨*global* `codebuf` 27b⟩≡ (197b)

```
// growing_array<ref_own<Code>>
code *codebuf;    /* compiler output */
```

A bytecode is represented by the `Code` union below. Each element in the `codebuf` array is either: a function pointer `f` (the bytecode itself, e.g., `Xsimple`[68c], `Xpipe`[137c]), an integer `i` (used for jump offsets or file descriptor numbers), or a string `s` (used for inline string arguments like in `Xword`[64b]). The first element of any code vector is always a reference count integer, managed by `codecopy()`[27d] and `codefree()`[27e] below. This reference counting is needed because bytecode vectors can be shared, for instance when a function is defined with `fn`.

⟨*struct* `Code` 27c⟩≡ (194c)

```
/*
 * The first word of any code vector is a reference count.
 * Always create a new reference to a code vector by calling codecopy(.).
 * Always call codefree(.) when deleting a reference.
 */
union Code {
    void (*f)(void); // Xxxx() bytecode
    int i;
    char *s;
};
```

⟨*function* `codecopy` 27d⟩≡ (207a)

```
code*
codecopy(code *cp)
{
    cp[0].i++;
    return cp;
}
```

⟨*function* `codefree` 27e⟩≡ (207a)

```
void
codefree(code *cp)
{
    code *p;
    // check ref count
```

```
    if(--cp[0].i != 0)
        return;

    for(p = cp+1; p->f; p++){
        ⟨codefree() in loop over code cp, switch bytecode cases 28a⟩
    }
    efree((char *)cp);
}
```

Uses `efree()`.

⟨`codefree()` *in loop over code* `cp`*, switch bytecode cases* 28a⟩≡ (27e) 68b ▷

```
if(p->f==Xfalse || p->f==Xtrue
|| p->f==Xread || p->f==Xwrite || p->f==Xrdwr
|| p->f==Xappend || p->f==Xclose
|| p->f==Xasync || p->f==Xbackq || p->f==Xcase
|| p->f==Xfor || p->f==Xjump
|| p->f==Xsubshell)
    p++;
```

Uses `Xappend()` 80c, `Xasync()`, `Xbackq()` 204a, `Xcase()` 91c, `Xclose()` 135g, `Xfalse()` 75a, `Xfor()` 89a, `Xjump()` 88c, `Xrdwr()` 133c, `Xread()` 80a, `Xsubshell()`, `Xtrue()` 74e, and `Xwrite()` 79a.

3.4 Thread and runqeue

Once the bytecodes are compiled, `rc` needs a data structure to keep track of the current execution state. This is the role of the `Thread` structure. A `Thread` is similar to a stack frame in a regular program: it contains a pointer to a code vector (`code`) and a program counter (`pc`) indexing the next bytecode to execute. The name "thread" might be confusing since these are not OS-level threads or processes. They are *units of execution* inside the `rc` interpreter. When a bytecode like `Xpipe()`[137c] or `Xasync()`[85b] needs to run commands concurrently, it creates actual OS processes (with `fork()`), but within the interpreter itself, threads are managed as a stack.

The global `runq`[28c] (for "run queue") points to the top of this stack of threads. Threads are linked through the `ret` field: when the current thread finishes (via `Xreturn()`[82d]), `rc` pops it and resumes the thread pointed to by `ret`. At the bottom of the stack sits the bootstrap thread, which loops forever reading new commands via `Xrdcmds()`[39a].

⟨*struct* `Thread` 28b⟩≡ (197a)

```
struct Thread {
    union Code *code;  /* code for this thread */
    int pc;   /* code[pc] is the next instruction */

    ⟨Thread other fields 29a⟩

    // Extra
    ⟨Thread extra fields 28d⟩
};
```

Uses `Code` 25a.

⟨*global* `runq` 28c⟩≡ (197b)

```
// stack<ref_own<Thread>> (next = Thread.ret)
thread *runq;
```

⟨`Thread` *extra fields* 28d⟩≡ (28b)

```
thread *ret;  /* who continues when this finishes */
```

The most important field of a thread is argv, the argument stack. It is a list of lists of words: bytecodes like Xmark()[64c] push a new empty list on this stack, and Xword()[64b] pushes a word onto the current list. When a bytecode like Xsimple()[68c] needs to run a command, it pops the top list from argv and uses it as the command and its arguments.

⟨Thread *other fields* 29a⟩≡ (28b) 29b ▷

```
// list<list<ref_own<word>>> (next = List.next)
struct List *argv;  /* argument stack */
```

Uses List 31b.

Each thread also has a list of local variables. These are created by the local builtin or by constructs like for that bind a variable for the duration of a block. When looking up a variable with vlook()[32b], rc first searches the current thread's locals before falling back to the global variable table.

⟨Thread *other fields* 29b⟩+≡ (28b) ◁ 29a 39b ▷

```
// list<ref_own<Var>> (next = Var.next)
struct Var *local;  /* list of local variables */
```

Uses Var 50d.

3.5 Words and lists of words

I mentioned the argv field in the Thread structure above, which is a list of lists of words. Let me now present the Word and List structures that make up this two-level list.

3.5.1 Words

A Word is simply a string with a next pointer, forming a linked list. This is the fundamental data type in rc: a command like ls -l /tmp is represented as a list of three words. Variables in rc can also hold multiple values (e.g., x=(a b c)), which are represented as a list of words.

⟨*struct* Word 29c⟩≡ (194c)

```
/*
 * word lists are in correct order,
 * i.e. word0->word1->word2->word3->nil
 */
struct Word {
    char *word;

    // Extra
    word *next;
};
```

The functions below provide the usual linked-list operations for word lists: newword()[29d] allocates a new word and prepends it to a list, count()[30a] returns the length, copywords()[30c] duplicates a list, and freewords()[30d] deallocates it.

⟨*function* newword 29d⟩≡ (210a)

```
word*
newword(char *wd, word *next)
{
    word *p = new(struct Word);
    p->word = strdup(wd);
    p->next = next;
    return p;
}
```

⟨*function* count 30a⟩≡ (210a)

```
int
count(word *w)
{
    int n;
    for(n = 0;w;n++)
        w = w->next;
    return n;
}
```

`copynwords()`[30b] and `copywords()` use the classic C pointer-to-pointer tail linking idiom: `end` starts as `&v` (address of the head pointer), and after each node is created, it advances to `&(*end)->next` (address of the new node's `next` field). This avoids special-casing the first element and builds the list in forward order without reversing at the end.

⟨*function* copynwords 30b⟩≡ (210a)

```
word*
copynwords(word *a, word *tail, int n)
{
    word *v = nil;
    word **end = &v;

    while(n-- > 0){
        *end = newword(a->word, 0);
        end = &(*end)->next;
        a = a->next;
    }
    *end = tail;
    return v;
}
```

Uses `newword()` 29d and `nil`.

⟨*function* copywords 30c⟩≡ (210a)

```
/*
 * copy arglist a, adding the copy to the front of tail
 */
word*
copywords(word *a, word *tail)
{
    word *v = nil;
    word **end;

    for(end=&v;a;a = a->next,end=&(*end)->next)
        *end = newword(a->word, nil);
    *end = tail;
    return v;
}
```

Uses `newword()` 29d and `nil`.

⟨*function* freewords 30d⟩≡ (210a)

```
void
freewords(word *w)
{
    word *nw;
    while(w){
        efree(w->word);
        nw = w->next;
        efree((char *)w);
        w = nw;
    }
```

```
}
```

Uses `efree()`.

⟨*function* `freelist` 31a⟩≡ (210a)

```
void
freelist(word *w)
{
    word *nw;
    while(w){
        nw = w->next;
        efree(w->word);
        efree((char *)w);
        w = nw;
    }
}
```

Uses `efree()`.

3.5.2 List of sequence of words

The `List` structure adds one more level of indirection on top of word lists. It is used for `Thread.argv`, the argument stack: each element in this stack is a `List` node containing a word list. For example, when the interpreter processes `ls -l /tmp`, the bytecodes `Xmark()`[64c] and `Xword()`[64b] will first push the words `ls`, `-l`, `/tmp` onto a word list, and this word list sits inside a `List` node on the `argv` stack.

⟨*struct* `List` 31b⟩≡ (197a)

```
struct List {
    // list<ref_own<Word>> (next = Word.next)
    word *words;

    // Extra
    list *next;
};
```

3.6 Variables

I already mentioned local variables in `Thread.local` above. Let me now present the `Var` structure itself and the global variable table. Variables are needed whenever you write something like `x=1` or `path=(/bin /usr/bin)` in the shell. They are also used internally by `rc` for special variables like `status` (the exit status of the last command) or `*` (the script arguments).

A `Var` has a `name` (the variable name as a string), a `val` (the variable value as a list of words, since in `rc` all variables can hold multiple values), and a `next` pointer for chaining in hash buckets or local lists.

⟨*struct* `Var` 31c⟩≡ (194c)

```
struct Var {
    // key
    char *name;  /* ascii name */
    // value
    word *val;  /* value */

    ⟨Var other fields 92a⟩
    // Extra
    ⟨Var extra fields 31d⟩
};
```

⟨`Var` *extra fields* 31d⟩≡ (31c)

```
var *next;  /* next on hash or local list */
```

The main interface to the variable system consists of `setvar()`[32a] to assign a value, and `vlook()`[32b] to look up a variable by name. `vlook()` first searches the local variables of the current thread (stored in `runq->local`), then falls back to the global variable table via `gvlook()`[32e].

⟨*function* `setvar` 32a⟩≡ (200a)

```
void
setvar(char *name, word *val)
{
    struct Var *v = vlook(name);
    freewords(v->val);
    v->val = val;
    v->changed = true;
}
```

Uses `Var` 50d, `freewords()` 30d, `true`, and `vlook()` 32b.

⟨*function* `vlook` 32b⟩≡ (200a)

```
var*
vlook(char *name)
{
    var *v;
    if(runq)
        for(v = runq->local;v;v = v->next)
            if(strcmp(v->name, name)==0)
                return v;

    return gvlook(name);
}
```

Uses `gvlook()` 32e and `runq` 28c.

Global variables are stored in a simple hash table of 521 buckets, with chaining through `Var.next` for collisions. `gvlook()` looks up a global variable and automatically creates it (with a `nil` value) if it does not exist yet.

⟨*constant* `NVAR` 32c⟩≡ (194c)

```
#define NVAR 521
```

⟨*global* `gvar` 32d⟩≡ (200a)

```
// map<string, ref_own<Var>> (next = Var.next in bucket list)
var *gvar[NVAR];  /* hash for globals */
% vs locals
```

Uses `NVAR` 29c.

⟨*function* `gvlook` 32e⟩≡ (200a)

```
var*
gvlook(char *name)
{
    int h = hash(name, NVAR);
    var *v;

    for(v = gvar[h];v;v = v->next)
        if(strcmp(v->name, name)==0)
            return v;
    gvar[h] = newvar(strdup(name), gvar[h]);
    return gvar[h];
}
```

Uses `NVAR` 29c, `gvar` 32d, `hash()` 33a, and `newvar()` 33b.

⟨*function* hash 33a⟩≡ (200a)

```
unsigned
hash(char *as, int n)
{
    int i = 1;
    unsigned h = 0;
    uchar *s;

    s = (uchar *)as;
    while (*s)
        h += *s++ * i++;
    return h % n;
}
```

⟨*function* newvar 33b⟩≡ (200a)

```
var*
newvar(char *name, var *next)
{
    var *v = new(var);
    v->name = name;
    v->val = nil;

    v->fn = nil;
    v->changed = false;
    v->fnchanged = false;

    v->next = next;
    return v;
}
```

Uses `false` and `nil`.

Chapter 4

main()

I now switch from the bottom-up approach of Chapter 3 to a top-down approach: I will describe in the following chapters the main functions of `rc`, starting in this chapter with `main()`[35e], the entry point of `rc`.

4.1 Overview

The `main()`[35e] function of `rc` performs four tasks: it processes the command-line arguments, it initializes the shell (keywords, traps, environment), it sets up the bootstrap bytecodes and the initial thread, and then it enters the bytecode interpreter loop. I will present here a simplified version of the bootstrap; the actual bootstrap code (which sources `rcmain`) is described in Chapter 12.

⟨`main()` *locals* 34a⟩≡ (34b) ?? ▷

```
code bootstrap[17];
```

Uses `true`.

⟨*function main (rc/exec.c)* 34b⟩≡ (211a)

```
void main(int argc, char *argv[])
{
    ⟨main() locals 34a⟩

    ⟨main() argc argv processing, modify flags 35c⟩

    ⟨main() initialisation 36a⟩
    ⟨main() initialize boostrap 115⟩
    ⟨main() initialize runq with bootstrap code 36d⟩
    ⟨main() initialize runq->argv 37b⟩

    ⟨main() interpreter loop 38c⟩
}
```

Uses `Xsimple()` 68c.

4.2 Command-line arguments processing

`rc` accepts many flags (e.g., `-i` for interactive mode, `-l` for login mode, `-e` for exit-on-error). Flags are stored in a global array indexed by the flag character, where a non-nil entry means the flag is set. Some flags like `-c` and `-m` take a string argument, which is why `flag` stores string arrays rather than simple booleans.

⟨*global* `flag` 34c⟩≡ (198a)

```
// map<char, option<array<string>>>
char **flag[NFLAG];
```

Uses `NFLAG` 35a.

⟨*constant* NFLAG 35a⟩≡ (194a)

```
#define NFLAG 128
```

⟨*global* flagset 35b⟩≡ (198a)

```
char *flagset[] = {"<flag>"};
```

Uses flagset 35b.

⟨main() *argc argv processing, modify flags* 35c⟩≡ (34b) 35d ▷

```
argc = getflags(argc, argv, "SsrdiIlxepvVc:1m:1[command]", 1);
if(argc==-1)
    usage("[file [arg ...]]");
```

getflags()[156h] is defined in Appendix C. The format string encodes which flags exist and which ones take an argument (c:1 means -c takes one argument). It populates the global flag array declared above.

4.2.1 Login mode: rc -l

When the kernel starts rc as a login shell, it sets argv[0] to "-rc" (starting with a dash) rather than passing an explicit -l flag. This is a UNIX convention: the code below detects this and sets flag['l'] accordingly, so that rc will later source the user's profile ($home/lib/profile).

⟨main() *argc argv processing, modify flags* 35d⟩+≡ (34b) ◁35c 35e ▷

```
if(argv[0][0]=='-')
    flag['l'] = flagset;
```

4.2.2 Interactive mode: rc -i

Interactive mode determines whether rc will display a prompt and handle signals differently (e.g., not exiting on ^C). The logic below is subtle: rc is interactive by default when no script file is given (argc==1) and standard input is a terminal. The -I flag forces non-interactive mode (useful when piping commands into rc from another program).

⟨main() *argc argv processing, modify flags* 35e⟩+≡ (34b) ◁35d

```
if(flag['I'])
    flag['i'] = nil;
else
    if(flag['i']==nil && argc==1 && Isatty(STDIN))
       flag['i'] = flagset;
```

Under Plan 9, there is no isatty() system call. Instead, Isatty()[211a] checks whether the file descriptor corresponds to /dev/cons (the console device) by inspecting its path with fd2path().

⟨*function* Isatty 35f⟩≡ (211b)

```
bool
Isatty(fdt fd)
{
    char buf[64];

    if(fd2path(fd, buf, sizeof buf) != 0)
        return false;

    /* might be #c/cons during boot - fixed 22 april 2005, remove this later */
    if(strcmp(buf, "#c/cons") == 0)
        return true;

    /* might be /mnt/term/dev/cons */
    return strlen(buf) >= 9 && strcmp(buf+strlen(buf)-9, "/dev/cons") == 0;
}
```

5.2.3

4.3 Initialization

The initialization first sets up `err`, a buffered IO handle for standard error (see Appendix C for `openfd()`[160f]). Then it initializes three subsystems: `kinit()`[53e] registers the shell keywords (`if`, `for`, etc.) so the lexer can recognize them, `Trapinit()`[112d] sets up the signal handler (see Chapter 11), and `Vinit()`[109b] reads the process environment (from `/env`) and populates the global variable table (see Chapter 10).

⟨`main()` *initialisation* 36a⟩≡ (34b) 36b ▷
```
err = openfd(STDERR);
```

⟨`main()` *initialisation* 36b⟩+≡ (34b) ◁36a 116c ▷
```
kinit();    // initialize keywords
Trapinit(); // notify() function setup
Vinit();    // read environment variables and add them in gvar
```

4.4 Bootstrapping bytecodes (simplified)

As mentioned earlier, `rc` does not start by compiling source code; it starts by *interpreting* a small series of hand-crafted bytecodes called the bootstrap. In its simplest form, the bootstrap just contains `Xrdcmds`[39a], the bytecode that reads a command, compiles it, and starts a new thread to execute it. The first element is the reference count (set to 1), and the last element is a zero sentinel marking the end of the code vector. The actual bootstrap is more complex (see Chapter 12): it sources the `rcmain` initialization script, which sets up the default `$path` and runs the user's profile.

⟨`main()` *initialize* `boostrap` *(simplified)* 36c⟩≡
```
memset(bootstrap, 0, sizeof bootstrap);

i = 0;
bootstrap[i++].i = 1; // reference count
bootstrap[i++].f = Xrdcmds;
bootstrap[i].i = 0;
```

4.5 Setting `runq`

`start()`[36e] creates a new thread and pushes it on top of `runq`[28c]. The program counter starts at 1 (not 0) because element 0 of any code vector is reserved for the reference count. Note how `start()` links the new thread to the old `runq` through `p->ret = runq`: this is how the interpreter knows where to resume after the current thread finishes.

⟨`main()` *initialize* `runq` *with bootstrap code* 36d⟩≡ (34b) 39c ▷
```
start(bootstrap, 1, (var *)nil);
```
Uses `Xword()` 64b.

⟨*function* `start` 36e⟩≡ (201a)
```
void
start(code *c, int pc, var *local)
{
    struct Thread *p = new(struct Thread);

    p->code = codecopy(c);
    p->pc = pc;
```

```
    p->argv = nil;
    p->local = local;

    p->cmdfile = nil;
    p->cmdfd = nil;
    p->lineno = 1;
    p->eof = false;
    p->iflag = false;

    ⟨start() set redir 76c⟩

    // add_stack(runq, p)
    p->ret = runq;
    runq = p;
}
```

Uses `Thread` 28b, `codecopy()` 27d, `false`, `nil`, and `runq` 28c.

4.6 Setting `runq->argv`

Now that `runq`[28c] is set, `main()`[35e] populates its argument stack with the command-line arguments. These arguments are pushed in reverse order because `pushword()`[38b] prepends to the list, so the first argument ends up at the top. The actual bootstrap code will later assign this list to `\$*` (the script arguments variable).

⟨*global* `argv0` 37a⟩≡ (201a)

```
/*
 * Start executing the given code at the given pc with the given redirection
 */
char *argv0="rc";
```

Uses `argv0` 37a.

⟨`main()` *initialize* `runq->argv` 37b⟩≡ (34b)

```
/* prime bootstrap argv */
pushlist();
argv0 = strdup(argv[0]);
for(i = argc-1; i!=0; --i)
    pushword(argv[i]);
```

Uses `Xdol()` 96b and `Xword()` 64b.

For example, after `rc myscript.rc foo bar` is initialized, the data structures look like this:

```
runq ----> Thread
             code = bootstrap [1:Xrdcmds, 0]
             pc   = 1
             argv ----> List
                          words -> "myscript.rc" -> "foo" -> "bar" -> nil
                          next  = nil
             ret  = nil   (bottom of stack)
```

The bootstrap code will then assign this word list to `\$*` and source `rcmain`, which sets up `\$path` and runs the script.

`pushlist()`[38a] pushes a new empty word list onto `runq->argv`, and `pushword()` adds a word to the current top list. These two functions are the main interface for building the argument stack and are used extensively by

bytecodes like Xmark()[64c] and Xword()[64b].

⟨*function* pushlist 38a⟩≡ (201a)

```
void
pushlist(void)
{
    list *p = new(list);

    // add_list(p, runq->argv)
    p->next = runq->argv;
    p->words = nil;
    runq->argv = p;
}
```

Uses nil and runq 28c.

⟨*function* pushword 38b⟩≡ (201a)

```
void
pushword(char *wd)
{
    if(runq->argv==nil)
        panic("pushword but no argv!", 0);
    runq->argv->words = newword(wd, runq->argv->words);
}
```

Uses newword() 29d, nil, panic() 154b, and runq 28c.

4.7 Bytecode interpreter loop

The heart of rc is this infinite loop. It increments the program counter, then calls the function pointer at the previous position. The increment-then-call-at-minus-one pattern is a common idiom: the pc is advanced *before* calling the bytecode function, so that if the bytecode needs to read inline arguments (e.g., a string after Xword[64b]), it can read them at runq->code[runq->pc] and advance pc itself. Between bytecodes, the loop also checks for pending signals (traps).

⟨main() *interpreter loop* 38c⟩≡ (34b)

```
for(;;){
    ⟨main() debug runq in interpreter loop 150b⟩

    runq->pc++;
    (*runq->code[runq->pc-1].f)();

    ⟨main() handing trap if necessary in interpreter loop 113a⟩
}
```

Uses STDIN, Xexit() 85c, nil, openfd() 160f, pushlist() 38a, runq 28c, and start() 36e.

4.8 Reading commands: Xrdcmds()

Xrdcmds()[39a] is the bytecode that implements the read-eval loop of rc. It calls yyparse()[55b] to read and compile one command line, then calls start()[36e] to push a new thread executing the compiled bytecodes. The key subtlety is the --p->pc before start(): this makes the bootstrap thread re-execute Xrdcmds() when the command thread finishes (via Xreturn()[82d]). This is how rc loops forever reading commands without an explicit loop in the bootstrap bytecodes themselves. Note that yyparse() internally calls compile()[63a], which

modifies the global `codebuf`[27b]. So when `Xrdcmds()` calls `start(codebuf, ...)`, it starts a thread on the freshly compiled bytecodes.

⟨*function* `Xrdcmds` 39a⟩≡ (201b)

```
void
Xrdcmds(void)
{
    struct Thread *p = runq;
    bool error;
    ⟨Xrdcmds() other locals 43d⟩

    ⟨Xrdcmds() flush errors and reset error count 153d⟩
    ⟨Xrdcmds() print status if -s 152a⟩
    ⟨Xrdcmds() set promptstr if interactive mode 43e⟩

    ⟨Xrdcmds() calls Noerror() before yyparse() 113e⟩
    // read one cmd line, compiles it, and modifies codebuf global
    error = yyparse();

    ⟨Xrdcmds() if yyparse() returned an error 39d⟩
    else{
        ⟨Xrdcmds() reset ntrap 114b⟩
        --p->pc; /* re-execute Xrdcmds after codebuf runs */
        // modifies runq, new thread (linked to bootstrap one)
        start(codebuf, 1, runq->local);
    }
    freenodes(); // allocated in yyparse()
}
```

Uses `Thread` 28b, `codebuf` 27b, `freenodes()` 26a, `runq` 28c, and `start()` 36e.

⟨`Thread` *other fields* 39b⟩+≡ (28b) ◁29b 42c▷

```
struct Io *cmdfd; /* file descriptor for Xrdcmd */
char *cmdfile;  /* file name in Xrdcmd */
bool iflag;  /* interactive? */
```

Uses `Io` 160e.

Each thread also carries input-related fields: `cmdfd` is the buffered IO handle from which the lexer reads characters, `cmdfile` is the filename (for error messages), and `iflag` determines whether this thread displays a prompt. These are per-thread fields (not globals) because the `.` (dot) builtin and `-c` flag create new threads that read from different sources (a script file, a string), and each thread needs its own input state.

⟨`main()` *initialize* `runq` *with bootstrap code* 39c⟩+≡ (34b) ◁36d

```
runq->cmdfd = openfd(STDIN); // reading from stdin
runq->cmdfile = "<stdin>";
runq->iflag = flag['i']? true : false;// interactive mode; will print a prompt
```

Uses `Xassign()` 95a and `Xmark()` 64c.

When `yyparse()` returns an error, the behavior depends on the mode. In non-interactive mode (or on a genuine EOF), `rc` closes the input and returns to the parent thread. In interactive mode, it simply prints a newline after an interrupt (`^C`) and goes back to read the next command.

⟨`Xrdcmds()` *if yyparse() returned an error* 39d⟩≡ (39a)

```
if(error){
    if(!p->iflag  ||  p->eof && !Eintr()){
        if(p->cmdfile)
            efree(p->cmdfile);
        closeio(p->cmdfd);

        Xreturn(); /* should this be omitted? */
    }else{
```

```
        if(Eintr()){
            pchr(err, '\n');
            p->eof = false;
        }
        --p->pc; /* go back for next command */
    }
}
```

Uses `Eintr()`, `Xreturn()` 82d, `closeio()` 162b, `efree()`, `err` 153b, `false`, and `pchr()` 161a.

Chapter 5

Input

Recall the main flow from Chapter 4: `main()`[35e] → `Xrdcmds()`[39a] → `yyparse()`[55b] → `yylex()`[46a]. Before describing the lexer, I will present the input routines it relies on. These routines handle reading characters from the current input source (terminal or script file), managing the prompt, and providing the one-character lookahead that the lexer needs.

5.1 Overview

The call chain for reading input is shown below:

```
yylex()
  |
  +---> nextc(); advance()   one-character lookahead (cached in 'future')
          |
          +---> getnext()      raw character read + prompt + escaped newlines
                  |
                  +---> rchr(runq->cmdfd)    buffered IO read
                          |
                          +---> read()       system call (terminal or file)
```

`getnext()`[41] reads a raw character and handles escaped newlines (for multi-line commands) and prompt display. `nextc()`[44e] adds a one-character lookahead on top of `getnext()`, and `advance()`[45a] consumes that lookahead.

5.2 Reading a character: `getnext()`

`getnext()`[41] reads one character from the current thread's input stream (`runq->cmdfd`). Besides the actual read (via `rchr()`[161c], the buffered IO read function), it handles three concerns: returning a previously peeked character (`peekc`), displaying the prompt before blocking on input, and converting escaped newlines (`\<newline>`) into spaces to support multi-line commands.

⟨*function* `getnext` 41⟩≡ (199)

```
/*
 * read a character from the input stream
 */
int
getnext(void)
{
    int c;
    ⟨getnext() other locals 42f⟩
```

```
    ⟨getnext() peekc handling 43a⟩
    ⟨getnext() return if already at EOF 42d⟩
    ⟨getnext() prompt management before reading the character 43f⟩

    c = rchr(runq->cmdfd);

    ⟨getnext() handle backslash 42e⟩
    ⟨getnext() prompt management after the character is read 44b⟩
    ⟨getnext() if character read is EOF 42b⟩
    ⟨getnext() if not EOF but verbose mode, print character read 152c⟩

    return c;
}
```

Uses `rchr()` 161c and `runq` 28c.

5.2.1 End-of-file management

When `rchr()`[161c] returns `EOF`, `getnext()`[41] sets the `eof` flag on the current thread. On subsequent calls, `getnext()` returns `EOF` immediately without trying to read again. This flag is also checked in `Xrdcmds()`[39a] to decide whether to close the input and return to the parent thread (see Chapter 4).

⟨*constant EOF (rc/io.h)* 42a⟩≡ (194b)

```
#define EOF (-1)
```

⟨`getnext()` *if character read is EOF* 42b⟩≡ (41)

```
if(c==EOF)
    runq->eof = true;
```

Uses `EOF` 42a, `runq` 28c, and `true`.

⟨`Thread` *other fields* 42c⟩+≡ (28b) ◁39b 44a▷

```
bool eof;  /* is cmdfd at eof? */
```

⟨`getnext()` *return if already at EOF* 42d⟩≡ (41)

```
if(runq->eof)
    return EOF;
```

Uses `EOF` 42a and `runq` 28c.

5.2.2 Multiple lines commands and escaped newlines

⟨`getnext()` *handle backslash* 42e⟩≡ (41)

```
if(!inquote && c=='\\'){

    c = rchr(runq->cmdfd);

    if(c=='\n' && !incomm){  /* don't continue a comment */
        ⟨getnext() when backslash and newline, set doprompt 44c⟩
        c=' ';
    }
    else{
        peekc = c;
        c='\\';
    }
}
```

Uses `incomm` 47e, `inquote` 51c, `rchr()` 161c, and `runq` 28c.

⟨`getnext()` *other locals* 42f⟩≡ (41)

```
static int peekc = EOF;
```

Uses `EOF` 42a.

⟨getnext() *peekc handling* 43a⟩≡ (41)

```
if(peekc!=EOF){
    c = peekc;
    peekc = EOF;
    return c;
}
```

Uses EOF 42a.

5.2.3 Displaying the prompt

In rc, the $prompt variable is a two-element list: the first element is the main prompt (e.g., "% "), and the second element is the continuation prompt displayed when a command spans multiple lines (e.g., "\t"). The prompt is displayed just before reading a character, not at the end of executing a command. This is done through the global doprompt flag, which is set to true after each newline (or EOF) and reset to false after the prompt is printed.

⟨*global* doprompt 43b⟩≡ (199)

```
bool doprompt = true;
```

Uses doprompt 43b and true.

⟨*global* promptstr 43c⟩≡ (197b)

```
char *promptstr;
```

⟨Xrdcmds() *other locals* 43d⟩≡ (39a)

```
word *prompt;
```

⟨Xrdcmds() *set promptstr if interactive mode* 43e⟩≡ (39a)

```
if(runq->iflag){
    prompt = vlook("prompt")->val;
    if(prompt)
        promptstr = prompt->word;
    else
        promptstr="% ";
}
```

Uses promptstr 43c, runq 28c, and vlook() 32b.

⟨getnext() *prompt management before reading the character* 43f⟩≡ (41)

```
if(doprompt)
    // pprompt() internally set doprompt back to false at the end
    pprompt();
```

Uses doprompt 43b and pprompt() 43g.

The subtlety in pprompt()[43g] is that after printing the current prompt string, it switches promptstr to the *second* element of $prompt (the continuation prompt). This way, if the lexer needs more input for a multi-line command, the next call to pprompt() will display the continuation prompt instead of the main one. The main prompt is restored in Xrdcmds()[39a] before each new command.

⟨*function* pprompt 43g⟩≡ (199)

```
void
pprompt(void)
{
    var *prompt;

    if(runq->iflag){
        // printing the prompt
        pstr(err, promptstr);
        flush(err);
```

```
        // set promptstr for the next pprompt()
        prompt = vlook("prompt");
        if(prompt->val && prompt->val->next)
            promptstr = prompt->val->next->word;
        else
            promptstr="\t";
    }
    runq->lineno++;
    doprompt = false;
}
```

Uses `doprompt` 43b, `err` 153b, `false`, `flush()` 160h, `promptstr` 43c, `pstr()` 164a, `runq` 28c, and `vlook()` 32b.

Each thread also tracks the current line number, incremented in `pprompt()` after each newline. This is used for error reporting when running scripts: a syntax error in line 42 of a script is much easier to find than one at an unknown location.

⟨`Thread` *other fields* 44a⟩+≡ (28b) ◁42c 77b▷

```
int lineno;   /* linenumber */
```

⟨`getnext()` *prompt management after the character is read* 44b⟩≡ (41)

```
doprompt = doprompt || c=='\n' || c==EOF;
```

Uses `EOF` 42a and `doprompt` 43b.

⟨`getnext()` *when backslash and newline, set doprompt* 44c⟩≡ (42e)

```
doprompt = true;
```

Uses `doprompt` 43b and `true`.

5.3 Looking ahead: `nextc()` and `advance()`

Lexers often need to look one character ahead to decide what token to produce. For example, when the lexer sees `&`, it must peek at the next character to decide whether it is a lone `&` (background operator) or part of `&&` (logical and). `nextc()`[44e] provides this lookahead by caching the next character in the global `future`. Multiple calls to `nextc()` return the same character until `advance()`[45a] is called to consume it. The helper `nextis()`[45b] combines both: it checks whether the next character matches, and consumes it if so.

⟨*global* `future` 44d⟩≡ (199)

```
int future = EOF;
```

Uses `EOF` 42a and `future` 44d.

⟨*function* `nextc` 44e⟩≡ (199)

```
/*
 * Look ahead in the input stream
 */
int
nextc(void)
{
    if(future==EOF)
        future = getnext();
    return future;
}
```

Uses `EOF` 42a, `future` 44d, and `getnext()` 41.

⟨*function* advance 45a⟩≡ (199)

```
/*
 * Consume the lookahead character.
 */
int
advance(void)
{
    int c = nextc();
    ⟨advance() save future in lastc 154a⟩
    future = EOF;
    return c;
}
```

Uses EOF 42a, future 44d, and nextc() 44e.

⟨*function* nextis 45b⟩≡ (199)

```
bool
nextis(int c)
{
    if(nextc()==c){
        advance();
        return true;
    }
    return false;
}
```

Uses advance() 45a, false, nextc() 44e, and true.

Chapter 6

Lexing

The lexer of `rc` is hand-written (not generated by `lex`), which gives more control over subtle interactions with the rest of the shell, at the cost of some complexity. The main function is `yylex()`[46a], called by the `yacc`-generated parser whenever it needs the next token.

6.1 yylex()

The structure of `yylex()`[46a] is a switch on the next character. After skipping whitespace, it dispatches on the character read: operators and special characters are handled as individual cases, and anything else is treated as a word character. There are a few lexing subtleties worth noting: newlines are sometimes skipped (via `skipnl()`[48a]) after certain operators to support multi-line commands; keywords like `if` and `for` are recognized as keywords only in command position (not as arguments); and the `(` character is handled specially depending on whether it follows a `$` (array subscript) or not (grouping/function definition).

⟨*function* `yylex` 46a⟩≡ (205a)

```
//@Scheck: called from yyparse()
int yylex(void)
{
    int c;
    ⟨yylex() other locals 49d⟩

    ⟨yylex() hack for SUB 54b⟩
    ⟨yylex() initialisations 49f⟩

    skipwhite();

    switch(c = advance()){
    ⟨yylex() switch c cases 47b⟩
    }
    // else
    ⟨yylex() if c is not a word character 51b⟩
    // else
    ⟨yylex() if c is a word character 52c⟩
}
```

Uses `advance()` 45a and `skipwhite()` 47f.

⟨*function* `wordchr` 46b⟩≡ (205a)

```
int
wordchr(int c)
{
    return !strchr("\n \t#;&|^$=`'{}()<>", c) && c!=EOF;
}
```

Uses `EOF` 42a.

⟨*function* idchr 47a⟩≡ (205a)

```
int
idchr(int c)
{
    /*
     * Formerly:
     * return 'a'<=c && c<='z' || 'A'<=c && c<='Z' || '0'<=c && c<='9'
     * || c=='_' || c=='*';
     */
    return c > ' ' && !strchr("!\"#$%&'()+,-./:;<=>?@[\\]^`{|}~", c);
}
```

⟨yylex() *switch c cases* 47b⟩≡ (46a) 48b ▷

```
case EOF:
    lastdol = false;
    strcpy(tok, "EOF");
    return EOF;
```

Uses EOF 42a, false, lastdol 49b, and tok 47c.

⟨*global* tok 47c⟩≡ (205a)

```
char tok[NTOK + UTFmax];
```

Uses NTOK<> 47d and UTFmax.

⟨*constant* NTOK 47d⟩≡ (205a)

```
#define NTOK 8192  /* maximum bytes in a word (token) */
```

6.2 Spaces and comments ('#')

skipwhite()[47f] skips spaces, tabs, and comments. In rc, comments start with # and extend to the end of the line. Note that the incomm global is needed by getnext()[41]: inside a comment, a backslash-newline should *not* be treated as a line continuation.

⟨*global* incomm 47e⟩≡ (199)

```
bool incomm;
```

⟨*function* skipwhite 47f⟩≡ (205a)

```
void
skipwhite(void)
{
    int c;
    for(;;){
        c = nextc();
        /* Why did this used to be  if(!inquote && c=='#') ?? */
        if(c=='#'){
            incomm = true;
            for(;;){
                c = nextc();
                if(c=='\n' || c==EOF) {
                    incomm = false;
                    break;
                }
                advance();
            }
        }
        if(c==' ' || c=='\t')
            advance();
        else
            return;
```

```
    }
}
```

Uses EOF 42a, advance() 45a, false, incomm 47e, nextc() 44e, and true.

6.3 Newlines

In rc, a newline normally acts as a command terminator (like ; in C). However, after certain operators like |, &&, or ||, a newline should be ignored because the user is clearly continuing the command on the next line. skipnl() 48a is called by the lexer after those operators to consume any whitespace and newlines until meaningful input arrives. This is why you can write things like:

```
ls |
wc -l
```

and have it work as expected.

⟨*function* skipnl 48a⟩≡ (205a)

```
void
skipnl(void)
{
    int c;
    for(;;){
        skipwhite();
        c = nextc();
        if(c!='\n')
            return;
        // consume the newline
        advance();
    }
}
```

Uses advance() 45a, nextc() 44e, and skipwhite() 47f.

6.4 Operators ('&', '&&', '|', '||', '<', '>', '$', ...)

Each operator is handled as a case in the main switch of yylex() 46a. For multi-character operators like && or ||, the lexer uses nextis() 45b to check the second character. Note that after binary operators like &&, ||, and |, the lexer calls skipnl() 48a so that newlines are treated as whitespace, allowing the command to continue on the next line. The lastdol flag tracks whether the previous token was $, which affects how (is lexed (see Section 6.7).

⟨yylex() *switch c cases* 48b⟩+≡ (46a) ◁47b 49a▷

```
case '&':
    lastdol = false;
    if(nextis('&')){
        skipnl();
        strcpy(tok, "&&");
        return ANDAND;
    }
    strcpy(tok, "&");
    return '&';
```

Uses false, lastdol 49b, nextis() 45b, skipnl() 48a, and tok 47c.

⟨yylex() *switch c cases* 49a⟩+≡ (46a) ◁48b 49c▷

```
  case '$':
      lastdol = true;
      if(nextis('#')){
          strcpy(tok, "$#");
          return COUNT;
      }
      if(nextis('"')){
          strcpy(tok, "$\"");
          return '"';
      }
      strcpy(tok, "$");
      return '$';
```

Uses `lastdol` 49b, `nextis()` 45b, `tok` 47c, and `true`.

⟨*global* `lastdol` 49b⟩≡ (205a)

```
  bool lastdol; /* was the last token read '$' or '$#' or '"'? */
```

⟨yylex() *switch c cases* 49c⟩+≡ (46a) ◁49a 51f▷

```
  case '|':
      lastdol = false;
      if(nextis('|')){
          skipnl();
          strcpy(tok, "||");
          return OROR;
      }
      // FALLTHROUGH
  case '<':
  case '>':
      lastdol = false;
      ⟨yylex() in switch when redirection character 49g⟩
```

Uses `false`, `lastdol` 49b, `nextis()` 45b, `skipnl()` 48a, and `tok` 47c.

Redirections and pipes are the most complex part of the lexer. Unlike simple operators like `&&` or `||` which map directly to a token, redirections carry extra information: the type (`WRITE`, `READ`, `APPEND`, `HERE`), and which file descriptors are involved (`fd0`, `fd1`). The lexer builds a `Tree` node right here (rather than in the parser), because the bracket syntax `>[2=1]` requires parsing digits and `=` signs that would be awkward to express in the `yacc` grammar.

⟨yylex() *other locals* 49d⟩≡ (46a) 49e▷

```
  char *w = tok;
```

Uses `tok` 47c.

⟨yylex() *other locals* 49e⟩+≡ (46a) ◁49d 54a▷

```
  struct Tree *t;
```

Uses `Tree` 194c.

⟨yylex() *initialisations* 49f⟩≡ (46a) 51d▷

```
  yylval.tree = nil;
```

Uses `nil`.

⟨yylex() *in switch when redirection character* 49g⟩≡ (49c)

```
  /*
   * funny redirection tokens:
   * redir: arrow | arrow '[' fd ']'
   * arrow: '<' | '<<' | '>' | '>>' | '|'
   * fd: digit | digit '=' | digit '=' digit
   * digit: '0'|'1'|'2'|'3'|'4'|'5'|'6'|'7'|'8'|'9'
   * some possibilities are nonsensical and get a message.
```

```
 */
*w++=c;
t = newtree();
switch(c){
⟨yylex() in switch when redirection character, switch c cases 50e⟩
}
⟨yylex() in switch when redirection character, if bracket after 133d⟩
*w='\0';
yylval.tree = t;
if(t->type==PIPE)
    skipnl();
return t->type;
```

Uses `newtree()` 25e and `skipnl()` 48a.

⟨`Tree` *redirection and pipe specific fields* 50a⟩≡ (25a)

```
//enum<Redirection_kind>
int rtype;

// For a pipe, fd0 is the left fd of the pipe, and fd1 the right fd.
// For a redirection, fd0 is what we redirect (stdout for >, stdin for <)
// and fd1 is what we possibly redirect to (when DUP).
fdt fd0;
fdt fd1; /* details of REDIR PIPE DUP tokens */
```

⟨*constant* `WRITE` 50b⟩≡ (194c)

```
#define WRITE 2
```

⟨*constant* `READ` 50c⟩≡ (194c)

```
#define READ 3
```

⟨*constant* `APPEND` 50d⟩≡ (194c)

```
#define APPEND 1
```

Each redirection character is handled as a case below. For a pipe, `fd0` defaults to 1 (stdout of the left command) and `fd1` to 0 (stdin of the right command), which is the standard `cmd1 | cmd2` behavior. For `>`, `fd0` defaults to 1 (redirect stdout); for `<`, `fd0` defaults to 0 (redirect stdin). A double `>>` sets the type to `APPEND`. These defaults can be overridden with the bracket syntax (e.g., `>[2]` to redirect stderr); see Section 14.4.4.

⟨`yylex()` *in switch when redirection character, switch c cases* 50e⟩≡ (49g) 50f▷

```
case '|':
    t->type = PIPE;
    t->fd0 = 1; // left fd of pipe (stdout of left cmd)
    t->fd1 = 0; // right fd of pipe (stdin of right cmd)
    break;
```

⟨`yylex()` *in switch when redirection character, switch c cases* 50f⟩+≡ (49g) ◁50e 51a▷

```
case '>':
    t->type = REDIR;
    if(nextis('>')){
        t->rtype = APPEND;
        *w++=c;
    }
    else
        t->rtype = WRITE;
    t->fd0 = 1;
    break;
```

Uses `APPEND` 50a, `WRITE`, and `nextis()` 45b.

⟨yylex() *in switch when redirection character, switch c cases* 51a⟩+≡ (49g) ◁50f

```
case '<':
    t->type = REDIR;
    ⟨yylex() in switch when redirection character, if here document 129b⟩
    ⟨yylex() in switch when redirection character, if read/write redirect 133a⟩
    else
        t->rtype = READ;
    t->fd0 = 0;
    break;
```

Uses READ.

⟨yylex() *if c is not a word character* 51b⟩≡ (46a)

```
if(!wordchr(c)){
    lastdol = false;
    tok[0] = c;
    tok[1]='\0';
    return c;
}
```

Uses false, lastdol 49b, tok 47c, and wordchr() 46b.

6.5 Quoted strings ('...')

In rc, strings are quoted with single quotes. To include a literal single quote inside a quoted string, you double it: 'it''s' produces it's. The inquote flag is needed by getnext()[41] because inside a quoted string, backslash-newline should *not* be treated as a line continuation. The lastword flag tracks whether the previous token was a word, which the grammar uses to decide whether adjacent tokens should be concatenated (e.g., a^b).

⟨*global* inquote 51c⟩≡ (199)

```
bool inquote;
```

⟨yylex() *initialisations* 51d⟩+≡ (46a) ◁49f

```
inquote = false;
```

Uses false and inquote 51c.

⟨*global* lastword 51e⟩≡ (205a)

```
// used also by syn.y
bool lastword; /* was the last token read a word or compound word terminator? */
```

⟨yylex() *switch c cases* 51f⟩+≡ (46a) ◁49c

```
case '\'':
    inquote = true;
    lastword = true;
    lastdol = false;
    for(;;){
        c = advance();
        if(c==EOF)
            break;
        if(c=='\''){
            if(nextc()!='\'')
                break;
            advance();
        }
        w = addutf(w, c);
    }
    if(w != nil)
        *w='\0';
```

```
    t = token(tok, WORD);
    t->quoted = true;

    yylval.tree = t;
    return t->type;
```

Uses EOF 42a, addutf() 127b, advance() 45a, false, inquote 51c, lastdol 49b, lastword 51e, nextc() 44e, nil, tok 47c, token() 52b, and true.

⟨Tree *word specific fields* 52a⟩≡ (25a)

```
bool quoted;
```

⟨*function* token 52b⟩≡ (199)

```
tree*
token(char *str, int type)
{
    tree *t = newtree();

    t->type = type;
    t->str = strdup(str);
    return t;
}
```

Uses newtree() 25e.

6.6 Keywords and identifiers (if, for, while, switch, fn, ...)

When the lexer encounters a word character (anything not an operator or whitespace), it reads characters until a non-word character appears. After a $, only identifier characters (idchr) are accepted, which excludes slashes and other path characters. Once the word is assembled, klook()[53a] checks whether it matches a keyword. Keywords are stored in a small hash table initialized by kinit()[53e] at startup. If the word is a keyword, klook() returns the appropriate token type; otherwise it returns WORD.

⟨yylex() *if c is a word character* 52c⟩≡ (46a)

```
for(;;){
    ⟨yylex() when c is a word character, if glob character 117b⟩
    w = addutf(w, c);

    c = nextc();
    if(lastdol ? !idchr(c) : !wordchr(c))
        break;
    advance();
}

lastword = true;
lastdol = false;
if(w!=nil)
    *w='\0';

t = klook(tok);
if(t->type != WORD)
    lastword = false;

t->quoted = false;

yylval.tree = t;
return t->type;
```

Uses addutf() 127b, advance() 45a, false, idchr() 47a, klook() 53a, lastdol 49b, lastword 51e, nextc() 44e, nil, tok 47c, true, and wordchr() 46b.

⟨*function* klook 53a⟩≡ (199)

```
tree*
klook(char *name)
{
    struct Kw *p;
    tree *t = token(name, WORD);

    for(p = kw[hash(name, NKW)];p;p = p->next)
        if(strcmp(p->name, name)==0){
            t->type = p->type;
            break;
        }
    return t;
}
```

Uses Kw 53b, NKW<> 53d, hash() 33a, kw 53c, and token() 52b.

⟨*struct* Kw 53b⟩≡ (199)

```
struct Kw {
    char *name;
    int type;

    struct Kw *next;
};
```

Uses Kw 53b.

⟨*global* kw 53c⟩≡ (199)

```
struct Kw *kw[NKW];
```

Uses Kw 53b and NKW<> 53d.

⟨*constant* NKW 53d⟩≡ (199)

```
#define NKW 30
```

⟨*function* kinit 53e⟩≡ (199)

```
void
kinit(void)
{
    kenter(FOR, "for");
    kenter(IN, "in");
    kenter(WHILE, "while");
    kenter(IF, "if");
    kenter(NOT, "not");
    kenter(SWITCH, "switch");
    kenter(FN, "fn");

    kenter(TWIDDLE, "~");
    kenter(BANG, "!");
    kenter(SUBSHELL, "@");
}
```

Uses kenter() 53f.

⟨*function* kenter 53f⟩≡ (199)

```
void
kenter(int type, char *name)
{
    int h = hash(name, NKW);
    struct Kw *p = new(struct Kw);
    p->type = type;
    p->name = name;
    p->next = kw[h];
```

```
    kw[h] = p;
}
```

Uses Kw 53b, NKW<> 53d, hash() 33a, and kw 53c.

6.7 Array subscript (<arr>(<n>))

This is one of the trickiest parts of the lexer. When a word is immediately followed by (, as in $x(1), it must be treated as an array subscript (SUB) rather than a regular parenthesis. Additionally, when two words are adjacent without whitespace (e.g., -x$flag), the lexer inserts an implicit ^ (caret, the concatenation operator) between them. Without this, the lexer would produce three separate tokens and there would be no way to distinguish -x$flag (one argument) from -x \$flag (two arguments).

⟨yylex() *other locals* 54a⟩+≡ (46a) ◁ 49e

```
int d = nextc();
```

Uses nextc() 44e.

⟨yylex() *hack for SUB* 54b⟩≡ (46a)

```
/*
 * Embarassing sneakiness:  if the last token read was a quoted or unquoted
 * WORD then we alter the meaning of what follows.  If the next character
 * is '(', we return SUB (a subscript paren) and consume the '('. Otherwise,
 * if the next character is the first character of a simple or compound word,
 * we insert a '^' before it.
 */
if(lastword){
    lastword = false;
    if(d=='('){
        advance();
        strcpy(tok, "( [SUB]");
        return SUB;
    }
    if(wordchr(d) || d=='\'' || d=='`' || d=='$' || d=='"'){
        strcpy(tok, "^");
        return '^';
    }
}
```

Uses advance() 45a, false, lastword 51e, tok 47c, and wordchr() 46b.

Chapter 7

Parsing

Now that we have seen how the lexer produces tokens from the input stream, we can describe the parser, which assembles those tokens into an abstract syntax tree.

7.1 Overview

The parser of `rc` is written using `yacc` (see COMPILER book [Pad16b] for an introduction to `yacc`). The grammar is quite compact, which reflects the simplicity of the `rc` language. The top-level rule `rc` parses a single line: either an empty line (EOF), or a `line` followed by a newline. When a complete line is parsed, the action calls `compile()`[63a] to transform the AST into bytecodes. Note that newlines have a semantic role in `rc`: they terminate commands (like `;` in C). This is what makes the shell convenient for interactive use — you do not need an explicit semicolon to run a command.

The `.y` file follows the standard `yacc` layout: a prologue section (`%...%`) with C includes, then declarations (`\%union`, `\%token`, `\%left`, `\%type`), and finally the grammar rules after `%%`. The `\%union` declaration tells `yacc` that semantic values (`yylval`, `\$1`, `\$2`, etc.) are `Tree` pointers, since every grammar rule builds or passes along AST nodes.

⟨*rc/syn.y* 55a⟩≡
```
%{
#include "rc.h"
#include "fns.h"
%}

%union {
 struct Tree *tree;
};

⟨token declarations 24⟩
⟨priority and associativity declarations 55c⟩
⟨type declarations 56a⟩

%%
⟨grammar 56b⟩
```

⟨*function* `yyparse` 55b⟩≡
```
... generated code from syn.y by yacc ...
```

⟨*priority and associativity declarations* 55c⟩≡ (55a)
```
/* operator priorities -- lowest first */
%left IF WHILE FOR SWITCH ')' NOT
%left ANDAND OROR
%left BANG SUBSHELL
```

```
%left PIPE
%left '^'
%right '$' COUNT '"'
%left SUB
```

⟨*type* declarations 56a⟩≡ (55a) ?? ▷

```
%type<tree> line cmd simple word comword
```

⟨*grammar* 56b⟩≡ (55a)

```
rc:
    /*empty*/       { return ERROR_1;}
|   line '\n'       { return !compile($1);}

⟨line rule 56c⟩
⟨cmd rule 56d⟩
⟨simple rule 56e⟩

⟨word rule 57a⟩
⟨comword rule 56g⟩

⟨other rules 56f⟩
```

7.2 Simple commands (<cmd> <arg1>...<argn>)

The grammar is layered: a `line` is one or more `cmd`s separated by `;` or `&`, a `cmd` is a `simple` command (or a control flow statement), and a `simple` command is a `first` word followed by zero or more `word` arguments. The distinction between `first` and `word` is important: a `first` cannot be a keyword, so that `if` in command position is recognized as a keyword, but `echo if` treats `if` as a regular word argument.

⟨*line rule* 56c⟩≡ (56b)

```
line:
    cmd
⟨line rule other cases 58a⟩
```

⟨*cmd rule* 56d⟩≡ (56b)

```
cmd:
    /*empty*/           {$$=nil;}
|   simple              {$$=simplemung($1);}
⟨cmd rule other cases 58c⟩
```

⟨*simple rule* 56e⟩≡ (56b)

```
simple:
    first
|   simple word         {$$=tree2(ARGLIST, $1, $2);}
⟨simple rule other cases 59d⟩
```

⟨*other rules* 56f⟩≡ (56b) 57b ▷

```
first:
    comword
|   first '^' word      {$$=tree2('^', $1, $3);}
```

⟨*comword rule* 56g⟩≡ (56b)

```
comword:
    WORD
⟨comword rule other cases 61g⟩
```

⟨*word rule* 57a⟩≡ (56b)

```
word:
    comword
|   word '^' word       {$$=tree2('^', $1, $3);}
|   keyword             {lastword=true; $1->type=WORD;}
```

⟨*other rules* 57b⟩+≡ (56b) ◁56f 57c▷

```
keyword: FOR|IN|WHILE|IF|NOT|TWIDDLE|BANG|SUBSHELL|SWITCH|FN
```

⟨*other rules* 57c⟩+≡ (56b) ◁57b 58b▷

```
words:
    /*empty*/       {$$=(struct Tree*)nil;}
|   words word      {$$=tree2(WORDS, $1, $2);}
```

`simplemung()`[57d] wraps a simple command in a `SIMPLE` node and also stores a string representation of the whole command in `Tree.str`. This string is used later to store function definitions in the environment (see Chapter 10). It also "percolates" redirections to the root of the subtree, so that the bytecode generator can handle them before executing the command.

⟨*function* `simplemung` 57d⟩≡ (204b)

```
/*
 * Add a SIMPLE node at the root of t and percolate all the redirections
 * up to the root.
 */
//@Scheck: used by syn.y
tree* simplemung(tree *t)
{
    tree *u;
    struct Io *s;

    t = tree1(SIMPLE, t);

    s = openstr();
    pfmt(s, "%t", t);
    t->str = strdup((char *)s->strp);
    closeio(s);

    ⟨simplemung() percolate redirections up to the root 57e⟩
    return t;
}
```

Uses `Io` 160e, `closeio()` 162b, `openstr()` 162c, `pfmt()` 163b, and `tree1()` 26b.

⟨`simplemung()` *percolate redirections up to the root* 57e⟩≡ (57d)

```
for(u = t->child[0]; u->type==ARGLIST; u = u->child[0]){
    if(u->child[1]->type==REDIR || u->child[1]->type==DUP){
        u->child[1]->child[1] = t;
        t = u->child[1];
        u->child[1] = nil;
    }
}
```

Uses `nil`.

For example, when parsing `grep pattern < input > output`, the parser first builds a flat `ARGLIST` tree with the redirections mixed among the arguments. Then `simplemung()` percolates the redirections up to the root, producing an AST where redirections wrap the command:

```
Before simplemung():            After simplemung():

SIMPLE                          > "output" fd0=1
```

```
 |                                  |
ARGLIST                           < "input" fd0=0
 / \                                |
ARGLIST       > "output"          SIMPLE
 / \                                |
ARGLIST  < "input"                ARGLIST
 / \                                / \
"grep"  "pattern"                "grep"  "pattern"
```

This way, the bytecode generator encounters the redirections first and can set them up before executing the command.

7.3 Operators

Operators are spread across different grammar levels, which implicitly defines their priorities. Sequencing (`;` and `&`) is at the `line` level (lowest priority), logical operators (`&&`, `||`) and pipes (`|`) are at the `cmd` level, and redirections are at the `simple` level (highest priority, closest to the command words).

7.3.1 Sequences (';', '&')

The `;` operator sequences two commands. The `&` operator runs a command asynchronously (in the background) and is treated as a unary postfix operator: `x & y` is parsed as `(x&); y`.

⟨*line rule other cases* 58a⟩≡ (56c)

```
|   cmdsa line      {$$=tree2(';', $1, $2);}
```

⟨*other rules* 58b⟩+≡ (56b) ◁57c 59e▷

```
cmdsa:
    cmd ';'
|   cmd '&'         {$$=tree1('&', $1);}
```

7.3.2 Logical operators ('&&', '||', '!')

⟨*cmd rule other cases* 58c⟩≡ (56d) 58d▷

```
|   cmd ANDAND cmd      {$$=tree2(ANDAND, $1, $3);}
|   cmd OROR cmd        {$$=tree2(OROR, $1, $3);}
```

⟨*cmd rule other cases* 58d⟩+≡ (56d) ◁58c 59a▷

```
|   BANG cmd            {$$=mung1($1, $2);}
```

⟨*function* mung1 58e⟩≡ (204b)

```
//@Scheck: used by syn.y
tree* mung1(tree *t, tree *c0)
{
    t->child[0] = c0;
    return t;
}
```

⟨*function* mung3 58f⟩≡ (204b)

```
//@Scheck: used by syn.y
tree* mung3(tree *t, tree *c0, tree *c1, tree *c2)
{
    t->child[0] = c0;
    t->child[1] = c1;
    t->child[2] = c2;
    return t;
}
```

7.3.3 String matching ('~')

The ~ (twiddle) operator is the fundamental comparison operator in `rc`. Since all values in `rc` are strings (or lists of strings), there are no arithmetic comparisons; instead, ~ performs pattern matching: `~ $x foo*` tests whether `$x` matches the pattern `foo*`[1]. It can also be used for simple equality tests (without wildcards) and even for counting: `~ $#list 0` tests if a list is empty. This is a major simplification compared to the Bourne shell, which relies on the external `test` program (or `[ ]`).

⟨*cmd rule other cases* 59a⟩+≡ (56d) ◁58d 59c▷

```
|   TWIDDLE word words  {$$=mung2($1, $2, $3);}
```

⟨*function* mung2 59b⟩≡ (204b)

```
//@Scheck: used by syn.y
tree* mung2(tree *t, tree *c0, tree *c1)
{
    t->child[0] = c0;
    t->child[1] = c1;
    return t;
}
```

7.3.4 Pipe ('|')

⟨*cmd rule other cases* 59c⟩+≡ (56d) ◁59a 59f▷

```
|   cmd PIPE cmd        {$$=mung2($2, $1, $3);}
```

7.3.5 Redirections ('>', '<')

Redirections are at the `simple` level in the grammar, which means they bind tighter than pipes and logical operators. For example, `ls | grep bi* > foo` is parsed as `ls | (grep bi* > foo)`, not `(ls | grep bi*) > foo`—the redirection applies only to `grep`, not to the whole pipeline. If you want to redirect the output of an entire pipeline, you need braces: `{ls | grep bi*} > foo`. A command can have multiple redirections (e.g., `ls < foo > bar`), and they can appear mixed with arguments. The `simplemung()` function later percolates them to the root of the AST subtree.

⟨*simple rule other cases* 59d⟩≡ (56e)

```
|   simple redir        {$$=tree2(ARGLIST, $1, $2);}
```

⟨*other rules* 59e⟩+≡ (56b) ◁58b 59g▷

```
redir:
    REDIR word      {$$=mung1($1, $1->rtype==HERE ? heredoc($2) : $2);}
```

⟨*redir rule other cases* 134c⟩

⟨*cmd rule other cases* 59f⟩+≡ (56d) ◁59c 60b▷

```
|   brace epilog        {$$=epimung($1, $2);}
```

⟨*other rules* 59g⟩+≡ (56b) ◁59e 60d▷

```
epilog:
    /*empty*/           {$$=nil;}
|   redir epilog        {$$=mung2($1, $1->child[0], $2);}
```

[1] The * here is interpreted by the twiddle operator itself using `match()`[122b] (see Chapter 13), not by the shell's globbing mechanism which expands filenames.

⟨*function* epimung 60a⟩≡ (204b)

```
//@Scheck: used by syn.y
tree* epimung(tree *comp, tree *epi)
{
    tree *p;
    if(epi==0)
        return comp;
    for(p = epi;p->child[1];p = p->child[1]);
    p->child[1] = comp;
    return epi;
}
```

⟨*cmd rule other cases* 60b⟩+≡ (56d) ◁59f 60c▷

```
|   redir cmd  %prec BANG
        {$$=mung2($1, $1->child[0], $2);}
```

7.4 Control flow statements (`if`, `if not`, `while`, `switch`, `for`)

Note that `rc` uses `if not` instead of `else`. This is because `rc` parses and executes one line at a time. When the user types `if(test) cmd` and presses Enter, `rc` must parse, compile, and execute this line immediately. The problem with `else` is that it would need to appear on the *next* line:

```
if(test) cmd1
else cmd2
```

But `rc` has already fully processed the first line before it even sees `else`. It cannot retroactively turn `if(test) cmd1` into the first half of an if/else. The Bourne shell solves this with `if...then...fi`: the explicit `fi` terminator tells the parser when the `if` block ends, so `else` can appear unambiguously between `then` and `fi`. But this requires more keywords. `rc` takes a different approach: `if not` is a completely separate command that checks the exit status left by the previous `if`. No lookahead is needed, no extra keywords, and each line is self-contained. The `for` loop has two forms: `for(x in a b c)` iterates over the explicit list, and `for(x)` iterates over `$*` (the script arguments).

⟨*cmd rule other cases* 60c⟩+≡ (56d) ◁60b 61d▷

```
|   IF paren {skipnl();} cmd  {$$=mung2($1, $2, $4);}
|   IF NOT   {skipnl();} cmd  {$$=mung1($2, $4);}

|   WHILE paren {skipnl();} cmd    {$$=mung2($1, $2, $4);}
|   SWITCH word {skipnl();} brace  {$$=tree2(SWITCH, $2, $4);}

 /*
  * if ``words'' is nil, we need a tree element to distinguish between
  * for(i in ) and for(i), the former being a loop over the empty set
  * and the latter being the implicit argument loop.  so if $5 is nil
  * (the empty set), we represent it as "()".  don't parenthesize non-nil
  * functions, to avoid growing parentheses every time we reread the
  * definition.
  */
|   FOR '(' word IN words ')' {skipnl();} cmd
    {$$=mung3($1, $3,    $5 ? $5 : tree1(PAREN, $5), $8);}

|   FOR '(' word ')' {skipnl();} cmd
    {$$=mung3($1, $3, (struct Tree *)0, $6);}
```

⟨*other rules* 60d⟩+≡ (56b) ◁59g 61b▷

```
paren:  '(' body ')'        {$$=tree1(PCMD, $2);}
```

⟨*token declarations* 61a⟩+≡ (55a) ◁25b 128a▷

```
%token PCMD
```

⟨*other rules* 61b⟩+≡ (56b) ◁60d 61c▷

```
brace:  '{' body '}'        {$$=tree1(BRACE, $2);}
```

⟨*other rules* 61c⟩+≡ (56b) ◁61b 61f▷

```
body:
    cmd
|   cmdsan body     {$$=tree2(';', $1, $2);}

cmdsan:
    cmdsa
|   cmd '\n'
```

7.5 Functions (fn)

Function definition in `rc` uses `fn name {body}`. The second form, `fn name` without a body, *deletes* the function definition. Note that `words` (not just a single word) is accepted as the function name: this allows defining a function for multiple names at once (e.g., `fn sigint sighup {...\}` to handle multiple signals with the same handler).

⟨*cmd rule other cases* 61d⟩+≡ (56d) ◁60c 61e▷

```
|   FN words brace  {$$=tree2(FN, $2, $3);}
|   FN words        {$$=tree1(FN, $2);}
```

7.6 Variables (<x> = ...)

Variable assignment in `rc` is written `x=value`. The interesting aspect is that an assignment can be followed by a command: `x=value cmd` makes the assignment local to that command only. When `cmd` is empty, the assignment is global. The assignment appears *before* the command, following the Bourne shell convention (which was itself motivated by programs like `make` that accept `x=y` on the command line). The `%prec BANG` directive tells `yacc` to resolve ambiguities (e.g., `x=1 ls && echo done`) using the priority of `BANG`, giving assignment lower priority than pipes but higher than logical operators.

⟨*cmd rule other cases* 61e⟩+≡ (56d) ◁61d 128b▷

```
|   assign cmd                    %prec BANG
      {$$=mung3($1, $1->child[0], $1->child[1], $2);}
```

⟨*other rules* 61f⟩+≡ (56b) ◁61c

```
assign: first '=' word      {$$=tree2('=', $1, $3);}
```

⟨*comword rule other cases* 61g⟩≡ (56g) 61h▷

```
|   '$' word        {$$=tree1('$', $2);}
|   COUNT word      {$$=tree1(COUNT, $2);}
|   '$' word SUB words ')'  {$$=tree2(SUB, $2, $4);}
```

7.7 Lists ((...))

In `rc`, parentheses create lists: `(a b c)` is a list of three words. This is how you assign multiple values to a variable: `x=(a b c)`. Lists are first-class values in `rc`, which is one of its main improvements over the Bourne shell.

⟨*comword rule other cases* 61h⟩+≡ (56g) ◁61g 136a▷

```
|   '(' words ')'   {$$=tree1(PAREN, $2);}
%$
% a bit hard to call it word then ... when it is really a list of words.
```

Chapter 8

Bytecode Generation and Interpretation

I merged bytecode generation and interpretation in one chapter because it helps to see the bytecodes generated from a given AST node together with the functions that interpret those bytecodes. We could also group them with the related parsing, but the parser can be understood in isolation.

8.1 Overview

Recall the main flow: `main()`[35e] → `Xrdcmds()`[39a] → `yyparse()`[55b] → `compile()`[63a]. After `yyparse()` builds the AST, it calls `compile()` which walks the tree and emits bytecodes into `codebuf`[27b]. The bytecodes are then interpreted by the main loop in `main()`.

8.1.1 emitxxx()

The emit functions append elements to `codebuf`[27b], growing it as needed. There are three variants: `emitf()`[62d] for function pointers (the bytecodes), `emiti()`[62c] for integers (jump offsets, file descriptors), and `emits()`[62e] for strings (inline arguments like the string after `Xword`[64b]). `codep` is the current write position in the buffer, and `ncode` is the allocated capacity.

⟨*global* codep 62a⟩≡ (207a)
```
// idx in codebuf
int codep;
```

⟨*global* ncode 62b⟩≡ (207a)
```
int ncode;
```

⟨*function* emiti 62c⟩≡ (207a)
```
#define emiti(x) ((codep!=ncode || morecode()), codebuf[codep].i = (x), codep++)
```

⟨*function* emitf 62d⟩≡ (207a)
```
#define emitf(x) ((codep!=ncode || morecode()), codebuf[codep].f = (x), codep++)
```

⟨*function* emits 62e⟩≡ (207a)
```
#define emits(x) ((codep!=ncode || morecode()), codebuf[codep].s = (x), codep++)
```

⟨*function* morecode 62f⟩≡ (207a)
```
//@Scheck: used by the macros above (why marked as dead then??? TODO)
int morecode(void)
{
    ncode+=100;
    codebuf = (code *)realloc((char *)codebuf, ncode*sizeof codebuf[0]);
    if(codebuf==nil)
        panic("Can't realloc %d bytes in morecode!", ncode*sizeof(code));
    return OK_0;
}
```

Uses `OK_0`, `codebuf` 27b, `ncode` 62b, `nil`, and `panic()` 154b.

8.1.2 compile()

`compile()`[63a] allocates a fresh code buffer, then calls `outcode()`[64a] to recursively walk the AST and emit bytecodes. It appends `Xreturn`[82d] at the end so that when the bytecodes finish executing, the interpreter returns to the previous thread. Note that `compile()` is called for *each line* entered interactively (or each line of a script). So each line gets its own bytecode sequence ending with `Xreturn`.

⟨*function* `compile` 63a⟩≡ (207a)

```
//@Scheck: called from syn.y
error0 compile(tree *t)
{
    ncode = 100;
    codep = 0;
    codebuf = (code *)emalloc(ncode*sizeof(code));

    emiti(0);   /* reference count */
    outcode(t, flag['e'] ? true : false);

    ⟨compile() check nerror 63b⟩
    ⟨compile() after outcode and error management, read heredoc 129g⟩

    emitf(Xreturn);
    emitf(nil);

    return OK_1;
}
```

Uses `OK_1`, `Xreturn()` 82d, `codebuf` 27b, `codep` 62a, `emalloc()`, `emitf<>` 62d, `emiti<>` 62c, `false`, `flag` 34c, `ncode` 62b, `nil`, `outcode()` 64a, and `true`.

⟨`compile()` *check nerror* 63b⟩≡ (63a)

```
if(nerror){
    efree((char *)codebuf);
    return ERROR_0;
}
```

Uses `ERROR_0`, `codebuf` 27b, `efree()`, and `nerror` 153c.

8.1.3 outcode()

`outcode()`[64a] is the main recursive function that walks the AST and emits bytecodes. It switches on the node type and emits the appropriate sequence of bytecodes for each construct. The `eflag` parameter propagates the `-e` flag (exit on error): when set, an `Xeflag`[127a] bytecode is emitted after certain commands to check if they failed and exit accordingly.

⟨*constant c0 (rc/code.c)* 63c⟩≡ (207a)

```
#define c0 t->child[0]
```

⟨*constant c1 (rc/code.c)* 63d⟩≡ (207a)

```
#define c1 t->child[1]
```

⟨*constant c2 (rc/code.c)* 63e⟩≡ (207a)

```
#define c2 t->child[2]
```

⟨*function* outcode 64a⟩≡ (207a)

```
void
outcode(tree *t, bool eflag)
{
    ⟨outcode() locals 74a⟩

    if(t==nil)
        return;

    ⟨outcode() set iflast before switch 87g⟩
    switch(t->type){
    ⟨outcode() cases 67c⟩
    default:
        pfmt(err, "bad type %d in outcode\n", t->type);
        break;
    }
    ⟨outcode() set iflast after switch 87f⟩
}
```

Uses err 153b, nil, and pfmt() 163b.

8.1.4 argv management

Before describing the compilation of specific AST nodes, let me present the bytecodes used to manage the argument stack. Most compilation schemes follow the same pattern: emit Xmark[64c] to start a new word list on argv, emit a series of Xword[64b] to push words onto it, then emit a bytecode (like Xsimple[68c]) that consumes the list. For example, ls -l /tmp compiles to: Xmark, Xword "ls", Xword "-l", Xword "/tmp", Xsimple.

⟨*function* Xword 64b⟩≡ (201b)

```
void
Xword(void)
{
    pushword(runq->code[runq->pc++].s);
}
```

Uses pushword() 38b and runq 28c.

⟨*function* Xmark 64c⟩≡ (201b)

```
void
Xmark(void)
{
    pushlist();
}
```

Uses pushlist() 38a.

⟨*function* poplist 64d⟩≡ (201a)

```
void
poplist(void)
{
    list *p = runq->argv;
    if(p==nil)
        panic("poplist but no argv", 0);
    freelist(p->words);
    runq->argv = p->next;
    efree((char *)p);
}
```

Uses efree(), freelist() 31a, nil, panic() 154b, and runq 28c.

⟨*function* popword 65a⟩≡ (201a)

```
void
popword(void)
{
    word *p;
    ⟨popword() sanity check argv 65b⟩
    p = runq->argv->words;
    ⟨popword() sanity check argv words 65c⟩
    runq->argv->words = p->next;
    efree(p->word);
    efree((char *)p);
}
```

Uses `efree()` and `runq` 28c.

⟨popword() *sanity check argv* 65b⟩≡ (65a)

```
if(runq->argv==nil)
    panic("popword but no argv!", 0);
```

Uses `nil`, `panic()` 154b, and `runq` 28c.

⟨popword() *sanity check argv words* 65c⟩≡ (65a)

```
if(p==nil)
    panic("popword but no word!", 0);
```

Uses `nil` and `panic()` 154b.

8.1.5 Process status management

The exit status of the last command is stored in the `$status` variable. `setstatus()`[65d] sets it, and `getstatus()`[65e] and `truestatus()`[66a] read it. In `rc`, a status is a string (not an integer like in the Bourne shell). An empty string or the string `"0"` means success; anything else means failure. This mirrors the Plan 9 system call `exits()` (see KERNEL book [Pad14]), which takes a string argument rather than a numeric exit code.

⟨*function* setstatus 65d⟩≡ (208b)

```
void
setstatus(char *s)
{
    setvar("status", newword(s, (word *)nil));
}
```

Uses `newword()` 29d, `nil`, and `setvar()` 32a.

⟨*function* getstatus 65e⟩≡ (208b)

```
char*
getstatus(void)
{
    var *status = vlook("status");
    return status->val ? status->val->word : "";
}
```

Uses `vlook()` 32b.

The `|` check in `truestatus()` below deserves explanation. When a pipeline like `ls | wc` finishes, `rc` combines the exit statuses of all commands into a single string separated by `|`, for example `"0|0"` if both succeed or `"error|0"` if the first fails. So `truestatus()` considers a status "true" if it contains only `0` and `|` characters—meaning every stage of the pipeline succeeded. Why not reduce the pipeline status to a single value earlier? Because keeping the full string lets the user inspect `\$status` and see exactly which stage failed (e.g., `"0|error|0"` tells you the middle command had a problem). With numeric exit codes (as in UNIX), reducing is easy: just take the maximum or the last non-zero value. But with string statuses, there is no natural way to merge `"can't open"` and `"0"` into a single meaningful string, so `rc` preserves them all. In `bash`, `\$?` only

reports the exit status of the *last* command in the pipeline—a failure in an earlier stage is silently lost. `bash` later added the `PIPESTATUS` array to work around this, but it requires explicit use.

This is another example of `rc` getting the design right from the start, while `bash` accumulates workarounds: pipeline statuses are visible by default in `\$status` (vs. `PIPESTATUS` added later in `bash`), all variables are automatically exported via `/env/` (vs. the error-prone `export` command), `if not` avoids the parsing ambiguity of `else` (vs. the heavyweight `if...then...fi` syntax), and string exit statuses naturally carry diagnostic information (vs. opaque numeric codes that require `strerror()` or a manual to decode).

⟨*function* `truestatus` 66a⟩≡ (208b)

```
bool
truestatus(void)
{
    char *s;
    for(s = getstatus();*s;s++)
        if(*s!='|' && *s!='0')
            return false;
    return true;
}
```

Uses `false`, `getstatus()` 65e, and `true`.

8.1.6 Subprocesses management

`rc` keeps track of all child processes it has spawned in the `waitpids` array. This is needed for the `wait` builtin (which waits for all children) and for proper cleanup when `rc` exits.

⟨*global* `waitpids` 66b⟩≡ (204a)

```
// growing_array<pid> (but really a list)
int *waitpids;
```

⟨*global* `nwaitpids` 66c⟩≡ (204a)

```
int nwaitpids;
```

⟨*function* `addwaitpid` 66d⟩≡ (204a)

```
void
addwaitpid(int pid)
{
    ⟨addwaitpid() grow waitpids if needed 66e⟩
    waitpids[nwaitpids++] = pid;
}
```

Uses `nil`, `nwaitpids` 66d, and `waitpids` 66c.

⟨`addwaitpid()` *grow* `waitpids` *if needed* 66e⟩≡ (66d)

```
waitpids = realloc(waitpids, (nwaitpids+1)*sizeof waitpids[0]);
if(waitpids == nil)
    panic("Can't realloc %d waitpids", nwaitpids+1);
```

⟨*function* `delwaitpid` 66f⟩≡ (211b)

```
void
delwaitpid(int pid)
{
    int r, w;

    for(r=w=0; r<nwaitpids; r++)
        if(waitpids[r] != pid)
            waitpids[w++] = waitpids[r];
    nwaitpids = w;
}
```

⟨*function* clearwaitpids 67a⟩≡ (204a)

```
void
clearwaitpids(void)
{
    nwaitpids = 0;
}
```

Uses nwaitpids 66d and waitpids 66c.

⟨*function* havewaitpid 67b⟩≡ (211b)

```
bool
havewaitpid(int pid)
{
    int i;

    for(i=0; i<nwaitpids; i++)
        if(waitpids[i] == pid)
            return true;
    return false;
}
```

8.2 Simple commands

Now that we have seen the helper infrastructure—argument stack management, process status, and subprocess tracking—we can describe the bytecode generation and interpretation for simple commands, which is the core of the shell.

8.2.1 Bytecode generation

A SIMPLE command compiles to: Xmark[64c] (start a new word list), followed by the bytecodes for the arguments (which push words onto the stack), followed by Xsimple[68c] (which pops the word list and executes the command). Note that ARGLIST and WORDS emit their children in reverse order: since Xword[64b] prepends to the word list, emitting the last word first ensures the list ends up in the correct order.

⟨outcode() *cases* 67c⟩≡ (64a) 67d ▷

```
case SIMPLE:
    emitf(Xmark);
    outcode(c0, eflag); // the arguments and argv0
    emitf(Xsimple);
    ⟨outcode() emit Xeflag after Xsimple 126b⟩
    break;
```

Uses Xmark() 64c, Xsimple() 68c, c0<> 63c 148b, emitf<> 62d, and outcode() 64a.

⟨outcode() *cases* 67d⟩+≡ (64a) ◁ 67c 67e ▷

```
case ARGLIST:
    outcode(c1, eflag);
    outcode(c0, eflag);
    break;
```

Uses c0<> 63c 148b, c1<> 63d 148c, and outcode() 64a.

⟨outcode() *cases* 67e⟩+≡ (64a) ◁ 67d 68a ▷

```
case WORDS:
    outcode(c1, eflag);
    outcode(c0, eflag);
    break;
```

Uses c0<> 63c 148b, c1<> 63d 148c, and outcode() 64a.

⟨outcode() *cases* 68a⟩+≡ (64a) ◁67e 73c▷

```
case WORD:
    emitf(Xword);
    emits(strdup(t->str));
    break;
```

Uses Xword() 64b, emitf<> 62d, and emits<> 62e.

⟨codefree() *in loop over code* cp*, switch bytecode cases* 68b⟩+≡ (27e) ◁28a 82c▷

```
else if(p->f==Xword || p->f==Xdelhere)
        efree((++p)->s);
```

Uses Xdelhere() 130b, Xword() 64b, and efree().

For example, `echo hello world` produces the following AST and bytecodes:

```
AST:                          codebuf:

SIMPLE                        Xmark
 |                            Xword "world"
ARGLIST                       Xword "hello"
 / \                          Xword "echo"
ARGLIST  "world"              Xsimple
 / \
"echo"  "hello"
```

Note how the arguments are emitted in reverse order (ARGLIST emits c1 before c0), so that after Xword prepends each word to the list, argv ends up in the correct order: echo, hello, world.

8.2.2 Xsimple()

Xsimple()[68c] is the bytecode that executes a simple command. It is the heart of rc and handles three cases, in order:

1. If the command name matches a shell *function* (defined with fn), it runs the function's bytecodes in a new thread.
2. If the command name matches a *builtin* (e.g., cd, exit, .), it calls the corresponding C function directly (without forking).
3. Otherwise, it forks a child process with execforkexec()[70a] and waits for it to finish.

The while(Waitfor(...) < 0) loop retries the wait if it was interrupted by a signal (e.g., ^C): the signal kills the child, but the parent still needs to reap it.

⟨*function* Xsimple 68c⟩≡ (206a)

```
void
Xsimple(void)
{
    word *a;
    thread *p = runq;
    int pid;
    ⟨Xsimple() other locals 93c⟩

    ⟨Xsimple() initializations, globlist() 118c⟩

    a = runq->argv->words;
    ⟨Xsimple() sanity check a 69a⟩
```

```
    ⟨Xsimple() if -x 152b⟩

    ⟨Xsimple() if argv0 is a function 94a⟩
    else{
        ⟨Xsimple() if argv0 is a builtin 101a⟩
        ⟨Xsimple() if exitnext() 73a⟩
        else{
            flush(err);
            Updenv(); /* necessary so changes don't go out again */
            if((pid = execforkexec()) < 0){
                Xerror("try again");
                return;
            }

            /* interrupts don't get us out */
            poplist();
            while(Waitfor(pid, true) < 0)
                ;
        }
    }
}
```

Uses `Updenv()`, `Waitfor()` 85b, `Xerror()` 69b, `err` 153b, `execforkexec()`, `flush()` 160h, `poplist()` 64d, and `runq` 28c.

⟨`Xsimple()` *sanity check a* 69a⟩≡ (68c)

```
if(a==nil){
    Xerror1("empty argument list");
    return;
}
```

Uses `Xerror1()` 69c and `nil`.

⟨*function* `Xerror` 69b⟩≡ (201a)

```
void
Xerror(char *s)
{
    if(strcmp(argv0, "rc")==0 || strcmp(argv0, "/bin/rc")==0)
        pfmt(err, "rc: %s: %r\n", s);
    else
        pfmt(err, "rc (%s): %s: %r\n", argv0, s);
    flush(err);
    setstatus("error");

    while(!runq->iflag)
        Xreturn();
}
```

Uses `Xreturn()` 82d, `argv0` 37a, `err` 153b, `flush()` 160h, `pfmt()` 163b, `runq` 28c, and `setstatus()` 65d.

⟨*function* `Xerror1` 69c⟩≡ (201a)

```
void
Xerror1(char *s)
{
    if(strcmp(argv0, "rc")==0 || strcmp(argv0, "/bin/rc")==0)
        pfmt(err, "rc: %s\n", s);
    else
        pfmt(err, "rc (%s): %s\n", argv0, s);
    flush(err);
    setstatus("error");

    while(!runq->iflag)
        Xreturn();
```

```
}
```

Uses `Xreturn()` 82d, `argv0` 37a, `err` 153b, `flush()` 160h, `pfmt()` 163b, `runq` 28c, and `setstatus()` 65d.

8.2.3 Fork

`execforkexec()`[70a] implements the fundamental `fork+exec+wait` pattern that is the essence of a shell. It forks a child process, and in the child, calls `execexec()`[70b] to replace the child with the actual command. If `exec` fails (e.g., the command does not exist), the child exits with an error; the parent continues normally. The `pushword("exec")` before `execexec()` is a trick: `execexec()` is also used as the `exec` builtin, where `exec` appears as the first word in `argv`. Here we add it artificially so `execexec()` can `popword()` it uniformly.

⟨*function* `execforkexec` 70a⟩≡ (204a)

```
int
execforkexec(void)
{
    int pid;
    int n;
    char buf[ERRMAX];

    // fork()!!
    switch(pid = fork()){
    case -1:
        return -1;
    case 0: // child
        clearwaitpids();
        pushword("exec");
        execexec();

        // should not be reached! unless the command did not exist
        strcpy(buf, "can't exec: ");
        n = strlen(buf);
        errstr(buf+n, ERRMAX-n);
        Exit(buf, __LOC__);
    }
    // parent
    addwaitpid(pid);
    return pid;
}
```

Uses `PRD` 31d, `PWR`, `READ`, `Thread` 28b, `Xerror()` 69b, `clearwaitpids()` 72a, `runq` 28c, and `start()` 36e.

8.2.4 Exec

⟨*function* `execexec` 70b⟩≡ (206a)

```
void
execexec(void)
{
    popword(); /* "exec" */
    ⟨execexec() sanity check arguments 71a⟩
    ⟨execexec() perform the redirections 76b⟩

    Execute(runq->argv->words, searchpath(runq->argv->words->word));
    // should not be reached! unless command did not exist
    poplist();
}
```

Uses `Execute()` 71d, `poplist()` 64d, `popword()` 65a, `runq` 28c, and `searchpath()` 71b.

⟨execexec() *sanity check arguments* 71a⟩≡ (70b)

```
if(runq->argv->words==nil){
    Xerror1("empty argument list");
    return;
}
```

Uses Xerror1() 69c, nil, and runq 28c.

8.2.5 $path management

When the command name starts with /, ./, ../, or # (device path in Plan 9), it is treated as an absolute path and no search is performed. Otherwise, searchpath()[71b] returns the list of directories in $path to try. Execute()[71d] then iterates over these directories, prepending each to the command name and calling exec(). If exec() fails, it tries the next directory. This is the standard UNIX path search mechanism.

⟨*function* searchpath 71b⟩≡ (209a)

```
word*
searchpath(char *w)
{
    word *path;

    if(strncmp(w, "/", 1)==0
    || strncmp(w, "#", 1)==0
    || strncmp(w, "./", 2)==0
    || strncmp(w, "../", 3)==0
    || (path = vlook("path")->val)==nil)
        path=&nullpath;
    return path;
}
```

Uses nil, nullpath 71c, and vlook() 32b.

⟨*global* nullpath 71c⟩≡ (209a)

```
struct Word nullpath = { "", nil};
```

Uses Word 194c, nil, and nullpath 71c.

⟨*function* Execute 71d⟩≡ (204a)

```
void
Execute(word *args, word *path)
{
    char **argv = mkargv(args);
    char file[1024];
    char errstr[1024];
    int nc;

    Updenv();
    errstr[0] = '\0';

    for(;path;path = path->next){
        nc = strlen(path->word);
        if(nc < sizeof file - 1){ /* 1 for / */
            strcpy(file, path->word);
            if(file[0]){
                strcat(file, "/");
                nc++;
            }
            if(nc + strlen(argv[1]) < sizeof file){
                strcat(file, argv[1]);

                // The actual exec() system call!
```

```
                exec(file, argv+1);

                // reached if the file does not exist

                rerrstr(errstr, sizeof errstr);
                /*
                 * if file exists and is executable, exec should
                 * have worked, unless it's a directory or an
                 * executable for another architecture.  in
                 * particular, if it failed due to lack of
                 * swap/vm (e.g., arg. list too long) or other
                 * allocation failure, stop searching and print
                 * the reason for failure.
                 */
                if (strstr(errstr, " allocat") != nil ||
                    strstr(errstr, " full") != nil)
                    break;
            }
            else werrstr("command name too long");
        }
    }
    // should not be reached if found an actual binary to exec
    pfmt(err, "%s: %s\n", argv[1], errstr);
    efree((char *)argv);
}
```

Uses `Updenv()`, `count()` 30a, `emalloc()`, `false`, `mkargv()` 71d, `nil`, `nwaitpids` 66d, `true`, and `waitpids` 66c.

⟨*function* `mkargv` 72a⟩≡ (204a)

```
char **
mkargv(word *a)
{
    char **argv = (char **)emalloc((count(a)+2) * sizeof(char *));
    char **argp = argv+1; /* leave one at front for runcoms */

    for(;a;a = a->next)
        *argp++=a->word;
    *argp = nil;
    return argv;
}
```

Uses `nwaitpids` 66d and `waitpids` 66c.

8.2.6 Wait

Back in the parent process, `Waitfor()`[85b] loops calling `wait()` until it reaps the child with the expected `pid`. If `wait()` returns a different child (from a background process started with `&`), its status is recorded but the loop continues. If `wait()` is interrupted by a signal, `Waitfor()` returns `-1`, and the caller (`Xsimple`) retries.

⟨*function* `Waitfor` 72b⟩≡ (211b)

```
int
Waitfor(int pid, bool _persist)
{
    thread *p;
    Waitmsg *w;
    char errbuf[ERRMAX];

    if(pid >= 0 && !havewaitpid(pid))
        return 0;
```

```
    // wait()!! until we found it
    while((w = wait()) != nil){
        delwaitpid(w->pid);

        if(w->pid==pid){
            setstatus(w->msg);
            free(w);
            return 0;
        }
        ⟨Waitfor() in while loop, if wait returns another pid 84e⟩
    }

    errstr(errbuf, sizeof errbuf);
    if(strcmp(errbuf, "interrupted")==0)
        return -1;
    return 0;
}
```

8.2.7 Fork optimization

If the next bytecode after `Xsimple`[68c] is `Xexit`[85c] (possibly preceded by `Xpopredir`[79b] calls), the shell is about to exit anyway, so forking a child and waiting for it is pointless. In that case, `Xsimple` calls `execexec()`[70b] directly, replacing the current process with the command. This is the same optimization that the `exec` builtin provides, applied automatically. This matters in practice for scripts like `rc -c 'ls'`: without this optimization, `rc` would fork, exec `ls` in the child, wait, and then exit—wasting a process.

⟨`Xsimple()` *if exitnext()* 73a⟩≡ (68c)

```
if(exitnext()){
    /* fork and wait is redundant */
    pushword("exec");
    execexec();
    Xexit();
}
```

Uses `Xexit()` 85c, `execexec()` 70b, `exitnext()` 73b, and `pushword()` 38b.

⟨*function* `exitnext` 73b⟩≡ (206a)

```
/*
 * Search through the following code to see if we're just going to exit.
 */
int
exitnext(void){
    union Code *c = &runq->code[runq->pc];
    while(c->f==Xpopredir)
        c++;
    return c->f==Xexit;
}
```

Uses `Code` 25a, `Xexit()` 85c, `Xpopredir()` 79b, and `runq` 28c.

8.3 Operators

8.3.1 Basic sequence

The sequencing operator `;` simply emits the bytecodes of both children one after the other.

⟨`outcode()` *cases* 73c⟩+≡ (64a) ◁68a 74b▷

```
case ';':
```

```
    outcode(c0, eflag);
    outcode(c1, eflag);
    break;
```

Uses `c0<>` 63c 148b, `c1<>` 63d 148c, and `outcode()` 64a.

8.3.2 Logical operators

`&&` and `||` use a forward jump technique. For `cmd1 &\ cmd2`, the compiler emits: the bytecodes of `cmd1`, then `Xtrue`[74e] followed by a placeholder jump offset, then the bytecodes of `cmd2`. After emitting `cmd2`, `stuffdot()`[74c] patches the placeholder with the actual address, so `Xtrue` can jump past `cmd2` if the status is false. `Xtrue` and `Xfalse`[75a] are symmetric: `Xtrue` jumps if the status is *false* (skip the second command), and `Xfalse` jumps if the status is *true*.

⟨`outcode()` *locals* 74a⟩≡ (64a) 82a ▷

```
int p;
```

⟨`outcode()` *cases* 74b⟩+≡ (64a) ◁ 73c 74d ▷

```
case ANDAND:
    outcode(c0, false);
    emitf(Xtrue);
    p = emiti(0);
    outcode(c1, eflag);
    stuffdot(p);
    break;
```

Uses `Xtrue()` 74e, `c0<>` 63c 148b, `c1<>` 63d 148c, `emitf<>` 62d, `emiti<>` 62c, `false`, `outcode()` 64a, and `stuffdot()` 74c.

⟨*function* `stuffdot` 74c⟩≡ (207a)

```
void
stuffdot(int a)
{
    if(a<0 || codep<=a)
        panic("Bad address %d in stuffdot", a);
    codebuf[a].i = codep;
}
```

Uses `codebuf` 27b, `codep` 62a, and `panic()` 154b.

⟨`outcode()` *cases* 74d⟩+≡ (64a) ◁ 74b 75b ▷

```
case OROR:
    outcode(c0, false);
    emitf(Xfalse);
    p = emiti(0);
    outcode(c1, eflag);
    stuffdot(p);
    break;
```

Uses `Xfalse()` 75a, `c0<>` 63c 148b, `c1<>` 63d 148c, `emitf<>` 62d, `emiti<>` 62c, `false`, `outcode()` 64a, and `stuffdot()` 74c.

⟨*function* `Xtrue` 74e⟩≡ (201b)

```
void
Xtrue(void)
{
    if(truestatus()) runq->pc++;
    else runq->pc = runq->code[runq->pc].i;
}
```

Uses `runq` 28c and `truestatus()` 66a.

⟨*function* Xfalse 75a⟩≡ (201b)

```
void
Xfalse(void)
{
    if(truestatus()) runq->pc = runq->code[runq->pc].i;
    else runq->pc++;
}
```

Uses runq 28c and truestatus() 66a.

⟨outcode() *cases* 75b⟩+≡ (64a) ◁74d 75d▷

```
case BANG:
    outcode(c0, eflag);
    emitf(Xbang);
    break;
```

Uses Xbang() 75c, c0<> 63c 148b, emitf<> 62d, and outcode() 64a.

⟨*function* Xbang 75c⟩≡ (201b)

```
void
Xbang(void)
{
    setstatus(truestatus()? "false" : "");
}
```

Uses setstatus() 65d and truestatus() 66a.

8.3.3 String matching

The ~ operator pushes two word lists onto argv (the subject and the patterns), then calls Xmatch[75e]. Two Xmark[64c] bytecodes are needed because both the subject and the patterns can be multi-word (e.g., if they come from variable expansion), and they must be kept separate. Xmatch()[75e] converts the subject words to a single string (joining with spaces), then tries each pattern word against it using match()[122b] (described in Chapter 13).

⟨outcode() *cases* 75d⟩+≡ (64a) ◁75b 78c▷

```
case TWIDDLE:
    emitf(Xmark);
    outcode(c1, eflag);
    emitf(Xmark);
    outcode(c0, eflag);
    emitf(Xmatch);
    if(eflag)
        emitf(Xeflag);
    break;
```

Uses Xeflag() 127a, Xmark() 64c, Xmatch() 75e, c0<> 63c 148b, c1<> 63d 148c, emitf<> 62d, and outcode() 64a.

⟨*function* Xmatch 75e⟩≡ (201b)

```
void
Xmatch(void)
{
    word *p;
    char *subject;

    subject = list2str(runq->argv->words);
    setstatus("no match");
    for(p = runq->argv->next->words;p;p = p->next)
        if(match(subject, p->word, '\0')){
            setstatus("");
            break;
        }
    efree(subject);
```

```
    poplist();
    poplist();
}
```

Uses efree(), list2str() 76a, match() 122b, poplist() 64d, runq 28c, and setstatus() 65d.

⟨*function* list2str 76a⟩≡ (201b)

```
char*
list2str(word *words)
{
    char *value, *s, *t;
    int len = 0;
    word *ap;

    for(ap = words;ap;ap = ap->next)
        len += 1+strlen(ap->word);
    value = emalloc(len+1);

    s = value;
    for(ap = words;ap;ap = ap->next){
        for(t = ap->word;*t;)
            *s++=*t++;
        *s++=' ';
    }
    if(s==value)
        *s='\0';
    else s[-1]='\0';
    return value;
}
```

Uses emalloc().

8.3.4 Redirection

Redirections in rc are handled in two phases. When the interpreter encounters a redirection bytecode (e.g., Xwrite[79a]), it opens the target file and pushes a Redir record onto the thread's redirection stack. But it does *not* actually redirect the file descriptors yet. The actual redirection happens later, in execexec()[70b] (inside the forked child process), just before exec(): doredir()[77e] walks the redirection stack and performs the dup() and close() calls. This two-phase design is necessary because the parent shell must not modify its own file descriptors; only the child (which is about to be replaced by exec()) should have its stdin/stdout/stderr redirected.

⟨execexec() *perform the redirections* 76b⟩≡ (70b)

```
doredir(runq->redir);
```

Uses doredir() 77e and runq 28c.

⟨start() *set redir* 76c⟩≡ (36e)

```
p->redir = p->startredir = runq ? runq->redir : nil;
```

Uses nil and runq 28c.

Redir

The Redir structure records a pending redirection: a type (e.g., ROPEN for a regular file redirection), a from file descriptor (the one that was opened), and a to file descriptor (the one that should be replaced, such as 1 for stdout). The thread's redir field points to a linked list of these records, built up as the interpreter encounters redirection bytecodes. The startredir field in start()[36e] deserves attention: when a new thread is created, it inherits its parent's redirection stack (so subshells see the parent's redirections), but startredir remembers

where the inherited portion ends. This way, if an error occurs, the thread can pop only the redirections it added, without disturbing the parent's state.

⟨*struct* Redir 77a⟩≡ (197a)

```
struct Redir {
    // enum<redirection_kind_bis>
    char type; /* what to do */

    /* what to do it to */
    short from;
    short to;

    // Extra
    ⟨Redir extra fields 77c⟩
};
```

⟨Thread *other fields* 77b⟩+≡ (28b) ◁44a 81a▷

```
// list<ref_own<Redir>> (next = Redir.next)
struct Redir *redir; /* redirection stack */
```

Uses Redir 77a.

⟨Redir *extra fields* 77c⟩≡ (77a)

```
struct Redir *next;  /* what else to do (reverse order) */
```

Uses Redir 77a.

⟨*function* pushredir 77d⟩≡ (201a)

```
void
pushredir(int type, int from, int to)
{
    redir * rp = new(redir);
    rp->type = type;
    rp->from = from;
    rp->to = to;

    // add_list(runq->redir, rp)
    rp->next = runq->redir;
    runq->redir = rp;
}
```

Uses runq 28c.

doredir()

Because pushredir()[77d] prepends each new record to the front of the list, the list is in reverse order relative to the command line. For cmd > foo >[2=1], the list has >[2=1] first, then > foo. But the user expects the redirections to be applied left-to-right: first redirect stdout to foo, then duplicate fd 2 from fd 1. doredir()[77e] solves this by recursing to the end of the list before performing any dup() calls, so the deepest (oldest) redirection executes first—restoring the original left-to-right order.

⟨*function* doredir 77e⟩≡ (206a)

```
void
doredir(redir *rp)
{
    if(rp){
        // recurse first, so do them in the reverse order of the list
        doredir(rp->next);

        switch(rp->type){
        ⟨doredir() switch redir type cases 78a⟩
        }
```

```
    }
}
```

Uses doredir() 77e.

⟨doredir() *switch redir type cases* 78a⟩≡ (77e) 135h ▷

```
case ROPEN:
    if(rp->from != rp->to){
        dup(rp->from, rp->to);
        close(rp->from);
    }
    break;
```

Uses Dup() 155b and ROPEN 78b.

⟨*constant* ROPEN 78b⟩≡ (197a)

```
/*
 * redir types
 */
#define ROPEN 1   /* dup2(from, to); close(from); */
```

Bytecode generation

The bytecode for a REDIR node first evaluates the filename (with globbing), then emits the appropriate redirection bytecode (e.g., Xwrite[79a] for >), followed by the file descriptor number. After the redirected command (c1) finishes, Xpopredir[79b] cleans up by closing the opened file descriptor and removing the Redir record from the stack. This scoping ensures that in cmd1 > foo; cmd2, the redirection applies only to cmd1—once Xpopredir runs, cmd2 sees the original file descriptors.

⟨outcode() *cases* 78c⟩+≡ (64a) ◁ 75d 82b ▷

```
case REDIR:
    emitf(Xmark);
    outcode(c0, eflag);
    emitf(Xglob);

    switch(t->rtype){
    ⟨outcode() when REDIR case, switch redirection type cases 78d⟩
    }
    emiti(t->fd0);
    outcode(c1, eflag);
    emitf(Xpopredir);
    break;
```

Uses Xglob() 119a, Xmark() 64c, Xpopredir() 79b, c0<> 63c 148b, c1<> 63d 148c, emitf<> 62d, emiti<> 62c, and outcode() 64a.

Xwrite()

Xwrite()[79a] handles the > redirection at runtime. It pops the filename from the argv stack, opens (or creates) the file with Creat()[155b], and pushes an ROPEN record mapping the newly opened file descriptor to the target fd (typically 1 for stdout, read from the bytecode stream). The actual dup() will happen later in doredir()[77e].

⟨outcode() *when REDIR case, switch redirection type cases* 78d⟩≡ (78c) 79c ▷

```
case WRITE:
    emitf(Xwrite);
    break;
```

Uses WRITE, Xwrite() 79a, and emitf<> 62d.

⟨*function* Xwrite 79a⟩≡ (201b)

```
void
Xwrite(void)
{
    char *file;
    fdt f;
    switch(count(runq->argv->words)){
    default:
        Xerror1("> requires singleton\n");
        return;
    case 0:
        Xerror1("> requires file\n");
        return;
    case 1:
        break;
    }
    file = runq->argv->words->word;
    if((f = Creat(file))<0){
        pfmt(err, "%s: ", file);
        Xerror("can't open");
        return;
    }
    pushredir(ROPEN, f, runq->code[runq->pc].i);
    runq->pc++;
    poplist();
}
```

Uses `Creat()` 155b, `ROPEN` 78b, `Xerror()` 69b, `Xerror1()` 69c, `count()` 30a, `err` 153b, `pfmt()` 163b, `poplist()` 64d, `pushredir()` 77d, and `runq` 28c.

`Xpopredir()`[79b] is the cleanup counterpart: it pops the top `Redir` record and, if it was an `ROPEN`, closes the file descriptor that was opened for the redirection. This is important because the opened fd (from `Creat()` or `Open()`) is a temporary—after `doredir()` has `dup()`'d it onto the target fd in the child process, the parent still holds the original, and it must be closed to avoid leaking file descriptors.

⟨*function* Xpopredir 79b⟩≡ (201a)

```
void
Xpopredir(void)
{
    struct Redir *rp = runq->redir;

    if(rp==nil)
        panic("turfredir null!", 0);

    // pop_list(runq->redir);
    runq->redir = rp->next;

    if(rp->type==ROPEN)
        close(rp->from);

    efree((char *)rp);
}
```

Uses `ROPEN` 78b, `Redir` 77a, `efree()`, `nil`, `panic()` 154b, and `runq` 28c.

Xread()

⟨`outcode()` *when REDIR case, switch redirection type cases* 79c⟩+≡ (78c) ◁78d 80b▷

```
case READ:
case HERE:
    emitf(Xread);
```

```
    break;
```

Uses HERE, READ, Xread() 80a, and emitf<> 62d.

⟨*function* Xread 80a⟩≡ (201b)

```
void
Xread(void)
{
    char *file;
    int f;
    switch(count(runq->argv->words)){
    default:
        Xerror1("< requires singleton\n");
        return;
    case 0:
        Xerror1("< requires file\n");
        return;
    case 1:
        break;
    }
    file = runq->argv->words->word;
    if((f = open(file, 0))<0){
        pfmt(err, "%s: ", file);
        Xerror("can't open");
        return;
    }
    pushredir(ROPEN, f, runq->code[runq->pc].i);
    runq->pc++;
    poplist();
}
```

Uses ROPEN 78b, Xerror() 69b, Xerror1() 69c, count() 30a, err 153b, pfmt() 163b, poplist() 64d, pushredir() 77d, and runq 28c.

Xappend()

⟨outcode() *when REDIR case, switch redirection type cases* 80b⟩+≡ (78c) ◁79c 133b▷

```
case APPEND:
    emitf(Xappend);
    break;
```

Uses APPEND 50a, Xappend() 80c, and emitf<> 62d.

⟨*function* Xappend 80c⟩≡ (201b)

```
void
Xappend(void)
{
    char *file;
    int f;
    switch(count(runq->argv->words)){
    default:
        Xerror1(">> requires singleton");
        return;
    case 0:
        Xerror1(">> requires file");
        return;
    case 1:
        break;
    }
    file = runq->argv->words->word;
    if((f = open(file, 1))<0 && (f = Creat(file))<0){
        pfmt(err, "%s: ", file);
```

```
        Xerror("can't open");
        return;
    }
    seek(f, 0L, SEEK__END);
    pushredir(ROPEN, f, runq->code[runq->pc].i);
    runq->pc++;
    poplist();
}
```

Uses `Creat()` 155b, `ROPEN` 78b, `Seek()` 167, `Xerror()` 69b, `Xerror1()` 69c, `count()` 30a, `err` 153b, `pfmt()` 163b, `poplist()` 64d, `pushredir()` 77d, and `runq` 28c.

Closing redirection opened files

⟨`Thread` *other fields* 81a⟩+≡ (28b) ◁77b 84a▷

```
struct Redir *startredir; /* redir inheritance point */
```

Uses `Redir` 77a.

⟨*function* `turfredir` 81b⟩≡ (201a)

```
void
turfredir(void)
{
    while(runq->redir != runq->startredir)
        Xpopredir();
}
```

Uses `Xpopredir()` 79b and `runq` 28c.

⟨`Xreturn()` *pop the redirections from this thread* 81c⟩≡ (82d)

```
turfredir();
```

Uses `turfredir()` 81b.

8.3.5 Pipe

Pipes are arguably the most important operator in a shell. For `cmd1 | cmd2`, four processes end up being involved: the parent `rc`, a child `rc` that interprets `cmd1`'s bytecodes, and then each side fork+exec's the actual command (e.g., `ls` and `wc`). The left side (`cmd1`) runs in a forked child `rc` process. The right side (`cmd2`) runs in the parent `rc`'s new thread. This asymmetry means that the right side can modify the shell's state (e.g., variable assignments), while the left side cannot. The compiled bytecodes contain two jump offsets: one for the right side's code, and one for the parent's `Xpipewait`[84d].

Here is the overall picture for `ls | wc -l`:

```
                pipe(pfd)
                    |
                 fork()
               /        \
        child rc          parent rc
        close(pfd[PRD])   close(pfd[PWR])
        redir: PWR->1     redir: PRD->0
             |                 |
       start thread      start thread
       for "ls"          for "wc -l"
             |                 |
        Xsimple:          Xsimple:
        fork+exec         fork+exec
```

```
        |                  |
+------+------+   +------+------+
| process "ls"|   | process "wc"|
|  stdout=pfd |   |  stdin=pfd  |
+------+------+   +------+------+
        |                  |
  wait for ls       wait for wc
        |                  |
   Xexit             Xreturn
 (child rc exits)   (thread done)
                           |
                  Xpipewait:
                  wait for child rc
                  concat statuses
```

The child runs `cmd1`'s bytecodes and ends with `Xexit` (it must terminate, not return to the parent's thread stack). The parent creates a new thread for `cmd2` that ends with `Xreturn`, then falls through to `Xpipewait` which waits for the child process and concatenates both exit statuses into a pipe status string like `"0|0"`.

⟨`outcode()` *locals* 82a⟩+≡ (64a) ◁74a 94c▷

```
int q;
```

⟨`outcode()` *cases* 82b⟩+≡ (64a) ◁78c 85a▷

```
case PIPE:
    emitf(Xpipe);
    emiti(t->fd0); // 1 in the normal case
    emiti(t->fd1); // 0 in the normal case
    p = emiti(0);
    q = emiti(0);

    // for first child
    outcode(c0, eflag);
    emitf(Xexit);

    // for second child
    stuffdot(p);
    outcode(c1, eflag);
    emitf(Xreturn);

    // for parent (rc)
    stuffdot(q);
    emitf(Xpipewait);
    break;
```

Uses `Xexit()` 85c, `Xpipe()` 137c, `Xpipewait()` 84d, `Xreturn()` 82d, `c0<>` 63c 148b, `c1<>` 63d 148c, `emitf<>` 62d, `emiti<>` 62c, `outcode()` 64a, and `stuffdot()` 74c.

⟨`codefree()` *in loop over code* `cp`, *switch bytecode cases* 82c⟩+≡ (27e) ◁68b 92c▷

```
else if(p->f==Xpipe)
        p+=4;
```

Uses `Xpipe()` 137c.

⟨*function* `Xreturn` 82d⟩≡ (201a)

```
void
Xreturn(void)
{
    struct Thread *p = runq;
```

```
    ⟨Xreturn() pop the redirections from this thread 81c⟩
    // free p
    while(p->argv)
        poplist();
    codefree(p->code);
    // pop(runq)
    runq = p->ret;
    efree((char *)p);

    if(runq==nil)
        Exit(getstatus(), __LOC__);
}
```

Uses Exit() 154c, Thread 28b, codefree() 27e, efree(), getstatus() 65e, nil, poplist() 64d, and runq 28c.

⟨*constant* PRD 83a⟩≡ (194c)

```
/*
 * Which fds are the reading/writing end of a pipe?
 * Unfortunately, this can vary from system to system.
 * 9th edition Unix doesn't care, the following defines
 * work on plan 9.
 */
#define PRD 0
```

⟨*constant* PWR 83b⟩≡ (194c)

```
#define PWR 1
```

⟨*function* Xpipe 83c⟩≡ (204a)

```
void
Xpipe(void)
{
    struct Thread *p = runq;
    int pc = p->pc;
    int forkid;
    fdt lfd = p->code[pc++].i;
    fdt rfd = p->code[pc++].i;
    fdt pfd[2];

    if(pipe(pfd)<0){
        Xerror("can't get pipe");
        return;
    }
    switch(forkid = fork()){
    case -1:
        Xerror("try again");
        break;
    case 0: // child
        clearwaitpids();
        // pc+2 so jump the jump addresses
        start(p->code, pc+2, runq->local);
        runq->ret = nil;
        close(pfd[PRD]);
        pushredir(ROPEN, pfd[PWR], lfd);
        break;
    default: // parent
        addwaitpid(forkid);
        start(p->code, p->code[pc].i, runq->local);
        close(pfd[PWR]);
        pushredir(ROPEN, pfd[PRD], rfd);
        p->pc = p->code[pc+1].i;
        p->pid = forkid;
```

```
        break;
    }
}
```

Uses ERRMAX, delwaitpid() 204a, havewaitpid() 71d, nil, runq 28c, and setstatus() 65d.

⟨Thread *other fields* 84a⟩+≡ (28b) ◁81a 84b▷

```
// option<int> (None = -1)
int pid;  /* process for Xpipewait to wait for */
```

⟨Thread *other fields* 84b⟩+≡ (28b) ◁84a 87e▷

```
char status[NSTATUS]; /* status for Xpipewait */
```

Uses NSTATUS 84c.

⟨*constant* NSTATUS 84c⟩≡ (197a)

```
#define NSTATUS ERRMAX   /* length of status (from plan 9) */
```

⟨*function* Xpipewait 84d⟩≡ (201b)

```
void
Xpipewait(void)
{
    char status[NSTATUS+1];
    if(runq->pid==-1)
        setstatus(concstatus(runq->status, getstatus()));
    else{
        strncpy(status, getstatus(), NSTATUS);
        status[NSTATUS]='\0';
        Waitfor(runq->pid, true);
        runq->pid=-1;
        setstatus(concstatus(getstatus(), status));
    }
}
```

Uses NSTATUS 84c, Waitfor() 85b, concstatus() 84f, getstatus() 65e, runq 28c, and setstatus() 65d.

⟨Waitfor() *in while loop, if wait returns another pid* 84e⟩≡ (72b)

```
// else
for(p = runq->ret;p;p = p->ret)
    if(p->pid==w->pid){
        p->pid=-1;
        strcpy(p->status, w->msg);
    }
free(w);
```

⟨*function* concstatus 84f⟩≡ (208b)

```
char*
concstatus(char *s, char *t)
{
    static char v[NSTATUS+1];
    int n = strlen(s);
    strncpy(v, s, NSTATUS);
    if(n<NSTATUS){
        v[n]='|';
        strncpy(v+n+1, t, NSTATUS-n-1);
    }
    v[NSTATUS]='\0';
    return v;
}
```

Uses NSTATUS 84c.

8.3.6 Asynchronous execution

The & operator runs a command in the background. The child's stdin is redirected to /dev/null (so it does not compete with the shell for terminal input), and the child is put in a new note group (RFNOTEG) so that interrupts sent to the shell do not kill it. The parent records the child's PID in $apid (for "asynchronous PID"), allowing the user to wait for it later or send signals to it.

⟨outcode() *cases* 85a⟩+≡ (64a) ◁82b 86a▷

```
  case '&':
      emitf(Xasync);
      p = emiti(0);
      outcode(c0, eflag);
      emitf(Xexit);
      stuffdot(p);
      break;
```

Uses Xasync(), Xexit() 85c, c0<> 63c 148b, emitf<> 62d, emiti<> 62c, outcode() 64a, and stuffdot() 74c.

⟨*function* Xasync 85b⟩≡ (204a)

```
  void
  Xasync(void)
  {
      fdt null = open("/dev/null", 0);
      int pid;
      char npid[10];
      if(null<0){
          Xerror("Can't open /dev/null\n");
          return;
      }
      switch(pid = rfork(RFFDG|RFPROC|RFNOTEG)){
      case -1:
          close(null);
          Xerror("try again");
          break;
      case 0: // child
          clearwaitpids();
          pushredir(ROPEN, null, 0);
          // start a new Thread runq->pc+1 so skip pointer to code after &
          start(runq->code, runq->pc+1, runq->local);
          runq->ret = 0;
          break;
      default: // parent
          addwaitpid(pid);
          close(null);
          // jump to code after &
          runq->pc = runq->code[runq->pc].i;
          inttoascii(npid, pid);
          setvar("apid", newword(npid, (word *)nil));
          break;
      }
  }
```

Uses efree(), err 153b, nil, and pfmt() 163b.

⟨*function* Xexit 85c⟩≡ (201b)

```
  void
  Xexit(void)
  {
      struct Var *trapreq;
      struct Word *starval;
      static bool beenhere = false;
```

```
    if(getpid()==mypid && !beenhere){
        trapreq = vlook("sigexit");
        if(trapreq->fn){
            beenhere = true;
            --runq->pc;
            starval = vlook("*")->val;
            start(trapreq->fn, trapreq->pc, (struct Var *)0);
            runq->local = newvar(strdup("*"), runq->local);
            runq->local->val = copywords(starval, (struct Word *)0);
            runq->local->changed = true;
            runq->redir = runq->startredir = nil;
            return;
        }
    }
    Exit(getstatus(), __LOC__);
}
```

Uses `Exit()` 154c, `Var` 50d, `Word` 194c, `copywords()` 30c, `false`, `getstatus()` 65e, `mypid` 116d, `newvar()` 33b, `nil`, `runq` 28c, `start()` 36e, `true`, and `vlook()` 32b.

8.4 Control flow statements

8.4.1 `if`

The `if` compilation uses the same forward-jump technique as `&&`: emit the condition, then `Xif`[86b] with a jump offset to skip the body if the condition is false. The `Xwastrue`[87a] bytecode at the end resets the `ifnot` flag. This flag is how `rc` connects `if` and `if not`: `Xif` sets `ifnot` to `true`, and `Xifnot`[87c] checks it to decide whether to execute the else-body. If any other command is executed between `if` and `if not`, `Xwastrue` resets the flag, making `if not` a no-op.

⟨`outcode()` *cases* 86a⟩+≡ (64a) ◁85a 87b▷

```
case IF:
    outcode(c0, false);
    emitf(Xif);
    p = emiti(0);
    outcode(c1, eflag);
    emitf(Xwastrue);
    stuffdot(p);
    break;
```

Uses `Xif()` 86b, `Xwastrue()` 87a, `c0<>` 63c 148b, `c1<>` 63d 148c, `emitf<>` 62d, `emiti<>` 62c, `false`, `outcode()` 64a, and `stuffdot()` 74c.

⟨*function* `Xif` 86b⟩≡ (201b)

```
void
Xif(void)
{
    ifnot = true;
    if(truestatus())
        runq->pc++;
    else
        runq->pc = runq->code[runq->pc].i;
}
```

Uses `ifnot` 86c, `runq` 28c, `true`, and `truestatus()` 66a.

⟨*global* `ifnot` 86c⟩≡ (201b)

```
bool ifnot;  /* dynamic if not flag */
```

⟨*function* Xwastrue 87a⟩≡ (201b)

```
void
Xwastrue(void)
{
    ifnot = false;
}
```

Uses false and ifnot 86c.

⟨outcode() *cases* 87b⟩+≡ (64a) ◁86a 88a▷

```
case NOT:
    ⟨outcode() when NOT, sanity check last command was an if 87d⟩
    emitf(Xifnot);
    p = emiti(0);
    outcode(c0, eflag);
    stuffdot(p);
    break;
```

Uses Xifnot() 87c, c0<> 63c 148b, emitf<> 62d, emiti<> 62c, outcode() 64a, and stuffdot() 74c.

⟨*function* Xifnot 87c⟩≡ (201b)

```
void
Xifnot(void)
{
    if(ifnot)
        runq->pc++;
    else
        runq->pc = runq->code[runq->pc].i;
}
```

Uses ifnot 86c and runq 28c.

⟨outcode() *when NOT, sanity check last command was an if* 87d⟩≡ (87b)

```
if(!runq->iflast)
    yyerror("‘if not’ does not follow ‘if(...)’");
```

Uses runq 28c and yyerror() 153e.

⟨Thread *other fields* 87e⟩+≡ (28b) ◁84b

```
bool iflast;  /* static ‘if not’ checking */
```

⟨outcode() *set iflast after switch* 87f⟩≡ (64a)

```
if(t->type!=NOT && t->type!=';')
    runq->iflast = t->type==IF;
else
   if(c0)
       runq->iflast = c0->type==IF;
```

Uses c0<> 63c 148b and runq 28c.

⟨outcode() *set iflast before switch* 87g⟩≡ (64a)

```
if(t->type!=NOT && t->type!=';')
    runq->iflast = false;
```

Uses false and runq 28c.

8.4.2 while

The while loop compiles to: the condition bytecodes, Xtrue[74e] (jump out if false), the body bytecodes, Xjump[88c] (jump back to the condition). If the condition is empty (e.g., while()), an Xsettrue[88b] bytecode is emitted to

make it loop forever.

⟨outcode() *cases* 88a⟩+≡ (64a) ◁87b 88d▷

```
case WHILE:
    q = codep;
    outcode(c0, false);
    if(q==codep)
        emitf(Xsettrue); /* empty condition == while(true) */
    emitf(Xtrue);
    p = emiti(0);
    outcode(c1, eflag);
    emitf(Xjump);
    emiti(q);
    stuffdot(p);
    break;
```

Uses Xjump() 88c, Xsettrue() 88b, Xtrue() 74e, c0<> 63c 148b, c1<> 63d 148c, codep 62a, emitf<> 62d, emiti<> 62c, false, outcode() 64a, and stuffdot() 74c.

⟨*function* Xsettrue 88b⟩≡ (201b)

```
void
Xsettrue(void)
{
    setstatus("");
}
```

Uses setstatus() 65d.

⟨*function* Xjump 88c⟩≡ (201b)

```
void
Xjump(void)
{
    runq->pc = runq->code[runq->pc].i;
}
```

Uses runq 28c.

8.4.3 for

The for loop is more complex. It first evaluates the list to iterate over (or expands \$* if no list is given), creates a local variable for the loop variable (via Xlocal[95b]), and then loops with Xfor[89a]. Xfor()[89a] pops the next word from the list, assigns it to the local variable, and executes the body. When the list is exhausted, it jumps past the body and Xunlocal[95c] removes the local binding.

⟨outcode() *cases* 88d⟩+≡ (64a) ◁88a 89b▷

```
case FOR:
    emitf(Xmark);
    if(c1){
        outcode(c1, eflag);
        emitf(Xglob);
    }
    else{
        emitf(Xmark);
        emitf(Xword);
        emits(strdup("*"));
        emitf(Xdol);
    }
    emitf(Xmark);  /* dummy value for Xlocal */
    emitf(Xmark);
    outcode(c0, eflag);
    emitf(Xlocal);
```

```
    p = emitf(Xfor);
    q = emiti(0);
    outcode(c2, eflag);
    emitf(Xjump);
    emiti(p);
    stuffdot(q);
    emitf(Xunlocal);
    break;
```

Uses Xdol() 96b, Xfor() 89a, Xglob() 119a, Xjump() 88c, Xlocal() 95b, Xmark() 64c, Xunlocal() 95c, Xword() 64b, c0<> 63c 148b, c1<> 63d 148c, c2<> 63e 148d, emitf<> 62d, emiti<> 62c, emits<> 62e, outcode() 64a, and stuffdot() 74c.

⟨*function* Xfor 89a⟩≡ (201b)

```
void
Xfor(void)
{
    if(runq->argv->words==0){
        poplist();
        runq->pc = runq->code[runq->pc].i;
    }
    else{
        freelist(runq->local->val);
        runq->local->val = runq->argv->words;
        runq->local->changed = true;
        runq->argv->words = runq->argv->words->next;
        runq->local->val->next = 0;
        runq->pc++;
    }
}
```

Uses freelist() 31a, poplist() 64d, runq 28c, and true.

8.4.4 switch

⟨outcode() *cases* 89b⟩+≡ (64a) ◁88d 91d▷

```
case SWITCH:
    codeswitch(t, eflag);
    break;
```

Uses codeswitch() 89c.

⟨*function* codeswitch 89c⟩≡ (207a)

```
/*
 * switch code looks like this:
 * Xmark
 * (get switch value)
 * Xjump 1f
 * out: Xjump leave
 * 1: Xmark
 * (get case values)
 * Xcase 1f
 * (commands)
 * Xjump out
 * 1: Xmark
 * (get case values)
 * Xcase 1f
 * (commands)
 * Xjump out
 * 1:
 * leave:
 * Xpopm
```

```
 */
void
codeswitch(tree *t, bool eflag)
{
    int leave;  /* patch jump address to leave switch */
    int out;  /* jump here to leave switch */
    int nextcase; /* patch jump address to next case */
    tree *tt;

    // c1 is BRACE { ; ; ; }
    if(c1->child[0]==nil
    || c1->child[0]->type != ';'
    || !iscase(c1->child[0]->child[0])){
        yyerror("case missing in switch");
        return;
    }

    emitf(Xmark);
    outcode(c0, eflag);
    emitf(Xjump);

    nextcase = emiti(0);
    out = emitf(Xjump);
    leave = emiti(0);

    stuffdot(nextcase);

    // from now on c0, c1, ... refer to this new t
    t = c1->child[0];
    while(t->type==';'){
        tt = c1;
        emitf(Xmark);
        for(t = c0->child[0];t->type==ARGLIST;t = c0)
            outcode(c1, eflag);
        emitf(Xcase);
        nextcase = emiti(0);
        t = tt;
        for(;;){
            if(t->type==';'){
                if(iscase(c0))
                    break;
                outcode(c0, eflag);
                t = c1;
            }
            else{
                if(!iscase(t))
                    outcode(t, eflag);
                break;
            }
        }
        emitf(Xjump);
        emiti(out);
        stuffdot(nextcase);
    }
    stuffdot(leave);
    emitf(Xpopm);
}
```

Uses `Xcase()` 91c, `Xjump()` 88c, `Xmark()` 64c, `Xpopm()` 91a, `c0<>` 63c 148b, `c1<>` 63d 148c, `emitf<>` 62d, `emiti<>` 62c, `iscase()` 91b, `nil`, `outcode()` 64a, `stuffdot()` 74c, and `yyerror()` 153e.

⟨*function* Xpopm 91a⟩≡ (201b)

```
void
Xpopm(void)
{
    poplist();
}
```

Uses poplist() 64d.

⟨*function* iscase 91b⟩≡ (207a)

```
bool
iscase(tree *t)
{
    if(t->type!=SIMPLE)
        return false;
    do { t = c0; } while(t->type==ARGLIST);
    return t->type==WORD && !t->quoted && strcmp(t->str, "case")==0;
}
```

Uses c0<> 63c 148b and false.

⟨*function* Xcase 91c⟩≡ (201b)

```
void
Xcase(void)
{
    word *p;
    char *s;
    bool ok = false;

    s = list2str(runq->argv->next->words);
    for(p = runq->argv->words;p;p = p->next){
        if(match(s, p->word, '\0')){
            ok = true;
            break;
        }
    }
    efree(s);
    if(ok)
        runq->pc++;
    else
        runq->pc = runq->code[runq->pc].i;
    poplist();
}
```

Uses efree(), false, list2str() 76a, match() 122b, poplist() 64d, runq 28c, and true.

8.4.5 Blocks: '{...}'

⟨outcode() *cases* 91d⟩+≡ (64a) ◁89b 91e▷

```
case PAREN:
    outcode(c0, eflag);
    break;
```

Uses c0<> 63c 148b and outcode() 64a.

⟨outcode() *cases* 91e⟩+≡ (64a) ◁91d 92b▷

```
case PCMD:
case BRACE:
    outcode(c0, eflag);
    break;
```

Uses c0<> 63c 148b and outcode() 64a.

8.5 Functions

In rc, functions are stored in variables: the Var structure has an fn field pointing to the function's bytecode vector and a pc field indicating where in that vector the function body starts. This design unifies functions and variables in the same namespace and allows functions to be exported to the environment (see Chapter 10).

⟨Var *other fields* 92a⟩≡ (31c) 108d ▷

```
code *fn;  /* pointer to function's code vector */
int pc;  /* pc of start of function */
bool fnchanged;
```

8.5.1 Function definitions (fn <foo> ...)

When defining a function with fn foo {body}, Xfn[93a] stores the current code vector (via codecopy[27d]) in the variable named foo. This is why bytecode vectors are reference-counted: the function's code vector is shared with the code that defined it. When fn foo is used without a body, Xdelfn[93b] deletes the function by setting v->fn to nil.

⟨outcode() *cases* 92b⟩+≡ (64a) ◁91e 94d ▷

```
case FN:
    emitf(Xmark);
    outcode(c0, eflag);
    if(c1){
        emitf(Xfn);
        p = emiti(0);
        emits(fnstr(c1));
        outcode(c1, eflag); // body of the function
        emitf(Xunlocal); /* get rid of $* */ //$
        emitf(Xreturn);
        stuffdot(p);
    }
    else
        emitf(Xdelfn);
    break;
```

Uses Xdelfn() 93b, Xfn() 93a, Xmark() 64c, Xreturn() 82d, Xunlocal() 95c, c0<> 63c 148b, c1<> 63d 148c, emitf<> 62d, emiti<> 62c, emits<> 62e, fnstr() 92d, outcode() 64a, and stuffdot() 74c.

⟨codefree() *in loop over code* cp*, switch bytecode cases* 92c⟩+≡ (27e) ◁82c 135d ▷

```
else if(p->f==Xfn){
        efree(p[2].s);
        p+=2;
    }
```

Uses Xfn() 93a and efree().

⟨*function* fnstr 92d⟩≡ (207a)

```
char*
fnstr(tree *t)
{
    void *v;
    extern char nl;
    char svnl = nl;
    io *f = openstr();

    nl = ';';
    pfmt(f, "%t", t);
    nl = svnl;
    v = f->strp;
    f->strp = nil;
```

```
    closeio(f);
    return v;
}
```

Uses closeio() 162b, nil, nl 148a, openstr() 162c, and pfmt() 163b.

⟨*function* Xfn 93a⟩≡ (201b)

```
void
Xfn(void)
{
    var *v;
    word *a;
    int end;

    end = runq->code[runq->pc].i;
    globlist();
    for(a = runq->argv->words;a;a = a->next){
        v = gvlook(a->word);
        if(v->fn)
            codefree(v->fn);
        v->fn = codecopy(runq->code);
        v->pc = runq->pc+2;
        v->fnchanged = true;
    }
    runq->pc = end;
    poplist();
}
```

Uses codecopy() 27d, codefree() 27e, globlist(), gvlook() 32e, poplist() 64d, runq 28c, and true.

⟨*function* Xdelfn 93b⟩≡ (201b)

```
void
Xdelfn(void)
{
    var *v;
    word *a;

    for(a = runq->argv->words;a;a = a->next){
        v = gvlook(a->word);
        if(v->fn)
            codefree(v->fn);
        v->fn = nil;
        v->fnchanged = true;
    }
    poplist();
}
```

Uses codefree() 27e, gvlook() 32e, nil, poplist() 64d, runq 28c, and true.

8.5.2 Function uses (<foo>(...))

When Xsimple()[68c] detects that the command name matches a function, it calls execfunc()[94b]. This function pops the command arguments from argv, starts a new thread with the function's bytecodes, and binds the arguments to a local $* variable. This is how function arguments work in rc: inside a function, $* contains the arguments passed to the function, shadowing the script's global $*.

⟨Xsimple() *other locals* 93c⟩≡ (68c) 100c ▷

```
var *v;
```

⟨Xsimple() *if argv0 is a function* 94a⟩≡ (68c)

```
v = gvlook(a->word);
if(v->fn)
    execfunc(v);
```

Uses execfunc() 94b and gvlook() 32e.

⟨*function* execfunc 94b⟩≡ (206a)

```
void
execfunc(var *func)
{
    word *starval;

    popword();
    starval = runq->argv->words;
    runq->argv->words = nil;
    poplist();
    start(func->fn, func->pc, runq->local);
    runq->local = newvar(strdup("*"), runq->local);
    runq->local->val = starval;
    runq->local->changed = true;
}
```

Uses newvar() 33b, nil, poplist() 64d, popword() 65a, runq 28c, start() 36e, and true.

8.6 Variables

8.6.1 Variable definitions (<x>=...)

Variable assignment has two modes: when followed by a command (e.g., x=1 cmd), the assignment is local to that command (using Xlocal[95b]/Xunlocal[95c]); when standalone (e.g., x=1), it is global (using Xassign[95a]). The code detects which mode by following the chain of '=' nodes through child[2]: if it eventually reaches a non-assignment node, there is a command and the assignments are local. Multiple chained assignments like A=b C=d cmd are supported: each creates a local binding, and they are all unlocalized after the command finishes.

⟨outcode() *locals* 94c⟩+≡ (64a) ◁82a

```
tree *tt;
```

⟨outcode() *cases* 94d⟩+≡ (64a) ◁92b 96a▷

```
case '=':
    tt = t;
    for(;t && t->type=='='; t = c2);

    if(t){     /* var=value cmd */
        for(t = tt;t->type=='=';t = c2){
            emitf(Xmark);
            outcode(c1, eflag);
            emitf(Xmark);
            outcode(c0, eflag);
            emitf(Xlocal);  /* push var for cmd */
        }
        outcode(t, eflag);  /* gen. code for cmd */
        for(t = tt; t->type == '='; t = c2)
            emitf(Xunlocal); /* pop var */
    }
    else{     /* var=value */
        for(t = tt;t;t = c2){
            emitf(Xmark);
            outcode(c1, eflag);
```

```
            emitf(Xmark);
            outcode(c0, eflag);
            emitf(Xassign); /* set var permanently */
        }
    }
    t = tt; /* so tests below will work */
    break;
```

Uses Xassign() 95a, Xlocal() 95b, Xmark() 64c, Xunlocal() 95c, c0<> 63c 148b, c1<> 63d 148c, c2<> 63e 148d, emitf<> 62d, and outcode() 64a.

⟨*function* Xassign 95a⟩≡ (201b)

```
void
Xassign(void)
{
    var *v;
    if(count(runq->argv->words)!=1){
        Xerror1("variable name not singleton!");
        return;
    }
    deglob(runq->argv->words->word); // remove the special \001 mark
    v = vlook(runq->argv->words->word);
    poplist();

    globlist();
    freewords(v->val);
    v->val = runq->argv->words;
    v->changed = true;
    runq->argv->words = nil;
    poplist();
}
```

Uses Xerror1() 69c, count() 30a, deglob() 120b, freewords() 30d, globlist(), nil, poplist() 64d, runq 28c, true, and vlook() 32b.

⟨*function* Xlocal 95b⟩≡ (201b)

```
void
Xlocal(void)
{
    if(count(runq->argv->words)!=1){
        Xerror1("variable name must be singleton\n");
        return;
    }
    deglob(runq->argv->words->word);
    runq->local = newvar(strdup(runq->argv->words->word), runq->local);
    poplist();
    globlist();
    runq->local->val = runq->argv->words;
    runq->local->changed = true;
    // change ownership
    runq->argv->words = nil;
    poplist();
}
```

Uses Xerror1() 69c, count() 30a, deglob() 120b, globlist(), newvar() 33b, nil, poplist() 64d, runq 28c, and true.

⟨*function* Xunlocal 95c⟩≡ (201b)

```
void
Xunlocal(void)
{
    var *v = runq->local, *hid;
    if(v==0)
```

```
        panic("Xunlocal: no locals!", 0);
    runq->local = v->next;
    hid = vlook(v->name);
    hid->changed = true;
    efree(v->name);
    freewords(v->val);
    efree((char *)v);
}
```

Uses `efree()`, `freewords()` 30d, `panic()` 154b, `runq` 28c, `true`, and `vlook()` 32b.

8.6.2 Variable uses ($<x>)

`Xdol()`[96b] (dollar) handles variable expansion. It looks up the variable name and pushes its value (a list of words) onto the argument stack. A subtle feature: if the variable name is a number (e.g., `$1`, `$2`), `Xdol()` treats it as a positional parameter, indexing into `$*` instead of looking up a named variable.

⟨`outcode()` *cases* 96a⟩+≡ (64a) ◁94d 97a▷

```
case '$': //$
    emitf(Xmark);
    outcode(c0, eflag);
    emitf(Xdol);
    break;
```

Uses `Xdol()` 96b, `Xmark()` 64c, `c0<>` 63c 148b, `emitf<>` 62d, and `outcode()` 64a.

⟨*function* `Xdol` 96b⟩≡ (201b)

```
void
Xdol(void)
{
    word *a, *star;
    char *s, *t;
    int n;

    if(count(runq->argv->words)!=1){
        Xerror1("variable name not singleton!");
        return;
    }
    s = runq->argv->words->word;
    deglob(s);

    n = 0;
    for(t = s;'0'<=*t && *t<='9';t++)
        n = n*10+*t-'0';

    a = runq->argv->next->words;
    if(n==0 || *t)
        a = copywords(vlook(s)->val, a);
    else{
        star = vlook("*")->val;
        if(star && 1 <= n && n <= count(star)){
            while(--n)
                star = star->next;
            a = newword(star->word, a);
        }
    }
    poplist();
    runq->argv->words = a;
}
```

Uses `Xerror1()` 69c, `copywords()` 30c, `count()` 30a, `deglob()` 120b, `newword()` 29d, `poplist()` 64d, `runq` 28c, and `vlook()` 32b.

⟨outcode() *cases* 97a⟩+≡ (64a) ◁96a 97c▷

```
case COUNT:
    emitf(Xmark);
    outcode(c0, eflag);
    emitf(Xcount);
    break;
```

Uses Xcount() 97b, Xmark() 64c, c0<> 63c 148b, emitf<> 62d, and outcode() 64a.

⟨*function* Xcount 97b⟩≡ (201b)

```
void
Xcount(void)
{
    word *a;
    char *s, *t;
    int n;
    char num[12];

    if(count(runq->argv->words)!=1){
        Xerror1("variable name not singleton!");
        return;
    }
    s = runq->argv->words->word;
    deglob(s);

    // n = int_of_string(s)
    n = 0;
    for(t = s;'0'<=*t && *t<='9';t++)
        n = n*10 + *t - '0';
    if(n==0 || *t){
        a = vlook(s)->val;
        inttoascii(num, count(a));
    }
    else{
        a = vlook("*")->val;
        inttoascii(num, (a && 1<=n && n<=count(a)) ? 1 : 0);
    }
    poplist();
    pushword(num);
}
```

Uses Xerror1() 69c, count() 30a, deglob() 120b, inttoascii(), poplist() 64d, pushword() 38b, runq 28c, and vlook() 32b.

Subscripting ($x(2) or $x(2-5)) extracts elements from a list variable by index or range. subwords()[98a] does the heavy lifting: it parses each subscript (which can be a single number or a n-m range, where a bare n- means "from n to the end"), walks n-1 links into the variable's value list, and calls copynwords()[30b] to extract the slice. The recursion on sub->next handles multiple subscripts like $x(1 3 5), accumulating results in reverse so the final list comes out in the right order.

⟨outcode() *cases* 97c⟩+≡ (64a) ◁97a 98b▷

```
case SUB:
    emitf(Xmark);
    outcode(c0, eflag);
    emitf(Xmark);
    outcode(c1, eflag);
    emitf(Xsub);
    break;
```

Uses Xmark() 64c, Xsub() 97d, c0<> 63c 148b, c1<> 63d 148c, emitf<> 62d, and outcode() 64a.

⟨*function* Xsub 97d⟩≡ (201b)

```
void
```

```
Xsub(void)
{
    word *a, *v;
    char *s;
    if(count(runq->argv->next->words)!=1){
        Xerror1("variable name not singleton!");
        return;
    }
    s = runq->argv->next->words->word;
    deglob(s);
    a = runq->argv->next->next->words;
    v = vlook(s)->val;
    a = subwords(v, count(v), runq->argv->words, a);
    poplist();
    poplist();
    runq->argv->words = a;
}
```

Uses `Xerror1()` 69c, `count()` 30a, `deglob()` 120b, `poplist()` 64d, `runq` 28c, `subwords()` 98a, and `vlook()` 32b.

⟨*function* `subwords` 98a⟩≡ (201b)

```
word*
subwords(word *val, int len, word *sub, word *a)
{
    int n, m;
    char *s;
    if(!sub)
        return a;
    a = subwords(val, len, sub->next, a);
    s = sub->word;
    deglob(s);
    m = 0;
    n = 0;
    while('0'<=*s && *s<='9')
        n = n*10+ *s++ -'0';
    if(*s == '-'){
        if(*++s == 0)
            m = len - n;
        else{
            while('0'<=*s && *s<='9')
                m = m*10+ *s++ -'0';
            m -= n;
        }
    }
    if(n<1 || n>len || m<0)
        return a;
    if(n+m>len)
        m = len-n;
    while(--n > 0)
        val = val->next;
    return copynwords(val, a, m+1);
}
```

Uses `copynwords()` 30b, `deglob()` 120b, and `subwords()` 98a.

⟨`outcode()` *cases* 98b⟩+≡ (64a) ◁97c 128c▷

```
case '^':
    emitf(Xmark);
    outcode(c1, eflag);
    emitf(Xmark);
    outcode(c0, eflag);
    emitf(Xconc);
```

```
    break;
```

Uses Xconc() 99a, Xmark() 64c, c0<> 63c 148b, c1<> 63d 148c, emitf<> 62d, and outcode() 64a.

⟨*function* Xconc 99a⟩≡ (201b)

```
void
Xconc(void)
{
    word *lp = runq->argv->words;
    word *rp = runq->argv->next->words;
    word *vp = runq->argv->next->next->words;
    int lc = count(lp), rc = count(rp);
    if(lc!=0 || rc!=0){
        if(lc==0 || rc==0){
            Xerror1("null list in concatenation");
            return;
        }
        if(lc!=1 && rc!=1 && lc!=rc){
            Xerror1("mismatched list lengths in concatenation");
            return;
        }
        vp = conclist(lp, rp, vp);
    }
    poplist();
    poplist();
    runq->argv->words = vp;
}
```

Uses Xerror1() 69c, conclist() 99b, count() 30a, poplist() 64d, and runq 28c.

⟨*function* conclist 99b⟩≡ (201b)

```
word*
conclist(word *lp, word *rp, word *tail)
{
    char *buf;
    word *v;
    if(lp->next || rp->next)
        tail = conclist(lp->next==0? lp: lp->next,
            rp->next==0? rp: rp->next, tail);
    buf = emalloc(strlen(lp->word)+strlen((char *)rp->word)+1);
    strcpy(buf, lp->word);
    strcat(buf, rp->word);
    v = newword(buf, tail);
    efree(buf);
    return v;
}
```

Uses conclist() 99b, efree(), emalloc(), and newword() 29d.

8.6.3 Special variables

Chapter 9

Builtins

Shell builtins are commands that must be executed inside the shell process itself, rather than in a forked child. The most obvious example is `cd`: if it were an external program, the `chdir()` would happen in the child process and the parent (the shell) would remain in the old directory.

9.1 Overview

Builtins are stored in a simple array of name/function pairs. When `Xsimple()`[68c] detects that the command name matches a builtin, it calls the corresponding function directly. The `builtin` keyword allows bypassing a function that shadows a builtin: `builtin cd` always calls the builtin `cd`, even if a function named `cd` exists.

⟨*struct* `Builtin` 100a⟩≡ (197a)

```
struct Builtin {
    char *name;
    void (*fnc)(void);
};
```

⟨*global* `builtins` 100b⟩≡ (208c)

```
builtin builtins[] = {
    "cd"     ,      execcd,
    "exit"   ,      execexit,
    "."      ,      execdot,
    "eval"   ,      execeval,

    "whatis" ,      execwhatis,

    "exec"   ,      execexec,   /* but with popword first */
    "rfork"  ,      execnewpgrp,
    "wait"   ,      execwait,

    "shift"  ,      execshift,
    "finit"  ,      execfinit,
    "flag"   ,      execflag,
    //TODO: unix-specific "umask", execumask,
    0
};
```

Uses `execdot()` 103d, `execeval()` 105e, `execexec()` 70b, `execfinit()` 144c, `execflag()` 143, `execnewpgrp()` 142, `execshift()` 144a, `execwait()` 106c, and `execwhatis()` 140.

⟨`Xsimple()` *other locals* 100c⟩+≡ (68c) ◁93c

```
struct Builtin *bp;
```

Uses `Builtin` 100a.

⟨Xsimple() *if argv0 is a builtin* 101a⟩≡ (68c)

```
⟨Xsimple() if argv0 is the builtin keyword 101b⟩
for(bp = builtins;bp->name;bp++)
    if(strcmp(a->word, bp->name)==0){
        (*bp->fnc)();
        return;
    }
```

Uses `builtins` 208c.

⟨Xsimple() *if argv0 is the builtin keyword* 101b⟩≡ (101a)

```
if(strcmp(a->word, "builtin")==0){
    if(count(a)==1){
        pfmt(err, "builtin: empty argument list\n");
        setstatus("empty arg list");
        poplist();
        return;
    }
    a = a->next;
    popword();
}
```

Uses `count()` 30a, `err` 153b, `pfmt()` 163b, `poplist()` 64d, `popword()` 65a, and `setstatus()` 65d.

9.2 $ cd

`cd` with no argument changes to `$home`. With one argument, it searches the `$cdpath` directories (similar to how `$path` works for commands). When the directory is found via `$cdpath`, `cd` prints the resolved path so the user knows where they ended up. Under `rio`, `cd` also writes the new directory to `/dev/wdir` so the window manager can track it.

⟨*function* `execcd` 101c⟩≡ (208c)

```
void
execcd(void)
{
    word *a = runq->argv->words;
    word *cdpath;
    char *dir;

    setstatus("can't cd");
    cdpath = vlook("cdpath")->val;

    switch(count(a)){
    case 1:
        a = vlook("home")->val;
        if(count(a)>=1){
            if(dochdir(a->word)>=0)
                setstatus("");
            else
                pfmt(err, "Can't cd %s: %r\n", a->word);
        }
        else
            pfmt(err, "Can't cd -- $home empty\n"); //$
        break;
    case 2:
        if(a->next->word[0]=='/' || cdpath==nil)
            cdpath = &nullpath;
        for(; cdpath; cdpath = cdpath->next){
            if(cdpath->word[0] != '\0')
```

```
                dir = appfile(cdpath->word, a->next->word);
            else
                dir = strdup(a->next->word);

            if(dochdir(dir) >= 0){
                if(cdpath->word[0] != '\0' &&
                    strcmp(cdpath->word, ".") != 0)
                    pfmt(err, "%s\n", dir);
                free(dir);
                setstatus("");
                break;
            }
            free(dir);
        }
        if(cdpath==nil)
            pfmt(err, "Can't cd %s: %r\n", a->next->word);
        break;
    default:
        pfmt(err, "Usage: cd [directory]\n");
        break;
    }
    poplist();
}
```

Uses `appfile<>()` 102a, `count()` 30a, `dochdir()` 102b, `err` 153b, `nil`, `nullpath` 71c, `pfmt()` 163b, `poplist()` 64d, `runq` 28c, `setstatus()` 65d, and `vlook()` 32b.

⟨*function* `appfile` 102a⟩≡ (208c)

```
static char *
appfile(char *dir, char *comp)
{
    int dirlen, complen;
    char *s, *p;

    dirlen = strlen(dir);
    complen = strlen(comp);
    s = emalloc(dirlen + 1 + complen + 1);
    memmove(s, dir, dirlen);
    p = s + dirlen;
    *p++ = '/';
    memmove(p, comp, complen);
    p[complen] = '\0';
    return s;
}
```

Uses `emalloc()`.

⟨*function* `dochdir` 102b⟩≡ (208c)

```
errorneg1
dochdir(char *word)
{
    ⟨dochdir() locals 103a⟩

    // the actual syscall
    if(chdir(word)<0)
        return ERROR_NEG1;

    ⟨dochdir() adjust /dev/wdir if run under rio 103b⟩
    return OK_1;
}
```

Uses `ERROR_NEG1` and `OK_1`.

In Plan 9, the window manager `rio` (see WINDOWS book [Pad16c]) tracks the current working directory of each window via the `/dev/wdir` file. When `cd` succeeds in an interactive shell, it writes the new directory to this file so that `rio` can display it in the window title or use it when spawning new windows. The file descriptor is cached in a static variable and opened only once (the `-2` sentinel means "not yet tried").

⟨`dochdir()` *locals* 103a⟩≡ (102b)
```
/* report to /dev/wdir if it exists and we're interactive */
static fdt wdirfd = -2;
```

⟨`dochdir()` *adjust* `/dev/wdir` *if run under* `rio` 103b⟩≡ (102b)
```
if(flag['i']!=nil){
    if(wdirfd==-2)  /* try only once */
        wdirfd = open("/dev/wdir", OWRITE); // TODO: |OCEXEC but plan9 specific?
    if(wdirfd>=0) {
        //fcntl(wdirfd, F_SETFD, FD_CLOEXEC);
        write(wdirfd, word, strlen(word));
    }
}
```
Uses `OCEXEC`, `OWRITE`, `flag` 34c, and `nil`.

9.3 $ exit

⟨*function* `execexit` 103c⟩≡ (208c)
```
void
execexit(void)
{
    switch(count(runq->argv->words)){
    default:
        pfmt(err, "Usage: exit [status]\nExiting anyway\n");
        // FALLTHROUGH
    case 2:
        setstatus(runq->argv->words->next->word);
        // FALLTHROUGH
    case 1:
        Xexit();
    }
}
```
Uses `Xexit()` 85c, `count()` 30a, `err` 153b, `pfmt()` 163b, `runq` 28c, and `setstatus()` 65d.

9.4 $.

The `.` (dot) builtin sources a script: it reads and executes the commands from a file in the current shell (without forking a new process). This is how `rc` loads initialization files like `rcmain` and the user's profile. It creates a new thread whose `cmdfd` reads from the sourced file, so `Xrdcmds()`[39a] will read commands from that file instead of the terminal.

⟨*function* `execdot` 103d⟩≡ (208c)
```
void
execdot(void)
{
    thread *p = runq;
    bool iflag = false;
    fdt fd;
    list *av;
    char *zero; // new stdin
```

```
char *file;
word *path;
⟨execdot() other locals 105c⟩

⟨execdot() if first 105d⟩
⟨execdot() if not first execution 126d⟩

popword(); // "."
if(p->argv->words && strcmp(p->argv->words->word, "-i")==0){
    iflag = true;
    popword();
}

/* get input file */
if(p->argv->words==nil){
    Xerror1("Usage: . [-i] file [arg ...]");
    return;
}
zero = strdup(p->argv->words->word);
popword();

fd = -1;
for(path = searchpath(zero); path; path = path->next){
    if(path->word[0] != '\0')
        file = appfile(path->word, zero);
    else
        file = strdup(zero);

    fd = open(file, 0);
    free(file);
    if(fd >= 0)
        break;
}
if(fd<0){
    pfmt(err, "%s: ", zero);
    setstatus("can't open"); // what for? it is reseted by Xerror anyway
    Xerror(".: can't open");
    return;
}

/* set up for a new command loop */
start(dotcmds, 1, (struct Var *)nil);

pushredir(RCLOSE, fd, 0);
runq->cmdfile = zero;
runq->cmdfd = openfd(fd);
runq->iflag = iflag;

runq->iflast = false;

/* push $* value */
pushlist();
runq->argv->words = p->argv->words;

/* free caller's copy of $* */
av = p->argv;
p->argv = av->next;
efree((char *)av);

/* push $0 value */
```

```
    pushlist();
    pushword(zero);

    ndot++;
}
```

Uses RCLOSE 135f, Var 50d, Xerror() 69b, Xerror1() 69c, appfile<>() 102a, dotcmds 105b, efree(), err 153b, false, ndot 105a, nil, openfd() 160f, pfmt() 163b, popword() 65a, pushlist() 38a, pushredir() 77d, pushword() 38b, runq 28c, searchpath() 71b, setstatus() 65d, start() 36e, and true.

⟨*global* ndot 105a⟩≡ (197b)

```
/*
 * How many dot commands have we executed?
 * Used to ensure that -v flag doesn't print rcmain.
 */
int ndot;
```

⟨*global* dotcmds 105b⟩≡ (208c)

```
union Code dotcmds[14];
```

Uses Code 25a.

⟨execdot() *other locals* 105c⟩≡ (103d)

```
static bool first = true;
```

Uses true.

⟨execdot() *if first* 105d⟩≡ (103d)

```
if(first){
    dotcmds[0].i = 1;
    dotcmds[1].f = Xmark;
      dotcmds[2].f = Xword;
      dotcmds[3].s = "0";
    dotcmds[4].f = Xlocal; // will pop_list twice

    dotcmds[5].f = Xmark;
      dotcmds[6].f = Xword;
      dotcmds[7].s="*";
    dotcmds[8].f = Xlocal; // will pop_list twice

    dotcmds[9].f = Xrdcmds; // =~ a REPL

    dotcmds[10].f = Xunlocal;
    dotcmds[11].f = Xunlocal;
    dotcmds[12].f = Xreturn;

    first = false;
}
```

Uses Xlocal() 95b, Xmark() 64c, Xrdcmds() 39a, Xreturn() 82d, Xunlocal() 95c, Xword() 64b, dotcmds 105b, and false.

9.5 $ eval

eval takes a string and executes it as a command. This is useful when the command to execute is constructed dynamically (e.g., from a variable). It joins the arguments into a single string, then creates a new thread reading from that string (via opencore()[162d], which creates a buffered IO from memory).

⟨*function* execeval 105e⟩≡ (208c)

```
void
execeval(void)
{
```

```
    char *cmdline, *s, *t;
    int len = 0;
    word *ap;

    if(count(runq->argv->words)<=1){
        Xerror1("Usage: eval cmd ...");
        return;
    }
    eflagok = true;
    for(ap = runq->argv->words->next;ap;ap = ap->next)
        len+=1+strlen(ap->word);

    cmdline = emalloc(len);
    s = cmdline;
    for(ap = runq->argv->words->next;ap;ap = ap->next){
        for(t = ap->word;*t;) *s++=*t++;
        *s++=' ';
    }
    s[-1]='\n';
    poplist();

    execcmds(opencore(cmdline, len));
    efree(cmdline);
}
```

Uses `Xerror1()` 69c, `count()` 30a, `eflagok` 126c, `efree()`, `emalloc()`, `execcmds()` 106b, `opencore()` 162d, `poplist()` 64d, `runq` 28c, and `true`.

⟨*global* `rdcmds` 106a⟩≡ (206a)

```
union Code rdcmds[4];
```

Uses `Code` 25a.

⟨*function* `execcmds` 106b⟩≡ (206a)

```
void
execcmds(io *f)
{
    static bool first = true;
    if(first){
        rdcmds[0].i = 1;
        rdcmds[1].f = Xrdcmds;
        rdcmds[2].f = Xreturn;
        first = false;
    }

    start(rdcmds, 1, runq->local);
    runq->cmdfd = f;
    runq->iflast = false;
}
```

Uses `Xrdcmds()` 39a, `Xreturn()` 82d, `false`, `rdcmds` 106a, `runq` 28c, `start()` 36e, and `true`.

9.6 $ wait

⟨*function* `execwait` 106c⟩≡ (208c)

```
void
execwait(void)
{
    switch(count(runq->argv->words)){
    default:
```

```
        Xerror1("Usage: wait [pid]");
        return;
    case 2:
        Waitfor(atoi(runq->argv->words->next->word), false);
        break;
    case 1:
        Waitfor(-1, false);
        break;
    }
    poplist();
}
```

Uses `Waitfor()` 85b, `Xerror1()` 69c, `count()` 30a, `poplist()` 64d, and `runq` 28c.

Chapter 10

Environment

Under Plan 9, the environment is not passed via the `envp` parameter of `main()` as in UNIX. Instead, each environment variable is stored as a file under `/env/`: the variable `$path` is stored in `/env/path`. This is a more elegant design because child processes automatically inherit the environment through the namespace, and changes are visible to all processes sharing the same namespace. `rc` must synchronize its internal variable table with these files in both directions: `Updenv()`[108a] writes changed variables to `/env/`, and `Vinit()`[109b] reads variables from `/env/` at startup.

10.1 Updenv()

`Updenv()`[108a] iterates over all global and local variables and writes any that have changed (with `v->changed == true`) to the `/env/` filesystem. The `changed` flag avoids redundant writes.

⟨*function* `Updenv` 108a⟩≡ (211b)

```
void
Updenv(void)
{
    var *v, **h;

    for(h = gvar;h!=&gvar[NVAR];h++)
        for(v=*h;v;v = v->next)
            addenv(v);
    if(runq)
        updenvlocal(runq->local);
}
```

⟨*function* `updenvlocal` 108b⟩≡ (211b)

```
void
updenvlocal(var *v)
{
    if(v){
        updenvlocal(v->next);
        addenv(v);
    }
}
```

⟨*enum* `MiscPlan9` 108c⟩≡ (211b)

```
enum {
    Maxenvname = 256, /* undocumented limit */
};
```

⟨`Var` *other fields* 108d⟩+≡ (31c) ◁92a

```
bool changed;
```

⟨*function* addenv 109a⟩≡ (211b)

```
void
addenv(var *v)
{
    char envname[Maxenvname];
    word *w;
    int f;
    io *fd;

    if(v->changed){
        v->changed = false;
        snprint(envname, sizeof envname, "/env/%s", v->name);
        if((f = Creat(envname))<0)
            pfmt(err, "rc: can't open %s: %r\n", envname);
        else{
            for(w = v->val;w;w = w->next)
                write(f, w->word, strlen(w->word)+1L);
            close(f);
        }
    }
    if(v->fnchanged){
        v->fnchanged = false;
        snprint(envname, sizeof envname, "/env/fn#%s", v->name);
        if((f = Creat(envname))<0)
            pfmt(err, "rc: can't open %s: %r\n", envname);
        else{
            if(v->fn){
                fd = openfd(f);
                pfmt(fd, "fn %q %s\n", v->name, v->fn[v->pc-1].s);
                closeio(fd);
            }
            close(f);
        }
    }
}
```

10.2 Vinit()

`Vinit()`[109b] reads the `/env/` directory at startup and populates the global variable table. Each environment file contains the variable's words separated by null bytes. `Vinit()` reads the file, splits on nulls, and calls `setvar()`[32a] for each variable. It then resets the `changed` flag so that `Updenv()`[108a] does not write them back immediately. The null byte separator is the only safe choice: unlike spaces or newlines, null bytes cannot appear in filenames or variable values, so the encoding is unambiguous. This is the same insight behind UNIX tools like `find -print0` and `xargs -0`.

⟨*function* Vinit 109b⟩≡ (211b)

```
void
Vinit(void)
{
    int dir, f, len, i, n, nent;
    char *buf, *s;
    char envname[Maxenvname];
    word *val;
    Dir *ent;

    dir = open("/env", OREAD);
    if(dir<0){
```

```
        pfmt(err, "rc: can't open /env: %r\n");
        return;
    }
    ent = nil;
    for(;;){
        nent = dirread(dir, &ent);
        if(nent <= 0)
            break;
        for(i = 0; i<nent; i++){
            len = ent[i].length;
            if(len && strncmp(ent[i].name, "fn#", 3)!=0){
                snprint(envname, sizeof envname, "/env/%s", ent[i].name);
                if((f = open(envname, 0))>=0){
                    buf = emalloc(len+1);
                    n = readn(f, buf, len);
                    if (n <= 0)
                        buf[0] = '\0';
                    else
                        buf[n] = '\0';
                    val = 0;
                    /* Charitably add a 0 at the end if need be */
                    if(buf[len-1])
                        buf[len++]='\0';
                    s = buf+len-1;
                    for(;;){
                        while(s!=buf && s[-1]!='\0') --s;
                        val = newword(s, val);
                        if(s==buf)
                            break;
                        --s;
                    }
                    setvar(ent[i].name, val);
                    vlook(ent[i].name)->changed = false;
                    close(f);
                    efree(buf);
                }
            }
        }
        free(ent);
    }
    close(dir);
}
```

⟨*global* envdir 110⟩≡ (201b)

```
fdt envdir;
```

Chapter 11

Signals

Signal handling is critical for a shell because the shell sits between the user and every running program. When the user presses `^C`, the interrupt signal is sent to all processes in the foreground group—including the shell itself. If the shell simply died along with the interrupted command, the user would lose their terminal session. Instead, the shell must catch the signal, cancel the current command, and return to its prompt, ready for the next input. Beyond `^C`, a shell must also handle hangup signals (so it can clean up or notify background jobs when a terminal disconnects), and let users define their own trap handlers for scripting (e.g., cleaning up temporary files on exit). See KERNEL book [Pad14] for how the kernel delivers notes to processes.

11.1 Overview

Under Plan 9, signals are called *notes*. When a note is received (e.g., an interrupt from `^C`), the kernel calls the handler registered by `Trapinit()`[112d]. The handler does not execute the trap function immediately (which would be unsafe in the middle of interpreting bytecodes). Instead, it increments a counter (`ntrap`) and the main interpreter loop checks this counter after each bytecode (see the `handing trap if necessary` chunk in `main()`[35e]). Users can define signal handlers with `fn sigint {...}`, `fn sighup {...\}`, etc. If no handler is defined, the default behavior applies (usually terminating the process).

⟨*constant* NSIG 111a⟩≡ (194c)
```
#define NSIG 32
```

⟨*constant* SIGINT 111b⟩≡ (194c)
```
#define SIGINT 2
```

⟨*constant* SIGQUIT 111c⟩≡ (194c)
```
#define SIGQUIT 3
```

⟨*global* syssigname 111d⟩≡ (211b)
```
char *syssigname[] = {
    "exit",  /* can't happen */
    "hangup",
    "interrupt",
    "quit",  /* can't happen */
    "alarm",
    "kill",
    "sys: fp: ",
    "term",
    0
};
```

⟨*global* signame 112a⟩≡ (211b)

```
char *signame[] = {
    "sigexit",
    "sighup",
    "sigint",
    "sigquit",
    "sigalrm",
    "sigkill",
    "sigfpe",
    "sigterm",
    0
};
```

⟨*global* ntrap 112b⟩≡ (205b)

```
int ntrap;    /* number of outstanding traps */
```

⟨*global* trap 112c⟩≡ (205b)

```
int trap[NSIG];    /* number of outstanding traps per type */
```

11.2 Registering handlers: Trapinit()

⟨*function* Trapinit 112d⟩≡ (211b)

```
void
Trapinit(void)
{
    notify(notifyf);
}
```

⟨*function* notifyf 112e⟩≡ (211b)

```
void
notifyf(void*, char *s)
{
    int i;
    for(i = 0;syssigname[i];i++)
     if(strncmp(s, syssigname[i], strlen(syssigname[i]))==0){
        if(strncmp(s, "sys: ", 5)!=0)
            interrupted = true;
        goto Out;
    }
    pfmt(err, "rc: note: %s\n", s);
    noted(NDFLT);
    return;
Out:
    if(strcmp(s, "interrupt")!=0 || trap[i]==0){
        trap[i]++;
        ntrap++;
    }
    if(ntrap>=32){ /* rc is probably in a trap loop */
        pfmt(err, "rc: Too many traps (trap %s), aborting\n", s);
        abort();
    }
    noted(NCONT);
}
```

11.3 Trap dispatch: dotrap()

⟨main() *handing trap if necessary in interpreter loop* 113a⟩≡ (38c)

```
if(ntrap)
    dotrap();
```

⟨*function* dotrap 113b⟩≡ (205b)

```
void
dotrap(void)
{
    int i;
    struct Var *trapreq;
    struct Word *starval;

    starval = vlook("*")->val;
    while(ntrap)
     for(i = 0;i!=NSIG;i++)
      while(trap[i]){
        --trap[i];
        --ntrap;
        if(getpid()!=mypid)
            Exit(getstatus(), __LOC__);
        trapreq = vlook(signame[i]);
        if(trapreq->fn){
            start(trapreq->fn, trapreq->pc, (struct Var *)nil);
            runq->local = newvar(strdup("*"), runq->local);
            runq->local->val = copywords(starval, (struct Word *)nil);
            runq->local->changed = true;
            runq->redir = runq->startredir = nil;
        }
        else if(i==SIGINT || i==SIGQUIT){
            /*
             * run the stack down until we uncover the
             * command reading loop.  Xreturn will exit
             * if there is none (i.e. if this is not
             * an interactive rc.)
             */
            while(!runq->iflag)
                Xreturn();
        }
        else
            Exit(getstatus(), __LOC__);
    }
}
```

Uses NSIG<> 205b, false, interrupted 113b, signame 113b, and syssigname 113b.

⟨*global* interrupted 113c⟩≡ (211b)

```
bool interrupted = false;
```

⟨*function* Eintr 113d⟩≡ (211b)

```
bool
Eintr(void)
{
    return interrupted;
}
```

⟨Xrdcmds() *calls Noerror() before yyparse()* 113e⟩≡ (39a)

```
Noerror();
```

Uses Noerror().

⟨*function* `Noerror` 114a⟩≡ (211b)

```
void
Noerror(void)
{
    interrupted = false;
}
```

⟨`Xrdcmds()` *reset ntrap* 114b⟩≡ (39a)

```
ntrap = 0; /* avoid double-interrupts during blocked writes */
```

Uses `ntrap` 205b.

Chapter 12

Initialization

In Chapter 4, we showed a simplified version of `main()` to introduce the interpreter loop. This chapter reveals the real initialization sequence: how `rc` fabricates its own bootstrap bytecodes, sources the `rcmain` script to set up the default environment, and transitions into the read-eval-print loop that makes an interactive shell usable.

12.1 Actual bootstrapping code

The bootstrap performs two operations encoded as hand-crafted bytecodes: first, it assigns the command-line arguments to `$*` (with `*=(argv)`), then it executes `. rcmain $*` to source the initialization script. The bootstrap is essentially the bytecode equivalent of:

```
*=(argv)
. /rc/lib/rcmain $*
```

The two-line script above is encoded as a sequence of bytecodes filling the `bootstrap` array. The first group (`Xmark`, `Xword "*"`, `Xassign`) assigns the command-line arguments to `$*`. The second group builds the argument list for the `.` builtin: it pushes `$*` (via `Xdol`), the path to `rcmain`, and the `.` command itself, then calls `Xsimple`.

⟨`main()` *initialize* `boostrap` 115⟩≡ (34b)

```
memset(bootstrap, 0, sizeof bootstrap);

i = 0;
bootstrap[i++].i=1;
// runq->argv is populated with the arguments to rc
// we just need to add '*=(argv)'
bootstrap[i++].f = Xmark;
  bootstrap[i++].f = Xword;
  bootstrap[i++].s="*";
bootstrap[i++].f = Xassign; // will pop_list() x2

bootstrap[i++].f = Xmark;
  bootstrap[i++].f = Xmark;
    bootstrap[i++].f = Xword;
    bootstrap[i++].s="*";
  bootstrap[i++].f = Xdol; // will pop_list()
  bootstrap[i++].f = Xword;
  bootstrap[i++].s = rcmain;
  bootstrap[i++].f = Xword;
  bootstrap[i++].s=".";
bootstrap[i++].f = Xsimple; // will pop_list()

bootstrap[i++].f = Xexit;
```

```
bootstrap[i].i = 0;
```

Uses Rcmain 211a, STDERR, Trapinit(), Vinit(), err 153b, flag 34c, inttoascii(), kinit() 53e, mypid 116d, newword() 29d, nil, openfd() 160f, and setvar() 32a.

12.2 Intitialization script and rc -m /path/to/rcmain

The default initialization script is /rc/lib/rcmain, which sets up $path and, for login shells, sources the user's $home/lib/profile. The -m flag allows overriding this path, which is useful during system bootstrap when the standard rcmain may not be available yet.

⟨*global* Rcmain 116a⟩≡ (211b)

```
char *Rcmain = "/rc/lib/rcmain";
```

⟨main() *locals* 116b⟩+≡ (34b) ◁?? 116e▷

```
char *rcmain;
```

⟨main() *initialisation* 116c⟩+≡ (34b) ◁36b 116f▷

```
rcmain = flag['m'] ? flag['m'][0] : Rcmain;
```

12.3 Actual environment

Before entering the interpreter loop, rc also sets up a few built-in variables: $pid (the shell's own process id, useful for creating unique temporary filenames) and $rcname (the name rc was invoked as, i.e., argv[0]).

⟨*global* mypid 116d⟩≡ (197b)

```
int mypid;
```

⟨main() *locals* 116e⟩+≡ (34b) ◁116b

```
char num[12];
```

⟨main() *initialisation* 116f⟩+≡ (34b) ◁116c 116g▷

```
mypid = getpid();
inttoascii(num, mypid);
setvar("pid", newword(num, (word *)nil));
```

Uses flag 34c and flagset 35b.

⟨main() *initialisation* 116g⟩+≡ (34b) ◁116f 126a▷

```
setvar("rcname", newword(argv[0], (word *)nil));
```

Uses flag 34c.

Chapter 13

Globbing

Globbing (also called wildcard expansion or pathname expansion) is one of the key features of a shell: when you write `ls *.c`, the shell expands `*.c` into the list of matching filenames *before* passing them to `ls`. This is a deliberate design choice: the expansion is done by the shell, not by the individual programs, which factorizes the functionality. In the original Unix shell, globbing was actually handled by a separate external program called `glob`. In MS-DOS, the opposite approach was taken: wildcards are passed unexpanded to each program, which must interpret them itself. This is generally worse (every program must reimplement the logic), but it does allow commands like `ren *.txt *.bak` to have a meaning—the program sees both patterns and can pair them up, something that is impossible when the shell expands both arguments independently.

The glob patterns `*`, `?`, and `[]` are different from regular expressions: `*` matches any string (not "any number of the preceding character"), and `?` matches any single character. This is more practical for filenames, where `*.c` is more natural than the regex `.*\.c`.

Globbing touches several phases of the interpreter. The lexer marks glob characters with a special escape byte so they can be distinguished from literal characters. The bytecode generator emits `Xglob`[119a] instructions at the right points. And at runtime, the actual matching is done by `globlist()`[118d], which walks directories and filters filenames against the pattern using `matchfn()`[122b].

13.1 Lexing globbing characters

Glob characters (`*`, `?`, `[]`) are marked during lexing by inserting a `GLOB` byte (`\001`) before them. This way, the rest of the shell can distinguish a literal `*` (inside quotes, no `GLOB` prefix) from a wildcard `*` (with a `GLOB` prefix). The `deglob()`[120b] function removes these markers when they are no longer needed.

⟨*constant* `GLOB` 117a⟩≡ (194c)

```
/*
 * Glob character escape in strings:
 * In a string, GLOB must be followed by *?[ or GLOB.
 * GLOB* matches any string
 * GLOB? matches any single character
 * GLOB[...] matches anything in the brackets
 * GLOBGLOB matches GLOB
 */
#define GLOB '\001'
```

⟨`yylex()` *when c is a word character, if glob character* 117b⟩≡ (52c)

```
if(c=='*' || c=='[' || c=='?' || c==GLOB)
    w = addtok(w, GLOB);
```

Uses `GLOB` 31c and `addtok()` 118a.

⟨*function* addtok 118a⟩≡ (205a)

```
char*
addtok(char *p, int val)
{
    if(p==nil)
        return nil;
    if(p >= &tok[NTOK]){
        *p = '\0';
        yyerror("token buffer too short");
        return nil;
    }
    *p++=val;
    return p;
}
```

Uses NTOK<> 47d, nil, tok 47c, and yyerror() 153e.

13.2 Expanding globbing characters

The expansion happens at runtime, not at compile time. The Xglob[119a] bytecode (emitted after evaluating command arguments) scans the word list in argv and replaces any word containing GLOB markers with the list of matching filenames from the filesystem.

⟨*global* globv 118b⟩≡ (200c)

```
struct Word *globv;
```

⟨Xsimple() *initializations,* globlist() 118c⟩≡ (68c)

```
globlist();
```

Uses globlist().

⟨*function* globlist 118d⟩≡ (200c)

```
void
globlist(void)
{
    word *a;
    globv = nil;

    globlist1(runq->argv->words);
    poplist();
    pushlist();

    if(globv){
        for(a = globv;a->next;a = a->next)
            ;
        a->next = runq->argv->words;
        runq->argv->words = globv;
    }
}
```

Uses false, glob(), and globlist1() 118d.

⟨*function* globlist1 118e⟩≡ (200c)

```
void
globlist1(word *gl)
{
    if(gl){
        globlist1(gl->next);
        glob(gl->word);
    }
}
```

Uses false, nextutf() 122a, and unicode() 122a.

⟨*function* Xglob 119a⟩≡ (201b)

```
void
Xglob(void)
{
    globlist();
}
```

Uses globlist().

13.3 glob()

glob()[119c] takes a pattern string and pushes all matching filenames onto globv. If no files match, the original pattern (with GLOB markers removed) is kept as-is. globdir()[120c] is the recursive workhorse: it scans the pattern for the first component containing a metacharacter, reads the corresponding directory, and recurses for each matching entry. The results are sorted alphabetically by globsort()[120c].

⟨*global* globname 119b⟩≡ (200c)

```
char *globname;
```

⟨*function* glob 119c⟩≡ (200c)

```
/*
 * Push all file names matched by p on the current thread's stack.
 * If there are no matches, the list consists of p.
 */
void
glob(void *ap)
{
    uchar *p = ap;
    word *svglobv = globv;
    int globlen = Globsize(ap);

    if(!globlen){
        deglob(p);
        globv = newword((char *)p, globv);
        return;
    }
    globname = emalloc(globlen);
    globname[0]='\0';
    globdir(p, (uchar *)globname);
    efree(globname);
    if(svglobv==globv){
        deglob(p);
        globv = newword((char *)p, globv);
    }
    else
        globsort(globv, svglobv);
}
```

Uses Closedir(), Opendir() 166c, Readdir(), globdir() 120c, globname, globv 120a, matchfn() 122b, and newword() 29d.

⟨*function* deglob 119d⟩≡ (200c)

```
/*
 * delete all the GLOB marks from s, in place
 */
void
deglob(void *as)
{
    char *s = as;
    char *t = s;
```

```
    do{
        if(*t==GLOB)
            t++;
        *s++=*t;
    }while(*t++);
}
```

Uses GLOB 31c and NDIR<> 200c.

⟨*function* globcmp 120a⟩≡ (200c)

```
int
globcmp(const void *s, const void *t)
{
    return strcmp(*(char**)s, *(char**)t);
}
```

Uses Word 194c.

⟨*function* globsort 120b⟩≡ (200c)

```
void
globsort(word *left, word *right)
{
    char **list;
    word *a;
    int n = 0;
    for(a = left;a!=right;a = a->next) n++;
    list = (char **)emalloc(n*sizeof(char *));
    for(a = left,n = 0;a!=right;a = a->next,n++) list[n] = a->word;
    qsort((void *)list, n, sizeof(void *), globcmp);
    for(a = left,n = 0;a!=right;a = a->next,n++) a->word = list[n];
    efree((char *)list);
}
```

Uses GLOB 31c.

⟨*function* globdir 120c⟩≡ (200c)

```
/*
 * Push names prefixed by globname and suffixed by a match of p onto the astack.
 * namep points to the end of the prefix in globname.
 */
void
globdir(uchar *p, uchar *namep)
{
    uchar *t, *newp;
    int f;
    /* scan the pattern looking for a component with a metacharacter in it */
    if(*p=='\0'){
        globv = newword(globname, globv);
        return;
    }
    t = namep;
    newp = p;
    while(*newp){
        if(*newp==GLOB)
            break;
        *t=*newp++;
        if(*t++=='/'){
            namep = t;
            p = newp;
        }
    }
    /* If we ran out of pattern, append the name if accessible */
```

```
    if(*newp=='\0'){
        *t='\0';
        if(access(globname, 0)==0)
            globv = newword(globname, globv);
        return;
    }
    /* read the directory and recur for any entry that matches */
    *namep='\0';
    if((f = Opendir(globname[0]?globname:"."))<0) return;
    while(*newp!='/' && *newp!='\0') newp++;
    while(Readdir(f, namep, *newp=='/')){
        if(matchfn(namep, p)){
            for(t = namep;*t;t++);
            globdir(newp, t);
        }
    }
    Closedir(f);
}
```

Uses GLOB 31c, efree(), emalloc(), globcmp() 120c, globname, globv 120a, and newword() 29d.

⟨*function* equtf 121a⟩≡ (200c)

```
/*
 * Do p and q point at equal utf codes
 */
bool
equtf(uchar *p, uchar *q)
{
    Rune pr, qr;
    if(*p!=*q)
        return false;

    chartorune(&pr, (char*)p);
    chartorune(&qr, (char*)q);
    return pr == qr;
}
```

Uses Globsize(), deglob() 120b, efree(), emalloc(), globdir() 120c, globname, globv 120a, and newword() 29d.

⟨*function* nextutf 121b⟩≡ (200c)

```
/*
 * Return a pointer to the next utf code in the string,
 * not jumping past nuls in broken utf codes!
 */
uchar*
nextutf(uchar *p)
{
    Rune dummy;
    return p + chartorune(&dummy, (char*)p);
}
```

Uses globsort() 120c and globv 120a.

⟨*function* unicode 121c⟩≡ (200c)

```
/*
 * Convert the utf code at *p to a unicode value
 */
int
unicode(uchar *p)
{
    Rune r;
```

```
    chartorune(&r, (char*)p);
    return r;
}
```

Uses `false`.

13.4 match()

`matchfn()`[122b] is the entry point called from `glob()`[119c]. It enforces one subtle rule: the dot-files `.` and `..` are only matched if the pattern explicitly starts with a dot. This prevents `*` from matching `.` and `..`, which would cause recursive globbing to loop or produce confusing results.

The actual matching is done by `match()`[122b], which walks the pattern and the string in lockstep. Non-`GLOB` bytes must match literally. `GLOB` bytes introduce the special characters: `*` tries every possible suffix via recursion (backtracking), `?` matches exactly one UTF-8 character, and `[...]` matches a character class with optional ranges (`a-z`) and complement (`~`). Note that `rc` uses `~` instead of `^` for complement inside character classes, since `^` is already the concatenation operator.

The `stop` parameter is `/`: a single glob pattern is matched one path component at a time by `globdir()`[120c], so `match()` stops at slashes rather than consuming them.

⟨*function* `matchfn` 122a⟩≡ (200c)

```
/*
 * Does the string s match the pattern p
 * . and .. are only matched by patterns starting with .
 * * matches any sequence of characters
 * ? matches any single character
 * [...] matches the enclosed list of characters
 */
bool
matchfn(void *as, void *ap)
{
    uchar *s = as, *p = ap;

    if(s[0]=='.' && (s[1]=='\0' || s[1]=='.' && s[2]=='\0') && p[0]!='.')
        return false;
    return match(s, p, '/');
}
```

⟨*function* `match` 122b⟩≡ (200c)

```
bool
match(void *as, void *ap, int stop)
{
    int compl, hit, lo, hi, t, c;
    uchar *s = as, *p = ap;

    for(; *p!=stop && *p!='\0'; s = nextutf(s), p = nextutf(p)){
        if(*p!=GLOB){
            if(!equtf(p, s)) return false;
        }
        else switch(*++p){
        case GLOB:
            if(*s!=GLOB)
                return false;
            break;
        case '*':
            for(;;){
                if(match(s, nextutf(p), stop)) return 1;
                if(!*s)
```

```
                break;
            s = nextutf(s);
        }
        return false;
    case '?':
        if(*s=='\0')
            return false;
        break;
    case '[':
        if(*s=='\0')
            return false;
        c = unicode(s);
        p++;
        compl=*p=='~';
        if(compl)
            p++;
        hit = 0;
        while(*p!=']'){
            if(*p=='\0')
                return false;  /* syntax error */
            lo = unicode(p);
            p = nextutf(p);
            if(*p!='-')
                hi = lo;
            else{
                p++;
                if(*p=='\0')
                    return false; /* syntax error */
                hi = unicode(p);
                p = nextutf(p);
                if(hi<lo){ t = lo; lo = hi; hi = t; }
            }
            if(lo<=c && c<=hi)
                hit = 1;
        }
        if(compl)
            hit=!hit;
        if(!hit)
            return false;
        break;
    }
}
return *s=='\0';
}
```

Uses GLOB 31c, equtf() 121b, false, match() 122b, nextutf() 122a, and unicode() 122a.

⟨*constant* NDIR 123a⟩≡ (211b)

```
#define NDIR 256  /* shoud be a better way */
```

⟨*function* Globsize 123b⟩≡ (211b)

```
int
Globsize(char *p)
{
    int isglob = 0, globlen = NDIR+1;
    for(;*p;p++){
        if(*p==GLOB){
            p++;
            if(*p!=GLOB)
                isglob++;
            globlen+=*p=='*'?NDIR:1;
```

```
        }
        else
            globlen++;
    }
    return isglob?globlen:0;
}
```

⟨*constant* NFD 124a⟩≡ (211b)

```
#define NFD 50
```

⟨*struct* DirEntryWrapper 124b⟩≡ (211b)

```
struct DirEntryWrapper {
    Dir *dbuf;
    int i;
    int n;
};
```

⟨*global* dir 124c⟩≡ (211b)

```
struct DirEntryWrapper dir[NFD];
```

⟨*function* Opendir 124d⟩≡ (211b)

```
int
Opendir(char *name)
{
    Dir *db;
    int f;
    f = open(name, 0);
    if(f==-1)
        return f;
    db = dirfstat(f);
    if(db!=nil && (db->mode&DMDIR)){
        if(f<NFD){
            dir[f].i = 0;
            dir[f].n = 0;
        }
        free(db);
        return f;
    }
    free(db);
    close(f);
    return -1;
}
```

⟨*function* trimdirs 124e⟩≡ (211b)

```
static int
trimdirs(Dir *d, int nd)
{
    int r, w;

    for(r=w=0; r<nd; r++)
        if(d[r].mode&DMDIR)
            d[w++] = d[r];
    return w;
}
```

⟨*function* Readdir 125a⟩≡ (211b)

```
/*
 * onlydirs is advisory -- it means you only
 * need to return the directories.  it's okay to
 * return files too (e.g., on unix where you can't
 * tell during the readdir), but that just makes
 * the globber work harder.
 */
int
Readdir(int f, void *p, int onlydirs)
{
    int n;

    if(f<0 || f>=NFD)
        return 0;
Again:
    if(dir[f].i==dir[f].n){ /* read */
        free(dir[f].dbuf);
        dir[f].dbuf = 0;
        n = dirread(f, &dir[f].dbuf);
        if(n>0){
            if(onlydirs){
                n = trimdirs(dir[f].dbuf, n);
                if(n == 0)
                    goto Again;
            }
            dir[f].n = n;
        }else
            dir[f].n = 0;
        dir[f].i = 0;
    }
    if(dir[f].i == dir[f].n)
        return 0;
    strcpy(p, dir[f].dbuf[dir[f].i].name);
    dir[f].i++;
    return 1;
}
```

⟨*function* Closedir 125b⟩≡ (211b)

```
void
Closedir(int f)
{
    if(f>=0 && f<NFD){
        free(dir[f].dbuf);
        dir[f].i = 0;
        dir[f].n = 0;
        dir[f].dbuf = 0;
    }
    close(f);
}
```

Chapter 14

Advanced Topics

In this chapter I present features that are not essential for understanding the core architecture of `rc`, but that are important in practice. Many of these features—here documents, command substitution, process substitution, general redirections—are what make a shell truly useful for scripting.

Note that in Plan 9, many shell-like features that other systems implement inside the shell are instead provided by the window manager `rio`: command-line editing, history, filename completion, and job control. This keeps `rc` simple.

14.1 Reading commands from a string: `rc -c`

The `-c` flag lets you pass a command string directly on the command line, as in `rc -c 'echo hello'`. This is how other programs (like `mk`, see the Build book) invoke the shell to execute a recipe: they build a command string and pass it via `-c` rather than writing it to a temporary file.

⟨`main()` *initialisation* 126a⟩+≡ (34b) ◁116g

```
setvar("cflag", flag['c']? newword(flag['c'][0], (word *)nil) : (word *)nil);
```

14.2 Failing fast: `rc -e`

The `-e` flag makes `rc` exit immediately when a simple command fails (returns non-null status). This is extremely important for build systems: `mk` (see the Build book) invokes `rc -e` for recipes, so that a failure in any command—especially inside a loop iterating over subdirectories—stops the entire build rather than silently continuing.

Note that `-e` is a command-line flag, not a `set` option, so you cannot enable it from within a script. The check is only inserted after simple commands, not after control flow constructs like `if` or `||`, since a false condition in `if(test ...)` should not abort the script.

⟨`outcode()` *emit Xeflag after Xsimple* 126b⟩≡ (67c)

```
if(eflag)
    emitf(Xeflag);
```

Uses `Xeflag()` 127a and `emitf<>` 62d.

⟨*global* `eflagok` 126c⟩≡ (197b)

```
bool eflagok; /* kludge flag so that -e doesn't exit in startup */
```

⟨`execdot()` *if not first execution* 126d⟩≡ (103d)

```
else
    eflagok = true;
```

Uses `eflagok` 126c and `true`.

⟨*function* Xeflag 127a⟩≡ (201b)

```
void
Xeflag(void)
{
    if(eflagok && !truestatus())
        Xexit();
}
```

Uses Xexit() 85c, eflagok 126c, and truestatus() 66a.

The eflagok guard prevents -e from triggering during startup (while sourcing rcmain). It is set to true only after the first dot command finishes, so initialization errors do not cause an immediate exit.

14.3 Unicode

Plan 9 was the birthplace of UTF-8, and rc handles Unicode transparently. The key insight is that UTF-8 is self-synchronizing: the lexer can consume multi-byte runes by reading the first byte (which encodes the length) and then pulling the appropriate number of continuation bytes. addutf()[127b] does exactly this during lexing: after addtok stores the first byte, it reads continuation bytes by checking the leading-bit pattern.

⟨*function* addutf 127b⟩≡ (205a)

```
char*
addutf(char *p, int c)
{
    uchar b, m;
    int i;

    p = addtok(p, c); /* 1-byte UTF runes are special */
    if(c < Runeself)
        return p;

    m = 0xc0;
    b = 0x80;
    for(i=1; i < UTFmax; i++){
        if((c&m) == b)
            break;
        p = addtok(p, advance());
        b = m;
        m = (m >> 1)|0x80;
    }
    return p;
}
```

Uses Runeself, UTFmax, addtok() 118a, and advance() 45a.

14.4 Advanced constructs

14.4.1 Subshell: @ <cmd>

The @ operator runs a command in a child process, so that side effects like cd or variable assignments do not affect the parent. Braces {...} group commands but execute them in the current process—so {cd /tmp; ls} would change the parent's directory. With @{cd /tmp; ls}, the directory change is confined to the subshell.

This is particularly useful in mkfile recipes that iterate over subdirectories:

```
for(i in $DIRS) @{
    cd $i
    mk $MKFLAGS $stem
```

```
}
```

The implementation is straightforward: fork, run the body in the child (which will `Xexit`), and `Waitfor` in the parent. The forward-jump past the body (via `stuffdot`) lets the parent skip the child's code.

⟨*token declarations* 128a⟩+≡ (55a) ◁61a 134b▷

```
%token SUBSHELL /** @ */
```

⟨*cmd rule other cases* 128b⟩+≡ (56d) ◁61e

```
|   SUBSHELL cmd        {$$=mung1($1, $2);}
```

⟨`outcode()` *cases* 128c⟩+≡ (64a) ◁98b 135b▷

```
case SUBSHELL:
    emitf(Xsubshell);
    p = emiti(0);
    outcode(c0, eflag);
    emitf(Xexit);
    stuffdot(p);
    if(eflag)
        emitf(Xeflag);
    break;
```

Uses `Xeflag()` 127a, `Xexit()` 85c, `Xsubshell()`, `c0<>` 63c 148b, `emitf<>` 62d, `emiti<>` 62c, `outcode()` 64a, and `stuffdot()` 74c.

⟨*function* `Xsubshell` 128d⟩≡ (204a)

```
void
Xsubshell(void)
{
    int pid;
    switch(pid = fork()){
    case -1:
        Xerror("try again");
        break;
    case 0: // child
        clearwaitpids();
        start(runq->code, runq->pc+1, runq->local);
        runq->ret = nil;
        break;
    default: // parent
        addwaitpid(pid);
        Waitfor(pid, true);
        runq->pc = runq->code[runq->pc].i;
        break;
    }
}
```

Uses `Waitfor()` 85b, `closeio()` 162b, `newword()` 29d, and `runq` 28c.

14.4.2 Here documents: `<< <HERE>`

Here documents let you embed multi-line *data* directly in a script, avoiding external file dependencies. The syntax `<<TAG` reads input until a line matching `TAG` appears. For example, a phone-lookup script can carry its own database inline:

```
grep $1 <<EOF
Alice 555-1234
Bob   555-5678
EOF
```

The program and its data live in the same file, so the script is self-contained and easy to move around. A variable would not work as well here: shell variables are word lists, so multi-line structure (newlines, whitespace, special characters) would be lost. A here document feeds the data as a raw byte stream to stdin, which is exactly what programs like `grep` or `sed` expect.

The implementation is clever: during parsing, `heredoc()` creates a temporary file (in `/tmp`) and records the tag and filename in a linked list. After the entire command line is parsed and compiled, `readhere()` goes back and reads the here-document bodies from the input, writing them to those temporary files. This two-phase approach is necessary because a command line can have multiple here documents (e.g., `cmd <<A <<B`), and the bodies appear after the full command line.

If the here-tag was not quoted, variable substitution is performed via `psubst()`[131f]; otherwise the text is copied literally. The temporary files are cleaned up by emitting `Xdelhere` bytecodes that `remove()` them after use.

⟨*constant* `HERE` 129a⟩≡ (194c)
```
#define HERE 4
```

⟨`yylex()` *in switch when redirection character, if here document* 129b⟩≡ (51a)
```
if(nextis(c)){
    t->rtype = HERE;
    *w++=c;
}
```
Uses `HERE` and `nextis()` 45b.

⟨*struct* `Here` 129c⟩≡ (194c)
```
struct Here {
    tree *tag;
    char *name;
    struct Here *next;
};
```

⟨*global* `here` 129d⟩≡ (206c)
```
struct Here *here;
```
Uses `Here`.

⟨*global* `ehere` 129e⟩≡ (206c)
```
struct Here **ehere;
```
Uses `Here`.

⟨*constant* `NLINE` 129f⟩≡ (206c)
```
/*
 * bug: lines longer than NLINE get split -- this can cause spurious
 * missubstitution, or a misrecognized EOF marker.
 */
#define NLINE 4096
```

⟨`compile()` *after outcode and error management, read heredoc* 129g⟩≡ (63a)
```
readhere();
```
Uses `readhere()` 129h.

⟨*function* `readhere` 129h⟩≡ (206c)
```
void
readhere(void)
{
    int c, subst;
    char *s, *tag;
    char line[NLINE+1];
    io *f;
    struct Here *h, *nexth;
```

```
    for(h = here; h; h = nexth){
        subst = !h->tag->quoted;
        tag = h->tag->str;
        c = Creat(h->name);
        if(c < 0)
            yyerror("can't create here document");
        f = openfd(c);
        s = line;
        pprompt();
        while((c = rchr(runq->cmdfd)) != EOF){
            if(c == '\n' || s == &line[NLINE]){
                *s = '\0';
                if(tag && strcmp(line, tag) == 0)
                    break;
                if(subst)
                    psubst(f, (uchar *)line);
                else
                    pstr(f, line);
                s = line;
                if(c == '\n'){
                    pprompt();
                    pchr(f, c);
                }else
                    *s++ = c;
            }else
                *s++ = c;
        }
        flush(f);
        closeio(f);
        cleanhere(h->name);
        nexth = h->next;
        efree((char *)h);
    }
    here = nil;
    doprompt = true;
}
```

Uses `Creat()` 155b, `EOF` 42a, `Here`, `NLINE<>` 129f, `cleanhere()` 130a, `closeio()` 162b, `doprompt` 43b, `efree()`, `flush()` 160h, `here` 129d, `nil`, `openfd()` 160f, `pchr()` 161a, `pprompt()` 43g, `pstr()` 164a, `psubst()` 131f, `rchr()` 161c, `runq` 28c, `true`, and `yyerror()` 153e.

⟨*function* `cleanhere` 130a⟩≡ (207a)

```
void
cleanhere(char *f)
{
    emitf(Xdelhere);
    emits(strdup(f));
}
```

Uses `Xdelhere()` 130b, `emitf<>` 62d, and `emits<>` 62e.

⟨*function* `Xdelhere` 130b⟩≡ (201b)

```
void
Xdelhere(void)
{
    remove(runq->code[runq->pc++].s);
}
```

Uses `Unlink()` and `runq` 28c.

⟨*function* `Unlink` 130c⟩≡ (208a)

⟨*global* ser 131a⟩≡ (206c)

```
int ser = 0;
```

Uses ser 131a.

⟨*global tmp (rc/here.c)* 131b⟩≡ (206c)

```
char tmp[] = "/tmp/here0000.0000";
```

Uses tmp 131b.

⟨*global* hex 131c⟩≡ (206c)

```
char hex[] = "0123456789abcdef";
```

Uses hex 131c.

⟨*function* hexnum 131d⟩≡ (206c)

```
void
hexnum(char *p, int n)
{
    *p++ = hex[(n>>12)&0xF];
    *p++ = hex[(n>>8)&0xF];
    *p++ = hex[(n>>4)&0xF];
    *p = hex[n&0xF];
}
```

Uses hex 131c.

⟨*function* heredoc 131e⟩≡ (206c)

```
//@Scheck: used by syn.y
tree* heredoc(tree *tag)
{
    struct Here *h = new(struct Here);

    if(tag->type != WORD)
        yyerror("Bad here tag");
    h->next = 0;
    if(here)
        *ehere = h;
    else
        here = h;
    ehere = &h->next;
    h->tag = tag;
    hexnum(&tmp[9], getpid());
    hexnum(&tmp[14], ser++);
    h->name = strdup(tmp);
    return token(tmp, WORD);
}
```

Uses Here, ehere 129e, here 129d, hexnum() 131d, ser 131a, tmp 131b, token() 52b, and yyerror() 153e.

⟨*function* psubst 131f⟩≡ (206c)

```
void
psubst(io *f, uchar *s)
{
    int savec, n;
    uchar *t, *u;
    Rune r;
    word *star;

    while(*s){
        if(*s != '$'){  /* copy plain text rune */
            if(*s < Runeself)
                pchr(f, *s++);
            else{
```

```
                n = chartorune(&r, (char *)s);
                while(n-- > 0)
                    pchr(f, *s++);
            }
        }else{   /* $something -- perform substitution */
            t = ++s;
            if(*t == '$')
                pchr(f, *t++);
            else{
                while(*t && idchr(*t))
                    t++;
                savec = *t;
                *t = '\0';
                n = 0;
                for(u = s; *u && '0' <= *u && *u <= '9'; u++)
                    n = n*10 + *u - '0';
                if(n && *u == '\0'){
                    star = vlook("*")->val;
                    if(star && 1 <= n && n <= count(star)){
                        while(--n)
                            star = star->next;
                        pstr(f, star->word);
                    }
                }else
                    pstrs(f, vlook((char *)s)->val);
                *t = savec;
                if(savec == '^')
                    t++;
            }
            s = t;
        }
    }
}
```

Uses `Runeself`, `count()` 30a, `idchr()` 47a, `pchr()` 161a, `pstr()` 164a, `pstrs()` 132a, and `vlook()` 32b.

⟨*function* `pstrs` 132a⟩≡ (206c)

```
void
pstrs(io *f, word *a)
{
    if(a){
        while(a->next && a->next->word){
            pstr(f, a->word);
            pchr(f, ' ');
            a = a->next;
        }
        pstr(f, a->word);
    }
}
```

Uses `pchr()` 161a and `pstr()` 164a.

14.4.3 Read-write redirections: <> <file>

The <> operator opens a file for both reading and writing (`ORDWR`), useful for in-place modification of files.

⟨*constant* `RDWR` 132b⟩≡ (194c)

```
#define RDWR 7
```

⟨yylex() *in switch when redirection character, if read/write redirect* 133a⟩≡ (51a)

```
else if (nextis('>')){
    t->rtype = RDWR;
    *w++=c;
}
```

Uses RDWR 194c and nextis() 45b.

⟨outcode() *when REDIR case, switch redirection type cases* 133b⟩+≡ (78c) ◁80b

```
case RDWR:
    emitf(Xrdwr);
    break;
```

Uses RDWR 194c, Xrdwr() 133c, and emitf<> 62d.

⟨*function* Xrdwr 133c⟩≡ (201b)

```
void
Xrdwr(void)
{
    char *file;
    int f;

    switch(count(runq->argv->words)){
    default:
        Xerror1("<> requires singleton\n");
        return;
    case 0:
        Xerror1("<> requires file\n");
        return;
    case 1:
        break;
    }
    file = runq->argv->words->word;
    if((f = open(file, ORDWR))<0){
        pfmt(err, "%s: ", file);
        Xerror("can't open");
        return;
    }
    pushredir(ROPEN, f, runq->code[runq->pc].i);
    runq->pc++;
    poplist();
}
```

Uses ORDWR, ROPEN 78b, Xerror() 69b, Xerror1() 69c, count() 30a, err 153b, pfmt() 163b, poplist() 64d, pushredir() 77d, and runq 28c.

14.4.4 General redirections: >[2] <file>

The basic redirections > and < always use file descriptors 1 and 0 respectively. The bracket syntax >[2] or <[3] lets you redirect any file descriptor. For example, echo error >[2] /tmp/log redirects stderr.

The lexer handles the bracket syntax inline: after seeing > or <, if a [follows, it reads the file descriptor number. If it then sees =, the redirection becomes a DUP (file descriptor duplication or closing) rather than a file redirection.

⟨yylex() *in switch when redirection character, if bracket after* 133d⟩≡ (49g)

```
if(nextis('[')){
    *w++='[';
    c = advance();
    *w++=c;
    if(c<'0' || '9'<c){
    RedirErr:
```

```
        *w = 0;
        yyerror(t->type==PIPE?"pipe syntax"
                :"redirection syntax");
        return EOF;
    }
    t->fd0 = 0;
    do{
        t->fd0 = t->fd0*10 + c-'0';
        *w++=c;
        c = advance();
    }while('0'<=c && c<='9');

    if(c=='='){
        *w++='=';
        if(t->type==REDIR)
            // change the token type
            t->type = DUP;
        c = advance();
        if('0'<=c && c<='9'){
            t->rtype = DUPFD;
            t->fd1 = t->fd0;
            t->fd0 = 0;
            do{
                t->fd0 = t->fd0*10+c-'0';
                *w++=c;
                c = advance();
            }while('0'<=c && c<='9');
        }
        else{
            if(t->type==PIPE)
                goto RedirErr;
            t->rtype = CLOSE;
        }
    }
    if(c!=']'
    || t->type==DUP && (t->rtype==HERE || t->rtype==APPEND))
        goto RedirErr;
    *w++=']';
}
```

Uses `APPEND` 50a, `CLOSE` 194c, `DUPFD` 25a, `EOF` 42a, `HERE`, `advance()` 45a, `nextis()` 45b, and `yyerror()` 153e.

⟨*constant* `CLOSE` 134a⟩≡ (194c)

```
#define CLOSE 6
```

14.4.5 Advanced dup: `>[<fd0>=<fd1> ]`, `<>[<fd0>=<fd1>]` , `<[<fd0>=<fd1>]`

The `DUP` construct performs `dup2` between file descriptors. `>[1=2]` makes fd 1 point to the same destination as fd 2 (so stdout goes to stderr). `>[1=]` (with nothing after the equals) closes fd 1. This is cleaner than the Bourne shell's `2>&1` syntax.

Like other redirections, the implementation uses the two-phase approach: `Xdup` pushes a `RDUP` record onto the redir stack (undone by `Xpopredir` afterward), and the actual `dup2` happens in `doredir()` when a child process is forked.

⟨*token declarations* 134b⟩+≡ (55a) ◁128a 136c▷

```
%token DUP
```

⟨*redir rule other cases* 134c⟩≡ (59e)

```
|   DUP
```

⟨*constant* DUPFD 135a⟩≡ (194c)

```
#define DUPFD 5
```

⟨outcode() *cases* 135b⟩+≡ (64a) ◁128c 136d▷

```
case DUP:
    if(t->rtype==DUPFD){
        emitf(Xdup);
        emiti(t->fd0);
        emiti(t->fd1);
    }
    else{
        emitf(Xclose);
        emiti(t->fd0);
    }
    outcode(c1, eflag);
    emitf(Xpopredir);
    break;
```

Uses DUPFD 25a, Xclose() 135g, Xdup() 135c, Xpopredir() 79b, c1<> 63d 148c, emitf<> 62d, emiti<> 62c, and outcode() 64a.

⟨*function* Xdup 135c⟩≡ (201b)

```
void
Xdup(void)
{
    pushredir(RDUP, runq->code[runq->pc].i, runq->code[runq->pc+1].i);
    runq->pc+=2;
}
```

Uses RDUP 135e, pushredir() 77d, and runq 28c.

⟨codefree() *in loop over code* cp, *switch bytecode cases* 135d⟩+≡ (27e) ◁92c

```
else if(p->f==Xdup || p->f==Xpipefd)
        p+=2;
```

Uses Xdup() 135c and Xpipefd() 70a.

⟨*constant* RDUP 135e⟩≡ (197a)

```
#define RDUP 2   /* dup2(from, to); */
```

⟨*constant* RCLOSE 135f⟩≡ (197a)

```
#define RCLOSE 3   /* close(from); */
```

⟨*function* Xclose 135g⟩≡ (201b)

```
void
Xclose(void)
{
    pushredir(RCLOSE, runq->code[runq->pc].i, 0);
    runq->pc++;
}
```

Uses RCLOSE 135f, pushredir() 77d, and runq 28c.

⟨doredir() *switch redir type cases* 135h⟩+≡ (77e) ◁78a 135i▷

```
case RDUP:
    dup(rp->from, rp->to);
    break;
```

Uses Dup() 155b and RDUP 135e.

⟨doredir() *switch redir type cases* 135i⟩+≡ (77e) ◁135h

```
case RCLOSE:
    close(rp->from);
    break;
```

Uses RCLOSE 135f.

14.4.6 Advanced pipes: `|[<fd>]` , `|[<fd0>=<fd1>]`

Just as general redirections extend `>` and `<` with bracket syntax, pipes can also specify which file descriptors to connect. `cmd1 |[2] cmd2` pipes fd 2 of `cmd1` (stderr) to `cmd2`'s stdin.

14.4.7 Command output as a file: `'<{<cmd>}'`

⟨*comword rule other cases* 136a⟩+≡ (56g) ◁61h 136b▷

```
|   REDIR brace     {$$=mung1($1, $2); $$->type=PIPEFD;}
```

Process substitution (called "command output as a file" in Plan 9) lets you use a command's output as if it were a file. `cmp <{old} <{new}` runs the two commands and passes their outputs as filenames to `cmp`.

This appears in the `comword` rule because it really resolves to a filename (`/dev/fd/N`) connected to the command's output via a pipe. The implementation forks a child that runs the command with its stdout (or stdin for `>`) connected to one end of a pipe, while the parent pushes the filename `/fd/N` (the other end) as a word onto the argv stack.

⟨*comword rule other cases* 136b⟩+≡ (56g) ◁136a 137a▷

```
|   REDIR brace     {$$=mung1($1, $2); $$->type=PIPEFD;}
```

⟨*token declarations* 136c⟩+≡ (55a) ◁134b

```
%token PIPEFD
```

⟨`outcode()` *cases* 136d⟩+≡ (64a) ◁135b 137b▷

```
case PIPEFD:
    emitf(Xpipefd);
    emiti(t->rtype);
    p = emiti(0);
    outcode(c0, eflag);
    emitf(Xexit);
    stuffdot(p);
    break;
```

Uses `Xexit()` 85c, `Xpipefd()` 70a, `c0<>` 63c 148b, `emitf<>` 62d, `emiti<>` 62c, `outcode()` 64a, and `stuffdot()` 74c.

⟨*global* `Fdprefix` 136e⟩≡ (211b)

```
char *Fdprefix = "/fd/";
```

⟨*function* `Xpipefd` 136f⟩≡ (204a)

```
void
Xpipefd(void)
{
    struct Thread *p = runq;
    int pc = p->pc, pid;
    char name[40];
    int pfd[2];
    int sidefd, mainfd;
    if(pipe(pfd)<0){
        Xerror("can't get pipe");
        return;
    }
    if(p->code[pc].i==READ){
        sidefd = pfd[PWR];
        mainfd = pfd[PRD];
    }
    else{
        sidefd = pfd[PRD];
        mainfd = pfd[PWR];
    }
    switch(pid = fork()){
```

```
    case -1:
        Xerror("try again");
        break;
    case 0: // child
        clearwaitpids();
        start(p->code, pc+2, runq->local);
        close(mainfd);
        pushredir(ROPEN, sidefd, p->code[pc].i==READ?1:0);
        runq->ret = 0;
        break;
    default: // parent
        addwaitpid(pid);
        close(sidefd);
        pushredir(ROPEN, mainfd, mainfd); /* isn't this a noop? */
        strcpy(name, Fdprefix);
        inttoascii(name+strlen(name), mainfd);
        pushword(name);
        p->pc = p->code[pc+1].i;
        break;
    }
}
```

Uses EOF 42a, Io 160e, PRD 31d, PWR, ROPEN 78b, UTFmax, Xerror() 69b, addwaitpid() 66e, clearwaitpids() 72a, nil, openfd() 160f, pushredir() 77d, runq 28c, rutf() 162a, start() 36e, and vlook() 32b.

14.4.8 Command substitution: ‘{<cmd>}

Command substitution captures a command's stdout as a list of words. The Bourne shell uses backticks ‘cmd‘, which cannot nest; rc uses ‘{cmd} with braces, which nests naturally.

The output is split into words using $ifs (inter-field separators, defaulting to space, tab, newline). The implementation forks a child connected via a pipe; the parent reads the output, splits on $ifs characters, and pushes the resulting word list onto the argv stack.

⟨*comword rule other cases* 137a⟩+≡ (56g) ◁136b 139a▷

```
| ’‘’ brace      {$$=tree1(’‘’, $2);}
```

⟨outcode() *cases* 137b⟩+≡ (64a) ◁136d 139b▷

```
case ’‘’:
    emitf(Xbackq);
    p = emiti(0);
    outcode(c0, false);
    emitf(Xexit);
    stuffdot(p);
    break;
```

Uses Xbackq() 204a, Xexit() 85c, c0<> 63c 148b, emitf<> 62d, emiti<> 62c, false, outcode() 64a, and stuffdot() 74c.

⟨*function* Xbackq 137c⟩≡ (204a)

```
/*
 * Who should wait for the exit from the fork?
 */
void
Xbackq(void)
{
    int n, pid;
    int pfd[2];
    char *stop;
    char utf[UTFmax+1];
    struct Io *f;
    var *ifs = vlook("ifs");
```

```
word *v, *nextv;
Rune r;
//String *word;

stop = ifs->val? ifs->val->word: "";
if(pipe(pfd)<0){
    Xerror("can't make pipe");
    return;
}
switch(pid = fork()){
case -1:
    Xerror("try again");
    close(pfd[PRD]);
    close(pfd[PWR]);
    return;
case 0: // child
    clearwaitpids();
    close(pfd[PRD]);
    start(runq->code, runq->pc+1, runq->local);
    pushredir(ROPEN, pfd[PWR], 1);
    return;
default: // parent
    Exit("TODO", __LOC__);
    //pad: commented because use the string.h s_xxx library
    // that I didn't want to port to goken
    //addwaitpid(pid);
    //close(pfd[PWR]);
    //f = openfd(pfd[PRD]);
    //word = s_new();
    //v = nil;
    ///* rutf requires at least UTFmax+1 bytes in utf */
    //while((n = rutf(f, utf, &r)) != EOF){
    //    utf[n] = '\0';
    //    if(utfutf(stop, utf) == nil)
    //        s_nappend(word, utf, n);
    //    else
    //        /*
    //         * utf/r is an ifs rune (e.g., \t, \n), thus
    //         * ends the current word, if any.
    //         */
    //        if(s_len(word) > 0){
    //            v = newword(s_to_c(word), v);
    //            s_reset(word);
    //        }
    //}
    //if(s_len(word) > 0)
    //    v = newword(s_to_c(word), v);
    //s_free(word);
    //closeio(f);
    //Waitfor(pid, false);
    ///* v points to reversed arglist -- reverse it onto argv */
    //while(v){
    //    nextv = v->next;
    //    v->next = runq->argv->words;
    //    runq->argv->words = v;
    //    v = nextv;
    //}
    //runq->pc = runq->code[runq->pc].i;
    //return;
}
```

```
}
```

Uses PRD 31d, PWR, RFFDG, RFNOTEG, RFPROC, ROPEN 78b, Thread 28b, Xerror() 69b, addwaitpid() 66e, clearwaitpids() 72a, inttoascii(), newword() 29d, nil, pushredir() 77d, runq 28c, setvar() 32a, and start() 36e.

14.4.9 Stringification of variables: $"<foo>

The $" operator converts a list variable into a single string with spaces between elements. If $x is (a b c), then $"x is a b c as a single word. This is the inverse of $ifs-splitting in command substitution: command substitution splits strings into lists, while $" joins lists back into strings.

⟨*comword rule other cases* 139a⟩+≡ (56g) ◁137a

```
|   '"' word        {$$=tree1('"', $2);}
```

⟨outcode() *cases* 139b⟩+≡ (64a) ◁137b

```
case '"':
    emitf(Xmark);
    outcode(c0, eflag);
    emitf(Xqdol);
    break;
```

Uses Xmark() 64c, Xqdol() 139c, c0<> 63c 148b, emitf<> 62d, and outcode() 64a.

⟨*function* Xqdol 139c⟩≡ (201b)

```
void
Xqdol(void)
{
    word *a, *p;
    char *s;
    int n;
    if(count(runq->argv->words)!=1){
        Xerror1("variable name not singleton!");
        return;
    }
    s = runq->argv->words->word;
    deglob(s);
    a = vlook(s)->val;
    poplist();
    n = count(a);
    if(n==0){
        pushword("");
        return;
    }
    for(p = a;p;p = p->next) n+=strlen(p->word);
    s = emalloc(n);
    if(a){
        strcpy(s, a->word);
        for(p = a->next;p;p = p->next){
            strcat(s, " ");
            strcat(s, p->word);
        }
    }
    else
        s[0]='\0';
    pushword(s);
    efree(s);
}
```

Uses Xerror1() 69c, count() 30a, deglob() 120b, efree(), emalloc(), poplist() 64d, pushword() 38b, runq 28c, and vlook() 32b.

14.5 Advanced builtins

These builtins could in principle be external programs (like `echo` and `test` are), but they need access to shell internals: `exec` replaces the shell process, `whatis` inspects variables and functions, `rfork` manipulates namespaces, `flag` queries command-line flags, and `shift` modifies `$*`.

14.5.1 `$ exec`

The `exec` builtin replaces the current shell process with the given command (no fork). It is called from `Xsimple()`[68c] when the first word of the command is `exec`.

14.5.2 `$ whatis`

`execwhatis()`[140] is the shell's introspection tool. Given a name, it checks (in order): is it a variable? a function? a builtin? an executable on `$path`? It prints the definition in a form that can be fed back to `rc`—variables as `x=value`, functions as `fn name {body}`, builtins as `builtin name`.

The subtle `mapfd()` helper is needed because `whatis` writes to fd 1, but fd 1 might have been redirected; `mapfd` walks the redir stack to find where fd 1 actually points.

⟨*function* `execwhatis` 140⟩≡ (208c)

```
void
execwhatis(void){   /* mildly wrong -- should fork before writing */
    word *a, *b, *path;
    var *v;
    struct Builtin *bp;
    char *file;
    struct Io out[1];
    int found, sep;
    a = runq->argv->words->next;
    if(a==0){
        Xerror1("Usage: whatis name ...");
        return;
    }
    setstatus("");
    memset(out, 0, sizeof out);
    out->fd = mapfd(1);
    out->bufp = out->buf;
    out->ebuf = &out->buf[NBUF];
    out->strp = nil;
    for(;a;a = a->next){
        v = vlook(a->word);
        if(v->val){
            pfmt(out, "%s=", a->word);
            if(v->val->next==0)
                pfmt(out, "%q\n", v->val->word);
            else{
                sep='(';
                for(b = v->val;b && b->word;b = b->next){
                    pfmt(out, "%c%q", sep, b->word);
                    sep=' ';
                }
                pfmt(out, ")\n");
            }
            found = 1;
        }
        else
            found = 0;
```

```
        v = gvlook(a->word);
        if(v->fn)
            pfmt(out, "fn %q %s\n", v->name, v->fn[v->pc-1].s);
        else{
            for(bp = builtins;bp->name;bp++)
                if(strcmp(a->word, bp->name)==0){
                    pfmt(out, "builtin %s\n", a->word);
                    break;
                }
            if(!bp->name){
                for(path = searchpath(a->word); path;
                    path = path->next){
                    if(path->word[0] != '\0')
                        file = appfile(path->word, a->word);
                    else
                        file = strdup(a->word);
                    if(Executable(file)){
                        pfmt(out, "%s\n", file);
                        free(file);
                        break;
                    }
                    free(file);
                }
                if(!path && !found){
                    pfmt(err, "%s: not found\n", a->word);
                    setstatus("not found");
                }
            }
        }
    }
    poplist();
    flush(err);
}
```

Uses `Builtin` 100a, `Executable()` 141b, `Io` 160e, `NBUF` 160d, `Xerror1()` 69c, `appfile<>()` 102a, `builtins` 208c, `err` 153b, `flush()` 160h, `gvlook()` 32e, `mapfd()` 141a, `nil`, `pfmt()` 163b, `poplist()` 64d, `runq` 28c, `searchpath()` 71b, `setstatus()` 65d, and `vlook()` 32b.

⟨*function* `mapfd` 141a⟩≡ (208c)

```
int
mapfd(int fd)
{
    redir *rp;
    for(rp = runq->redir;rp;rp = rp->next){
        switch(rp->type){
        case RCLOSE:
            if(rp->from==fd)
                fd=-1;
            break;
        case RDUP:
        case ROPEN:
            if(rp->to==fd)
                fd = rp->from;
            break;
        }
    }
    return fd;
}
```

Uses `RCLOSE` 135f, `RDUP` 135e, `ROPEN` 78b, and `runq` 28c.

⟨*function* `Executable` 141b⟩≡ (208c)

```
bool
Executable(char *file)
{
    Dir *statbuf;
    bool ret;

    statbuf = dirstat(file);
    if(statbuf == nil)
        return false;
    ret = ((statbuf->mode&0111)!=0 && (statbuf->mode&DMDIR)==0);
    free(statbuf);
    return ret;
}
```

Uses DMDIR, false, and nil.

14.5.3 $ rfork

The rfork builtin exposes Plan 9's rfork(2) system call, which selectively shares or unshares resources between parent and child. The flags control which namespaces to copy or clear: n/N for the mount namespace, e/E for environment, s for note group, f/F for file descriptors. This is the Plan 9 equivalent of Linux containers and namespaces—but from 1992.

The environment flag is particularly interesting. As mentioned in Chapter 10, all rc variables are automatically exported to /env/—there is no export command like in bash. But this raises the question: how do you *prevent* a variable from being visible to children? The answer is rfork: rfork e creates a *copy* of the current /env/, so subsequent changes in either the parent or the child are invisible to the other; rfork E creates an entirely *empty* /env/, starting with a clean slate. Note that the lowercase flags (n, e, f) copy the resource, while the uppercase flags (N, E, F) clear it entirely. When called with no arguments, rfork defaults to RFENVG|RFNAMEG|RFNOTEG, creating new environment, mount namespace, and note groups all at once.

⟨*function* execnewpgrp 142⟩≡ (208c)

```
void
execnewpgrp(void)
{
    int arg;
    char *s;
    switch(count(runq->argv->words)){
    case 1:
        arg = RFENVG|RFNAMEG|RFNOTEG;
        break;
    case 2:
        arg = 0;
        for(s = runq->argv->words->next->word;*s;s++) switch(*s){
        default:
            goto Usage;
        case 'n':
            arg|=RFNAMEG;  break;
        case 'N':
            arg|=RFCNAMEG;
            break;
        //pad: commented for goken
        //case 'm':
        //    arg|=RFNOMNT;  break;
        case 'e':
            arg|=RFENVG;   break;
        case 'E':
            arg|=RFCENVG;  break;
        case 's':
```

```
            arg|=RFNOTEG;  break;
        case 'f':
            arg|=RFFDG;    break;
        case 'F':
            arg|=RFCFDG;   break;
        }
        break;
    default:
    Usage:
        pfmt(err, "Usage: %s [fnesFNEm]\n", runq->argv->words->word);
        setstatus("rfork usage");
        poplist();
        return;
    }
    if(rfork(arg)==-1){
        pfmt(err, "rc: %s failed\n", runq->argv->words->word);
        setstatus("rfork failed");
    }
    else
        setstatus("");
    poplist();
}
```

Uses RFCENVG, RFCFDG, RFCNAMEG, RFENVG, RFFDG, RFNAMEG, RFNOMNT, RFNOTEG, count() 30a, err 153b, pfmt() 163b, poplist() 64d, runq 28c, and setstatus() 65d.

14.5.4 $ flag

The flag builtin queries and sets rc's own command-line flags at runtime. flag e + enables -e (exit on error), flag e - disables it, and flag e alone tests whether it is set (via the exit status).

⟨*function* execflag 143⟩≡ (208c)

```
void
execflag(void)
{
    char *letter, *val;

    switch(count(runq->argv->words)){
    case 2:
        setstatus(flag[(uchar)runq->argv->words->next->word[0]]?"":"flag not set");
        break;
    case 3:
        letter = runq->argv->words->next->word;
        val = runq->argv->words->next->next->word;
        if(strlen(letter)==1){
            if(strcmp(val, "+")==0){
                flag[(uchar)letter[0]] = flagset;
                break;
            }
            if(strcmp(val, "-")==0){
                flag[(uchar)letter[0]] = 0;
                break;
            }
        }
    default:
        Xerror1("Usage: flag [letter] [+-]");
        return;
    }
    poplist();
}
```

Uses `Xerror1()` 69c, `count()` 30a, `flag` 34c, `flagset` 35b, `poplist()` 64d, `runq` 28c, and `setstatus()` 65d.

14.5.5 $ shift

`shift` removes the first `n` elements (default 1) from `$*`, the positional parameters. It modifies the variable in place by walking the linked list and freeing the removed nodes.

⟨*function* `execshift` 144a⟩≡ (208c)

```
void
execshift(void)
{
    int n;
    word *a;
    var *star;
    switch(count(runq->argv->words)){
    default:
        pfmt(err, "Usage: shift [n]\n");
        setstatus("shift usage");
        poplist();
        return;
    case 2:
        n = atoi(runq->argv->words->next->word);
        break;
    case 1:
        n = 1;
        break;
    }
    star = vlook("*");
    for(;n && star->val;--n){
        a = star->val->next;
        efree(star->val->word);
        efree((char *)star->val);
        star->val = a;
        star->changed = true;
    }
    setstatus("");
    poplist();
}
```

Uses `count()` 30a, `efree()`, `err` 153b, `pfmt()` 163b, `poplist()` 64d, `runq` 28c, `setstatus()` 65d, `true`, and `vlook()` 32b.

14.5.6 $ finit

The `finit` builtin re-reads function definitions from the `/env` filesystem. This is Plan 9-specific: when another process (e.g., a parent shell) defines a function, it appears as a file `/env/fn#name`. `finit` scans `/env` for these `fn#` files and executes them to import the function definitions into the current shell. It uses a small hand-crafted bytecode loop (`rdfns`) similar to the bootstrap code, with `Xrdfn` reading one function file per iteration.

⟨*global* `rdfns` 144b⟩≡ (208c)

```
union Code rdfns[4];
```

⟨*function* `execfinit` 144c⟩≡ (208c)

```
void
execfinit(void)
{
    //TODO: commented for goken for now because Linux does not have a /env
    // probably need to move to Plan9.c
    //static bool first = true;
```

```
    //if(first){
    //    rdfns[0].i = 1;
    //    rdfns[1].f = Xrdfn;
    //    rdfns[2].f = Xjump;
    //    rdfns[3].i = 1;
    //    first = false;
    //}
    //Xpopm(); // pop_list()
    //envdir = open("/env", 0);
    //if(envdir<0){
    //    pfmt(err, "rc: can't open /env: %r\n");
    //    return;
    //}
    //start(rdfns, 1, runq->local);
}
```

Uses `Xjump()` 88c, `Xpopm()` 91a, `Xrdfn()`, `envdir` 110, `err` 153b, `false`, `pfmt()` 163b, `rdfns`, `runq` 28c, `start()` 36e, and `true`.

⟨*function* `Xrdfn` 145⟩≡ (211b)

```
void
Xrdfn(void)
{
    int f, len;
    Dir *e;
    char envname[Maxenvname];
    static Dir *ent, *allocent;
    static int nent;

    for(;;){
        if(nent == 0){
            free(allocent);
            nent = dirread(envdir, &allocent);
            ent = allocent;
        }
        if(nent <= 0)
            break;
        while(nent){
            e = ent++;
            nent--;
            len = e->length;
            if(len && strncmp(e->name, "fn#", 3)==0){
                snprint(envname, sizeof envname, "/env/%s", e->name);
                if((f = open(envname, 0))>=0){
                    execcmds(openfd(f));
                    return;
                }
            }
        }
    }
    close(envdir);
    Xreturn();
}
```

Chapter 15

Conclusion

You now know how the Plan 9 shell `rc` works, to the smallest details, and more generally how many shells work.

Despite being only about 6700 lines of C (plus a small Yacc grammar), `rc` implements a complete programming language: a hand-written lexer, a Yacc parser, a bytecode compiler, and a bytecode interpreter—the same architecture found in much larger language implementations like Python or Java. Along the way, you have seen how `rc` uses only a handful of system calls—`rfork()`, `exec()`, `wait()`, `pipe()`, `open()`, `close()`, `dup()`—to implement pipes, redirections, subshells, and background jobs. The beauty of the UNIX process model is that these few primitives are enough to build a surprisingly powerful command-line environment.

15.1 Connections to other books

A shell sits at the intersection of many topics covered in other books of Principia Softwarica:

- The Kernel book [Pad14] explains what lies behind the system calls you have seen throughout this book: `rfork()`, `exec()`, `wait()`, `chdir()`, and the `/env` and `/dev/cons` device files. Understanding the kernel will deepen your understanding of how pipes are implemented at the OS level, how the `#!` shebang mechanism works, and how signals (notes) are delivered.
- The Compiler book [Pad16b] uses the same compilation pipeline as `rc`—lexing, parsing, bytecode generation—but for the C language and targeting real machine code rather than a bytecode interpreter. Having seen `rc`'s simpler compiler first should make the C compiler easier to follow.
- The Generators book [**?**] (Lex and Yacc) explains what the `yacc`-generated parser in `rc` actually does behind the scenes: the shift-reduce algorithm, the parse tables, and the interaction between `yylex()` and `yyparse()`.
- The Builder book [Pad16a] describes `mk`, the Plan 9 build system, which relies heavily on `rc` for executing recipes. Features like `-e` (exit on error) and `-c` (command from string) exist largely because `mk` needs them.

15.2 Missing features

`rc` is deliberately minimal. Many features that other shells provide are instead handled by the Plan 9 window manager `rio`:

- *Command-line editing and history:* Under `rio`, you can scroll back in the terminal window and copy-paste previous commands. There is no need for a readline library.
- *Filename completion:* Provided by `rio`'s terminal, not by `rc` itself.

- *Job control (*`bg`*,* `fg`*,* `C-z`*):* Less necessary when you have multiple windows. The `&` operator and `wait $apid` cover most use cases.
- *Aliases:* Not needed because `fn` (functions) serve the same purpose and are stored persistently in the environment.

This separation of concerns—keeping the shell simple and letting the terminal handle interactive features—is a characteristic Plan 9 design decision. It contrasts with shells like `bash` or `zsh`, which include tens of thousands of lines of code for features that `rc` delegates elsewhere.

Appendix A

Debugging

A.1 AST dumper

⟨*global* nl 148a⟩≡ (206b)

```
char nl='\n';  /* change to semicolon for bourne-proofing */
```

Uses nl 148a.

⟨*constant* c0 148b⟩≡ (206b)

```
#define c0 t->child[0]
```

⟨*constant* c1 148c⟩≡ (206b)

```
#define c1 t->child[1]
```

⟨*constant* c2 148d⟩≡ (206b)

```
#define c2 t->child[2]
```

⟨*function* pcmd 148e⟩≡ (206b)

```
void
pcmd(io *f, tree *t)
{
    if(t==nil)
        return;
    assert(f != nil);

    switch(t->type){
    case '$': pfmt(f, "$%t", c0); break;
    case '"': pfmt(f, "$\"%t", c0); break;
    case '&': pfmt(f, "%t&", c0); break;
    case '^': pfmt(f, "%t^%t", c0, c1); break;
    case '`': pfmt(f, "`%t", c0); break;
    case ANDAND:pfmt(f, "%t && %t", c0, c1); break;
    case OROR: pfmt(f, "%t || %t", c0, c1); break;
    case BANG: pfmt(f, "! %t", c0); break;
    case BRACE: pfmt(f, "{%t}", c0); break;
    case COUNT: pfmt(f, "$#%t", c0); break;
    case FN: pfmt(f, "fn %t %t", c0, c1); break;
    case IF: pfmt(f, "if%t%t", c0, c1); break;
    case NOT: pfmt(f, "if not %t", c0); break;
    case PCMD:
    case PAREN: pfmt(f, "(%t)", c0); break;
    case SUB: pfmt(f, "$%t(%t)", c0, c1); break;
    case SIMPLE: pfmt(f, "%t", c0); break;
    case SUBSHELL: pfmt(f, "@ %t", c0); break;
    case SWITCH: pfmt(f, "switch %t %t", c0, c1); break;
    case TWIDDLE: pfmt(f, "~ %t %t", c0, c1); break;
```

```
case WHILE:  pfmt(f, "while %t%t", c0, c1); break;

case ARGLIST:
    if(c0==nil)
        pfmt(f, "%t", c1);
    else if(c1==nil)
        pfmt(f, "%t", c0);
    else
        pfmt(f, "%t %t", c0, c1);
    break;
case ';':
    if(c0){
        if(c1)
            pfmt(f, "%t%c%t", c0, nl, c1);
        else pfmt(f, "%t", c0);
    }
    else pfmt(f, "%t", c1);
    break;
case WORDS:
    if(c0)
        pfmt(f, "%t ", c0);
    pfmt(f, "%t", c1);
    break;
case FOR:
    pfmt(f, "for(%t", c0);
    if(c1)
        pfmt(f, " in %t", c1);
    pfmt(f, ")%t", c2);
    break;
case WORD:
    if(t->quoted)
        pfmt(f, "%Q", t->str);
    else pdeglob(f, t->str);
    break;
case DUP:
    if(t->rtype==DUPFD)
        pfmt(f, ">[%d=%d]", t->fd1, t->fd0); /* yes, fd1, then fd0; read lex.c */
    else
        pfmt(f, ">[%d=]", t->fd0);
    pfmt(f, "%t", c1);
    break;
case PIPEFD:
case REDIR:
    switch(t->rtype){
    case HERE:
        pchr(f, '<');
    case READ:
    case RDWR:
        pchr(f, '<');
        if(t->rtype==RDWR)
            pchr(f, '>');
        if(t->fd0!=0)
            pfmt(f, "[%d]", t->fd0);
        break;
    case APPEND:
        pchr(f, '>');
    case WRITE:
        pchr(f, '>');
        if(t->fd0!=1)
            pfmt(f, "[%d]", t->fd0);
```

```
            break;
        }
        pfmt(f, "%t", c0);
        if(c1)
            pfmt(f, " %t", c1);
        break;
    case '=':
        pfmt(f, "%t=%t", c0, c1);
        if(c2)
            pfmt(f, " %t", c2);
        break;
    case PIPE:
        pfmt(f, "%t|", c0);
        if(t->fd1==0){
            if(t->fd0!=1)
                pfmt(f, "[%d]", t->fd0);
        }
        else pfmt(f, "[%d=%d]", t->fd0, t->fd1);
        pfmt(f, "%t", c1);
        break;

    default:
        pfmt(f, "bad cmd %d %p %p %p", t->type, c0, c1, c2);
        break;
    }
}
```

Uses APPEND 50a, DUPFD 25a, HERE, RDWR 194c, READ, WRITE, assert, c0<> 63c 148b, c1<> 63d 148c, c2<> 63e 148d, nil, nl 148a, pchr() 161a, pdeglob() 150a, and pfmt() 163b.

⟨*function* pdeglob 150a⟩≡ (206b)

```
void
pdeglob(io *f, char *s)
{
    while(*s){
        if(*s==GLOB)
            s++;
        pchr(f, *s++);
    }
}
```

Uses GLOB 31c and pchr() 161a.

A.2 Bytecode generator trace: rc -r

⟨main() *debug runq in interpreter loop* 150b⟩≡ (38c)

```
if(flag['r'])
    pfnc(err, runq);
```

⟨*global* fname 150c⟩≡ (207b)

```
struct{
    void (*f)(void);
    char *name;
} fname[] = {
    Xappend, "Xappend",
    Xassign, "Xassign",
    Xasync, "Xasync",
    Xbackq, "Xbackq",
    Xbang, "Xbang",
    Xcase, "Xcase",
```

```
    Xclose, "Xclose",
    Xconc, "Xconc",
    Xcount, "Xcount",
    Xdelfn, "Xdelfn",
    Xdelhere, "Xdelhere",
    Xdol, "Xdol",
    Xdup, "Xdup",
    Xeflag, "Xeflag",
    (void (*)(void))Xerror, "Xerror",
    Xexit, "Xexit",
    Xfalse, "Xfalse",
    Xfn, "Xfn",
    Xfor, "Xfor",
    Xglob, "Xglob",
    Xif, "Xif",
    Xifnot, "Xifnot",
    Xjump, "Xjump",
    Xlocal, "Xlocal",
    Xmark, "Xmark",
    Xmatch, "Xmatch",
    Xpipe, "Xpipe",
    Xpipefd, "Xpipefd",
    Xpipewait, "Xpipewait",
    Xpopm, "Xpopm",
    Xpopredir, "Xpopredir",
    Xqdol, "Xqdol",
    Xrdcmds, "Xrdcmds",
    Xrdfn, "Xrdfn",
    Xrdwr, "Xrdwr",
    Xread, "Xread",
    Xreturn, "Xreturn",
    Xsimple, "Xsimple",
    Xsub, "Xsub",
    Xsubshell, "Xsubshell",
    Xtrue, "Xtrue",
    Xunlocal, "Xunlocal",
    Xwastrue, "Xwastrue",
    Xword, "Xword",
    Xwrite, "Xwrite",
    0
};
```

Uses Xappend() 80c, Xassign() 95a, Xasync(), Xbackq() 204a, Xbang() 75c, Xcase() 91c, Xclose() 135g, Xconc() 99a, Xcount() 97b, Xdelfn() 93b, Xdelhere() 130b, Xdol() 96b, Xdup() 135c, Xeflag() 127a, Xerror() 69b, Xexit() 85c, Xfalse() 75a, Xfn() 93a, Xfor() 89a, Xglob() 119a, Xif() 86b, Xifnot() 87c, Xjump() 88c, Xlocal() 95b, Xmark() 64c, Xmatch() 75e, Xpipe() 137c, Xpipefd() 70a, Xpipewait() 84d, Xpopm() 91a, Xpopredir() 79b, Xqdol() 139c, Xrdcmds() 39a, Xrdfn(), Xrdwr() 133c, Xread() 80a, Xreturn() 82d, Xsimple() 68c, Xsub() 97d, Xsubshell(), Xtrue() 74e, Xunlocal() 95c, Xwastrue() 87a, Xword() 64b, Xwrite() 79a, __anon_struct_7 150c, and fname 150c.

⟨*function* pfnc 151⟩≡ (207b)

```
void
pfnc(io *fd, thread *t)
{
    int i;
    void (*fn)(void) = t->code[t->pc].f;
    list *a;

    pfmt(fd, "pid %d cycle %p %d ", getpid(), t->code, t->pc);
    for (i = 0; fname[i].f; i++)
        if (fname[i].f == fn) {
            pstr(fd, fname[i].name);
```

```
            break;
        }
    if (!fname[i].f)
        pfmt(fd, "%p", fn);
    for (a = t->argv; a; a = a->next)
        pfmt(fd, " (%v)", a->words);
    pchr(fd, '\n');
    flush(fd);
}
```

Uses `flush()` 160h, `fname` 150c, `pchr()` 161a, `pfmt()` 163b, and `pstr()` 164a.

A.3 Printing subprocesses status: `rc -s`

⟨`Xrdcmds()` *print status if -s* 152a⟩≡ (39a)

```
if(flag['s'] && !truestatus())
    pfmt(err, "status=%v\n", vlook("status")->val);
```

Uses `err` 153b, `flag` 34c, `pfmt()` 163b, `truestatus()` 66a, and `vlook()` 32b.

A.4 Printing commands: `rc -x`

⟨`Xsimple()` *if -x* 152b⟩≡ (68c)

```
if(flag['x'])
    pfmt(err, "%v\n", p->argv->words); /* wrong, should do redirs */
```

Uses `err` 153b, `flag` 34c, and `pfmt()` 163b.

A.5 Printing characters: `rc -v`

⟨`getnext()` *if not EOF but verbose mode, print character read* 152c⟩≡ (41)

```
else
    if(flag['V'] || ndot>=2 && flag['v'])
        pchr(err, c);
```

Uses `err` 153b, `flag` 34c, `ndot` 105a, and `pchr()` 161a.

A.6 Printing all characters: `rc -V`

Appendix B

Error Management

⟨*constant* ERRMAX 153a⟩≡ (194c)
```
#define ERRMAX 128
```

⟨*global* err 153b⟩≡ (210b)
```
io *err;
```

⟨*global* nerror 153c⟩≡ (210b)
```
int nerror;  /* number of errors encountered during compilation */
```

⟨Xrdcmds() *flush errors and reset error count* 153d⟩≡ (39a)
```
flush(err);
nerror = 0;
```
Uses err 153b, flush() 160h, and nerror 153c.

⟨*function* yyerror 153e⟩≡ (205a)
```
void
yyerror(char *m)
{
    pfmt(err, "rc: ");
    if(runq->cmdfile && !runq->iflag)
        pfmt(err, "%s:%d: ", runq->cmdfile, runq->lineno);
    else if(runq->cmdfile)
        pfmt(err, "%s: ", runq->cmdfile);
    else if(!runq->iflag)
        pfmt(err, "line %d: ", runq->lineno);

    if(tok[0] && tok[0]!='\n')
        pfmt(err, "token %q: ", tok);
    pfmt(err, "%s\n", m);
    flush(err);
    lastword = false;
    lastdol = false;

    while(lastc!='\n' && lastc!=EOF)
        advance();

    nerror++;
    setvar("status", newword(m, (word *)nil));
}
```
Uses EOF 42a, advance() 45a, err 153b, false, flush() 160h, lastc 153f, lastdol 49b, lastword 51e, nerror 153c, newword() 29d, nil, pfmt() 163b, runq 28c, setvar() 32a, and tok 47c.

⟨*global* lastc 153f⟩≡ (199)
```
int lastc;
```

⟨advance() *save future in lastc* 154a⟩≡ (45a)

```
lastc = future;
```

Uses future 44d and lastc 153f.

⟨*function* panic 154b⟩≡ (210b)

```
void
panic(char *s, int n)
{
    pfmt(err, "rc: ");
    pfmt(err, s, n);
    pchr(err, '\n');
    flush(err);
    Abort();
}
```

Uses Abort(), err 153b, flush() 160h, pchr() 161a, and pfmt() 163b.

⟨*function* Abort 154c⟩≡ (210b)

```
void
Abort(void)
{
    pfmt(err, "aborting\n");
    flush(err);
    Exit("aborting", __LOC__);
}
```

Uses Updenv(), getstatus() 65e, setstatus() 65d, and truestatus() 66a.

⟨*function* Exit 154d⟩≡ (211b)

```
void
Exit(char *stat, char* _LOC)
{
    USED(_LOC);
    Updenv();
    setstatus(stat);
    exits(truestatus() ? "" : getstatus());
}
```

Appendix C

Utilities

⟨*function* _efgfmt 155a⟩≡ (211b)

```
/* avoid loading any floating-point library code */
//@Scheck: weird, probably linker trick
int _efgfmt(Fmt *)
{
    return -1;
}
```

C.1 Memory management

⟨*function* emalloc 155b⟩≡ (208a)

```
void *
emalloc(long n)
{
    void *p = Malloc(n);

    if(p==nil)
        panic("Can't malloc %d bytes", n);
    return p;
}
```

⟨*function* Malloc 155c⟩≡ (211b)

```
void*
Malloc(ulong n)
{
    return mallocz(n, 1);
}
```

⟨*function* efree 155d⟩≡ (208a)

```
void
efree(void *p)
{
    if(p)
        free(p);
    else pfmt(err, "free 0\n");
}
```

C.2 Command-line arguments

⟨*global* cmdname 155e⟩≡ (198a)

```
char *cmdname;
```

⟨*global* flagarg 156a⟩≡ (198a)

```
static char *flagarg="";
```

Uses flagarg<> 156a.

⟨*global* reason 156b⟩≡ (198a)

```
static int reason;
```

⟨*constant* RESET 156c⟩≡ (198a)

```
#define RESET 1
```

⟨*constant* FEWARGS 156d⟩≡ (198a)

```
#define FEWARGS 2
```

⟨*constant* FLAGSYN 156e⟩≡ (198a)

```
#define FLAGSYN 3
```

⟨*constant* BADFLAG 156f⟩≡ (198a)

```
#define BADFLAG 4
```

⟨*global* badflag 156g⟩≡ (198a)

```
static int badflag;
```

⟨*function* getflags 156h⟩≡ (198a)

```
int
getflags(int argc, char *argv[], char *flags, int stop)
{
    char *s;
    int i, j, c, count;
    flagarg = flags;
    if(cmdname==0)
        cmdname = argv[0];

    i = 1;
    while(i!=argc){
        if(argv[i][0] != '-' || argv[i][1] == '\0'){
            if(stop)  /* always true in rc */
                return argc;
            i++;
            continue;
        }
        s = argv[i]+1;
        while(*s){
            c=*s++;
            count = scanflag(c, flags);
            if(count==-1)
                return -1;
            if(flag[c]){ reason = RESET; badflag = c; return -1; }
            if(count==0){
                flag[c] = flagset;
                if(*s=='\0'){
                    for(j = i+1;j<=argc;j++)
                        argv[j-1] = argv[j];
                    --argc;
                }
            }
            else{
                if(*s=='\0'){
                    for(j = i+1;j<=argc;j++)
                        argv[j-1] = argv[j];
                    --argc;
```

```
                    s = argv[i];
                }
                if(argc-i<count){
                    reason = FEWARGS;
                    badflag = c;
                    return -1;
                }
                reverse(argv+i, argv+argc);
                reverse(argv+i, argv+argc-count);
                reverse(argv+argc-count+1, argv+argc);
                argc-=count;
                flag[c] = argv+argc+1;
                flag[c][0] = s;
                s="";
            }
        }
    }
    return argc;
}
```

Uses FEWARGS<> 156d, RESET<> 156c, badflag<> 156g, cmdname 155e, flag 34c, flagarg<> 156a, flagset 35b, reason<> 156b, reverse<>() 157a, and scanflag<>() 157b.

⟨*function* reverse 157a⟩≡ (198a)

```
static void
reverse(char **p, char **q)
{
    char *t;
    for(;p<q;p++,--q){ t=*p; *p=*q; *q = t; }
}
```

⟨*function* scanflag 157b⟩≡ (198a)

```
static int
scanflag(int c, char *f)
{
    int fc, count;
    if(0<=c && c<NFLAG)
        while(*f){
            if(*f==' '){
                f++;
                continue;
            }
            fc=*f++;
            if(*f==':'){
                f++;
                if(*f<'0' || '9'<*f){ reason = FLAGSYN; return -1; }
                count = 0;
                while('0'<=*f && *f<='9') count = count*10+*f++-'0';
            }
            else
                count = 0;
            if(*f=='['){
                do{
                    f++;
                    if(*f=='\0'){ reason = FLAGSYN; return -1; }
                }while(*f!=']');
                f++;
            }
            if(c==fc)
                return count;
        }
```

```
    reason = BADFLAG;
    badflag = c;
    return -1;
}
```

Uses BADFLAG<> 156f, FLAGSYN<> 156e, NFLAG 35a, badflag<> 156g, and reason<> 156b.

⟨*function* usage 158⟩≡ (198a)

```
void
usage(char *tail)
{
    char *s, *t, c;
    int count, nflag = 0;
    switch(reason){
    case RESET:
        errs("Flag -");
        errc(badflag);
        errs(": set twice\n");
        break;
    case FEWARGS:
        errs("Flag -");
        errc(badflag);
        errs(": too few arguments\n");
        break;
    case FLAGSYN:
        errs("Bad argument to getflags!\n");
        break;
    case BADFLAG:
        errs("Illegal flag -");
        errc(badflag);
        errc('\n');
        break;
    }
    errs("Usage: ");
    errs(cmdname);
    for(s = flagarg;*s;){
        c=*s;
        if(*s++==' ')
            continue;
        if(*s==':'){
            s++;
            count = 0;
            while('0'<=*s && *s<='9') count = count*10+*s++-'0';
        }
        else count = 0;
        if(count==0){
            if(nflag==0)
                errs(" [-");
            nflag++;
            errc(c);
        }
        if(*s=='['){
            s++;
            while(*s!=']' && *s!='\0') s++;
            if(*s==']')
                s++;
        }
    }
    if(nflag)
        errs("]");
    for(s = flagarg;*s;){
```

```
        c=*s;
        if(*s++==' ')
            continue;
        if(*s==':'){
            s++;
            count = 0;
            while('0'<=*s && *s<='9') count = count*10+*s++-'0';
        }
        else count = 0;
        if(count!=0){
            errs(" [-");
            errc(c);
            if(*s=='['){
                s++;
                t = s;
                while(*s!=']' && *s!='\0') s++;
                errs(" ");
                errn(t, s-t);
                if(*s==']')
                    s++;
            }
            else
                while(count--) errs(" arg");
            errs("]");
        }
        else if(*s=='['){
            s++;
            while(*s!=']' && *s!='\0') s++;
            if(*s==']')
                s++;
        }
    }
    if(tail){
        errs(" ");
        errs(tail);
    }
    errs("\n");
    Exit("bad flags", "getflags.c");
}
```

Uses BADFLAG<> 156f, Exit() 154c, FEWARGS<> 156d, FLAGSYN<> 156e, RESET<> 156c, badflag<> 156g, cmdname 155e, errc<>() 160c, errn<>() 159a, errs<>() 159b, flagarg<> 156a, and reason<> 156b.

⟨*function* errn 159a⟩≡ (198a)

```
static void
errn(char *s, int count)
{
    while(count){ errc(*s++); --count; }
}
```

Uses errc<>() 160c.

⟨*function* errs 159b⟩≡ (198a)

```
static void
errs(char *s)
{
    while(*s) errc(*s++);
}
```

Uses errc<>() 160c.

⟨*constant NBUF (rc/getflags.c)* 159c⟩≡ (198a)

```
#define NBUF 80
```

⟨*global* buf 160a⟩≡ (198a)

```
static char buf[NBUF];
```

Uses NBUF<> 159c.

⟨*global* bufp 160b⟩≡ (198a)

```
static char *bufp = buf;
```

Uses buf<> 160a and bufp<> 160b.

⟨*function* errc 160c⟩≡ (198a)

```
static void
errc(int c)
{
    *bufp++=c;
    if(bufp==&buf[NBUF] || c=='\n'){
        write(2, buf, bufp-buf);
        bufp = buf;
    }
}
```

Uses NBUF<> 159c, Write() 208a, buf<> 160a, and bufp<> 160b.

C.3 Buffered IO

⟨*constant* NBUF 160d⟩≡ (194b)

```
#define NBUF 512
```

⟨*struct* Io 160e⟩≡ (194b)

```
struct Io {
    fdt fd;
    byte *bufp, *ebuf, *strp;
    byte buf[NBUF];
};
```

Uses NBUF 160d.

⟨*function* openfd 160f⟩≡ (198b)

```
io*
openfd(fdt fd)
{
    io *f = new(struct Io);
    f->fd = fd;
    f->bufp = f->ebuf = f->buf;
    f->strp = nil;
    return f;
}
```

Uses nil.

⟨*enum* MiscConstants 160g⟩≡ (198b)

```
enum { Stralloc = 100, };
```

⟨*function* flush 160h⟩≡ (198b)

```
void
flush(io *f)
{
    int n;

    if(f->strp){
        n = f->ebuf - f->strp;
        f->strp = realloc(f->strp, n+Stralloc+1);
```

```
        if(f->strp==0)
            panic("Can't realloc %d bytes in flush!", n+Stralloc+1);
        f->bufp = f->strp + n;
        f->ebuf = f->bufp + Stralloc;
        memset(f->bufp, '\0', Stralloc+1);
    }
    else{
        n = f->bufp-f->buf;
        if(n && write(f->fd, f->buf, n) != n){
            write(2, "Write error\n", 12);
            if(ntrap)
                dotrap();
        }
        f->bufp = f->buf;
        f->ebuf = f->buf+NBUF;
    }
}
```

Uses NBUF 160d, Stralloc<> 160g, Write() 208a, dotrap(), ntrap 205b, and panic() 154b.

⟨*function* pchr 161a⟩≡ (209b)

```
void
pchr(io *b, int c)
{
    if(b->bufp==b->ebuf)
        fullbuf(b, c);
    else *b->bufp++=c;
}
```

Uses fullbuf() 161b.

⟨*function* fullbuf 161b⟩≡ (198b)

```
int
fullbuf(io *f, int c)
{
    flush(f);
    return *f->bufp++=c;
}
```

Uses flush() 160h.

⟨*function* rchr 161c⟩≡ (198b)

```
int
rchr(io *b)
{
    if(b->bufp==b->ebuf)
        return emptybuf(b);
    return *b->bufp++;
}
```

Uses emptybuf() 161d.

⟨*function* emptybuf 161d⟩≡ (198b)

```
int
emptybuf(io *f)
{
    int n;
    if(f->fd==-1 || (n = read(f->fd, f->buf, NBUF))<=0) return EOF;
    f->bufp = f->buf;
    f->ebuf = f->buf + n;
    return *f->bufp++;
}
```

Uses EOF 42a, NBUF 160d, and Read() 208a.

⟨*function* rutf 162a⟩≡ (198b)

```
int
rutf(io *b, char *buf, Rune *r)
{
    int n, i, c;

    c = rchr(b);
    if(c == EOF)
        return EOF;
    *buf = c;
    if(c < Runesync){
        *r = c;
        return 1;
    }
    for(i = 1; (c = rchr(b)) != EOF; ){
        buf[i++] = c;
        buf[i] = 0;
        if(fullrune(buf, i)){
            n = chartorune(r, buf);
            b->bufp -= i - n; /* push back unconsumed bytes */
            assert(b->fd == -1 || b->bufp > b->buf);
            return n;
        }
    }
    /* at eof */
    b->bufp -= i - 1;   /* consume 1 byte */
    *r = Runeerror;
    return runetochar(buf, r);
}
```

Uses `EOF` 42a, `Runeerror`, `Runesync`, `assert`, and `rchr()` 161c.

⟨*function* closeio 162b⟩≡ (198b)

```
void
closeio(io *io)
{
    if(io->fd>=0)
        close(io->fd);
    if(io->strp)
        efree(io->strp);
    efree(io);
}
```

Uses `efree()`.

⟨*function* openstr 162c⟩≡ (198b)

```
io*
openstr(void)
{
    io *f = new(struct Io);

    f->fd = -1;
    f->bufp = f->strp = emalloc(Stralloc+1);
    f->ebuf = f->bufp + Stralloc;
    memset(f->bufp, '\0', Stralloc+1);
    return f;
}
```

Uses `Stralloc<>` 160g and `emalloc()`.

⟨*function* opencore 162d⟩≡ (198b)

```
/*
```

```
 * Open a corebuffer to read.  EOF occurs after reading len
 * characters from buf.
 */

io*
opencore(char *s, int len)
{
    io *f = new(struct Io);
    uchar *buf = emalloc(len);

    f->fd = -1 /*open("/dev/null", 0)*/;
    f->bufp = f->strp = buf;
    f->ebuf = buf+len;
    Memcpy(buf, s, len);
    return f;
}
```

Uses `Memcpy()` and `emalloc()`.

C.4 Format

⟨*global* `pfmtnest` 163a⟩≡ (209b)

```
int pfmtnest = 0;
```

Uses `pfmtnest` 163a.

⟨*function* `pfmt` 163b⟩≡ (209b)

```
void
pfmt(io *f, char *fmt, ...)
{
    va_list ap;
    char err[ERRMAX];

    va_start(ap, fmt);
    pfmtnest++;
    for(;*fmt;fmt++) {
        if(*fmt!='%') {
            pchr(f, *fmt);
            continue;
        }
        if(*++fmt == '\0')  /* "blah%"? */
            break;
        switch(*fmt){
        case 'c':
            pchr(f, va_arg(ap, int));
            break;
        case 'd':
            pdec(f, va_arg(ap, int));
            break;
        case 'o':
            poct(f, va_arg(ap, unsigned int));
            break;
        case 'p':
            pptr(f, va_arg(ap, void*));
            break;
        case 'Q':
            pquo(f, va_arg(ap, char *));
            break;
        case 'q':
            pwrd(f, va_arg(ap, char *));
```

```
            break;
        case 's':
            pstr(f, va_arg(ap, char *));
            break;

        case 'r':
            errstr(err, sizeof err); pstr(f, err);
            break;

        // rc specific, TODO LP split here
        case 't':
            pcmd(f, va_arg(ap, struct Tree *));
            break;
        case 'v':
            pval(f, va_arg(ap, struct Word *));
            break;

        default:
            pchr(f, *fmt);
            break;
        }
    }
    va_end(ap);
    if(--pfmtnest==0)
        flush(f);
}
```

Uses ERRMAX, Tree 194c, Word 194c, flush() 160h, pchr() 161a, pcmd() 148e, pdec() 164b, pfmtnest 163a, poct() 165a, pptr() 165d, pquo() 165b, pstr() 164a, pval() 165e, and pwrd() 165c.

⟨*function* pstr 164a⟩≡ (209b)

```
void
pstr(io *f, char *s)
{
    if(s==0)
        s="(null)";
    while(*s) pchr(f, *s++);
}
```

Uses pchr() 161a.

⟨*function* pdec 164b⟩≡ (209b)

```
void
pdec(io *f, int n)
{
    if(n<0){
        n=-n;
        if(n>=0){
            pchr(f, '-');
            pdec(f, n);
            return;
        }
        /* n is two's complement minimum integer */
        n = 1-n;
        pchr(f, '-');
        pdec(f, n/10);
        pchr(f, n%10+'1');
        return;
    }
    if(n>9)
        pdec(f, n/10);
    pchr(f, n%10+'0');
```

```
}
```

Uses pchr() 161a and pdec() 164b.

⟨*function* poct 165a⟩≡ (209b)

```
void
poct(io *f, unsigned int n)
{
    if(n>7)
        poct(f, n>>3);
    pchr(f, (n&7)+'0');
}
```

Uses pchr() 161a and poct() 165a.

⟨*function* pquo 165b⟩≡ (209b)

```
void
pquo(io *f, char *s)
{
    pchr(f, '\'');
    for(;*s;s++)
        if(*s=='\'')
            pfmt(f, "'''");
        else pchr(f, *s);
    pchr(f, '\'');
}
```

Uses pchr() 161a and pfmt() 163b.

⟨*function* pwrd 165c⟩≡ (209b)

```
void
pwrd(io *f, char *s)
{
    char *t;
    for(t = s;*t;t++) if(*t >= 0 && needsrcquote(*t)) break;
    if(t==s || *t)
        pquo(f, s);
    else pstr(f, s);
}
```

Uses pquo() 165b and pstr() 164a.

⟨*function* pptr 165d⟩≡ (209b)

```
void
pptr(io *f, void *v)
{
    int n;
    uintptr p;

    p = (uintptr)v;
    if(sizeof(uintptr) == sizeof(uvlong) && p>>32)
        for(n = 60;n>=32;n-=4) pchr(f, "0123456789ABCDEF"[(p>>n)&0xF]);

    for(n = 28;n>=0;n-=4) pchr(f, "0123456789ABCDEF"[(p>>n)&0xF]);
}
```

Uses pchr() 161a.

⟨*function* pval 165e⟩≡ (209b)

```
void
pval(io *f, word *a)
{
    if(a){
        while(a->next && a->next->word){
```

```
            pwrd(f, (char *)a->word);
            pchr(f, ' ');
            a = a->next;
        }
        pwrd(f, (char *)a->word);
    }
}
```

Uses `pchr()` 161a and `pwrd()` 165c.

C.5 String conversions

⟨*global* `bp` 166a⟩≡ (208a)

```
char *bp;
```

⟨*function* `inttoascii` 166b⟩≡ (208a)

```
void
inttoascii(char *s, long n)
{
    bp = s;
    iacvt(n);
    *bp='\0';
}
```

Uses `NFD<>` 155d and `dir` 166c.

⟨*function* `iacvt` 166c⟩≡ (208a)

```
static void
iacvt(int n)
{
    if(n<0){
        *bp++='-';
        n=-n; /* doesn't work for n==-inf */
    }
    if(n/10)
        iacvt(n/10);
    *bp++=n%10+'0';
}
```

Uses `DirEntryWrapper` 208a and `NFD<>` 155d.

C.6 Misc

⟨*function* `Write` 166d⟩≡ (208a)

⟨*function* `Read` 166e⟩≡ (208a)

⟨*function* `Seek` 166f⟩≡ (208a)

⟨*function* `Creat` 166g⟩≡ (208a)

```
int
Creat(char *file)
{
    return create(file, 1, 0666L);
}
```

⟨*function* `Dup` 166h⟩≡ (208a)

⟨*function* Memcpy 167⟩≡ (208a)

```
void
Memcpy(void *a, void *b, long n)
{
    memmove(a, b, n);
}
```

Appendix D

Examples of rc scripts

D.1 /rc/lib/rcmain

This is the initialization script sourced by the bootstrap bytecodes (see Chapter 12). It sets up default values for $home, $ifs, $prompt, and $path, calls finit to import shell functions from the environment, and then dispatches based on how rc was invoked: with -c (run a command string), with -i (interactive, source the user's profile and read from the console), or with a script file argument.

```
# rcmain: Plan 9 version
if(~ $#home 0) home=/
if(~ $#ifs 0) ifs='
'

switch($#prompt){
case 0
 prompt=('% ' ' ')
case 1
 prompt=($prompt ' ')
}

if(~ $rcname ?.out) prompt=('broken! ' ' ')

if(flag p) path=/bin
if not{
 finit
 if(~ $#path 0) path=(. /bin)
}

fn sigexit

if(! ~ $#cflag 0){
 if(flag l && /bin/test -r $home/lib/profile) . $home/lib/profile
 status=''
 eval $cflag
}
if not if(flag i){
 if(flag l && /bin/test -r $home/lib/profile) . $home/lib/profile
 status=''
```

```
 if(! ~ $#* 0) . $*
 . -i '#d/0'
}
if not if(~ $#* 0) . '#d/0'
if not{
 status=''
 . $*
}
exit $status
```

D.2 /usr/glenda/lib/profile

This is Glenda's (the default Plan 9 user) login profile, sourced by `rcmain` when `rc` is invoked with the `-l` (login shell) flag (the `flag l` test in `rcmain` above). It uses `bind` to overlay the user's personal `bin` directories onto `/bin` (so user scripts and binaries are found automatically), sets up the font and mail filesystem (`upas/fs`), and then dispatches based on `$service`: on a terminal, it starts the window manager `rio`; on a CPU server, it connects to the terminal's devices via `/mnt/term`.

```
bind -a $home/bin/rc /bin
bind -a $home/bin/$cputype /bin
bind -c tmp /tmp
if(! syscall create /tmp/xxx 1 0666 >[2]/dev/null)
 ramfs # in case we're running off a cd
font = /lib/font/bit/pelm/euro.9.font
upas/fs
fn cd { builtin cd $* && awd }  # for acme
switch($service){
case terminal
 plumber
 echo -n accelerated > '#m/mousectl'
 echo -n 'res 3' > '#m/mousectl'
 prompt=('term% ' ' ')
 fn term%{ $* }
 exec rio -i riostart
case cpu
 if (test -e /mnt/term/mnt/wsys) { # rio already running
  bind -a /mnt/term/mnt/wsys /dev
  if(test -w /dev/label)
   echo -n $sysname > /dev/label
 }
 bind /mnt/term/dev/cons /dev/cons
 bind /mnt/term/dev/consctl /dev/consctl
 bind -a /mnt/term/dev /dev
 prompt=('cpu% ' ' ')
 fn cpu%{ $* }
 news
 if (! test -e /mnt/term/mnt/wsys) { # cpu call from drawterm
  font=/lib/font/bit/pelm/latin1.8.font
  exec rio
```

```
 }
case con
 prompt=('cpu% ' ' ')
 news
}
```

Appendix E

Shell Companion Utilities

This appendix presents the source code of a few small programs that are closely tied to the shell. They are too simple to deserve their own book, but they are used so frequently in shell scripts and interactive sessions that understanding their internals is worthwhile. For more general Plan 9 utilities, see UTILITIES book [Pad25].

E.1 echo

`echo` concatenates its arguments separated by spaces, followed by a newline. The `-n` flag suppresses the trailing newline. Note that it builds the entire output in a single buffer and writes it with one `write()` call, so the output is atomic—important when multiple processes write to the same file descriptor.

⟨*function* `main`*(echo)* 171⟩≡ (189a)

```
void
main(int argc, char *argv[])
{
    int nflag;
    int i, len;
    char *buf, *p;

    nflag = 0;
    if(argc > 1 && strcmp(argv[1], "-n") == 0)
        nflag = 1;

    len = 1;
    for(i = 1+nflag; i < argc; i++)
        len += strlen(argv[i])+1;

    buf = malloc(len);
    if(buf == 0)
        exits("no memory");

    p = buf;
    for(i = 1+nflag; i < argc; i++){
        strcpy(p, argv[i]);
        p += strlen(p);
        if(i < argc-1)
            *p++ = ' ';
    }

    if(!nflag)
        *p++ = '\n';

    if(write(1, buf, p-buf) < 0){
        fprint(2, "echo: write error: %r\n");
```

```
        exits("write error");
    }

    exits((char *)nil);
}
```

E.2 pwd

`pwd` simply calls `getwd()` and prints the result. In Plan 9, the current directory is a per-process kernel state, not an environment variable like `\$PWD` in some UNIX shells.

⟨*function main (misc/pwd.c)* 172a⟩≡ (189b)

```
/*
 * Print working (current) directory
 */
void
main(int argc, char *argv[])
{
    char pathname[512];

    USED(argc, argv);
    if(getwd(pathname, sizeof(pathname)) == 0) {
        fprint(2, "pwd: %r\n");
        exits("getwd");
    }
    print("%s\n", pathname);
    exits(nil);
}
```

E.3 test

`test` evaluates conditional expressions, used heavily in shell scripts (e.g., `if(test -f foo) ...`). It is an external program, not a builtin—unlike in `bash` where `test` and `[` are built in. The implementation is a small recursive-descent parser with three precedence levels: `-o` (or), `-a` (and), and `!` (not), followed by the primary tests (`-f`, `-d`, `-r`, `-eq`, etc.).

⟨*function* `EQ` 172b⟩≡ (189c)

```
#define EQ(a,b) ((tmp=a)==0?0:(strcmp(tmp,b)==0))
```

⟨*global* `ap` 172c⟩≡ (189c)

```
int ap;
```

⟨*global* `ac` 172d⟩≡ (189c)

```
int ac;
```

⟨*global* `av` 172e⟩≡ (189c)

```
char **av;
```

⟨*global* `tmp` 172f⟩≡ (189c)

```
static char *tmp;
```

⟨*function main (misc/test.c)* 173a⟩≡ (189c)

```
void
main(int argc, char *argv[])
{
    int r;
    char *c;

    ac = argc; av = argv; ap = 1;
    if(EQ(argv[0],"[")) {
        if(!EQ(argv[--ac],"]"))
            synbad("] missing","");
    }
    argv[ac] = 0;
    if (ac<=1)
        exits("usage");
    r = e();
    /*
     * nice idea but short-circuit -o and -a operators may have
     * not consumed their right-hand sides.
     */
    if(false && (c = nxtarg(1)) != nil)
        synbad("unexpected operator/operand: ", c);
    exits(r? nil : "false");
}
```

⟨*function* nxtarg 173b⟩≡ (189c)

```
char *
nxtarg(int mt)
{
    if(ap>=ac){
        if(mt){
            ap++;
            return(0);
        }
        synbad("argument expected","");
    }
    return(av[ap++]);
}
```

⟨*function* nxtintarg 173c⟩≡ (189c)

```
bool
nxtintarg(int *pans)
{
    if(ap<ac && isint(av[ap], pans)){
        ap++;
        return true;
    }
    return false;
}
```

⟨*function* e 173d⟩≡ (189c)

```
int
e(void)
{
    int p1;

    p1 = e1();
    if (EQ(nxtarg(1), "-o"))
        return(p1 || e());
    ap--;
```

```
    return(p1);
}
```

⟨*function* e1 174a⟩≡ (189c)

```
int
e1(void)
{
    int p1;

    p1 = e2();
    if (EQ(nxtarg(1), "-a"))
        return (p1 && e1());
    ap--;
    return(p1);
}
```

⟨*function* e2 174b⟩≡ (189c)

```
int
e2(void)
{
    if (EQ(nxtarg(0), "!"))
        return(!e2());
    ap--;
    return(e3());
}
```

⟨*function* e3 174c⟩≡ (189c)

```
bool
e3(void)
{
    int p1, int1, int2;
    char *a, *p2;

    a = nxtarg(0);
    if(EQ(a, "(")) {
        p1 = e();
        if(!EQ(nxtarg(0), ")"))
            synbad(") expected","");
        return(p1);
    }

    if(EQ(a, "-A"))
        return(hasmode(nxtarg(0), DMAPPEND));

    if(EQ(a, "-L"))
        return(hasmode(nxtarg(0), DMEXCL));

    if(EQ(a, "-T"))
        return(hasmode(nxtarg(0), DMTMP));

    if(EQ(a, "-f"))
        return(isreg(nxtarg(0)));

    if(EQ(a, "-d"))
        return(isdir(nxtarg(0)));

    if(EQ(a, "-r"))
        return(tio(nxtarg(0), 4));

    if(EQ(a, "-w"))
```

```
    return(tio(nxtarg(0), 2));

if(EQ(a, "-x"))
    return(tio(nxtarg(0), 1));

if(EQ(a, "-e"))
    return(tio(nxtarg(0), 0));

if(EQ(a, "-c"))
    return false;

if(EQ(a, "-b"))
    return false;

if(EQ(a, "-u"))
    return false;

if(EQ(a, "-g"))
    return false;

if(EQ(a, "-s"))
    return(fsizep(nxtarg(0)));

if(EQ(a, "-t"))
    if(ap>=ac)
        return(isatty(1));
    else if(nxtintarg(&int1))
        return(isatty(int1));
    else
        synbad("not a valid file descriptor number ", "");

if(EQ(a, "-n"))
    return(!EQ(nxtarg(0), ""));
if(EQ(a, "-z"))
    return(EQ(nxtarg(0), ""));

p2 = nxtarg(1);
if (p2==0)
    return(!EQ(a,""));
if(EQ(p2, "="))
    return(EQ(nxtarg(0), a));

if(EQ(p2, "!="))
    return(!EQ(nxtarg(0), a));

if(EQ(p2, "-older"))
    return(isolder(nxtarg(0), a));

if(EQ(p2, "-ot"))
    return(isolderthan(nxtarg(0), a));

if(EQ(p2, "-nt"))
    return(isnewerthan(nxtarg(0), a));

if(!isint(a, &int1))
    synbad("unexpected operator/operand: ", p2);

if(nxtintarg(&int2)){
    if(EQ(p2, "-eq"))
        return(int1==int2);
```

```
        if(EQ(p2, "-ne"))
            return(int1!=int2);
        if(EQ(p2, "-gt"))
            return(int1>int2);
        if(EQ(p2, "-lt"))
            return(int1<int2);
        if(EQ(p2, "-ge"))
            return(int1>=int2);
        if(EQ(p2, "-le"))
            return(int1<=int2);
    }

    synbad("unknown operator ",p2);
    return false;  /* to shut ken up */
}
```

⟨*function* tio 176a⟩≡ (189c)

```
bool
tio(char *a, int f)
{
    return access (a, f) >= 0;
}
```

⟨*function* hasmode 176b⟩≡ (189c)

```
/*
 * note that the name strings pointed to by Dir members are
 * allocated with the Dir itself (by the same call to malloc),
 * but are not included in sizeof(Dir), so copying a Dir won't
 * copy the strings it points to.
 */
bool
hasmode(char *f, ulong m)
{
    bool r;
    Dir *dir;

    dir = dirstat(f);
    if (dir == nil)
        return false;
    r = (dir->mode & m) != 0;
    free(dir);
    return r;
}
```

⟨*function* isdir 176c⟩≡ (189c)

```
bool
isdir(char *f)
{
    return hasmode(f, DMDIR);
}
```

⟨*function* isreg 176d⟩≡ (189c)

```
bool
isreg(char *f)
{
    bool r;
    Dir *dir;

    dir = dirstat(f);
    if (dir == nil)
```

```
        return false;
    r = (dir->mode & DMDIR) == 0;
    free(dir);
    return r;
}
```

⟨*function* isatty 177a⟩≡ (189c)

```
int
isatty(int fd)
{
    int r;
    Dir *d1, *d2;

    d1 = dirfstat(fd);
    d2 = dirstat("/dev/cons");
    if (d1 == nil || d2 == nil)
        r = 0;
    else
        r = d1->type == d2->type && d1->dev == d2->dev &&
            d1->qid.path == d2->qid.path;
    free(d1);
    free(d2);
    return r;
}
```

⟨*function* fsizep 177b⟩≡ (189c)

```
bool
fsizep(char *f)
{
    bool r;
    Dir *dir;

    dir = dirstat(f);
    if (dir == nil)
        return false;
    r = dir->length > 0;
    free(dir);
    return r;
}
```

⟨*function* synbad 177c⟩≡ (189c)

```
void
synbad(char *s1, char *s2)
{
    int len;

    write(2, "test: ", 6);
    if ((len = strlen(s1)) != 0)
        write(2, s1, len);
    if ((len = strlen(s2)) != 0)
        write(2, s2, len);
    write(2, "\n", 1);
    exits("bad syntax");
}
```

⟨*function* isint 177d⟩≡ (189c)

```
int
isint(char *s, int *pans)
{
    char *ep;
```

```
    *pans = strtol(s, &ep, 0);
    return (*ep == 0);
}
```

⟨*function* isolder 178⟩≡ (189c)

```
bool
isolder(char *pin, char *f)
{
    int r, rel;
    ulong n, m;
    char *p = pin;
    Dir *dir;

    dir = dirstat(f);
    if (dir == nil)
        return false;

    /* parse time */
    n = 0;
    rel = 0;
    while(*p){
        m = strtoul(p, &p, 0);
        switch(*p){
        case 0:
            n = m;
            break;
        case 'y':
            m *= 12;
            /* fall through */
        case 'M':
            m *= 30;
            /* fall through */
        case 'd':
            m *= 24;
            /* fall through */
        case 'h':
            m *= 60;
            /* fall through */
        case 'm':
            m *= 60;
            /* fall through */
        case 's':
            n += m;
            p++;
            rel = 1;
            break;
        default:
            synbad("bad time syntax, ", pin);
        }
    }
    if (!rel)
        m = n;
    else{
        m = time(0);
        if (n > m)  /* before epoch? */
            m = 0;
        else
            m -= n;
    }
```

```
    r = dir->mtime < m;
    free(dir);
    return r;
}
```

⟨*function* isolderthan 179a⟩≡ (189c)

```
int
isolderthan(char *a, char *b)
{
    int r;
    Dir *ad, *bd;

    ad = dirstat(a);
    bd = dirstat(b);
    if (ad == nil || bd == nil)
        r = 0;
    else
        r = ad->mtime > bd->mtime;
    free(ad);
    free(bd);
    return r;
}
```

⟨*function* isnewerthan 179b⟩≡ (189c)

```
int
isnewerthan(char *a, char *b)
{
    int r;
    Dir *ad, *bd;

    ad = dirstat(a);
    bd = dirstat(b);
    if (ad == nil || bd == nil)
        r = 0;
    else
        r = ad->mtime < bd->mtime;
    free(ad);
    free(bd);
    return r;
}
```

E.4 sh, the simple shell

This is a minimal shell, useful during early stages of porting Plan 9 to a new architecture when `rc` is not yet available. It supports basic command execution, pipes, redirections, and backgrounding, but has no variables, functions, or control flow. Comparing it to `rc` helps appreciate how much complexity the full shell adds.

⟨*constant* MAXLINE 179c⟩≡ (190)

```
#define MAXLINE 200  /* maximum line length */
```

⟨*constant* WORD 179d⟩≡ (190)

```
#define WORD 256  /* token code for words */
```

⟨*constant* EOF 179e⟩≡ (190)

```
#define EOF -1   /* token code for end of file */
```

⟨*function* ispunct 179f⟩≡ (190)

```
#define ispunct(c)  (c=='|' || c=='&' || c==';' || c=='<' || \
                     c=='>' || c=='(' || c==')' || c=='\n')
```

⟨*function* isspace 180a⟩≡ (190)
```
#define isspace(c)  (c==' ' || c=='\t')
```

⟨*function* execute 180b⟩≡ (190)
```
#define execute(np)  (ignored = (np? (*(np)->op)(np) : 0))
```

⟨*struct* Node 180c⟩≡ (190)
```
struct Node {   /* parse tree node */
    int (*op)(Node *);  /* operator function */
    Node *args[2];  /* argument nodes */
    char *argv[100]; /* argument pointers */
    char *io[3];  /* i/o redirection */
};
```

⟨*global* nodes 180d⟩≡ (190)
```
Node nodes[25];  /* node pool */
```

⟨*global* nfree 180e⟩≡ (190)
```
Node *nfree;   /* next available node */
```

⟨*global* strspace 180f⟩≡ (190)
```
char strspace[10*MAXLINE]; /* string storage */
```

⟨*global* sfree 180g⟩≡ (190)
```
char *sfree;   /* next free character in strspace */
```

⟨*global* t 180h⟩≡ (190)
```
int t;   /* current token code */
```

⟨*global* token 180i⟩≡ (190)
```
char  *token;   /* current token text (in strspace) */
```

⟨*global* putback 180j⟩≡ (190)
```
int putback = 0;  /* lookahead */
```

⟨*global* status 180k⟩≡ (190)
```
char status[256];  /* exit status of most recent command */
```

⟨*global* cflag 180l⟩≡ (190)
```
int cflag = 0;  /* command is argument to sh */
```

⟨*global* tflag 180m⟩≡ (190)
```
int tflag = 0;  /* read only one line */
```

⟨*global* interactive 180n⟩≡ (190)
```
bool interactive = false; /* prompt */
```

⟨*global* cflagp 180o⟩≡ (190)
```
char *cflagp;  /* command line for cflag */
```

⟨*global* path 180p⟩≡ (190)
```
char *path[] ={"/bin", 0};
```

⟨*global* ignored 180q⟩≡ (190)
```
int ignored;
```

⟨*function main (sh/sh.c)* 181a⟩≡ (190)

```
void
main(int argc, char *argv[])
{
    Node *np;

    if(argc>1 && strcmp(argv[1], "-t")==0)
        tflag++;
    else if(argc>2 && strcmp(argv[1], "-c")==0){
        cflag++;
        cflagp = argv[2];
    }else if(argc>1){
        close(0);
        if(open(argv[1], 0) != 0){
            error(": can't open", argv[1]);
            exits("argument");
        }
    }else
        interactive = true;
    for(;;){
        if(interactive)
            fprint(2, "%d$ ", getpid());
        nfree = nodes;
        sfree = strspace;
        if((t=gettoken()) == EOF)
            break;
        if(t != '\n')
            if(np = list())
                execute(np);
            else
                error("syntax error", "");
        while(t!=EOF && t!='\n') /* flush syntax errors */
            t = gettoken();
    }
    exits(status);
}
```

⟨*function* alloc 181b⟩≡ (190)

```
/* alloc - allocate for op and return a node */
Node*
alloc(int (*op)(Node *))
{
    if(nfree < nodes+sizeof(nodes)){
        nfree->op = op;
        nfree->args[0] = nfree->args[1] = 0;
        nfree->argv[0] = nfree->argv[1] = 0;
        nfree->io[0] = nfree->io[1] = nfree->io[2] = 0;
        return nfree++;
    }
    error("node storage overflow", "");
    exits("node storage overflow");
    return nil;
}
```

⟨*function* builtin 181c⟩≡ (190)

```
/* builtin - check np for builtin command and, if found, execute it */
bool
builtin(Node *np)
{
    int n = 0;
```

```
    char name[MAXLINE];
    Waitmsg *wmsg;

    if(np->argv[1])
        n = strtoul(np->argv[1], 0, 0);
    if(strcmp(np->argv[0], "cd") == 0){
        if(chdir(np->argv[1]? np->argv[1] : "/") == -1)
            error(": bad directory", np->argv[0]);
        return true;
    }else if(strcmp(np->argv[0], "exit") == 0)
        exits(np->argv[1]? np->argv[1] : status);
    else if(strcmp(np->argv[0], "bind") == 0){
        if(np->argv[1]==0 || np->argv[2]==0)
            error("usage: bind new old", "");
        else if(bind(np->argv[1], np->argv[2], 0)==-1)
            error("bind failed", "");
        return true;
//#ifdef asdf
// }else if(strcmp(np->argv[0], "unmount") == 0){
//  if(np->argv[1] == 0)
//   error("usage: unmount [new] old", "");
//  else if(np->argv[2] == 0){
//   if(unmount((char *)0, np->argv[1]) == -1)
//    error("unmount:", "");
//  }else if(unmount(np->argv[1], np->argv[2]) == -1)
//   error("unmount", "");
//  return true;
//#endif
    }else if(strcmp(np->argv[0], "wait") == 0){
        while((wmsg = wait()) != nil){
            strncpy(status, wmsg->msg, sizeof(status)-1);
            if(n && wmsg->pid==n){
                n = 0;
                free(wmsg);
                break;
            }
            free(wmsg);
        }
        if(n)
            error("wait error", "");
        return true;
    }else if(strcmp(np->argv[0], "rfork") == 0){
        char *p;
        int mask;

        p = np->argv[1];
        if(p == 0 || *p == 0)
            p = "ens";
        mask = 0;

        while(*p)
            switch(*p++){
            case 'n': mask |= RFNAMEG; break;
            case 'N': mask |= RFCNAMEG; break;
            case 'e': mask |= RFENVG; break;
            case 'E': mask |= RFCENVG; break;
            case 's': mask |= RFNOTEG; break;
            case 'f': mask |= RFFDG; break;
            case 'F': mask |= RFCFDG; break;
            case 'm': mask |= RFNOMNT; break;
```

```
            default: error(np->argv[1], "bad rfork flag");
            }
        rfork(mask);

        return OK_1;
    }else if(strcmp(np->argv[0], "exec") == 0){
        redirect(np);
        if(np->argv[1] == (char *) 0)
            return 1;
        exec(np->argv[1], &np->argv[1]);
        n = np->argv[1][0];
        if(n!='/' && n!='#' && (n!='.' || np->argv[1][1]!='/'))
            for(n = 0; path[n]; n++){
                sprint(name, "%s/%s", path[n], np->argv[1]);
                exec(name, &np->argv[1]);
            }
        error(": not found", np->argv[1]);
        return true;
    }
    // no builtin found
    return false;
}
```

⟨*function* command 183a⟩≡ (190)

```
/* command - ( list ) [ ( < | > | >> ) word ]* | simple */
Node*
command(void)
{
    Node *np;

    if(t != '(')
        return simple();
    np = alloc(xsubshell);
    t = gettoken();
    if((np->args[0]=list())==0 || t!=')')
        return nil;
    while((t=gettoken())=='<' || t=='>')
        if(!setio(np))
            return nil;
    return np;
}
```

⟨*function* getch 183b⟩≡ (190)

```
/* getch - get next, possibly pushed back, input character */
int
getch(void)
{
    unsigned char c;
    static int done=0;

    if(putback){
        c = putback;
        putback = 0;
    }else if(tflag){
        if(done || read(0, &c, 1)!=1){
            done = 1;
            return EOF;
        }
        if(c == '\n')
            done = 1;
```

```
    }else if(cflag){
        if(done)
            return EOF;
        if((c=*cflagp++) == 0){
            done = 1;
            c = '\n';
        }
    }else if(read(0, &c, 1) != 1)
        return EOF;
    return c;
}
```

⟨*function* gettoken 184a⟩≡ (190)

```
/* gettoken - get next token into string space, return token code */
int
gettoken(void)
{
    int c;

    while((c = getch()) != EOF)
        if(!isspace(c))
            break;
    if(c==EOF || ispunct(c))
        return c;
    token = sfree;
    do{
        if(sfree >= strspace+sizeof(strspace) - 1){
            error("string storage overflow", "");
            exits("string storage overflow");
        }
        *sfree++ = c;
    }while((c=getch()) != EOF && !ispunct(c) && !isspace(c));
    *sfree++ = 0;
    putback = c;
    return WORD;
}
```

⟨*function* list 184b⟩≡ (190)

```
/* list - pipeline ( ( ; | & ) pipeline )* [ ; | & ]  (not LL(1), but ok) */
Node*
list(void)
{
    Node *np, *np1;

    np = alloc(0);
    if((np->args[1]=pipeline()) == 0)
        return nil;
    while(t==';' || t=='&'){
        np->op = (t==';')? xwait : xnowait;
        t = gettoken();
        if(t==')' || t=='\n') /* tests ~first(pipeline) */
            break;
        np1 = alloc(0);
        np1->args[0] = np;
        if((np1->args[1]=pipeline()) == 0)
            return nil;
        np = np1;
    }
    if(np->op == 0)
        np->op = xwait;
```

```
    return np;
}
```

⟨*function* error 185a⟩≡ (190)

```
/* error - print error message s, prefixed by t */
void
error(char *s, char *t)
{
    char buf[256];

    fprint(2, "%s%s", t, s);
    errstr(buf, sizeof buf);
    fprint(2, ": %s\n", buf);
}
```

⟨*function* pipeline 185b⟩≡ (190)

```
/* pipeline - command ( | command )* */
Node*
pipeline(void)
{
    Node *np, *np1;

    if((np=command()) == 0)
        return nil;
    while(t == '|'){
        np1 = alloc(xpipeline);
        np1->args[0] = np;
        t = gettoken();
        if((np1->args[1]=command()) == 0)
            return nil;
        np = np1;
    }
    return np;
}
```

⟨*function* redirect 185c⟩≡ (190)

```
/* redirect - redirect i/o according to np->io[] values */
void
redirect(Node *np)
{
    int fd;

    if(np->io[0]){
        if((fd = open(np->io[0], 0)) < 0){
            error(": can't open", np->io[0]);
            exits("open");
        }
        dup(fd, 0);
        close(fd);
    }
    if(np->io[1]){
        if((fd = create(np->io[1], 1, 0666L)) < 0){
            error(": can't create", np->io[1]);
            exits("create");
        }
        dup(fd, 1);
        close(fd);
    }
    if(np->io[2]){
        if((fd = open(np->io[2], 1)) < 0 && (fd = create(np->io[2], 1, 0666L)) < 0){
```

```
            error(": can't write", np->io[2]);
            exits("write");
        }
        dup(fd, 1);
        close(fd);
        seek(1, 0, 2);
    }
}
```

⟨*function* setio 186a⟩≡ (190)

```
/* setio - ( < | > | >> ) word; fill in np->io[] */
error0
setio(Node *np)
{
    if(t == '<'){
        t = gettoken();
        np->io[0] = token;
    }else if(t == '>'){
        t = gettoken();
        if(t == '>'){
            t = gettoken();
            np->io[2] = token;
            }else
            np->io[1] = token;
    }else
        return ERROR_0;
    if(t != WORD)
        return ERROR_0;
    return OK_1;
}
```

⟨*function* simple 186b⟩≡ (190)

```
/* simple - word ( [ < | > | >> ] word )* */
Node*
simple(void)
{
    Node *np;
    int n = 1;

    if(t != WORD)
        return nil;
    np = alloc(xsimple);
    np->argv[0] = token;
    while((t = gettoken())==WORD || t=='<' || t=='>')
        if(t == WORD)
            np->argv[n++] = token;
        else if(!setio(np))
            return nil;
    np->argv[n] = 0;
    return np;
}
```

⟨*function* xpipeline 186c⟩≡ (190)

```
/* xpipeline - execute cmd | cmd */
int
xpipeline(Node *np)
{
    int pid, fd[2];

    if(pipe(fd) < 0){
```

```
        error("can't create pipe", "");
        return 0;
    }
    if((pid=fork()) == 0){ /* left side; redirect stdout */
        dup(fd[1], 1);
        close(fd[0]);
        close(fd[1]);
        execute(np->args[0]);
        exits(status);
    }else if(pid == -1){
        error("can't create process", "");
        return 0;
    }
    if((pid=fork()) == 0){ /* right side; redirect stdin */
        dup(fd[0], 0);
        close(fd[0]);
        close(fd[1]);
        pid = execute(np->args[1]); /*BUG: this is wrong sometimes*/
        if(pid > 0)
            while(waitpid()!=pid)
                ;
        exits(nil);
    }else if(pid == -1){
        error("can't create process", "");
        return 0;
    }
    close(fd[0]); /* avoid using up fd's */
    close(fd[1]);
    return pid;
}
```

⟨*function* xsimple 187⟩≡ (190)

```
/* xsimple - execute a simple command */
int
xsimple(Node *np)
{
    char name[MAXLINE];
    int pid, i;

    if(builtin(np))
        return 0;

    if(pid = fork()){
        if(pid == -1)
            error(": can't create process", np->argv[0]);
        return pid;
    }
    redirect(np); /* child process */
    exec(np->argv[0], &np->argv[0]);
    i = np->argv[0][0];

    if(i!='/' && i!='#' && (i!='.' || np->argv[0][1]!='/'))
        for(i = 0; path[i]; i++){
            sprint(name, "%s/%s", path[i], np->argv[0]);
            exec(name, &np->argv[0]);
        }
    error(": not found", np->argv[0]);
    exits("not found");
    return -1;  // suppress compiler warnings
}
```

⟨*function* xsubshell 188a⟩≡ (190)

```
/* xsubshell - execute (cmd) */
int
xsubshell(Node *np)
{
    int pid;

    if(pid = fork()){
        if(pid == -1)
            error("can't create process", "");
        return pid;
    }
    redirect(np); /* child process */
    execute(np->args[0]);
    exits(status);
    return -1;  // suppress compiler warnings
}
```

⟨*function* xnowait 188b⟩≡ (190)

```
/* xnowait - execute cmd & */
int
xnowait(Node *np)
{
    int pid;

    execute(np->args[0]);
    pid = execute(np->args[1]);
    if(interactive)
        fprint(2, "%d\n", pid);
    return 0;
}
```

⟨*function* xwait 188c⟩≡ (190)

```
/* xwait - execute cmd ; */
int xwait(Node *np)
{
    int pid;
    Waitmsg *wmsg;

    execute(np->args[0]);
    pid = execute(np->args[1]);
    if(pid > 0){
        while((wmsg = wait()) != nil){
            if(wmsg->pid == pid)
                break;
            free(wmsg);
        }
        if(wmsg == nil)
            error("wait error", "");
        else {
            strncpy(status, wmsg->msg, sizeof(status)-1);
            free(wmsg);
        }
    }
    return 0;
}
```

Appendix F

Extra Code

F.1 misc/

F.1.1 misc/echo.c

⟨misc/echo.c 189a⟩≡
```
#include <u.h>
#include <libc.h>

⟨function main(echo) 171⟩
```

F.1.2 misc/pwd.c

⟨misc/pwd.c 189b⟩≡
```
#include <u.h>
#include <libc.h>

⟨function main (misc/pwd.c) 172a⟩
```

F.1.3 misc/test.c

⟨misc/test.c 189c⟩≡
```
/*
 * POSIX standard
 * test expression
 * [ expression ]
 *
 * Plan 9 additions:
 * -A file exists and is append-only
 * -L file exists and is exclusive-use
 * -T file exists and is temporary
 */

#include <u.h>
#include <libc.h>

⟨function EQ 172b⟩

⟨global ap 172c⟩
⟨global ac 172d⟩
⟨global av 172e⟩
⟨global tmp 172f⟩
```

```
void synbad(char *, char *);
int fsizep(char *);
int isdir(char *);
int isreg(char *);
int isatty(int);
int isint(char *, int *);
int isolder(char *, char *);
int isolderthan(char *, char *);
int isnewerthan(char *, char *);
int hasmode(char *, ulong);
int tio(char *, int);
int e(void), e1(void), e2(void), e3(void);
char *nxtarg(int);
```

⟨*function main (misc/test.c)* 173a⟩

⟨*function* nxtarg 173b⟩

⟨*function* nxtintarg 173c⟩

⟨*function* e 173d⟩

⟨*function* e1 174a⟩

⟨*function* e2 174b⟩

⟨*function* e3 174c⟩

⟨*function* tio 176a⟩

⟨*function* hasmode 176b⟩

⟨*function* isdir 176c⟩

⟨*function* isreg 176d⟩

⟨*function* isatty 177a⟩

⟨*function* fsizep 177b⟩

⟨*function* synbad 177c⟩

⟨*function* isint 177d⟩

⟨*function* isolder 178⟩

⟨*function* isolderthan 179a⟩

⟨*function* isnewerthan 179b⟩

F.2 sh/

F.2.1 sh/sh.c

⟨sh/sh.c 190⟩≡

```
/* sh - simple shell - great for early stages of porting */
#include <u.h>
#include <libc.h>
```

⟨*constant* MAXLINE 179c⟩
⟨*constant* WORD 179d⟩
⟨*constant* EOF 179e⟩
⟨*function* ispunct 179f⟩
⟨*function* isspace 180a⟩
⟨*function* execute 180b⟩

```
typedef struct Node Node;
```
⟨*struct* Node 180c⟩

⟨*global* nodes 180d⟩
⟨*global* nfree 180e⟩
⟨*global* strspace 180f⟩
⟨*global* sfree 180g⟩
⟨*global* t 180h⟩
⟨*global* token 180i⟩
⟨*global* putback 180j⟩
⟨*global* status 180k⟩
⟨*global* cflag 180l⟩
⟨*global* tflag 180m⟩
⟨*global* interactive 180n⟩
⟨*global* cflagp 180o⟩
⟨*global* path 180p⟩
⟨*global* ignored 180q⟩

```
// lexer
int gettoken(void);
int getch(void);
// parser (recursive descent)
Node *list(void);
Node *command(void);
Node *simple(void);
Node *pipeline(void);
int setio(Node *np);
// interpreter
// see execute macro above
int builtin(Node *np);


void redirect(Node *np);

int xpipeline(Node *np);
int xsimple(Node *np);
int xsubshell(Node *np);
int xnowait(Node *np);
int xwait(Node *np);

// error management
void error(char *s, char *t);
// memory management
Node *alloc(int (*op)(Node *));
```

⟨*function main (sh/sh.c)* 181a⟩

⟨*function* alloc 181b⟩

⟨*function* builtin 181c⟩

⟨*function* command 183a⟩

⟨*function* getch 183b⟩

⟨*function* gettoken 184a⟩

⟨*function* list 184b⟩

⟨*function* error 185a⟩

⟨*function* pipeline 185b⟩

⟨*function* redirect 185c⟩

⟨*function* setio 186a⟩

⟨*function* simple 186b⟩

⟨*function* xpipeline 186c⟩

⟨*function* xsimple 187⟩

⟨*function* xsubshell 188a⟩

⟨*function* xnowait 188b⟩

⟨*function* xwait 188c⟩

F.3 rc/

F.3.1 rc/fns.h

⟨rc/fns.h 192⟩≡

```
// for var.c, words.c, tree.c: see rc.h
// see also io.h and getflags.h

// input.c
int getnext(void);
int advance(void);
int nextc(void);
bool nextis(int c);
void pprompt(void);
void kinit(void);
tree *token(char*, int);
tree *klook(char*);

// lex.c
int yylex(void);
//@Scheck: used in syn.y
void skipnl(void);
int idchr(int);

// syn.y
//@Scheck: defined in syn.y and y.tab.c
int yyparse(void);

// executils.c
void start(code*, int, var*);
```

```
void pushlist(void);
void poplist(void);
void pushword(char*);
void popword(void);
void pushredir(int, int, int);

// status.c
char *getstatus(void);
void setstatus(char*);
char* concstatus(char *s, char *t);
bool truestatus(void);

// path.c
word* searchpath(char*);

// env.c
void Updenv(void);
void Vinit(void);

// processes.c
int Waitfor(int, bool);
void Execute(word*,  word*);

// code.c
error0 compile(tree*);
code *codecopy(code*);
void codefree(code*);
void cleanhere(char*);

// trap.c
void dotrap(void);
void Trapinit(void);
bool Eintr(void);
void Noerror(void);

// here.c
void readhere(void);
tree *heredoc(tree*);

// glob.c
void deglob(void*);
void globlist(void);
bool match(void*, void*, int);

// utils.c
#define new(type) ((type *)emalloc(sizeof(type)))
void *emalloc(long);
void efree(void *);
void Memcpy(void*, void*, long);
int Creat(char*);
int Opendir(char*);
int Readdir(int, void*, int);
void Closedir(int);
void inttoascii(char*, long);

// error.c
void panic(char*, int);
void yyerror(char*);
void Exit(char*, char*);
```

F.3.2 rc/getflags.h

⟨rc/getflags.h 194a⟩≡

```
⟨constant NFLAG 35a⟩

extern char **flag[NFLAG];
extern char *flagset[];

int getflags(int, char*[], char*, int);
void usage(char*);
```

F.3.3 rc/io.h

⟨rc/io.h 194b⟩≡

```
⟨constant EOF (rc/io.h) 42a⟩
⟨constant NBUF 160d⟩

⟨struct Io 160e⟩

extern io *err;

// io.c
io *openfd(fdt);
io *openstr(void);
io *opencore(char *, int);
void closeio(io*);
void flush(io*);

int rchr(io*);
int rutf(io*, char*, Rune*);

// fmt.c
void pchr(io*, int);
void pstr(io*, char*);
void pfmt(io*, char*, ...);

// pcmd.c
void pcmd(io*, tree*);
// pfnc.c
void pfnc(io*, thread*);
```

F.3.4 rc/rc.h

⟨rc/rc.h 194c⟩≡

```
/*
 * Assume plan 9 by default; if Unix is defined, assume unix.
 * Please don't litter the code with ifdefs.  The five below should be enough.
 */
#ifndef Unix
/* plan 9 */
#include <u.h>
#include <libc.h>

// could be in trap.c
⟨constant NSIG 111a⟩
⟨constant SIGINT 111b⟩
⟨constant SIGQUIT 111c⟩
```

```
//???
//#define fcntl(fd, op, arg) /* unix compatibility */
//#define F_SETFD
//#define FD_CLOEXEC

//TODO:
#define __LOC__ "NO__LOC__INFO"

#else
#include "unix.h"

// magic incantation for cpp (found by chatGPT)
#define STR2(x) #x
#define STR(x) STR2(x)
#define __LOC__ __FILE__ ":" STR(__LINE__)

#endif

//#ifndef YYPREFIX
//#ifndef PAREN
//#include "x.tab.h"
//pad: better like that, otherwise get some "redefined FOR macro" error
//#endif
//#endif

//#ifndef Unix
//#pragma incomplete word
//#pragma incomplete io
//#endif


// MiscPlan9 is back in plan9.c

#ifndef ERRMAX
⟨constant ERRMAX 153a⟩
#endif

⟨constant YYMAXDEPTH ??⟩

// forward decls
typedef struct Tree tree;
typedef struct Word word;
typedef struct Io io;
typedef union Code code;
typedef struct Var var;
typedef struct List list;
typedef struct Redir redir;
typedef struct Thread thread;
typedef struct Builtin builtin;

⟨struct Tree 25a⟩

// tree.c
tree *newtree(void);
//@Scheck: useful, for syn.y, and not just for tree.c
tree *tree1(int, tree*);
tree *tree2(int, tree*, tree*);
//@Scheck: useful, for syn.y, and not just for tree.c
tree *tree3(int, tree*, tree*, tree*);
tree *mung1(tree*, tree*);
```

```
tree *mung2(tree*, tree*, tree*);
tree *mung3(tree*, tree*, tree*, tree*);
tree *epimung(tree*, tree*);
tree *simplemung(tree*);
void freenodes(void);

⟨struct Code 27c⟩

⟨constant APPEND 50d⟩
⟨constant WRITE 50b⟩
⟨constant READ 50c⟩
⟨constant HERE 129a⟩
⟨constant DUPFD 135a⟩
⟨constant CLOSE 134a⟩
⟨constant RDWR 132b⟩

⟨struct Word 29c⟩

// words.c
word *newword(char *, word *);
word *copywords(word *, word *);
word* copynwords(word *a, word *tail, int n);
void freelist(word *w);
void freewords(word*);
int count(word*);

⟨struct Var 31c⟩

// var.c
var *vlook(char*);
var *gvlook(char*);
var *newvar(char*, var*);
void setvar(char*, word*);

⟨constant NVAR 32c⟩
extern var *gvar[NVAR];  /* hash for globals */

⟨struct Here 129c⟩

⟨constant GLOB 117a⟩

⟨constant PRD 83a⟩
⟨constant PWR 83b⟩

// globals.c
extern int mypid;
extern char *promptstr;
extern int ndot;
// input.c
extern bool doprompt;
extern bool inquote;
extern bool incomm;
extern int lastc;
// lex.c
extern bool lastword;
// error.c
extern int nerror; /* number of errors encountered during compilation */
```

Uses Builtin 100a, Code 25a, Here, Io 160e, List 31b, Redir 77a, Thread 28b, Tree 194c, Var 50d, and Word 194c.

F.3.5 rc/exec.h

⟨rc/exec.h 197a⟩≡

```
/*
 * Definitions used in the interpreter
 */
extern void Xappend(void), Xasync(void), Xbackq(void), Xbang(void), Xclose(void);
extern void Xconc(void), Xcount(void), Xdelfn(void), Xdol(void), Xqdol(void), Xdup(void);
extern void Xexit(void), Xfalse(void), Xfn(void), Xfor(void), Xglob(void);
extern void Xjump(void), Xmark(void), Xmatch(void), Xpipe(void), Xread(void);
extern void Xrdwr(void);
extern void Xrdfn(void), Xreturn(void), Xsubshell(void);
extern void Xtrue(void), Xword(void), Xwrite(void), Xpipefd(void), Xcase(void);
extern void Xlocal(void), Xunlocal(void), Xassign(void), Xsimple(void), Xpopm(void);
extern void Xrdcmds(void), Xwastrue(void), Xif(void), Xifnot(void), Xpipewait(void);
extern void Xdelhere(void), Xpopredir(void), Xsub(void), Xeflag(void), Xsettrue(void);
extern void Xerror(char*);
extern void Xerror1(char*);

⟨struct Redir 77a⟩

⟨constant NSTATUS 84c⟩
⟨constant ROPEN 78b⟩
⟨constant RDUP 135e⟩
⟨constant RCLOSE 135f⟩

⟨struct List 31b⟩

⟨struct Thread 28b⟩

⟨struct Builtin 100a⟩

// globals.c
extern thread *runq;
extern code *codebuf;  /* compiler output */
extern bool eflagok;  /* kludge flag so that -e doesn't exit in startup */
// path.c
extern word nullpath;
// trap.c
extern int ntrap;  /* number of outstanding traps */
// builtins.c
extern struct Builtin builtins[];

// simple.c
void execexec(void);
void execcmds(io *);
// processes.c
int  execforkexec(void);
```

F.3.6 rc/globals.c

⟨rc/globals.c 197b⟩≡

```
⟨includes 18⟩

// was in rc.h
⟨global mypid 116d⟩
⟨global promptstr 43c⟩
⟨global ndot 105a⟩
```

```
// was in exec.h
⟨global runq 28c⟩
⟨global codebuf 27b⟩
⟨global eflagok 126c⟩
```

F.3.7 rc/getflags.c

```
⟨rc/getflags.c 198a⟩≡
  #include "getflags.h"

  extern void Exit(char*, char*);


  static void reverse(char**, char**);
  static int scanflag(int, char*);
  static void errn(char*, int);
  static void errs(char*);
  static void errc(int);

  ⟨global flagset 35b⟩
  ⟨global flag 34c⟩
  ⟨global cmdname 155e⟩
  ⟨global flagarg 156a⟩
  ⟨global reason 156b⟩

  ⟨constant RESET 156c⟩
  ⟨constant FEWARGS 156d⟩
  ⟨constant FLAGSYN 156e⟩
  ⟨constant BADFLAG 156f⟩

  ⟨global badflag 156g⟩

  ⟨function getflags 156h⟩

  ⟨function reverse 157a⟩

  ⟨function scanflag 157b⟩

  ⟨function usage 158⟩

  ⟨function errn 159a⟩

  ⟨function errs 159b⟩
  ⟨constant NBUF (rc/getflags.c) 159c⟩
  ⟨global buf 160a⟩
  ⟨global bufp 160b⟩

  ⟨function errc 160c⟩
```

F.3.8 rc/io.c

```
⟨rc/io.c 198b⟩≡
  ⟨includes 18⟩

  ⟨enum MiscConstants 160g⟩

  // forward decls
  int emptybuf(io*);
```

```
int fullbuf(io*, int);

⟨function rchr 161c⟩

⟨function rutf 162a⟩

⟨function fullbuf 161b⟩

⟨function flush 160h⟩

⟨function openfd 160f⟩

⟨function openstr 162c⟩

⟨function opencore 162d⟩

⟨function closeio 162b⟩

⟨function emptybuf 161d⟩
```

F.3.9 rc/input.c

```
⟨rc/input.c 199⟩≡
  ⟨includes 18⟩
  #include "y.tab.h"

  // was in lex.c

  ⟨global future 44d⟩
  ⟨global doprompt 43b⟩
  ⟨global inquote 51c⟩
  ⟨global incomm 47e⟩

  // was in rc.h
  ⟨global lastc 153f⟩

  // forward decl
  int getnext(void);

  ⟨function nextc 44e⟩
  ⟨function advance 45a⟩
  ⟨function getnext 41⟩

  ⟨function pprompt 43g⟩


  ⟨function nextis 45b⟩


  extern unsigned hash(char *as, int n);

  // was in tree.c

  ⟨function token 52b⟩

  // was in var.c

  ⟨constant NKW 53d⟩
  ⟨struct Kw 53b⟩
```

⟨*global* kw 53c⟩

⟨*function* kenter 53f⟩

⟨*function* kinit 53e⟩

⟨*function* klook 53a⟩

F.3.10 rc/var.c

⟨rc/var.c 200a⟩≡
⟨*includes* 18⟩

```
// was in rc.h
```
⟨*global* gvar 32d⟩

⟨*function* hash 33a⟩

⟨*function* gvlook 32e⟩

⟨*function* vlook 32b⟩

⟨*function* setvar 32a⟩

⟨*function* newvar 33b⟩

F.3.11 rc/env.c

⟨rc/env.c 200b⟩≡
⟨*includes* 18⟩

```
// the Vxxx are back in plan9.c
```

F.3.12 rc/glob.c

⟨rc/glob.c 200c⟩≡
⟨*includes* 18⟩

```
bool matchfn(void*, void*);

// NDIR and Globsize are back in plan9.c
extern int Globsize(char *p);
```

⟨*global* globname 119b⟩
⟨*global* globv 118b⟩
⟨*function* deglob 119d⟩

⟨*function* globcmp 120a⟩

⟨*function* globsort 120b⟩
⟨*function* globdir 120c⟩
⟨*function* glob 119c⟩

⟨*function* equtf 121a⟩

⟨*function* nextutf 121b⟩

⟨*function* unicode 121c⟩

⟨*function* matchfn 122a⟩

⟨*function* match 122b⟩

⟨*function* globlist1 118e⟩

⟨*function* globlist 118d⟩

F.3.13 rc/executils.c

⟨rc/executils.c 201a⟩≡
⟨*includes* 18⟩

⟨*global* argv0 37a⟩

⟨*function* start 36e⟩

⟨*function* pushword 38b⟩

⟨*function* popword 65a⟩

⟨*function* pushlist 38a⟩

⟨*function* poplist 64d⟩

⟨*function* pushredir 77d⟩

⟨*function* Xerror 69b⟩

⟨*function* Xerror1 69c⟩

⟨*function* turfredir 81b⟩

⟨*function* Xpopredir 79b⟩

⟨*function* Xreturn 82d⟩

F.3.14 rc/exec.c

⟨rc/exec.c 201b⟩≡
⟨*includes* 18⟩

```
/*
 * Opcode routines
 * Arguments on stack (...)
 * Arguments in line [...]
 * Code in line with jump around {...}
 *
 * Xappend(file)[fd]   open file to append
 * Xassign(name, val)   assign val to name
 * Xasync{... Xexit}   make thread for {}, no wait
```

```
 * Xbackq{... Xreturn}   make thread for {}, push stdout
 * Xbang    complement condition
 * Xcase(pat, value){...}  exec code on match, leave (value) on
 *      stack
 * Xclose[i]    close file descriptor
 * Xconc(left, right)   concatenate, push results
 * Xcount(name)    push var count
 * Xdelfn(name)    delete function definition
 * Xdelhere
 * Xdol(name)    get variable value
 * Xdup[i j]    dup file descriptor
 * Xeflag
 * Xerror
 * Xexit    rc exits with status
 * Xfalse{...}    execute {} if false
 * Xfn(name){... Xreturn}   define function
 * Xfor(var, list){... Xreturn}  for loop
 * Xglob
 * Xif
 * Xifnot
 * Xjump[addr]    goto
 * Xlocal(name, val)   create local variable, assign value
 * Xmark    mark stack
 * Xmatch(pat, str)   match pattern, set status
 * Xpipe[i j]{... Xreturn}{... Xreturn} construct a pipe between 2 new threads,
 *      wait for both
 * Xpipefd[type]{... Xreturn}  connect {} to pipe (input or output,
 *      depending on type), push /dev/fd/??
 * Xpipewait
 * Xpopm(value)    pop value from stack
 * Xpopredir
 * Xrdcmds
 * Xrdfn
 * Xrdwr(file)[fd]   open file for reading and writing
 * Xread(file)[fd]   open file to read
 * Xqdol(name)    concatenate variable components
 * Xreturn    kill thread
 * Xsimple(args)   run command and wait
 * Xsub
 * Xsubshell{... Xexit}   execute {} in a subshell and wait
 * Xtrue{...}    execute {} if true
 * Xunlocal    delete local variable
 * Xwastrue
 * Xword[string]   push string
 * Xwrite(file)[fd]   open file to write
 */
```

⟨*function* Xappend 80c⟩

⟨*function* Xsettrue 88b⟩

⟨*function* Xbang 75c⟩

⟨*function* Xclose 135g⟩

⟨*function* Xdup 135c⟩

⟨*function* Xeflag 127a⟩

⟨*function* Xexit 85c⟩

⟨*function* `Xfalse` 75a⟩
⟨*global* `ifnot` 86c⟩

⟨*function* `Xifnot` 87c⟩

⟨*function* `Xjump` 88c⟩

⟨*function* `Xmark` 64c⟩

⟨*function* `Xpopm` 91a⟩

⟨*function* `Xread` 80a⟩

⟨*function* `Xrdwr` 133c⟩

⟨*function* `Xtrue` 74e⟩

⟨*function* `Xif` 86b⟩

⟨*function* `Xwastrue` 87a⟩

⟨*function* `Xword` 64b⟩

⟨*function* `Xwrite` 79a⟩

⟨*function* `list2str` 76a⟩

⟨*function* `Xmatch` 75e⟩

⟨*function* `Xcase` 91c⟩

⟨*function* `conclist` 99b⟩

⟨*function* `Xconc` 99a⟩

⟨*function* `Xassign` 95a⟩

⟨*function* `Xdol` 96b⟩

⟨*function* `Xqdol` 139c⟩

⟨*function* `subwords` 98a⟩

⟨*function* `Xsub` 97d⟩

⟨*function* `Xcount` 97b⟩

⟨*function* `Xlocal` 95b⟩

⟨*function* `Xunlocal` 95c⟩

⟨*function* `Xfn` 93a⟩

⟨*function* `Xdelfn` 93b⟩

⟨*function* `Xpipewait` 84d⟩

⟨*function* `Xrdcmds` 39a⟩

⟨*function* Xdelhere 130b⟩

⟨*function* Xfor 89a⟩

⟨*function* Xglob 119a⟩

⟨*global* envdir 110⟩

```
// Xrdfn is back in plan9.c
```

F.3.15 rc/processes.c

⟨rc/processes.c 204a⟩≡

```
⟨includes 18⟩
//#include <string.h>

// Fdprefix, delwaitpid(), havewaitpid() Waitfor() are back in plan9.c
extern char *Fdprefix;

⟨global waitpids 66b⟩
⟨global nwaitpids 66c⟩
⟨function addwaitpid 66d⟩
⟨function clearwaitpids 67a⟩


⟨function mkargv 72a⟩

⟨function Execute 71d⟩

// was in havefork.c

⟨function Xasync 85b⟩

⟨function Xpipe 83c⟩

⟨function Xbackq 137c⟩

⟨function Xpipefd 136f⟩

⟨function Xsubshell 128d⟩

⟨function execforkexec 70a⟩
```

Uses Fdprefix 204a and nwaitpids 66d.

F.3.16 rc/tree.c

⟨rc/tree.c 204b⟩≡

```
⟨includes 18⟩
#include "y.tab.h"

⟨global treenodes 25c⟩
⟨function newtree 25e⟩

⟨function freenodes 26a⟩

⟨function tree1 26b⟩

⟨function tree2 26c⟩
```

⟨*function* tree3 26d⟩

⟨*function* mung1 58e⟩

⟨*function* mung2 59b⟩

⟨*function* mung3 58f⟩

⟨*function* epimung 60a⟩
⟨*function* simplemung 57d⟩

F.3.17 rc/lex.c

⟨rc/lex.c 205a⟩≡
```
⟨includes 18⟩
#include "y.tab.h"

⟨constant NTOK 47d⟩

// was used by subr.c
⟨global lastdol 49b⟩
⟨global lastword 51e⟩
// was in rc.h
⟨global tok 47c⟩

⟨function wordchr 46b⟩

⟨function idchr 47a⟩


⟨function yyerror 153e⟩


⟨function skipwhite 47f⟩

⟨function skipnl 48a⟩

⟨function addtok 118a⟩

⟨function addutf 127b⟩

⟨function yylex 46a⟩
```

F.3.18 rc/trap.c

⟨rc/trap.c 205b⟩≡
```
⟨includes 18⟩

// NSIG, SIGINT, SIGQUIT are back in rc.h (under an ifdef for plan9)

// was in exec.h
⟨global ntrap 112b⟩
⟨global trap 112c⟩

// signame, syssigname, interrputed are back in plan9.c
// notifyf(), Trapinit(), Eintr(), Noerror() are back in plan9.c
extern char *signame[];
```

```
// generic part independent of plan9
⟨function dotrap 113b⟩
```

F.3.19 rc/simple.c

```
⟨rc/simple.c 206a⟩≡
/*
 * Maybe 'simple' is a misnomer.
 */
⟨includes 18⟩

// forward decls
void execfunc(var*);

⟨function exitnext 73b⟩

⟨function Xsimple 68c⟩

⟨function doredir 77e⟩

⟨function execexec 70b⟩

⟨function execfunc 94b⟩

⟨global rdcmds 106a⟩

⟨function execcmds 106b⟩
```

F.3.20 rc/pcmd.c

```
⟨rc/pcmd.c 206b⟩≡
⟨includes 18⟩
#include "y.tab.h"

⟨global nl 148a⟩
⟨constant c0 148b⟩
⟨constant c1 148c⟩
⟨constant c2 148d⟩

⟨function pdeglob 150a⟩

⟨function pcmd 148e⟩
```

F.3.21 rc/here.c

```
⟨rc/here.c 206c⟩≡
⟨includes 18⟩
#include "y.tab.h"

⟨global here 129d⟩
⟨global ehere 129e⟩
⟨global ser 131a⟩
⟨global tmp (rc/here.c) 131b⟩
⟨global hex 131c⟩

void psubst(io*, uchar*);
```

```
void pstrs(io*, word*);

⟨function hexnum 131d⟩

⟨function heredoc 131e⟩

⟨constant NLINE 129f⟩

⟨function readhere 129h⟩

⟨function psubst 131f⟩

⟨function pstrs 132a⟩
```

F.3.22 rc/code.c

```
⟨rc/code.c 207a⟩≡
  ⟨includes 18⟩
  #include "y.tab.h"

  ⟨constant c0 (rc/code.c) 63c⟩
  ⟨constant c1 (rc/code.c) 63d⟩
  ⟨constant c2 (rc/code.c) 63e⟩
  ⟨global codep 62a⟩
  ⟨global ncode 62b⟩
  ⟨function emitf 62d⟩
  ⟨function emiti 62c⟩
  ⟨function emits 62e⟩


  // forward decls
  void outcode(tree*, bool);
  void codeswitch(tree*, bool);
  bool iscase(tree*);

  ⟨function morecode 62f⟩

  ⟨function stuffdot 74c⟩

  ⟨function compile 63a⟩

  ⟨function cleanhere 130a⟩

  ⟨function fnstr 92d⟩

  ⟨function outcode 64a⟩
  ⟨function codeswitch 89c⟩

  ⟨function iscase 91b⟩

  ⟨function codecopy 27d⟩

  ⟨function codefree 27e⟩
```

F.3.23 rc/pfnc.c

```
⟨rc/pfnc.c 207b⟩≡
  ⟨includes 18⟩
```

⟨*global* fname 150c⟩

⟨*function* pfnc 151⟩

F.3.24 rc/utils.c

⟨rc/utils.c 208a⟩≡
⟨*includes* 18⟩

```
// Opendir, Closedir and so on are back in plan9.c
```

⟨*function* Unlink 130c⟩
⟨*function* Write 166d⟩
⟨*function* Read 166e⟩
⟨*function* Seek 166f⟩

⟨*function* Creat 166g⟩

⟨*function* Dup 166h⟩

⟨*function* Memcpy 167⟩

```
// back in plan9.c
extern void* Malloc(ulong n);
```
⟨*function* emalloc 155b⟩

⟨*function* efree 155d⟩

⟨*global* bp 166a⟩

⟨*function* iacvt 166c⟩

⟨*function* inttoascii 166b⟩

F.3.25 rc/status.c

⟨rc/status.c 208b⟩≡
⟨*includes* 18⟩

⟨*function* setstatus 65d⟩

⟨*function* getstatus 65e⟩

⟨*function* truestatus 66a⟩

⟨*function* concstatus 84f⟩

F.3.26 rc/builtins.c

⟨rc/builtins.c 208c⟩≡
⟨*includes* 18⟩

⟨*function* dochdir 102b⟩

⟨*function* appfile 102a⟩

⟨*function* mapfd 141a⟩

⟨*function* execcd 101c⟩

⟨*function* execexit 103c⟩

⟨*function* execeval 105e⟩

⟨*function* execflag 143⟩

```
// was in plan9.c
```

⟨*function* Executable 141b⟩

⟨*function* execwhatis 140⟩

⟨*function* execshift 144a⟩

⟨*global* dotcmds 105b⟩

⟨*function* execdot 103d⟩

⟨*function* execwait 106c⟩

⟨*function* execnewpgrp 142⟩

⟨*global* rdfns 144b⟩

```
extern fdt envdir;
```

⟨*function* execfinit 144c⟩

⟨*global* builtins 100b⟩

Uses builtins 208c and execcd() 101c.

F.3.27 rc/path.c

⟨rc/path.c 209a⟩≡
⟨*includes* 18⟩

⟨*global* nullpath 71c⟩

⟨*function* searchpath 71b⟩

F.3.28 rc/fmt.c

⟨rc/fmt.c 209b⟩≡
⟨*includes* 18⟩

⟨*global* pfmtnest 163a⟩

```
// forward decls
void pdec(io*, int);
void poct(io*, unsigned int);
void pptr(io*, void*);
void pval(io*, word*);
void pquo(io*, char*);
void pwrd(io*, char*);

// in io.c
int fullbuf(io*, int);
```

⟨*function* pfmt 163b⟩

⟨*function* pchr 161a⟩

⟨*function* pquo 165b⟩

⟨*function* pwrd 165c⟩

⟨*function* pptr 165d⟩

⟨*function* pstr 164a⟩

⟨*function* pdec 164b⟩

⟨*function* poct 165a⟩

⟨*function* pval 165e⟩

```
// _efgfmt is back in plan9.c
```

F.3.29 rc/words.c

⟨rc/words.c 210a⟩≡
⟨*includes* 18⟩

⟨*function* newword 29d⟩

⟨*function* freewords 30d⟩

⟨*function* count 30a⟩

⟨*function* copynwords 30b⟩

⟨*function* copywords 30c⟩

⟨*function* freelist 31a⟩

F.3.30 rc/error.c

⟨rc/error.c 210b⟩≡
⟨*includes* 18⟩

```
// was in rc.h
```

⟨*global* nerror 153c⟩
⟨*global* err 153b⟩

```
// forward decls
void Abort(void);

⟨function panic 154b⟩

// Exit is back in plan9.c

⟨function Abort 154c⟩
```

F.3.31 rc/main.c

```
⟨rc/main.c 211a⟩≡
  ⟨includes 18⟩

  // Rcmain and Isatty are back in plan9.c
  extern char* Rcmain;
  extern bool Isatty(fdt fd);

  /*
   * get command line flags.
   * initialize keywords & traps.
   * get values from environment.
   * set $pid, $cflag, $*
   * fabricate bootstrap code and start it (*=(argv);. /usr/lib/rcmain $*)
   * start interpreting code
   */
  ⟨function main (rc/exec.c) 34b⟩
```

Uses Rcmain 211a.

F.3.32 rc/plan9.c

```
⟨rc/plan9.c 211b⟩≡
  /*
   * Plan 9 versions of system-specific functions
   * By convention, exported routines herein have names beginning with an
   * upper case letter.
   */
  ⟨includes 18⟩

  // could be in main.c
  ⟨global Rcmain 116a⟩
  ⟨function Isatty 35f⟩

  // could be in rc.h
  ⟨enum MiscPlan9 108c⟩

  // defined in trap.c
  extern int trap[NSIG];
  // defined in exec.c
  extern fdt envdir;

  // could be in env.c
  ⟨function Vinit 109b⟩
  ⟨function addenv 109a⟩
  ⟨function updenvlocal 108b⟩
  ⟨function Updenv 108a⟩
```

```
// could be in glob.c
⟨constant NDIR 123a⟩
⟨function Globsize 123b⟩

// could be in processes.c
⟨global Fdprefix 136e⟩

// in processes.c
extern int *waitpids;
extern int nwaitpids;

⟨function delwaitpid 66f⟩
⟨function havewaitpid 67b⟩
⟨function Waitfor 72b⟩

// could be in trap.c
⟨global signame 112a⟩
⟨global syssigname 111d⟩
⟨global interrupted 113c⟩
⟨function notifyf 112e⟩
⟨function Trapinit 112d⟩
⟨function Eintr 113d⟩
⟨function Noerror 114a⟩

// could be in fmt.c
⟨function _efgfmt 155a⟩

// could be in error.c
⟨function Exit 154d⟩

// could be in exec.c
⟨function Xrdfn 145⟩

// could be in utils.c
⟨function Malloc 155c⟩

// could be in utils.c
⟨constant NFD 124a⟩
⟨struct DirEntryWrapper 124b⟩
⟨global dir 124c⟩
⟨function trimdirs 124e⟩
⟨function Readdir 125a⟩
⟨function Opendir 124d⟩
⟨function Closedir 125b⟩
```

F.3.33 rc/unix.c

⟨rc/unix.c 212⟩≡

```
/*
 * Unix versions of system-specific functions
 * By convention, exported routines herein have names beginning with an
 * upper case letter.
 */
// to avoid conflict for wait(), waitpid() signatures
#define NOPLAN9DEFINES
#include "rc.h"
#include "io.h"
#include "exec.h"
#include "getflags.h"
```

```
//TODO? use plan9port errstr instead?
#include <errno.h>

// system-specific globals defined here but used in other files
char *Rcmain = "/usr/lib/rcmain";
char *Fdprefix = "/dev/fd/";

//*****************************************************************************
// Environment
//*****************************************************************************

#define SEP '\1'

word*
enval(char *s)
{
  char *t, c;
  word *v;
  for(t = s; *t && *t!=SEP; t++);
  c=*t;
  *t='\0';
  v = newword(s, c=='\0'?(word *)0:enval(t+1));
  *t = c;
  return v;
}

// set in Vinit()
char **environp;

void
Vinit(void)
{
  // see Unix environ(7) man page
  extern char **environ;
  char *s;
  char **env = environ;
  word* wd;
  environp = env;
  for(; *env; env++){
    for(s=*env; *s && *s!='(' && *s!='='; s++);
    switch(*s){
    case '\0':
      pfmt(err, "environment %q?\n", *env);
      break;
    case '=':
      *s='\0';
      wd = enval(s+1);
      setvar(*env, wd);
      //TODO: should generalize this in main.c so it also applies for Plan9
      if(strcmp(*env, "RCMAIN") == 0) {
 Rcmain = strdup(wd->word);
      }
      *s='=';
      break;
    case '(': /* ignore functions for now */
      break;
    }
  }
}
```

```
//TODO: should be set in execfinit
char **envp = nil;

void
Xrdfn(void)
{
  char *s;
  int len;
  for(;*envp;envp++){
    for(s=*envp;*s && *s!='(' && *s!='=';s++);
    switch(*s){
    case '\0':
      pfmt(err, "environment %q?\n", *envp);
      break;
    case '=': /* ignore variables */
      break;
    case '(':  /* Bourne again */
      s=*envp+3;
      envp++;
      len = strlen(s);
      s[len]='\n';
      execcmds(opencore(s, len+1));
      s[len]='\0';
      return;
    }
  }
  Xreturn();
}
//
//union code rdfns[4];
//
//void
//execfinit(void)
//{
// static int first = 1;
// if(first){
//  rdfns[0].i = 1;
//  rdfns[1].f = Xrdfn;
//  rdfns[2].f = Xjump;
//  rdfns[3].i = 1;
//  first = 0;
// }
// Xpopm();
// envp = environp;
// start(rdfns, 1, runq->local);
//}
//
//int
//cmpenv(const void *aa, const void *ab)
//{
// char **a = aa, **b = ab;
//
// return strcmp(*a, *b);
//}
//
//char **
//mkenv(void)
//{
// char **env, **ep, *p, *q;
```

```
// struct var **h, *v;
// struct word *a;
// int nvar = 0, nchr = 0, sep;
//
// /*
//  * Slightly kludgy loops look at locals then globals.
//  * locals no longer exist - geoff
//  */
// for(h = gvar-1; h != &gvar[NVAR]; h++)
// for(v = h >= gvar? *h: runq->local; v ;v = v->next){
//  if((v==vlook(v->name)) && v->val){
//   nvar++;
//   nchr+=strlen(v->name)+1;
//   for(a = v->val;a;a = a->next)
//    nchr+=strlen(a->word)+1;
//  }
//  if(v->fn){
//   nvar++;
//   nchr+=strlen(v->name)+strlen(v->fn[v->pc-1].s)+8;
//  }
// }
// env = (char **)emalloc((nvar+1)*sizeof(char *)+nchr);
// ep = env;
// p = (char *)&env[nvar+1];
// for(h = gvar-1; h != &gvar[NVAR]; h++)
// for(v = h >= gvar? *h: runq->local;v;v = v->next){
//  if((v==vlook(v->name)) && v->val){
//   *ep++=p;
//   q = v->name;
//   while(*q) *p++=*q++;
//   sep='=';
//   for(a = v->val;a;a = a->next){
//    *p++=sep;
//    sep = SEP;
//    q = a->word;
//    while(*q) *p++=*q++;
//   }
//   *p++='\0';
//  }
//  if(v->fn){
//   *ep++=p;
//   *p++='#'; *p++='('; *p++=')'; /* to fool Bourne */
//   *p++='f'; *p++='n'; *p++=' ';
//   q = v->name;
//   while(*q) *p++=*q++;
//   *p++=' ';
//   q = v->fn[v->pc-1].s;
//   while(*q) *p++=*q++;
//   *p++='\0';
//  }
// }
// *ep = 0;
// qsort((void *)env, nvar, sizeof ep[0], cmpenv);
// return env;
//}

void
Updenv(void)
{
}
```

```
//*****************************************************************************
// Signals/notes and Waitfor()
//*****************************************************************************

char *sigmsg[] = {
/*  0 normal  */ 0,
/*  1 SIGHUP  */ "Hangup",
/*  2 SIGINT  */ 0,
/*  3 SIGQUIT */ "Quit",
/*  4 SIGILL  */ "Illegal instruction",
/*  5 SIGTRAP */ "Trace/BPT trap",
/*  6 SIGIOT  */ "abort",
/*  7 SIGEMT  */ "EMT trap",
/*  8 SIGFPE  */ "Floating exception",
/*  9 SIGKILL */ "Killed",
/* 10 SIGBUS  */ "Bus error",
/* 11 SIGSEGV */ "Memory fault",
/* 12 SIGSYS  */ "Bad system call",
/* 13 SIGPIPE */ 0,
/* 14 SIGALRM */ "Alarm call",
/* 15 SIGTERM */ "Terminated",
/* 16 unused  */ "signal 16",
/* 17 SIGSTOP */ "Process stopped",
/* 18 unused  */ "signal 18",
/* 19 SIGCONT */ "Process continued",
/* 20 SIGCHLD */ "Child death",
};

//pad: Note that plan9 Waitfor(), that was in processes.c (now in plan9.c), was
// compiling correctly also under Unix and I originally used it and rc
// was partially working. However, when called from mk (via MKSHELL), I had
// weird bugs like the $status was containing weird strings and so
// failing mk even if the command was run correctly.
// Anyway, simpler to uncomment and use the Waitfor() below.
void
Waitfor(int pid, bool persist)
{
 int wpid, sig;
 thread *p;
 int wstat;
 char wstatstr[12];

 for(;;){
  errno = 0;
  wpid = wait(&wstat);
  if(errno==EINTR && persist)
   continue;
  if(wpid==-1)
   break;
  sig = wstat&0177;
  if(sig==0177){
   pfmt(err, "trace: ");
   sig = (wstat>>8)&0177;
  }
  if(sig>(sizeof sigmsg/sizeof sigmsg[0]) || sigmsg[sig]){
   if(pid!=wpid)
    pfmt(err, "%d: ", wpid);
   if(sig<=(sizeof sigmsg/sizeof sigmsg[0]))
    pfmt(err, "%s", sigmsg[sig]);
```

```
   else if(sig==0177) pfmt(err, "stopped by ptrace");
   else pfmt(err, "signal %d", sig);
   if(wstat&0200)pfmt(err, " -- core dumped");
   pfmt(err, "\n");
  }
  wstat = sig?sig+1000:(wstat>>8)&0xFF;
  if(wpid==pid){
   inttoascii(wstatstr, wstat);
   setstatus(wstatstr);
   break;
  }
  else{
   for(p = runq->ret;p;p = p->ret)
    if(p->pid==wpid){
     p->pid=-1;
     inttoascii(p->status, wstat);
     break;
    }
  }
 }
}

char *signame[] = {
  "sigexit", "sighup", "sigint", "sigquit",
  "sigill", "sigtrap", "sigiot", "sigemt",
  "sigfpe", "sigkill", "sigbus", "sigsegv",
  "sigsys", "sigpipe", "sigalrm", "sigterm",
  "sig16", "sigstop", "sigtstp", "sigcont",
  "sigchld", "sigttin", "sigttou", "sigtint",
  "sigxcpu", "sigxfsz", "sig26", "sig27",
  "sig28", "sig29", "sig30", "sig31",
  0,
};

// defined in trap.c
extern int trap[NSIG];

void
gettrap(int sig)
{
  signal(sig, gettrap);
  trap[sig]++;
  ntrap++;
  if(ntrap>=NSIG){
    pfmt(err, "rc: Too many traps (trap %d), dumping core\n", sig);
    signal(SIGABRT, (void (*)())0);
    kill(getpid(), SIGABRT);
  }
}

void
Trapinit(void)
{
  int i;
  void (*sig)();

  if(1 || flag['d']){ /* wrong!!! */
    sig = signal(SIGINT, gettrap);
    if(sig==SIG_IGN)
      signal(SIGINT, SIG_IGN);
```

```
  }
  else{
    for(i = 1;i<=NSIG;i++) if(i!=SIGCHLD){
 sig = signal(i, gettrap);
 if(sig==SIG_IGN)
   signal(i, SIG_IGN);
      }
  }
}

//*******************************************************************************
// Errno
//*******************************************************************************

bool
Eintr(void){
  return errno==EINTR;
}

void
Noerror(void)
{
  errno = 0;
}

//*******************************************************************************
// Directories
//*******************************************************************************

#define NDIR 14  /* should get this from param.h */
int
Globsize(char *p)
{
  int isglob = 0, globlen = NDIR+1;
  for(;*p;p++){
    if(*p==GLOB){
      p++;
      if(*p!=GLOB)
 isglob++;
      globlen+=*p=='*'?NDIR:1;
    }
    else
      globlen++;
  }
  return isglob?globlen:0;
}

#include <sys/types.h>
#include <dirent.h>

#define NDIRLIST 50

DIR *dirlist[NDIRLIST];

Opendir(name)
     char *name;
{
  DIR **dp;
  for(dp = dirlist;dp!=&dirlist[NDIRLIST];dp++)
    if(*dp==0){
```

```
      *dp = opendir(name);
      return *dp?dp-dirlist:-1;
    }
  return -1;
}

int
Readdir(int f, char *p, int onlydirs)
{
  struct dirent *dp = readdir(dirlist[f]);

  if(dp==0)
    return 0;
  strcpy(p, dp->d_name);
  return 1;
}

void
Closedir(int f)
{
  closedir(dirlist[f]);
  dirlist[f] = 0;
}

//*****************************************************************************
// Misc
//*****************************************************************************

//char **
//mkargv(a)
//register struct word *a;
//{
// char **argv = (char **)emalloc((count(a)+2)*sizeof(char *));
// char **argp = argv+1; /* leave one at front for runcoms */
//
// for(;a;a = a->next)
//  *argp++=a->word;
// *argp = 0;
// return argv;
//}
//

//void
//Execute(struct word *args, struct word *path)
//{
// char *msg="not found";
// int txtbusy = 0;
// char **env = mkenv();
// char **argv = mkargv(args);
// char file[512];
//
// for(;path;path = path->next){
//  strcpy(file, path->word);
//  if(file[0])
//   strcat(file, "/");
//  strcat(file, argv[1]);
//ReExec:
//  execve(file, argv+1, env);
//  switch(errno){
//  case ENOEXEC:
```

```
//   pfmt(err, "%s: Bourne again\n", argv[1]);
//   argv[0]="sh";
//   argv[1] = strdup(file);
//   execve("/bin/sh", argv, env);
//   goto Bad;
//  case ETXTBSY:
//   if(++txtbusy!=5){
//    sleep(txtbusy);
//    goto ReExec;
//   }
//   msg="text busy"; goto Bad;
//  case EACCES:
//   msg="no access";
//   break;
//  case ENOMEM:
//   msg="not enough memory"; goto Bad;
//  case E2BIG:
//   msg="too big"; goto Bad;
//  }
// }
//Bad:
// pfmt(err, "%s: %s\n", argv[1], msg);
// efree((char *)env);
// efree((char *)argv);
//}


/*
 * Wrong:  should go through components of a|b|c and return the maximum.
 */
void
Exit(char *stat, char* loc)
{
  int n = 0;
  USED(loc);
  //if(flag['s'])
  //  fprint(STDERR, "Exit from %s: %s\n", loc, stat);

  while(*stat){
    if(*stat!='|'){
      if(*stat<'0' || '9'<*stat)
 exit(1);
      else n = n*10+*stat-'0';
    }
    stat++;
  }
  exit(n);
}

bool Isatty(fdt fd){
  return isatty(fd);
}

//void
//execumask(void)  /* wrong -- should fork before writing */
//{
// int m;
// struct io out[1];
// switch(count(runq->argv->words)){
// default:
```

```
//  pfmt(err, "Usage: umask [umask]\n");
//  setstatus("umask usage");
//  poplist();
//  return;
// case 2:
//  umask(octal(runq->argv->words->next->word));
//  break;
// case 1:
//  umask(m = umask(0));
//  out->fd = mapfd(1);
//  out->bufp = out->buf;
//  out->ebuf=&out->buf[NBUF];
//  out->strp = 0;
//  pfmt(out, "%o\n", m);
//  break;
// }
// setstatus("");
// poplist();
//}


void*
Malloc(unsigned long n)
{
  return (void *)malloc(n);
}

int
needsrcquote(int c)
{
  if(c <= ' ')
    return 1;
  if(strchr("`^#*[]=|\\?${}()'<>&;", c))
    return 1;
  return 0;
}

//void
//delwaitpid(int pid)
//{
// int r, w;
//
// for(r=w=0; r<nwaitpids; r++)
//  if(waitpids[r] != pid)
//   waitpids[w++] = waitpids[r];
// nwaitpids = w;
//}

//int
//havewaitpid(int pid)
//{
// int i;
//
// for(i=0; i<nwaitpids; i++)
//  if(waitpids[i] == pid)
//   return 1;
// return 0;
//
//}
```

Glossary

```
LOC = Lines Of Code
CLI = Command-Line Interface
GUI = Graphical User Interface
IDE = Integrated Development Environment
```

Indexes

Here is a list of the identifiers used, and where they appear. Underlined entries indicate the place of definition. This index is generated automatically.

`Abort()`: 154b
`addtok()`: 117b, 118a, 127b
`addutf()`: 51f, 52c, 127b
`addwaitpid()`: 66e, 136f, 137c
`advance()`: 45a, 45b, 46a, 47f, 48a, 51f, 52c, 54b, 127b, 133d, 153e
`APPEND`: 50a, 50f, 80b, 133d, 148e
`appfile<>()`: 101c, 102a, 103d, 140
`argv0`: 37a, 37a, 69b, 69c
`assert`: 148e, 162a
`BADFLAG<>`: 156f, 157b, 158
`badflag<>`: 156g, 156h, 157b, 158
`buf<>`: 160a, 160b, 160c
`bufp<>`: 160b, 160b, 160c
`Builtin`: 100a, 100c, 140, 194c
`Builtin.fnc`: 100a
`Builtin.name`: 100a
`builtin` (typedef): 194c
`builtins`: 101a, 140, 208c, 208c
`c0<>`: 63c, 67c, 67d, 67e, 73c, 74b, 74d, 75b, 75d, 78c, 82b, 85a, 86a, 87b, 87f, 88a, 88d, 89c, 91b, 91d, 91e, 92b, 94d, 96a, 97a, 97c, 98b, 128c, 136d, 137b, 139b, 148b, 148e
`c1<>`: 63d, 67d, 67e, 73c, 74b, 74d, 75d, 78c, 82b, 86a, 88a, 88d, 89c, 92b, 94d, 97c, 98b, 135b, 148c, 148e
`c2<>`: 63e, 88d, 94d, 148d, 148e
`cleanhere()`: 129h, 130a
`clearwaitpids()`: 70a, 72a, 136f, 137c
`CLOSE`: 133d, 194c
`Closedir()`: 119c
`closeio()`: 39d, 57d, 92d, 128d, 129h, 162b
`cmdname`: 155e, 156h, 158
`Code`: 25a, 28b, 73b, 105b, 106a, 194c
`Code.f`: 25a
`Code.s`: 50a
`code` (typedef): 194c
`codebuf`: 27b, 39a, 62f, 63a, 63b, 74c
`codecopy()`: 27d, 36e, 93a
`codefree()`: 27e, 82d, 93a, 93b
`codep`: 62a, 63a, 74c, 88a
`codeswitch()`: 89b, 89c
`compile()`: 63a

conclist(): 99a, 99b, 99b
concstatus(): 84d, 84f
copynwords(): 30b, 98a
copywords(): 30c, 85c, 96b
count(): 30a, 71d, 79a, 80a, 80c, 95a, 95b, 96b, 97b, 97d, 99a, 101b, 101c, 103c, 105e, 106c, 131f, 133c, 139c, 142, 143, 144a
Creat(): 79a, 80c, 129h, 155b
deglob(): 95a, 95b, 96b, 97b, 97d, 98a, 120b, 121a, 139c
delwaitpid(): 83c, 204a
dir: 166b, 166c
DirEntryWrapper: 166c, 208a
DirEntryWrapper.dbuf: 208a
DirEntryWrapper.n: 166a
DMDIR: 141b
dochdir(): 101c, 102b
doprompt: 43b, 43b, 43f, 43g, 44b, 44c, 129h
doredir(): 76b, 77e, 77e
dotcmds: 103d, 105b, 105d
dotrap(): 160h
Dup(): 78a, 135h, 155b
DUPFD: 25a, 133d, 135b, 148e
eflagok: 105e, 126c, 126d, 127a
efree(): 26a, 27e, 30d, 31a, 39d, 63b, 64d, 65a, 68b, 75e, 79b, 82d, 85b, 91c, 92c, 95c, 99b, 103d, 105e, 120c, 121a, 129h, 139c, 144a, 162b
ehere: 129e, 131e
Eintr(): 39d
emalloc(): 63a, 71d, 76a, 99b, 102a, 105e, 120c, 121a, 139c, 162c, 162d
emitf<>: 62d, 63a, 67c, 68a, 74b, 74d, 75b, 75d, 78c, 78d, 79c, 80b, 82b, 85a, 86a, 87b, 88a, 88d, 89c, 92b, 94d, 96a, 97a, 97c, 98b, 126b, 128c, 130a, 133b, 135b, 136d, 137b, 139b
emiti<>: 62c, 63a, 74b, 74d, 78c, 82b, 85a, 86a, 87b, 88a, 88d, 89c, 92b, 128c, 135b, 136d, 137b
emits<>: 62e, 68a, 88d, 92b, 130a
emptybuf(): 161c, 161d
envdir: 110, 144c
EOF: 42a, 42b, 42d, 42f, 43a, 44b, 44d, 44e, 45a, 46b, 47b, 47f, 51f, 129h, 133d, 136f, 153e, 161d, 162a
epimung(): 60a
equtf(): 121b, 122b
err: 39d, 43g, 64a, 68c, 69b, 69c, 79a, 80a, 80c, 85b, 101b, 101c, 103c, 103d, 115, 133c, 140, 142, 144a, 144c, 152a, 152b, 152c, 153b, 153d, 153e, 154b
errc<>(): 158, 159a, 159b, 160c
ERRMAX: 83c, 163b
errn<>(): 158, 159a
ERROR_0: 63b
ERROR_NEG1: 102b
errs<>(): 158, 159b
execcd(): 101c, 208c
execcmds(): 105e, 106b
execdot(): 100b, 103d
execeval(): 100b, 105e
execexec(): 70b, 73a, 100b

execexit(): 103c
execfinit(): 100b, 144c
execflag(): 100b, 143
execforkexec(): 68c
execfunc(): 94a, 94b
execnewpgrp(): 100b, 142
execshift(): 100b, 144a
Executable(): 140, 141b
Execute(): 70b, 71d
execwait(): 100b, 106c
execwhatis(): 100b, 140
Exit(): 82d, 85c, 154c, 158
exitnext(): 73a, 73b
false: 33b, 36e, 39d, 43g, 45b, 47b, 47f, 48b, 49c, 51b, 51d, 51f, 52c, 54b, 63a, 66a, 71d, 74b, 74d, 85c, 86a, 87a, 87g, 88a, 91b, 91c, 103d, 105d, 106b, 113b, 118d, 118e, 121c, 122b, 137b, 141b, 144c, 153e
Fdprefix: 204a, 204a
FEWARGS<>: 156d, 156h, 158
flag: 34c, 63a, 103b, 115, 116f, 116g, 143, 152a, 152b, 152c, 156h
flagarg<>: 156a, 156a, 156h, 158
flagset: 35b, 35b, 116f, 143, 156h
FLAGSYN<>: 156e, 157b, 158
flush(): 43g, 68c, 69b, 69c, 129h, 140, 151, 153d, 153e, 154b, 160h, 161b, 163b
fname: 150c, 150c, 151
fnstr(): 92b, 92d
freelist(): 31a, 64d, 89a
freenodes(): 26a, 39a
freewords(): 30d, 32a, 95a, 95c
fullbuf(): 161a, 161b
future: 44d, 44d, 44e, 45a, 154a
getflags(): 156h
getnext(): 41, 44e
getstatus(): 65e, 66a, 82d, 84d, 85c, 154c
GLOB: 31c, 117b, 119d, 120b, 120c, 122b, 150a
glob(): 118d
globcmp(): 120c, 120c
globdir(): 119c, 120c, 121a
globlist(): 93a, 95a, 95b, 118c, 119a
globlist1(): 118d, 118d
globname: 119c, 120c, 121a
Globsize(): 121a
globsort(): 120c, 121b
globv: 119c, 120a, 120c, 121a, 121b
gvar: 32d, 32e
gvlook(): 32b, 32e, 93a, 93b, 94a, 140
hash(): 32e, 33a, 53a, 53f
havewaitpid(): 71d, 83c
HERE: 79c, 129b, 133d, 148e
Here: 129d, 129e, 129h, 131e, 194c
here: 129d, 129h, 131e

Here.name: 194c
Here.next: 194c
Here.tag: 194c
heredoc(): 131e
hex: 131c, 131c, 131d
hexnum(): 131d, 131e
idchr(): 47a, 52c, 131f
ifnot: 86b, 86c, 87a, 87c
incomm: 42e, 47e, 47f
inquote: 42e, 51c, 51d, 51f
interrupted: 113b, 113b
inttoascii(): 97b, 115, 137c
Io: 39b, 57d, 136f, 140, 160e, 194c
Io.buf: 160e
Io.bufp: 160e
Io.ebuf: 160e
Io.fd: 160e
Io.strp: 160e
io (typedef): 194c
Isatty(): 211a
iscase(): 89c, 91b
kenter(): 53e, 53f
kinit(): 53e, 115
klook(): 52c, 53a
Kw: 53a, 53b, 53b, 53c, 53f
kw: 53a, 53c, 53f
Kw.name: 53b
Kw.next: 53b
Kw.type: 53b
lastc: 153e, 153f, 154a
lastdol: 47b, 48b, 49a, 49b, 49c, 51b, 51f, 52c, 153e
lastword: 51e, 51f, 52c, 54b, 153e
List: 29a, 31b, 194c
List.next: 31b
List.words: 31b
list2str(): 75e, 76a, 91c
list (typedef): 194c
main<>(): 35e
mapfd(): 140, 141a
match(): 75e, 91c, 122b, 122b
matchfn(): 119c, 122b
Maxenvname: 194c
Memcpy(): 162d
mkargv(): 71d, 71d
morecode(): 62f
mung1(): 58e
mung2(): 59b
mung3(): 58f
mypid: 85c, 115, 116d

NBUF: 140, 160d, 160e, 160h, 161d
NBUF<>: 159c, 160a, 160c
ncode: 62b, 62f, 63a
NDIR<>: 119d, 200c
ndot: 103d, 105a, 152c
nerror: 63b, 153c, 153d, 153e
newtree(): 25e, 26d, 49g, 52b
newvar(): 32e, 33b, 85c, 94b, 95b
newword(): 29d, 30b, 30c, 38b, 65d, 96b, 99b, 115, 119c, 120c, 121a, 128d, 137c, 153e
nextc(): 44e, 45a, 45b, 47f, 48a, 51f, 52c, 54a
nextis(): 45b, 48b, 49a, 49c, 50f, 129b, 133a, 133d
nextutf(): 118e, 122a, 122b
NFD<>: 155d, 166b, 166c
NFLAG: 34c, 35a, 157b
nil: 25e, 26a, 26b, 26c, 27a, 30b, 30c, 33b, 36e, 38a, 38b, 38c, 49f, 51f, 52c, 57e, 62f, 63a, 64a, 64d, 65b, 65c, 65d, 66d, 69a, 71a, 71b, 71c, 71d, 76c, 79b, 82d, 83c, 85b, 85c, 89c, 92d, 93b, 94b, 95a, 95b, 101c, 103b, 103d, 115, 118a, 129h, 136f, 137c, 140, 141b, 148e, 153e, 160f
NKW<>: 53a, 53c, 53d, 53f
nl: 92d, 148a, 148a, 148e
NLINE<>: 129f, 129h
Noerror(): 113e
NSIG<>: 113b, 205b
NSTATUS: 84b, 84c, 84d, 84f
NTOK<>: 47c, 47d, 118a
ntrap: 114b, 160h, 205b
nullpath: 71b, 71c, 71c, 101c
NVAR: 29c, 32d, 32e
nwaitpids: 66d, 66d, 67a, 71d, 72a, 204a
OCEXEC: 103b
OK_0: 62f
OK_1: 63a, 102b
opencore(): 105e, 162d
Opendir(): 119c, 166c
openfd(): 38c, 103d, 115, 129h, 136f, 160f
openstr(): 57d, 92d, 162c
ORDWR: 133c
outcode(): 63a, 64a, 67c, 67d, 67e, 73c, 74b, 74d, 75b, 75d, 78c, 82b, 85a, 86a, 87b, 88a, 88d, 89c, 91d, 91e, 92b, 94d, 96a, 97a, 97c, 98b, 128c, 135b, 136d, 137b, 139b
OWRITE: 103b
panic(): 38b, 62f, 64d, 65b, 65c, 74c, 79b, 95c, 154b, 160h
pchr(): 39d, 129h, 131f, 132a, 148e, 150a, 151, 152c, 154b, 161a, 163b, 164a, 164b, 165a, 165b, 165d, 165e
pcmd(): 148e, 163b
pdec(): 163b, 164b, 164b
pdeglob(): 148e, 150a
pfmt(): 57d, 64a, 69b, 69c, 79a, 80a, 80c, 85b, 92d, 101b, 101c, 103c, 103d, 133c, 140, 142, 144a, 144c, 148e, 151, 152a, 152b, 153e, 154b, 163b, 165b
pfmtnest: 163a, 163a, 163b
pfnc(): 151
poct(): 163b, 165a, 165a

poplist(): 64d, 68c, 70b, 75e, 79a, 80a, 80c, 82d, 89a, 91a, 91c, 93a, 93b, 94b, 95a, 95b, 96b, 97b, 97d, 99a, 101b, 101c, 105e, 106c, 133c, 139c, 140, 142, 143, 144a
popword(): 65a, 70b, 94b, 101b, 103d
pprompt(): 43f, 43g, 129h
pptr(): 163b, 165d
pquo(): 163b, 165b, 165c
PRD: 31d, 70a, 136f, 137c
promptstr: 43c, 43e, 43g
pstr(): 43g, 129h, 131f, 132a, 151, 163b, 164a, 165c
pstrs(): 131f, 132a
psubst(): 129h, 131f
pushlist(): 38a, 38c, 64c, 103d
pushredir(): 77d, 79a, 80a, 80c, 103d, 133c, 135c, 135g, 136f, 137c
pushword(): 38b, 64b, 73a, 97b, 103d, 139c
pval(): 163b, 165e
PWR: 70a, 136f, 137c
pwrd(): 163b, 165c, 165e
rchr(): 41, 42e, 129h, 161c, 162a
RCLOSE: 103d, 135f, 135g, 135i, 141a
Rcmain: 115, 211a, 211a
rdcmds: 106a, 106b
rdfns: 144c
RDUP: 135c, 135e, 135h, 141a
RDWR: 133a, 133b, 148e, 194c
READ: 51a, 70a, 79c, 148e
Read(): 161d, 208a
Readdir(): 119c
readhere(): 129g, 129h
reason<>: 156b, 156h, 157b, 158
Redir: 77a, 77b, 77c, 79b, 81a, 194c
Redir.from: 77a
Redir.next: 77c
Redir.to: 77a
Redir.type: 77a
redir (typedef): 194c
RESET<>: 156c, 156h, 158
reverse<>(): 156h, 157a
RFCENVG: 142
RFCFDG: 142
RFCNAMEG: 142
RFENVG: 142
RFFDG: 137c, 142
RFNAMEG: 142
RFNOMNT: 142
RFNOTEG: 137c, 142
RFPROC: 137c
ROPEN: 78a, 78b, 79a, 79b, 80a, 80c, 133c, 136f, 137c, 141a
Runeerror: 162a
Runeself: 127b, 131f

Runesync: 162a
runq: 28c, 32b, 36e, 38a, 38b, 38c, 39a, 41, 42b, 42d, 42e, 43e, 43g, 64b, 64d, 65a, 65b, 68c, 69b, 69c, 70a, 70b, 71a, 73b, 74e, 75a, 75e, 76b, 76c, 77d, 79a, 79b, 80a, 80c, 81b, 82d, 83c, 84d, 85c, 86b, 87c, 87d, 87f, 87g, 88c, 89a, 91c, 93a, 93b, 94b, 95a, 95b, 95c, 96b, 97b, 97d, 99a, 101c, 103c, 103d, 105e, 106b, 106c, 128d, 129h, 130b, 133c, 135c, 135g, 136f, 137c, 139c, 140, 141a, 142, 143, 144a, 144c, 153e
rutf(): 136f, 162a
scanflag<>(): 156h, 157b
searchpath(): 70b, 71b, 103d, 140
Seek(): 80c, 167
ser: 131a, 131a, 131e
setstatus(): 65d, 69b, 69c, 75c, 75e, 83c, 84d, 88b, 101b, 101c, 103c, 103d, 140, 142, 143, 144a, 154c
setvar(): 32a, 65d, 115, 137c, 153e
signame: 113b, 113b
simplemung(): 57d
skipnl(): 48a, 48b, 49c, 49g
skipwhite(): 46a, 47f, 48a
start(): 36e, 38c, 39a, 70a, 85c, 94b, 103d, 106b, 136f, 137c, 144c
STDERR: 115
STDIN: 38c
Stralloc<>: 160g, 160h, 162c
stuffdot(): 74b, 74c, 74d, 82b, 85a, 86a, 87b, 88a, 88d, 89c, 92b, 128c, 136d, 137b
subwords(): 97d, 98a, 98a
syssigname: 113b, 113b
Thread: 28b, 36e, 39a, 70a, 82d, 137c, 194c
Thread.argv: 29a
Thread.cmdfd: 39b
Thread.cmdfile: 39b
Thread.code: 28b
Thread.eof: 42c
Thread.iflag: 39b
Thread.iflast: 87e
Thread.lineno: 44a
Thread.local: 29b
Thread.pc: 28b
Thread.pid: 84a
Thread.redir: 77b
Thread.ret: 28d
Thread.startredir: 81a
Thread.status: 84b
thread (typedef): 194c
tmp: 131b, 131b, 131e
tok: 47b, 47c, 48b, 49a, 49c, 49d, 51b, 51f, 52c, 54b, 118a, 153e
token(): 51f, 52b, 53a, 131e
trap: 113b
Trapinit(): 115
Tree: 49e, 163b, 194c, 194c
Tree.child: 194c
Tree.fd0: 194c
Tree.fd1: 194c

Tree.rtype: 194c
Tree.str: 194c
Tree.type: 194c
tree1(): 26b, 57d
tree2(): 26c
tree3(): 26b, 26c, 26d
tree (typedef): 194c
treenodes: 25c, 25e, 26a
true: 32a, 34a, 42b, 43b, 44c, 45b, 47f, 49a, 51f, 52c, 63a, 66a, 71d, 85c, 86b, 89a, 91c, 93a, 93b, 94b, 95a, 95b, 95c, 103d, 105c, 105e, 106b, 126d, 129h, 144a, 144c
truestatus(): 66a, 74e, 75a, 75c, 86b, 127a, 152a, 154c
turfredir(): 81b, 81c
unicode(): 118e, 122a, 122b
Unlink(): 130b
Updenv(): 68c, 71d, 154c
usage(): 158
UTFmax: 47c, 127b, 136f
Var: 29b, 32a, 50d, 85c, 103d, 194c
Var.fnchanged: 129a
Var.next: 134a
var (typedef): 194c
Vinit(): 115
vlook(): 32a, 32b, 43e, 43g, 65e, 71b, 85c, 95a, 95c, 96b, 97b, 97d, 101c, 131f, 136f, 139c, 140, 144a, 152a
Waitfor(): 68c, 84d, 85b, 106c, 128d
waitpids: 66c, 66d, 67a, 71d, 72a
Word: 71c, 85c, 120a, 163b, 194c, 194c
Word.next: 27c
Word.word: 194c
word (typedef): 194c
wordchr(): 46b, 51b, 52c, 54b
WRITE: 50f, 78d, 148e
Write(): 160c, 160h, 208a
Xappend(): 28a, 80b, 80c, 150c
Xassign(): 39c, 94d, 95a, 150c
Xasync(): 28a, 85a, 150c
Xbackq(): 28a, 137b, 150c, 204a
Xbang(): 75b, 75c, 150c
Xcase(): 28a, 89c, 91c, 150c
Xclose(): 28a, 135b, 135g, 150c
Xconc(): 98b, 99a, 150c
Xcount(): 97a, 97b, 150c
Xdelfn(): 92b, 93b, 150c
Xdelhere(): 68b, 130a, 130b, 150c
Xdol(): 37b, 88d, 96a, 96b, 150c
Xdup(): 135b, 135c, 135d, 150c
Xeflag(): 75d, 126b, 127a, 128c, 150c
Xerror(): 68c, 69b, 70a, 79a, 80a, 80c, 103d, 133c, 136f, 137c, 150c
Xerror1(): 69a, 69c, 71a, 79a, 80a, 80c, 95a, 95b, 96b, 97b, 97d, 99a, 103d, 105e, 106c, 133c, 139c, 140, 143
Xexit(): 38c, 73a, 73b, 82b, 85a, 85c, 103c, 127a, 128c, 136d, 137b, 150c

Xfalse(): 28a, 74d, 75a, 150c
Xfn(): 92b, 92c, 93a, 150c
Xfor(): 28a, 88d, 89a, 150c
Xglob(): 78c, 88d, 119a, 150c
Xif(): 86a, 86b, 150c
Xifnot(): 87b, 87c, 150c
Xjump(): 28a, 88a, 88c, 88d, 89c, 144c, 150c
Xlocal(): 88d, 94d, 95b, 105d, 150c
Xmark(): 39c, 64c, 67c, 75d, 78c, 88d, 89c, 92b, 94d, 96a, 97a, 97c, 98b, 105d, 139b, 150c
Xmatch(): 75d, 75e, 150c
Xpipe(): 82b, 82c, 137c, 150c
Xpipefd(): 70a, 135d, 136d, 150c
Xpipewait(): 82b, 84d, 150c
Xpopm(): 89c, 91a, 144c, 150c
Xpopredir(): 73b, 78c, 79b, 81b, 135b, 150c
Xqdol(): 139b, 139c, 150c
Xrdcmds(): 39a, 105d, 106b, 150c
Xrdfn(): 144c, 150c
Xrdwr(): 28a, 133b, 133c, 150c
Xread(): 28a, 79c, 80a, 150c
Xreturn(): 39d, 63a, 69b, 69c, 82b, 82d, 92b, 105d, 106b, 150c
Xsettrue(): 88a, 88b
Xsimple(): 34b, 67c, 68c, 150c
Xsub(): 97c, 97d, 150c
Xsubshell(): 28a, 128c, 150c
Xtrue(): 28a, 74b, 74e, 88a, 150c
Xunlocal(): 88d, 92b, 94d, 95c, 105d, 150c
Xwastrue(): 86a, 87a, 150c
Xword(): 36d, 37b, 64b, 68a, 68b, 88d, 105d, 150c
Xwrite(): 28a, 78d, 79a, 150c
yyerror(): 87d, 89c, 118a, 129h, 131e, 133d, 153e
yylex(): 46a
__anon_enum_6: 160g
__anon_enum_8: 194c
__anon_struct_7.f: 150c
__anon_struct_7.name: 150c
__anon_struct_7: 150c, 150c

Bibliography

[BK89] Morris Bolsky and David Korn. *The KornShell Command and Programming Language*. Prentice Hall, 1989. cited page(s) 10

[BKM86] Jon Bentley, Donald Knuth, and Doug McIlroy. Programming pearls: A literate program. *Communication of the ACM*, 29(6):471–483, June 1986. The end of the article contains a one-line shell script solving a problem that required Donald Knuth a hundred lines of Pascal code. cited page(s) 8

[BLM92] Doug Brown, John Levine, and Tony Mason. *Lex and Yacc*. O'Reilly, 1992. cited page(s) 11

[Bou79] Stephen R. Bourne. An introduction to the unix shell. In *Unix Programmer's Manual Vol 2a*, 1979. Also available at `shell/docs/sh.pdf`. cited page(s) 9, 11

[Bou15] Stephen R. Bourne. Early days of unix and design of sh. In *BSDCan - The BSD Conference*, 2015. `https://www.bsdcan.org/2015/schedule/attachments/306_srbBSDCan2015.pdf`, also in `shells/docs/early-days-unix-sh-bourne.pdf`. cited page(s) 12

[Duf90] Tom Duff. Rc – a shell for plan 9 and unix. In *Unix Research System: Papers (Vol 2) 10th ed.* Saunders College, 1990. cited page(s) 9

[Duf00] Tom Duff. Rc – the plan 9 shell. Technical report, Bell Labs, 2000. Also available at `shells/docs/rc.pdf`. cited page(s) 12

[Joh79] Stephen C. Johnson. Yacc: Yet another compiler-compiler. In *Unix Programmer's Manual Vol 2b*, 1979. Also available at `generators/docs/yacc.pdf`. cited page(s) 9

[Joy86] William Joy. An introduction to the c shell. In *UNIX Programmer's Supplementary Documents 1*, 1986. Available at `https://docs.freebsd.org/44doc/usd/04.csh/paper.pdf`. cited page(s) 10

[Kid05] Oliver Kiddle. *From Bash to Z Shell*. Apress, 2005. cited page(s) 10, 11

[Knu92] Donald E. Knuth. *Literate Programming*. Center for the Study of Language and Information, 1992. cited page(s) 12

[Kor94] David G. Korn. ksh - an extensible high level language. In *Proceedings of the USENIX 1994 Very High Level Languages Symposium*, 1994. cited page(s) 10

[KP84] Brian W. Kernighan and Rob Pike. *The UNIX Programming Environment*. Prentice Hall, 1984. cited page(s) 11

[KR88] Brian W. Kernighan and Dennis M. Ritchie. *The C Programming Language*. Prentice Hall, 1988. cited page(s) 11

[Mic08] Randal K. Michael. *Master UNIX Shell Scripting*. Wiley, 2008. cited page(s) 11

[New95] Cameron Newham. *Learning the bash Shell*. O'Reilly, 1995. cited page(s) 9, 11

[Pad09] Yoann Padioleau. Syncweb, literate programming meets unison, 2009. https://github.com/aryx/syncweb. cited page(s) 12

[Pad14] Yoann Padioleau. *Principia Softwarica: The Plan 9 Kernel 9*. 2014. cited page(s) 11, 12, 15, 16, 18, 65, 111, 146

[Pad16a] Yoann Padioleau. *Principia Softwarica: The Build System mk*. 2016. cited page(s) 146

[Pad16b] Yoann Padioleau. *Principia Softwarica: The Plan 9 C Compiler 5c*. 2016. cited page(s) 12, 21, 24, 55, 146

[Pad16c] Yoann Padioleau. *Principia Softwarica: The Plan 9 Windowing System rio*. 2016. cited page(s) 9, 103

[Pad25] Yoann Padioleau. *Principia Softwarica: The Plan 9 Utilities*. 2025. cited page(s) 171

[Pou00] Louis Pouzin. The origin of the shell, 2000. http://multicians.org/shell.html. cited page(s) 13, 14

[Ram94] Chet Ramey. Bash, the bourne-again shell, 1994. Available in bash/doc/rose94.pdf. cited page(s) 9

[Roc04] Marc J. Rochkind. *Advanced UNIX Programming*. Addison-Wesley, 2004. cited page(s) 8, 12

[Ste94] Richard W. Stevens. *Advanced Programming in the UNIX Environment*. Addison-Wesley, 1994. cited page(s) 12